# MathOptInterface

The JuMP core developers and contributors

October 29, 2026

# Contents


**Contents** ii

**I Introduction** 1

**1 Introduction** 2

1.1 What is MathOptInterface? . . . 2

1.2 How the documentation is structured . . . 2

1.3 Citing MathOptInterface . . . 3

**2 Motivation** 4

**II Tutorials** 5

**3 Solving a problem using MathOptInterface** 6

3.1 Required packages . . . 6

3.2 Define the data . . . 6

3.3 Add the variables . . . 7

3.4 Set the objective . . . 7

3.5 Add the constraints . . . 7

3.6 Optimize the model . . . 7

3.7 Understand why the solver stopped . . . 7

3.8 Understand what solution was returned . . . 8

3.9 Query the objective . . . 8

3.10 Query the primal solution . . . 8

**4 Implementing a solver interface** 9

4.1 A note on the API . . . 9

4.2 Preliminaries . . . 9

4.3 Structuring the package . . . 11

4.4 Initial code . . . 12

4.5 The big decision: incremental modification? . . . 15

4.6 Names . . . 18

4.7 Solutions . . . 19

4.8 Other tips . . . 21

**5 Transitioning from MathProgBase** 23

5.1 Transitioning a solver interface . . . 23

5.2 Transitioning the high-level functions . . . 23

**6 Implementing a constraint bridge** 25

6.1 Preliminaries . . . 25

6.2 Four mandatory parts in a constraint bridge . . . 25

6.3 Bridge registration . . . 28

6.4 Bridge improvements . . . 29

**7 Manipulating expressions** 32

7.1 Creating functions . . . 32

7.2 Canonicalizing functions . . . 33
7.3 Exploring functions . . . 33
**8 Latency 35**
8.1 Background . . . 35
8.2 Causes . . . 35
8.3 Resolutions . . . 38

**III Manual 41**
**9 Standard form problem 42**
9.1 Functions . . . 42
9.2 One-dimensional sets . . . 42
9.3 Vector cones . . . 42
9.4 Matrix cones . . . 44
9.5 Multi-dimensional sets with combinatorial structure . . . 44
**10 Models 46**
10.1 Attributes . . . 46
10.2 ModelLike API . . . 47
10.3 AbstractOptimizer API . . . 47
**11 Variables 49**
11.1 Add a variable . . . 49
11.2 Delete a variable . . . 49
11.3 Variable attributes . . . 50
**12 Constraints 51**
12.1 Add a constraint . . . 51
12.2 Delete a constraint . . . 52
12.3 Constraint attributes . . . 52
12.4 Constraints by function-set pairs . . . 52
12.5 JuMP mapping . . . 54
**13 Solutions 55**
13.1 Solving and retrieving the results . . . 55
13.2 Why did the solver stop? . . . 55
13.3 Primal solutions . . . 55
13.4 Dual solutions . . . 56
13.5 Common status situations . . . 56
13.6 Querying solution attributes . . . 57
**14 Problem modification 59**
14.1 Modify the set of a constraint . . . 59
14.2 Modify the function of a constraint . . . 60
14.3 Modify constant term in a scalar function . . . 61
14.4 Modify constant terms in a vector function . . . 61
14.5 Modify affine coefficients in a scalar function . . . 62
14.6 Modify quadratic coefficients in a scalar function . . . 62
14.7 Modify affine coefficients in a vector function . . . 63

**IV Background 65**
**15 Duality 66**
15.1 Duality and scalar product . . . 68
15.2 Dual for problems with quadratic functions . . . 68
15.3 Dual for square semidefinite matrices . . . 70
**16 Infeasibility certificates 72**
16.1 Conic duality . . . 72

16.2 Unbounded problems . . . 73
16.3 Infeasible problems . . . 74
**17 Naming conventions 75**
17.1 Sets . . . 75

**V API Reference 76**
**18 Standard form 77**
18.1 Functions . . . 77
18.2 Scalar functions . . . 78
18.3 Vector functions . . . 82
18.4 Sets . . . 86
18.5 Scalar sets . . . 89
18.6 Vector sets . . . 93
18.7 Constraint programming sets . . . 110
18.8 Matrix sets . . . 119
**19 Models 126**
19.1 Attribute interface . . . 126
19.2 Model interface . . . 132
19.3 Model attributes . . . 136
19.4 Optimizer interface . . . 146
19.5 Optimizer attributes . . . 148
**20 Variables 177**
20.1 Functions . . . 177
20.2 Attributes . . . 181
**21 Constraints 185**
21.1 Types . . . 185
21.2 Functions . . . 185
21.3 Attributes . . . 188
**22 Modifications 198**
**23 Nonlinear programming 203**
23.1 Types . . . 203
23.2 Attributes . . . 204
23.3 Functions . . . 206
**24 Callbacks 222**
24.1 Attributes . . . 223
24.2 Lazy constraints . . . 224
24.3 User cuts . . . 225
24.4 Heuristic solutions . . . 226
**25 Errors 229**

**VI Submodules 235**
**26 Benchmarks 236**
26.1 Overview . . . 236
26.2 API Reference . . . 237
**27 Bridges 240**
27.1 Overview . . . 240
27.2 Implementation . . . 243
27.3 List of bridges . . . 245
27.4 API Reference . . . 300
**28 FileFormats 325**
28.1 Overview . . . 325

28.2 API Reference . . . 330
28.3 The LP file format . . . 337
**29 Nonlinear 342**
29.1 Overview . . . 342
29.2 SymbolicAD . . . 352
29.3 API Reference . . . 357
**30 Utilities 376**
30.1 Overview . . . 376
30.2 API Reference . . . 388
**31 Test 434**
31.1 Overview . . . 434
31.2 API Reference . . . 439

**VII Developer Docs 444**
**32 Checklists 445**
32.1 Making a release . . . 445
32.2 Adding a new set . . . 445
32.3 Adding a new bridge . . . 446
32.4 Updating MathOptFormat . . . 447
**33 Contributing 448**
33.1 Obtain the source code . . . 448
33.2 Make changes . . . 448
33.3 Running the tests . . . 449
33.4 Building the documentation . . . 450
**34 Defining a new set 451**

# Part I

# Introduction

# Chapter 1

# Introduction

Welcome to the documentation for MathOptInterface.

**Note**

This documentation is also available in PDF format: MathOptInterface.pdf.

## 1.1 What is MathOptInterface?

MathOptInterface.jl (MOI) is an abstraction layer designed to provide a unified interface to mathematical optimization solvers so that users do not need to understand multiple solver-specific APIs.

**Tip**

This documentation is aimed at developers writing software interfaces to solvers and modeling languages using the MathOptInterface API. If you are a user interested in solving optimization problems, we encourage you instead to use MOI through a higher-level modeling interface like JuMP or Convex.jl.

## 1.2 How the documentation is structured

Having a high-level overview of how this documentation is structured will help you know where to look for certain things.

- The **Tutorials** section contains articles on how to use and implement the MathOptInteraface API. Look here if you want to write a model in MOI, or write an interface to a new solver.
- The **Manual** contains short code-snippets that explain how to use the MOI API. Look here for more details on particular areas of MOI.
- The **Background** section contains articles on the theory behind MathOptInterface. Look here if you want to understand why, rather than how.
- The **API Reference** contains a complete list of functions and types that comprise the MOI API. Look here if you want to know how to use (or implement) a particular function.
- The **Submodules** section contains stand-alone documentation for each of the submodules within MOI. These submodules are not required to interface a solver with MOI, but they make the job much easier.

## 1.3 Citing MathOptInterface

If you find MathOptInterface useful in your work, we kindly request that you cite the following paper:

```
@article{legat2021mathoptinterface,
    title={{MathOptInterface}: a data structure for mathematical optimization problems},
    author={Legat, Beno{\^\i}t and Dowson, Oscar and Garcia, Joaquim Dias and Lubin, Miles},
    journal={INFORMS Journal on Computing},
    year={2021},
    doi={10.1287/ijoc.2021.1067},
    publisher={INFORMS}
}
```

A preprint of this paper is freely available.

# Chapter 2

# Motivation

MathOptInterface (MOI) is a replacement for MathProgBase, the first-generation abstraction layer for mathematical optimization previously used by JuMP and Convex.jl.

To address a number of limitations of MathProgBase, MOI is designed to:

- Be simple and extensible
  - unifying linear, quadratic, and conic optimization,
  - seamlessly facilitating extensions to essentially arbitrary constraints and functions (for example, indicator constraints, complementarity constraints, and piecewise-linear functions)
- Be fast
  - by allowing access to a solver's in-memory representation of a problem without writing intermediate files (when possible)
  - by using multiple dispatch and avoiding requiring containers of non-concrete types
- Allow a solver to return multiple results (for example, a pool of solutions)
- Allow a solver to return extra arbitrary information via attributes (for example, variable- and constraint-wise membership in an irreducible inconsistent subset for infeasibility analysis)
- Provide a greatly expanded set of status codes explaining what happened during the optimization procedure
- Enable a solver to more precisely specify which problem classes it supports
- Enable both primal and dual warm starts
- Enable adding and removing both variables and constraints by indices that are not required to be consecutive
- Enable any modification that the solver supports to an existing model
- Avoid requiring the solver wrapper to store an additional copy of the problem data

# Part II

# Tutorials

# Chapter 3

# Solving a problem using MathOptInterface

In this tutorial we demonstrate how to use MathOptInterface to solve the binary-constrained knapsack problem:

$$\begin{aligned} \max\ & c^\top x \\ s.t.\ & w^\top x \leq C \\ & x_i \in \{0, 1\}, \quad \forall i = 1, \ldots, n \end{aligned}$$

## 3.1 Required packages

Load the MathOptInterface module and define the shorthand `MOI`:

```
import MathOptInterface as MOI
```

As an optimizer, we choose GLPK:

```
using GLPK
optimizer = GLPK.Optimizer()
```

## 3.2 Define the data

We first define the constants of the problem:

```
julia> c = [1.0, 2.0, 3.0]
3-element Vector{Float64}:
 1.0
 2.0
 3.0

julia> w = [0.3, 0.5, 1.0]
3-element Vector{Float64}:
 0.3
 0.5
 1.0

julia> C = 3.2
3.2
```

## 3.3 Add the variables

```
julia> x = MOI.add_variables(optimizer, length(c));
```

## 3.4 Set the objective

```
julia> MOI.set(
           optimizer,
           MOI.ObjectiveFunction{MOI.ScalarAffineFunction{Float64}}(),
           MOI.ScalarAffineFunction(MOI.ScalarAffineTerm.(c, x), 0.0),
       );

julia> MOI.set(optimizer, MOI.ObjectiveSense(), MOI.MAX_SENSE)
```

**Tip**

`MOI.ScalarAffineTerm.(c, x)` is a shortcut for `[MOI.ScalarAffineTerm(c[i], x[i]) for i = 1:3]`. This is Julia's broadcast syntax in action, and is used quite often throughout MOI.

## 3.5 Add the constraints

We add the knapsack constraint and integrality constraints:

```
julia> MOI.add_constraint(
           optimizer,
           MOI.ScalarAffineFunction(MOI.ScalarAffineTerm.(w, x), 0.0),
           MOI.LessThan(C),
       );
```

Add integrality constraints:

```
julia> for x_i in x
           MOI.add_constraint(optimizer, x_i, MOI.ZeroOne())
       end
```

## 3.6 Optimize the model

```
julia> MOI.optimize!(optimizer)
```

## 3.7 Understand why the solver stopped

The first thing to check after optimization is why the solver stopped, for example, did it stop because of a time limit or did it stop because it found the optimal solution?

```
julia> MOI.get(optimizer, MOI.TerminationStatus())
OPTIMAL::TerminationStatusCode = 1
```

Looks like we found an optimal solution.

## 3.8 Understand what solution was returned

```
julia> MOI.get(optimizer, MOI.ResultCount())
1

julia> MOI.get(optimizer, MOI.PrimalStatus())
FEASIBLE_POINT::ResultStatusCode = 1

julia> MOI.get(optimizer, MOI.DualStatus())
NO_SOLUTION::ResultStatusCode = 0
```

## 3.9 Query the objective

What is its objective value?

```
julia> MOI.get(optimizer, MOI.ObjectiveValue())
6.0
```

## 3.10 Query the primal solution

And what is the value of the variables x?

```
julia> MOI.get(optimizer, MOI.VariablePrimal(), x)
3-element Vector{Float64}:
 1.0
 1.0
 1.0
```

# Chapter 4

# Implementing a solver interface

This guide outlines the basic steps to implement an interface to MathOptInterface for a new solver.

**Danger**

Implementing an interface to MathOptInterface for a new solver is a lot of work. Before starting, we recommend that you join the Developer chatroom and explain a little bit about the solver you are wrapping. If you have questions that are not answered by this guide, please ask them in the Developer chatroom so we can improve this guide.

## 4.1 A note on the API

The API of MathOptInterface is large and varied. In order to support the diversity of solvers and use-cases, we make heavy use of duck-typing. That is, solvers are not expected to implement the full API, nor is there a well-defined minimal subset of what must be implemented. Instead, you should implement the API as necessary to make the solver function as you require.

The main reason for using duck-typing is that solvers work in different ways and target different use-cases.

For example:

- Some solvers support incremental problem construction, support modification after a solve, and have native support for things like variable names.
- Other solvers are "one-shot" solvers that require all of the problem data to construct and solve the problem in a single function call. They do not support modification or things like variable names.
- Other "solvers" are not solvers at all, but things like file readers. These may only support functions like `read_from_file`, and may not even support the ability to add variables or constraints directly.
- Finally, some "solvers" are layers which take a problem as input, transform it according to some rules, and pass the transformed problem to an inner solver.

## 4.2 Preliminaries

Before starting on your wrapper, you should do some background research and make the solver accessible via Julia.

### Decide if MathOptInterface is right for you

The first step in writing a wrapper is to decide whether implementing an interface is the right thing to do.

MathOptInterface is an abstraction layer for unifying constrained mathematical optimization solvers. If your solver doesn't fit in the category, for example, it implements a derivative-free algorithm for unconstrained objective functions, MathOptInterface may not be the right tool for the job.

**Tip**

If you're not sure whether you should write an interface, ask in the Developer chatroom.

### Find a similar solver already wrapped

The next step is to find (if possible) a similar solver that is already wrapped. Although not strictly necessary, this will be a good place to look for inspiration when implementing your wrapper.

The JuMP documentation has a good list of solvers, along with the problem classes they support.

**Tip**

If you're not sure which solver is most similar, ask in the Developer chatroom.

### Create a low-level interface

Before writing a MathOptInterface wrapper, you first need to be able to call the solver from Julia.

#### Wrapping solvers written in Julia

If your solver is written in Julia, there's nothing to do here. Go to the next section.

#### Wrapping solvers written in C

Julia is well suited to wrapping solvers written in C.

**Info**

This is not true for C++. If you have a solver written in C++, first write a C interface, then wrap the C interface.

Before writing a MathOptInterface wrapper, there are a few extra steps.

#### Create a JLL

If the C code is publicly available under an open source license, create a JLL package via Yggdrasil. The easiest way to do this is to copy an existing solver. Good examples to follow are the COIN-OR solvers.

**Warning**

Building the solver via Yggdrasil is non-trivial. Please ask the Developer chatroom for help.

If the code is commercial or not publicly available, the user will need to manually install the solver. See Gurobi.jl or CPLEX.jl for examples of how to structure this.

#### Use Clang.jl to wrap the C API

The next step is to use Clang.jl to automatically wrap the C API. The easiest way to do this is to follow an example. Good examples to follow are Cbc.jl and HiGHS.jl.

Sometimes, you will need to make manual modifications to the resulting files.

#### Solvers written in other languages

Ask the Developer chatroom for advice. You may be able to use one of the JuliaInterop packages to call out to the solver.

For example, SeDuMi.jl uses MATLAB.jl to call the SeDuMi solver written in MATLAB.

### 4.3 Structuring the package

Structure your wrapper as a Julia package. Consult the Julia documentation if you haven't done this before.

MOI solver interfaces may be in the same package as the solver itself (either the C wrapper if the solver is accessible through C, or the Julia code if the solver is written in Julia, for example), or in a separate package which depends on the solver package.

**Note**

The JuMP core contributors request that you do not use "JuMP" in the name of your package without prior consent.

Your package should have the following structure:

```
/.github
    /workflows
        ci.yml
        format_check.yml
        TagBot.yml
/gen
    gen.jl  # Code to wrap the C API
/src
    NewSolver.jl
    /gen
        libnewsolver_api.jl
        libnewsolver_common.jl
    /MOI_wrapper
        MOI_wrapper.jl
        other_files.jl
/test
    runtests.jl
    /MOI_wrapper
        MOI_wrapper.jl
.gitignore
.JuliaFormatter.toml
README.md
LICENSE.md
Project.toml
```

- The `/.github` folder contains the scripts for GitHub actions. The easiest way to write these is to copy the ones from an existing solver.

- The `/gen` and `/src/gen` folders are only needed if you are wrapping a solver written in C.
- The `/src/MOI_wrapper` folder contains the Julia code for the MOI wrapper.
- The `/test` folder contains code for testing your package. See Setup tests for more information.
- The `.JuliaFormatter.toml` and `.github/workflows/format_check.yml` enforce code formatting using JuliaFormatter.jl. Check existing solvers or JuMP.jl for details.

### Documentation

Your package must include documentation explaining how to use the package. The easiest approach is to include documentation in your `README.md`. A more involved option is to use Documenter.jl.

Examples of packages with README-based documentation include:

- Cbc.jl
- HiGHS.jl
- SCS.jl

Examples of packages with Documenter-based documentation include:

- Alpine.jl
- COSMO.jl
- Juniper.jl

### Setup tests

The best way to implement an interface to MathOptInterface is via test-driven development.

The `MOI.Test` submodule contains a large test suite to help check that you have implemented things correctly.

Follow the guide How to test a solver to set up the tests for your package.

**Tip**

Run the tests frequently when developing. However, at the start there is going to be a lot of errors. Start by excluding large classes of tests (for example, `exclude = ["test_basic_", "test_model_"]`, implement any missing methods until the tests pass, then remove an exclusion and repeat.

## 4.4 Initial code

By this point, you should have a package setup with tests, formatting, and access to the underlying solver. Now it's time to start writing the wrapper.

### The `Optimizer` object

The first object to create is a subtype of `AbstractOptimizer`. This type is going to store everything related to the problem.

By convention, these optimizers should not be exported and should be named `PackageName.Optimizer`.

```
import MathOptInterface as MOI

struct Optimizer <: MOI.AbstractOptimizer
    # Fields go here
end
```

### Optimizer objects for C solvers

**Warning**

This section is important if you wrap a solver written in C.

Wrapping a solver written in C will require the use of pointers, and for you to manually free the solver's memory when the `Optimizer` is garbage collected by Julia.

**Never pass a pointer directly to a Julia `ccall` function.**

Instead, store the pointer as a field in your `Optimizer`, and implement `Base.cconvert` and `Base.unsafe_convert`. Then you can pass `Optimizer` to any `ccall` function that expects the pointer.

In addition, make sure you implement a `finalizer` for each model you create.

If `newsolver_createProblem()` is the low-level function that creates the problem pointer in C, and `newsolver_freeProblem(::Pt` is the low-level function that frees memory associated with the pointer, your `Optimizer()` function should look like this:

```
struct Optimizer <: MOI.AbstractOptimizer
    ptr::Ptr{Cvoid}

    function Optimizer()
        ptr = newsolver_createProblem()
        model = Optimizer(ptr)
        finalizer(model) do m
            newsolver_freeProblem(m)
            return
        end
        return model
    end
end

Base.cconvert(::Type{Ptr{Cvoid}}, model::Optimizer) = model
Base.unsafe_convert(::Type{Ptr{Cvoid}}, model::Optimizer) = model.ptr
```

### Implement methods for `Optimizer`

All `Optimizers` must implement the following methods:

- `empty!`

- `is_empty`

Other methods, detailed below, are optional or depend on how you implement the interface.

**Tip**

For this and all future methods, read the docstrings to understand what each method does, what it expects as input, and what it produces as output. If it isn't clear, let us know and we will improve the docstrings. It is also very helpful to look at an existing wrapper for a similar solver.

You should also implement `Base.summary(::IO, ::Optimizer)` to print a nice string when someone shows your model. For example

```
function Base.summary(io::IO, model::Optimizer)
    return print(io, "NewSolver with the pointer $(model.ptr)")
end
```

### Implement attributes

MathOptInterface uses attributes to manage different aspects of the problem.

For each attribute

- `get` gets the current value of the attribute
- `set` sets a new value of the attribute. Not all attributes can be set. For example, the user can't modify the `SolverName`.
- `supports` returns a `Bool` indicating whether the solver supports the attribute.

**Info**

Use `attribute_value_type` to check the value expected by a given attribute. You should make sure that your `get` function correctly infers to this type (or a subtype of it).

Each column in the table indicates whether you need to implement the particular method for each attribute.

For example:

```
function MOI.get(model::Optimizer, ::MOI.Silent)
    return # true if MOI.Silent is set
end

function MOI.set(model::Optimizer, ::MOI.Silent, v::Bool)
    if v
        # Set a parameter to turn off printing
    else
        # Restore the default printing
    end
    return
end

MOI.supports(::Optimizer, ::MOI.Silent) = true
```

| Attribute | get | set | supports |
|---|---|---|---|
| SolverName | Yes | No | No |
| SolverVersion | Yes | No | No |
| RawSolver | Yes | No | No |
| Name | Yes | Yes | Yes |
| Silent | Yes | Yes | Yes |
| TimeLimitSec | Yes | Yes | Yes |
| ObjectiveLimit | Yes | Yes | Yes |
| SolutionLimit | Yes | Yes | Yes |
| NodeLimit | Yes | Yes | Yes |
| RawOptimizerAttribute | Yes | Yes | Yes |
| NumberOfThreads | Yes | Yes | Yes |
| AbsoluteGapTolerance | Yes | Yes | Yes |
| RelativeGapTolerance | Yes | Yes | Yes |

#### Define `supports_constraint`

The next step is to define which constraints and objective functions you plan to support.

For each function-set constraint pair, define `supports_constraint`:

```
function MOI.supports_constraint(
    ::Optimizer,
    ::Type{MOI.VariableIndex},
    ::Type{MOI.ZeroOne},
)
    return true
end
```

To make this easier, you may want to use `Unions`:

```
function MOI.supports_constraint(
    ::Optimizer,
    ::Type{MOI.VariableIndex},
    ::Type{<:Union{MOI.LessThan,MOI.GreaterThan,MOI.EqualTo}},
)
    return true
end
```

**Tip**

Only support a constraint if your solver has native support for it.

## 4.5 The big decision: incremental modification?

Now you need to decide whether to support incremental modification or not.

Incremental modification means that the user can add variables and constraints one-by-one without needing to rebuild the entire problem, and they can modify the problem data after an `optimize!` call. Supporting incremental modification means implementing functions like `add_variable` and `add_constraint`.

The alternative is to accept the problem data in a single `optimize!` or `copy_to` function call. Because these functions see all of the data at once, it can typically call a more efficient function to load data into the underlying solver.

Good examples of solvers supporting incremental modification are MILP solvers like GLPK.jl and Gurobi.jl. Examples of non-incremental solvers are AmplNLWriter.jl and SCS.jl

It is possible for a solver to implement both approaches, but you should probably start with one for simplicity.

**Tip**

Only support incremental modification if your solver has native support for it.

In general, supporting incremental modification is more work, and it usually requires some extra book-keeping. However, it provides a more efficient interface to the solver if the problem is going to be resolved multiple times with small modifications. Moreover, once you've implemented incremental modification, it's usually not much extra work to add a `copy_to` interface. The converse is not true.

**Tip**

If this is your first time writing an interface, start with the one-shot `optimize!`.

### The non-incremental interface

There are two ways to implement the non-incremental interface. The first uses a two-argument version of `optimize!`, the second implements `copy_to` followed by the one-argument version of `optimize!`.

If your solver does not support modification, and requires all data to solve the problem in a single function call, you should implement the "one-shot" `optimize!`.

- `optimize!(::ModelLike, ::ModelLike)`

If your solver separates data loading and the actual optimization into separate steps, implement the `copy_to` interface.

- `copy_to(::ModelLike, ::ModelLike)`
- `optimize!(::ModelLike)`

### The incremental interface

**Warning**

Writing this interface is a lot of work. The easiest way is to consult the source code of a similar solver.

To implement the incremental interface, implement the following functions:

- `add_variable`
- `add_variables`
- `add_constraint`

- `add_constraints`
- `is_valid`
- `delete`
- `optimize!(::ModelLike)`

**Info**

Solvers do not have to support `AbstractScalarFunction` in `GreaterThan`, `LessThan`, `EqualTo`, or `Interval` with a nonzero constant in the function. Throw `ScalarFunctionConstantNotZero` if the function constant is not zero.

In addition, you should implement the following model attributes:

| Attribute | get | set | supports |
|---|---|---|---|
| `ListOfModelAttributesSet` | Yes | No | No |
| `ObjectiveFunctionType` | Yes | No | No |
| `ObjectiveFunction` | Yes | Yes | Yes |
| `ObjectiveSense` | Yes | Yes | Yes |
| `Name` | Yes | Yes | Yes |

Variable-related attributes:

| Attribute | get | set | supports |
|---|---|---|---|
| `ListOfVariableAttributesSet` | Yes | No | No |
| `ListOfVariablesWithAttributeSet` | Yes | No | No |
| `NumberOfVariables` | Yes | No | No |
| `ListOfVariableIndices` | Yes | No | No |

Constraint-related attributes:

| Attribute | get | set | supports |
|---|---|---|---|
| `ListOfConstraintAttributesSet` | Yes | No | No |
| `ListOfConstraintsWithAttributeSet` | Yes | No | No |
| `NumberOfConstraints` | Yes | No | No |
| `ListOfConstraintTypesPresent` | Yes | No | No |
| `ConstraintFunction` | Yes | Yes | No |
| `ConstraintSet` | Yes | Yes | No |

#### Modifications

If your solver supports modifying data in-place, implement `modify` for the following `AbstractModifications`:

- `ScalarConstantChange`
- `ScalarCoefficientChange`
- `ScalarQuadraticCoefficientChange`

- `VectorConstantChange`
- `MultirowChange`

#### Variables constrained on creation

Some solvers require variables be associated with a set when they are created. This conflicts with the incremental modification approach, since you cannot first add a free variable and then constrain it to the set.

If this is the case, implement:

- `add_constrained_variable`
- `add_constrained_variables`
- `supports_add_constrained_variables`

By default, MathOptInterface assumes solvers support free variables. If your solver does not support free variables, define:

```
MOI.supports_add_constrained_variables(::Optimizer, ::Type{Reals}) = false
```

### Incremental and `copy_to`

If you implement the incremental interface, you have the option of also implementing `copy_to`.

If you don't want to implement `copy_to`, for example, because the solver has no API for building the problem in a single function call, define the following fallback:

```
MOI.supports_incremental_interface(::Optimizer) = true

function MOI.copy_to(dest::Optimizer, src::MOI.ModelLike)
    return MOI.Utilities.default_copy_to(dest, src)
end
```

## 4.6 Names

Regardless of which interface you implement, you have the option of implementing the `Name` attribute for variables and constraints:

| Attribute | `get` | `set` | `supports` |
|---|---|---|---|
| `VariableName` | Yes | Yes | Yes |
| `ConstraintName` | Yes | Yes | Yes |

If you implement names, you must also implement the following three methods:

```
function MOI.get(model::Optimizer, ::Type{MOI.VariableIndex}, name::String)
    return # The variable named `name`.
end

function MOI.get(model::Optimizer, ::Type{MOI.ConstraintIndex}, name::String)
```

```
    return # The constraint any type named `name`.
end

function MOI.get(
    model::Optimizer,
    ::Type{MOI.ConstraintIndex{F,S}},
    name::String,
) where {F,S}
    return # The constraint of type F-in-S named `name`.
end
```

These methods have the following rules:

- If there is no variable or constraint with the name, return `nothing`
- If there is a single variable or constraint with that name, return the variable or constraint
- If there are multiple variables or constraints with the name, throw an error.

**Warning**

You should not implement `ConstraintName` for `VariableIndex` constraints. If you implement `ConstraintName` for other constraints, you can add the following two methods to disable `ConstraintName` for `VariableIndex` constraints.

```
function MOI.supports(
    ::Optimizer,
    ::MOI.ConstraintName,
    ::Type{<:MOI.ConstraintIndex{MOI.VariableIndex,<:MOI.AbstractScalarSet}},
)
    return throw(MOI.VariableIndexConstraintNameError())
end
function MOI.set(
    ::Optimizer,
    ::MOI.ConstraintName,
    ::MOI.ConstraintIndex{MOI.VariableIndex,<:MOI.AbstractScalarSet},
    ::String,
)
    return throw(MOI.VariableIndexConstraintNameError())
end
```

## 4.7 Solutions

Implement `optimize!` to solve the model:

- `optimize!`

All `Optimizers` must implement the following attributes:

- `DualStatus`

- `PrimalStatus`
- `RawStatusString`
- `ResultCount`
- `TerminationStatus`

**Info**

You only need to implement `get` for solution attributes. Don't implement `set` or `supports`.

**Note**

Solver wrappers should document how the low-level statuses map to the MOI statuses. Statuses like `NEARLY_FEASIBLE_POINT` and `INFEASIBLE_POINT`, are designed to be used when the solver explicitly indicates that relaxed tolerances are satisfied or the returned point is infeasible, respectively.

You should also implement the following attributes:

- `ObjectiveValue`
- `SolveTimeSec`
- `VariablePrimal`

**Tip**

Attributes like `VariablePrimal` and `ObjectiveValue` are indexed by the result count. Use `MOI.check_result_index_bounds(model, attr)` to throw an error if the attribute is not available.

If your solver returns dual solutions, implement:

- `ConstraintDual`
- `DualObjectiveValue`

For integer solvers, implement:

- `ObjectiveBound`
- `RelativeGap`

If applicable, implement:

- `SimplexIterations`
- `BarrierIterations`
- `NodeCount`

If your solver uses the Simplex method, implement:

- `ConstraintBasisStatus`

If your solver accepts primal or dual warm-starts, implement:

- `VariablePrimalStart`
- `ConstraintDualStart`

## 4.8 Other tips

Here are some other points to be aware of when writing your wrapper.

### Unsupported constraints at runtime

In some cases, your solver may support a particular type of constraint (for example, quadratic constraints), but only if the data meets some condition (for example, it is convex).

In this case, declare that you `support` the constraint, and throw `AddConstraintNotAllowed`.

### Dealing with multiple variable bounds

MathOptInterface uses `VariableIndex` constraints to represent variable bounds. Defining multiple variable bounds on a single variable is not allowed.

Throw `LowerBoundAlreadySet` or `UpperBoundAlreadySet` if the user adds a constraint that results in multiple bounds.

Only throw if the constraints conflict. It is okay to add `VariableIndex`-in-`GreaterThan` and then `VariableIndex`-in-`LessThan`, but not `VariableIndex`-in-`Interval` and then `VariableIndex`-in-`LessThan`,

### Expect duplicate coefficients

Solvers must expect that functions such as `ScalarAffineFunction` and `VectorQuadraticFunction` may contain duplicate coefficients.

For example, `ScalarAffineFunction([ScalarAffineTerm(x, 1), ScalarAffineTerm(x, 1)], 0.0)`.

Use `Utilities.canonical` to return a new function with the duplicate coefficients aggregated together.

### Don't modify user-data

All data passed to the solver must be copied immediately to internal data structures. Solvers may not modify any input vectors and must assume that input vectors will not be modified by users in the future.

This applies, for example, to the `terms` vector in `ScalarAffineFunction`. Vectors returned to the user, for example, via `ObjectiveFunction` or `ConstraintFunction` attributes, must not be modified by the solver afterwards. The in-place version of `get!` can be used by users to avoid extra copies in this case.

### Column Generation

There is no special interface for column generation. If the solver has a special API for setting coefficients in existing constraints when adding a new variable, it is possible to queue modifications and new variables and then call the solver's API once all of the new coefficients are known.

### Solver-specific attributes

You don't need to restrict yourself to the attributes defined in the MathOptInterface.jl package.

Solver-specific attributes should be specified by creating an appropriate subtype of `AbstractModelAttribute`, `AbstractOptimizerAttribute`, `AbstractVariableAttribute`, or `AbstractConstraintAttribute`.

For example, Gurobi.jl adds attributes for multiobjective optimization by defining:

```
struct NumberOfObjectives <: MOI.AbstractModelAttribute end

function MOI.set(model::Optimizer, ::NumberOfObjectives, n::Integer)
    # Code to set NumberOfObjectives
    return
end

function MOI.get(model::Optimizer, ::NumberOfObjectives)
    n = # Code to get NumberOfObjectives
    return n
end
```

Then, the user can write:

```
model = Gurobi.Optimizer()
MOI.set(model, Gurobi.NumberofObjectives(), 3)
```

# Chapter 5

# Transitioning from MathProgBase

MathOptInterface is a replacement for MathProgBase.jl. However, it is not a direct replacement.

## 5.1 Transitioning a solver interface

MathOptInterface is more extensive than MathProgBase which may make its implementation seem daunting at first. There are however numerous utilities in MathOptInterface that the simplify implementation process.

For more information, read Implementing a solver interface.

## 5.2 Transitioning the high-level functions

MathOptInterface doesn't provide replacements for the high-level interfaces in MathProgBase. We recommend you use JuMP as a modeling interface instead.

**Tip**

If you haven't used JuMP before, start with the tutorial Getting started with JuMP

### `linprog`

Here is one way of transitioning from `linprog`:

```
using JuMP

function linprog(c, A, sense, b, l, u, solver)
    N = length(c)
    model = Model(solver)
    @variable(model, l[i] <= x[i=1:N] <= u[i])
    @objective(model, Min, c' * x)
    eq_rows, ge_rows, le_rows = sense .== '=', sense .== '>', sense .== '<'
    @constraint(model, A[eq_rows, :] * x .== b[eq_rows])
    @constraint(model, A[ge_rows, :] * x .>= b[ge_rows])
    @constraint(model, A[le_rows, :] * x .<= b[le_rows])
    optimize!(model)
    return (
        status = termination_status(model),
        objval = objective_value(model),
        sol = value.(x)
```

```
    )
end
```

### mixintprog

Here is one way of transitioning from `mixintprog`:

```
using JuMP

function mixintprog(c, A, rowlb, rowub, vartypes, lb, ub, solver)
    N = length(c)
    model = Model(solver)
    @variable(model, lb[i] <= x[i=1:N] <= ub[i])
    for i in 1:N
        if vartypes[i] == :Bin
            set_binary(x[i])
        elseif vartypes[i] == :Int
            set_integer(x[i])
        end
    end
    @objective(model, Min, c' * x)
    @constraint(model, rowlb .<= A * x .<= rowub)
    optimize!(model)
    return (
        status = termination_status(model),
        objval = objective_value(model),
        sol = value.(x)
    )
end
```

### quadprog

Here is one way of transitioning from `quadprog`:

```
using JuMP

function quadprog(c, Q, A, rowlb, rowub, lb, ub, solver)
    N = length(c)
    model = Model(solver)
    @variable(model, lb[i] <= x[i=1:N] <= ub[i])
    @objective(model, Min, c' * x + 0.5 * x' * Q * x)
    @constraint(model, rowlb .<= A * x .<= rowub)
    optimize!(model)
    return (
        status = termination_status(model),
        objval = objective_value(model),
        sol = value.(x)
    )
end
```

# Chapter 6

# Implementing a constraint bridge

This guide outlines the basic steps to create a new bridge from a constraint expressed in the formalism `Function`-in-`Set`.

## 6.1 Preliminaries

First, decide on the set you want to bridge. Then, study its properties: the most important one is whether the set is scalar or vector, which impacts the dimensionality of the functions that can be used with the set.

- A scalar function only has one dimension. MOI defines three types of scalar functions: a variable (`VariableIndex`), an affine function (`ScalarAffineFunction`), or a quadratic function (`ScalarQuadraticFunction`).
- A vector function has several dimensions (at least one). MOI defines three types of vector functions: several variables (`VectorOfVariables`), an affine function (`VectorAffineFunction`), or a quadratic function (`VectorQuadraticFunction`). The main difference with scalar functions is that the order of dimensions can be very important: for instance, in an indicator constraint (`Indicator`), the first dimension indicates whether the constraint about the second dimension is active.

To explain how to implement a bridge, we present the example of `Bridges.Constraint.FlipSignBridge`. This bridge maps <= (`LessThan`) constraints to >= (`GreaterThan`) constraints. This corresponds to reversing the sign of the inequality. We focus on scalar affine functions (we disregard the cases of a single variable or of quadratic functions). This example is a simplified version of the code included in MOI.

## 6.2 Four mandatory parts in a constraint bridge

The first part of a constraint bridge is a new concrete subtype of `Bridges.Constraint.AbstractBridge`. This type must have fields to store all the new variables and constraints that the bridge will add. Typically, these types are parametrized by the type of the coefficients in the model.

Then, three sets of functions must be defined:

1. `Bridges.Constraint.bridge_constraint`: this function implements the bridge and creates the required variables and constraints.
2. `supports_constraint`: these functions must return `true` when the combination of function and set is supported by the bridge. By default, the base implementation always returns `false` and the bridge does not have to provide this implementation.

3. `Bridges.added_constrained_variable_types` and `Bridges.added_constraint_types`: these functions return the types of variables and constraints that this bridge adds. They are used to compute the set of other bridges that are required to use the one you are defining, if need be.

More functions can be implemented, for instance to retrieve properties from the bridge or deleting a bridged constraint.

### 1. Structure for the bridge

A typical `struct` behind a bridge depends on the type of the coefficients that are used for the model (typically `Float64`, but coefficients might also be integers or complex numbers).

This structure must hold a reference to all the variables and the constraints that are created as part of the bridge.

The type of this structure is used throughout MOI as an identifier for the bridge. It is passed as argument to most functions related to bridges.

The best practice is to have the name of this type end with `Bridge`.

In our example, the bridge maps any `ScalarAffineFunction{T}`-in-`LessThan{T}` constraint to a single `ScalarAffineFunction{T}`-in-`GreaterThan{T}` constraint. The affine function has coefficients of type `T`. The bridge is parametrized with `T`, so that the constraint that the bridge creates also has coefficients of type `T`.

```
struct SignBridge{T<:Number} <: Bridges.Constraint.AbstractBridge
    constraint::ConstraintIndex{ScalarAffineFunction{T}, GreaterThan{T}}
end
```

### 2. Bridge creation

The function `Bridges.Constraint.bridge_constraint` is called whenever the bridge is instantiated for a specific model, with the given function and set. The arguments to `bridge_constraint` are similar to `add_constraint`, with the exception of the first argument: it is the `Type` of the struct defined in the first step (for our example, `Type{SignBridge{T}}`).

`bridge_constraint` returns an instance of the struct defined in the first step. the first step.

In our example, the bridge constraint could be defined as:

```
function Bridges.Constraint.bridge_constraint(
    ::Type{SignBridge{T}}, # Bridge to use.
    model::ModelLike, # Model to which the constraint is being added.
    f::ScalarAffineFunction{T}, # Function to rewrite.
    s::LessThan{T}, # Set to rewrite.
) where {T}
    # Create the variables and constraints required for the bridge.
    con = add_constraint(model, -f, GreaterThan(-s.upper))

    # Return an instance of the bridge type with a reference to all the
    # variables and constraints that were created in this function.
    return SignBridge(con)
end
```

### 3. Supported constraint types

The function `supports_constraint` determines whether the bridge type supports a given combination of function and set.

This function must closely match `bridge_constraint`, because it will not be called if `supports_constraint` returns `false`.

```
function supports_constraint(
    ::Type{SignBridge{T}}, # Bridge to use.
    ::Type{ScalarAffineFunction{T}}, # Function to rewrite.
    ::Type{LessThan{T}}, # Set to rewrite.
) where {T}
    # Do some computation to ensure that the constraint is supported.
    # Typically, you can directly return true.
    return true
end
```

### 4. Metadata about the bridge

To determine whether a bridge can be used, MOI uses a shortest-path algorithm that uses the variable types and the constraints that the bridge can create. This information is communicated from the bridge to MOI using the functions `Bridges.added_constrained_variable_types` and `Bridges.added_constraint_types`. Both return lists of tuples: either a list of 1-tuples containing the variable types (typically, `ZeroOne` or `Integer`) or a list of 2-tuples contained the functions and sets (like `ScalarAffineFunction{T}`-`GreaterThan`).

For our example, the bridge does not create any constrained variables, and only `ScalarAffineFunction{T}`-in-`GreaterThan{T}` constraints:

```
function Bridges.added_constrained_variable_types(::Type{SignBridge{T}}) where {T}
    # The bridge does not create variables, return an empty list of tuples:
    return Tuple{Type}[]
end

function Bridges.added_constraint_types(::Type{SignBridge{T}}) where {T}
    return Tuple{Type,Type}[
        # One element per F-in-S the bridge creates.
        (ScalarAffineFunction{T}, GreaterThan{T}),
    ]
end
```

A bridge that creates binary variables would rather have this definition of `added_constrained_variable_types`:

```
function Bridges.added_constrained_variable_types(::Type{SomeBridge{T}}) where {T}
    # The bridge only creates binary variables:
    return Tuple{Type}[(ZeroOne,)]
end
```

**Warning**

If you declare the creation of constrained variables in `added_constrained_variable_types`, the corresponding constraint type `VariableIndex` must not be indicated in `added_constraint_types`. This would restrict the use of the bridge to solvers that can add such a constraint after the variable is created.

More concretely, if you declare in `added_constrained_variable_types` that your bridge creates binary variables (`ZeroOne`), and if you never add such a constraint afterward (you do not call `add_constraint(model, var, ZeroOne())`), then you must not list `(VariableIndex, ZeroOne)` in `added_constraint_types`.

Typically, the function `Bridges.Constraint.concrete_bridge_type` does not have to be defined for most bridges.

## 6.3 Bridge registration

For a bridge to be used by MOI, it must be known by MOI.

### `SingleBridgeOptimizer`

The first way to do so is to create a single-bridge optimizer. This type of optimizer wraps another optimizer and adds the possibility to use only one bridge. It is especially useful when unit testing bridges.

It is common practice to use the same name as the type defined for the bridge (`SignBridge`, in our example) without the suffix `Bridge`.

```
const Sign{T,OT<: ModelLike} =
    SingleBridgeOptimizer{SignBridge{T}, OT}
```

In the context of unit tests, this bridge is used in conjunction with a `Utilities.MockOptimizer`:

```
mock = Utilities.MockOptimizer(
    Utilities.UniversalFallback(Utilities.Model{Float64}()),
)
bridged_mock = Sign{Float64}(mock)
```

### New bridge for a `LazyBridgeOptimizer`

Typical user-facing models for MOI are based on `Bridges.LazyBridgeOptimizer`. For instance, this type of model is returned by `Bridges.full_bridge_optimizer`. These models can be added more bridges by using `Bridges.add_bridge`:

```
inner_optimizer = Utilities.Model{Float64}()
optimizer = Bridges.full_bridge_optimizer(inner_optimizer, Float64)
Bridges.add_bridge(optimizer, SignBridge{Float64})
```

## 6.4 Bridge improvements

### Attribute retrieval

Like models, bridges have attributes that can be retrieved using `get` and `set`. The most important ones are the number of variables and constraints, but also the lists of variables and constraints.

In our example, we only have one constraint and only have to implement the `NumberOfConstraints` and `ListOfConstraintIndices` attributes:

```
function get(
    ::SignBridge{T},
    ::NumberOfConstraints{
        ScalarAffineFunction{T},
        GreaterThan{T},
    },
) where {T}
    return 1
end

function get(
    bridge::SignBridge{T},
    ::ListOfConstraintIndices{
        ScalarAffineFunction{T},
        GreaterThan{T},
    },
) where {T}
    return [bridge.constraint]
end
```

You must implement one such pair of functions for each type of constraint the bridge adds to the model.

**Warning**

Avoid returning a list from the bridge object without copying it. Users must be able to change the contents of the returned list without altering the bridge object.

For variables, the situation is simpler. If your bridge creates new variables, you must implement the `NumberOfVariables` and `ListOfVariableIndices` attributes. However, these attributes do not have parameters, unlike their constraint counterparts. Only two functions suffice:

```
function get(
    ::SignBridge{T},
    ::NumberOfVariables,
) where {T}
    return 0
end

function get(
    ::SignBridge{T},
    ::ListOfVariableIndices,
) where {T}
    return VariableIndex[]
end
```

In order for the user to be able to access the function and set of the original constraint, the bridge needs to implement getters for the `ConstraintFunction` and `ConstraintSet` attributes:

```
function get(
    model::MOI.ModelLike,
    attr::MOI.ConstraintFunction,
    bridge::SignBridge,
)
    return -MOI.get(model, attr, bridge.constraint)
end

function get(
    model::MOI.ModelLike,
    attr::MOI.ConstraintSet,
    bridge::SignBridge,
)
    set = MOI.get(model, attr, bridge.constraint)
    return MOI.LessThan(-set.lower)
end
```

**Warning**

Alternatively, one could store the original function and set in `SignBridge` during `Bridges.Constraint.bridge_constraint` to make these getters simpler and more efficient. On the other hand, this will increase the memory footprint of the bridges as the garbage collector won't be able to delete that object. The convention is to not store the function in the bridge and not care too much about the efficiency of these getters. If the user needs efficient getters for `ConstraintFunction` then they should use a `Utilities.CachingOptimizer`.

### Model modifications

To avoid copying the model when the user request to change a constraint, MOI provides `modify`. Bridges can also implement this API to allow certain changes, such as coefficient changes.

In our case, a modification of a coefficient in the original constraint (for example, replacing the value of the coefficient of a variable in the affine function) must be transmitted to the constraint created by the bridge, but with a sign change.

```
function modify(
    model::ModelLike,
    bridge::SignBridge,
    change::ScalarCoefficientChange,
)
    modify(
        model,
        bridge.constraint,
        ScalarCoefficientChange(change.variable, -change.new_coefficient),
    )
    return
end
```

### Bridge deletion

When a bridge is deleted, the constraints it added must be deleted too.

```
function delete(model::ModelLike, bridge::SignBridge)
    delete(model, bridge.constraint)
    return
end
```

# Chapter 7

# Manipulating expressions

This guide highlights a syntactically appealing way to build expressions at the MOI level, but also to look at their contents. It may be especially useful when writing models or bridge code.

## 7.1 Creating functions

This section details the ways to create functions with MathOptInterface.

### Creating scalar affine functions

The simplest scalar function is simply a variable:

```
julia> x = MOI.add_variable(model) # Create the variable x
MOI.VariableIndex(1)
```

This type of function is extremely simple; to express more complex functions, other types must be used. For instance, a `ScalarAffineFunction` is a sum of linear terms (a factor times a variable) and a constant. Such an object can be built using the standard constructor:

```
julia> f = MOI.ScalarAffineFunction([MOI.ScalarAffineTerm(1, x)], 2) # x + 2
(2) + (1) MOI.VariableIndex(1)
```

However, you can also use operators to build the same scalar function:

```
julia> f = x + 2
(2) + (1) MOI.VariableIndex(1)
```

### Creating scalar quadratic functions

Scalar quadratic functions are stored in `ScalarQuadraticFunction` objects, in a way that is highly similar to scalar affine functions. You can obtain a quadratic function as a product of affine functions:

```
julia> 1 * x * x
(0) + 1.0 MOI.VariableIndex(1)²

julia> f * f  # (x + 2)²
```

```
(4) + (2) MOI.VariableIndex(1) + (2) MOI.VariableIndex(1) + 1.0 MOI.VariableIndex(1)²

julia> f^2  # (x + 2)² too
(4) + (2) MOI.VariableIndex(1) + (2) MOI.VariableIndex(1) + 1.0 MOI.VariableIndex(1)²
```

### Creating vector functions

A vector function is a function with several values, irrespective of the number of input variables. Similarly to scalar functions, there are three main types of vector functions: `VectorOfVariables`, `VectorAffineFunction`, and `VectorQuadraticFunction`.

The easiest way to create a vector function is to stack several scalar functions using `Utilities.vectorize`. It takes a vector as input, and the generated vector function (of the most appropriate type) has each dimension corresponding to a dimension of the vector.

```
julia> g = MOI.Utilities.vectorize([f, 2 * f])
┌                            ┐
│(2) + (1) MOI.VariableIndex(1)│
│(4) + (2) MOI.VariableIndex(1)│
└                            ┘
```

**Warning**

`Utilities.vectorize` only takes a vector of similar scalar functions: you cannot mix `VariableIndex` and `ScalarAffineFunction`, for instance. In practice, it means that `Utilities.vectorize([x, f])` does not work; you should rather use `Utilities.vectorize([1 * x, f])` instead to only have `ScalarAffineFunction` objects.

## 7.2 Canonicalizing functions

In more advanced use cases, you might need to ensure that a function is "canonical." Functions are stored as an array of terms, but there is no check that these terms are redundant: a `ScalarAffineFunction` object might have two terms with the same variable, like `x + x + 1`. These terms could be merged without changing the semantics of the function: `2x + 1`.

Working with these objects might be cumbersome. Canonicalization helps maintain redundancy to zero.

`Utilities.is_canonical` checks whether a function is already in its canonical form:

```
julia> MOI.Utilities.is_canonical(f + f) # (x + 2) + (x + 2) is stored as x + x + 4
false
```

`Utilities.canonical` returns the equivalent canonical version of the function:

```
julia> MOI.Utilities.canonical(f + f) # Returns 2x + 4
(4) + (2) MOI.VariableIndex(1)
```

## 7.3 Exploring functions

At some point, you might need to dig into a function, for instance to map it into solver constructs.

### Vector functions

`Utilities.scalarize` returns a vector of scalar functions from a vector function:

```
julia> MOI.Utilities.scalarize(g) # Returns a vector [f, 2 * f].
2-element Vector{MathOptInterface.ScalarAffineFunction{Int64}}:
 (2) + (1) MOI.VariableIndex(1)
 (4) + (2) MOI.VariableIndex(1)
```

**Note**

`Utilities.eachscalar` returns an iterator on the dimensions, which serves the same purpose as `Utilities.scalarize`.

`output_dimension` returns the number of dimensions of the output of a function:

```
julia> MOI.output_dimension(g)
2
```

# Chapter 8

# Latency

MathOptInterface suffers the "time-to-first-solve" problem of start-up latency.

This hurts both the user- and developer-experience of MathOptInterface. In the first case, because simple models have a multi-second delay before solving, and in the latter, because our tests take so long to run.

This page contains some advice on profiling and fixing latency-related problems in the MathOptInterface.jl repository.

## 8.1 Background

Before reading this part of the documentation, you should familiarize yourself with the reasons for latency in Julia and how to fix them.

- Read the blogposts on julialang.org on precompilation and SnoopCompile
- Read the SnoopCompile documentation.
- Watch Tim Holy's talk at JuliaCon 2021
- Watch the package development workshop at JuliaCon 2021

## 8.2 Causes

There are three main causes of latency in MathOptInterface:

1. A large number of types
2. Lack of method ownership
3. Type-instability in the bridge layer

### A large number of types

Julia is very good at specializing method calls based on the input type. Each specialization has a compilation cost, but the benefit of faster run-time performance.

The best-case scenario is for a method to be called a large number of times with a single set of argument types. The worst-case scenario is for a method to be called a single time for a large set of argument types.

Because of MathOptInterface's function-in-set formulation, we fall into the worst-case situation.

This is a fundamental limitation of Julia, so there isn't much we can do about it. However, if we can precompile MathOptInterface, much of the cost can be shifted from start-up latency to the time it takes to precompile a package on installation.

However, there are two things which make MathOptInterface hard to precompile.

### Lack of method ownership

Lack of method ownership happens when a call is made using a mix of structs and methods from different modules. Because of this, no single module "owns" the method that is being dispatched, and so it cannot be precompiled.

**Tip**

This is a slightly simplified explanation. Read the precompilation tutorial for a more in-depth discussion on back-edges.

Unfortunately, the design of MOI means that this is a frequent occurrence: we have a bunch of types in `MOI.Utilities` that wrap types defined in external packages (for example, the `Optimizers`), which implement methods of functions defined in `MOI` (for example, `add_variable`, `add_constraint`).

Here's a simple example of method-ownership in practice:

```
module MyMOI
struct Wrapper{T}
    inner::T
end
optimize!(x::Wrapper) = optimize!(x.inner)
end  # MyMOI

module MyOptimizer
using ..MyMOI
struct Optimizer end
MyMOI.optimize!(x::Optimizer) = 1
end  # MyOptimizer

using SnoopCompile
model = MyMOI.Wrapper(MyOptimizer.Optimizer())

julia> tinf = @snoopi_deep MyMOI.optimize!(model)
InferenceTimingNode: 0.008256/0.008543 on InferenceFrameInfo for Core.Compiler.Timings.ROOT() with
↪  1 direct children
```

The result is that there was one method that required type inference. If we visualize `tinf`:

```
using ProfileView
ProfileView.view(flamegraph(tinf))
```

we see a flamegraph with a large red-bar indicating that the method `MyMOI.optimize(MyMOI.Wrapper{MyOptimizer.Optimizer}` cannot be precompiled.

To fix this, we need to designate a module to "own" that method (that is, create a back-edge). The easiest way to do this is for `MyOptimizer` to call `MyMOI.optimize(MyMOI.Wrapper{MyOptimizer.Optimizer})` during `using MyOptimizer`. Let's see that in practice:

```
module MyMOI
struct Wrapper{T}
    inner::T
end
optimize(x::Wrapper) = optimize(x.inner)
end  # MyMOI

module MyOptimizer
using ..MyMOI
struct Optimizer end
MyMOI.optimize(x::Optimizer) = 1
# The syntax of this let-while loop is very particular:
#  * `let ... end` keeps everything local to avoid polluting the MyOptimizer
#    namespace
#  * `while true ... break end` runs the code once, and forces Julia to compile
#    the inner loop, rather than interpret it.
let
    while true
        model = MyMOI.Wrapper(Optimizer())
        MyMOI.optimize(model)
        break
    end
end
end  # MyOptimizer

using SnoopCompile
model = MyMOI.Wrapper(MyOptimizer.Optimizer())

julia> tinf = @snoopi_deep MyMOI.optimize(model)
InferenceTimingNode: 0.006822/0.006822 on InferenceFrameInfo for Core.Compiler.Timings.ROOT() with
↪  0 direct children
```

There are now `0` direct children that required type inference because the method was already stored in `MyOptimizer`!

Unfortunately, this trick only works if the call-chain is fully inferrable. If there are breaks (due to type instability), then the benefit of doing this is reduced. And unfortunately for us, the design of MathOptInterface has a lot of type instabilities.

### Type instability in the bridge layer

Most of MathOptInterface is pretty good at ensuring type-stability. However, a key component is not type stable, and that is the bridging layer.

In particular, the bridging layer defines `Bridges.LazyBridgeOptimizer`, which has fields like:

```
struct LazyBridgeOptimizer
    constraint_bridge_types::Vector{Any}
    constraint_node::Dict{Tuple{Type,Type},ConstraintNode}
    constraint_types::Vector{Tuple{Type,Type}}
end
```

This is because the `LazyBridgeOptimizer` needs to be able to deal with any function-in-set type passed to it, and we also allow users to pass additional bridges that they defined in external packages.

So to recap, MathOptInterface suffers package latency because:

1. there are a large number of types and functions
2. and these are split between multiple modules, including external packages
3. and there are type-instabilities like those in the bridging layer.

## 8.3 Resolutions

There are no magic solutions to reduce latency. Issue #1313 tracks progress on reducing latency in MathOptInterface.

A useful script is the following (replace GLPK as needed):

```
import GLPK
import MathOptInterface as MOI

function example_diet(optimizer, bridge)
    category_data = [
        1800.0 2200.0;
          91.0    Inf;
           0.0   65.0;
           0.0 1779.0
    ]
    cost = [2.49, 2.89, 1.50, 1.89, 2.09, 1.99, 2.49, 0.89, 1.59]
    food_data = [
        410 24 26 730;
        420 32 10 1190;
        560 20 32 1800;
        380  4 19 270;
        320 12 10 930;
        320 15 12 820;
        320 31 12 1230;
        100  8 2.5 125;
        330  8 10 180
    ]
    bridge_model = if bridge
        MOI.instantiate(optimizer; with_bridge_type=Float64)
    else
        MOI.instantiate(optimizer)
    end
    model = MOI.Utilities.CachingOptimizer(
        MOI.Utilities.UniversalFallback(MOI.Utilities.Model{Float64}()),
        MOI.Utilities.AUTOMATIC,
    )
    MOI.Utilities.reset_optimizer(model, bridge_model)
    MOI.set(model, MOI.Silent(), true)
    nutrition = MOI.add_variables(model, size(category_data, 1))
    for (i, v) in enumerate(nutrition)
        MOI.add_constraint(model, v, MOI.GreaterThan(category_data[i, 1]))
        MOI.add_constraint(model, v, MOI.LessThan(category_data[i, 2]))
    end
    buy = MOI.add_variables(model, size(food_data, 1))
    MOI.add_constraint.(model, buy, MOI.GreaterThan(0.0))
    MOI.set(model, MOI.ObjectiveSense(), MOI.MIN_SENSE)
    f = MOI.ScalarAffineFunction(MOI.ScalarAffineTerm.(cost, buy), 0.0)
    MOI.set(model, MOI.ObjectiveFunction{typeof(f)}(), f)
```

```
    for (j, n) in enumerate(nutrition)
        f = MOI.ScalarAffineFunction(
            MOI.ScalarAffineTerm.(food_data[:, j], buy),
            0.0,
        )
        push!(f.terms, MOI.ScalarAffineTerm(-1.0, n))
        MOI.add_constraint(model, f, MOI.EqualTo(0.0))
    end
    MOI.optimize!(model)
    term_status = MOI.get(model, MOI.TerminationStatus())
    @assert term_status == MOI.OPTIMAL
    MOI.add_constraint(
        model,
        MOI.ScalarAffineFunction(
            MOI.ScalarAffineTerm.(1.0, [buy[end-1], buy[end]]),
            0.0,
        ),
        MOI.LessThan(6.0),
    )
    MOI.optimize!(model)
    @assert MOI.get(model, MOI.TerminationStatus()) == MOI.INFEASIBLE
    return
end

if length(ARGS) > 0
    bridge = get(ARGS, 2, "") != "--no-bridge"
    println("Running: $(ARGS[1]) $(get(ARGS, 2, ""))")
    @time example_diet(GLPK.Optimizer, bridge)
    @time example_diet(GLPK.Optimizer, bridge)
    exit(0)
end
```

You can create a flame-graph via

```
using SnoopCompile
tinf = @snoopi_deep example_diet(GLPK.Optimizer, true)
using ProfileView
ProfileView.view(flamegraph(tinf))
```

Here's how things looked in mid-August 2021:

There are a few opportunities for improvement (non-red flames, particularly on the right). But the main problem is a large red (non-precompilable due to method ownership) flame.

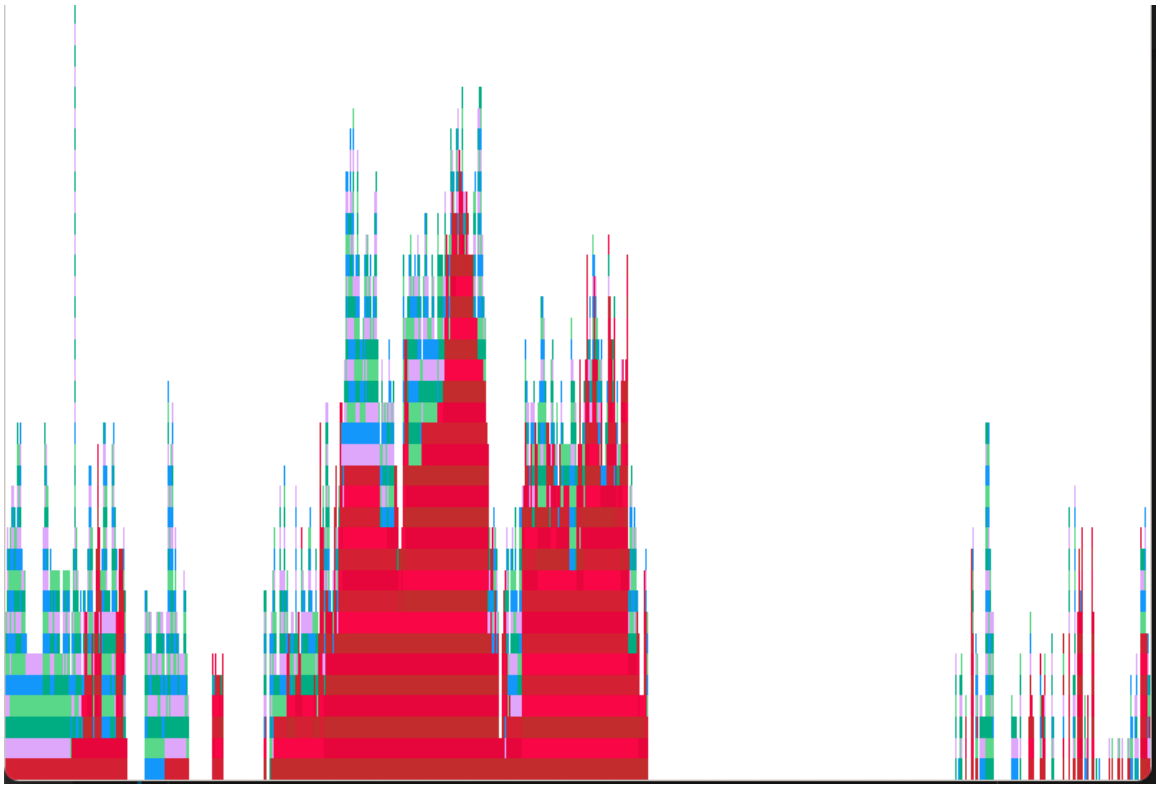


Figure 8.1: flamegraph

# Part III

# Manual

# Chapter 9

# Standard form problem

MathOptInterface represents optimization problems in the standard form:

$$\min_{x \in \mathbb{R}^n} \quad f_0(x) \tag{9.1}$$

$$\text{s.t.} \quad f_i(x) \in \mathcal{S}_i \qquad i = 1 \dots m \tag{9.2}$$

where:

- the functions $f_0, f_1, \dots, f_m$ are specified by `AbstractFunction` objects
- the sets $\mathcal{S}_1, \dots, \mathcal{S}_m$ are specified by `AbstractSet` objects

An important design consideration is that the sets are independent of the decision variables.

**Tip**

For more information on this standard form, read our paper.

MOI defines some commonly used functions and sets, but the interface is extensible to other sets recognized by the solver.

## 9.1 Functions

The function types implemented in MathOptInterface.jl are:

## 9.2 One-dimensional sets

The one-dimensional set types implemented in MathOptInterface.jl are:

## 9.3 Vector cones

The vector-valued set types implemented in MathOptInterface.jl are:

| Function | Description |
|---|---|
| `VariableIndex` | $x_j$, the projection onto a single coordinate defined by a variable index $j$ |
| `ScalarAffineFunction` | $a^T x + b$, where $a$ is a vector and $b$ scalar |
| `ScalarQuadraticFunction` | $\frac{1}{2}x^T Q x + a^T x + b$, where $Q$ is a symmetric matrix, $a$ is a vector, and $b$ is a constant |
| `ScalarNonlinearFunction` | $f(x)$, where $f$ is a nonlinear function |
| `VectorOfVariables` | The projection onto multiple coordinates (that is, extracting a sub-vector) |
| `VectorAffineFunction` | $Ax + b$, where $A$ is a matrix and $b$ is a vector |
| `VectorQuadraticFunction` | A vector of scalar-valued quadratic functions |
| `VectorNonlinearFunction` | $f(x)$, where $f$ is a vector-valued nonlinear function |

| Set | Description |
|---|---|
| `LessThan(u)` | $(-\infty, u]$ |
| `GreaterThan(l)` | $[l, \infty)$ |
| `EqualTo(v)` | $\{v\}$ |
| `Interval(l, u)` | $[l, u]$ |
| `Integer()` | $\mathbb{Z}$ |
| `ZeroOne()` | $\{0, 1\}$ |
| `Semicontinuous(l, u)` | $\{0\} \cup [l, u]$ |
| `Semiinteger(l, u)` | $\{0\} \cup \{l, l+1, \ldots, u-1, u\}$ |

| Set | Description |
|---|---|
| `Reals(d)` | $\mathbb{R}^d$ |
| `Zeros(d)` | $0^d$ |
| `Nonnegatives(d)` | $\{x \in \mathbb{R}^d : x \geq 0\}$ |
| `Nonpositives(d)` | $\{x \in \mathbb{R}^d : x \leq 0\}$ |
| `SecondOrderCone(d)` | $\{(t, x) \in \mathbb{R}^d : t \geq \|x\|_2\}$ |
| `RotatedSecondOrderCone(d)` | $\{(t, u, x) \in \mathbb{R}^d : 2tu \geq \|x\|_2^2, t \geq 0, u \geq 0\}$ |
| `ExponentialCone()` | $\{(x, y, z) \in \mathbb{R}^3 : y \exp(x/y) \leq z, y > 0\}$ |
| `DualExponentialCone()` | $\{(u, v, w) \in \mathbb{R}^3 : -u \exp(v/u) \leq \exp(1) w, u < 0\}$ |
| `GeometricMeanCone(d)` | $\{(t, x) \in \mathbb{R}^{1+n} : x \geq 0, t \leq \sqrt[n]{\prod_i x_i}\}$ where $n$ is $d-1$ |
| `DualGeometricMeanCone(d)` | $\{(t, x) \in \mathbb{R}^{1+n} : x \geq 0, 0 \geq t \geq -n \sqrt[n]{\prod_i x_i}\}$ where $n$ is $d-1$ |
| `PowerCone(α)` | $\{(x, y, z) \in \mathbb{R}^3 : x^\alpha y^{1-\alpha} \geq \|z\|, x \geq 0, y \geq 0\}$ |
| `DualPowerCone(α)` | $\{(u, v, w) \in \mathbb{R}^3 : \left(\frac{u}{\alpha}\right)^\alpha \left(\frac{v}{1-\alpha}\right)^{1-\alpha} \geq \|w\|, u, v \geq 0\}$ |
| `NormOneCone(d)` | $\{(t, x) \in \mathbb{R}^d : t \geq \sum_i \|x_i\|\}$ |
| `NormInfinityCone(d)` | $\{(t, x) \in \mathbb{R}^d : t \geq \max_i \|x_i\|\}$ |
| `RelativeEntropyCone(d)` | $\{(u, v, w) \in \mathbb{R}^d : u \geq \sum_i w_i \log(\frac{w_i}{v_i}), v_i > 0, w_i > 0\}$ |
| `DualRelativeEntropyCone(d)` | $\{(u, v, w) \in \mathbb{R}^d : \forall i, w_i \geq u(\log(\frac{u}{v_i}) - 1), v_i > 0, u > 0\}$ |
| `HyperRectangle(l, u)` | $\{x \in \bar{\mathbb{R}}^d : x_i \in [l_i, u_i] \forall i = 1, \ldots, d\}$ |
| `NormCone(p, d)` | $\{(t, x) \in \mathbb{R}^d : t \geq \left(\sum_i \|x_i\|^p\right)^{\frac{1}{p}}\}$ |

| Set | Description |
|---|---|
| `VectorNonlinearOracle` | $\{x \in \mathbb{R}^{dimension} : l \leq f(x) \leq u\}$ |

## 9.4 Matrix cones

The matrix-valued set types implemented in MathOptInterface.jl are:

| Set | Description |
|---|---|
| `RootDetConeTriangle(d)` | $\{(t, X) \in \mathbb{R}^{1+d(1+d)/2} : t \leq \det(X)^{1/d}, X$ is the upper triangle of a PSD matrix$\}$ |
| `RootDetConeSquare(d)` | $\{(t, X) \in \mathbb{R}^{1+d^2} : t \leq \det(X)^{1/d}, X$ is a PSD matrix$\}$ |
| `PositiveSemidefiniteConeTriangle(d)` | $\{X \in \mathbb{R}^{d(d+1)/2} : X$ is the upper triangle of a PSD matrix$\}$ |
| `PositiveSemidefiniteConeSquare(d)` | $\{X \in \mathbb{R}^{d^2} : X$ is a PSD matrix$\}$ |
| `LogDetConeTriangle(d)` | $\{(t, u, X) \in \mathbb{R}^{2+d(1+d)/2} : t \leq u \log(\det(X/u)), X$ is the upper triangle of a PSD matrix, $u > 0\}$ |
| `LogDetConeSquare(d)` | $\{(t, u, X) \in \mathbb{R}^{2+d^2} : t \leq u \log(\det(X/u)), X$ is a PSD matrix, $u > 0\}$ |
| `NormSpectralCone(r, c)` | $\{(t, X) \in \mathbb{R}^{1+r\times c} : t \geq \sigma_1(X), X$ is a $r \times c$ matrix$\}$ |
| `NormNuclearCone(r, c)` | $\{(t, X) \in \mathbb{R}^{1+r\times c} : t \geq \sum_i \sigma_i(X), X$ is a $r \times c$ matrix$\}$ |
| `HermitianPositiveSemidefiniteConeTriangle(d)` | The cone of Hermitian positive semidefinite matrices, with `side_dimension` rows and columns. |
| `Scaled(S)` | The set `S` scaled so that `Utilities.set_dot` corresponds to `LinearAlgebra.dot` |

Some of these cones can take two forms: `XXXConeTriangle` and `XXXConeSquare`.

In `XXXConeTriangle` sets, the matrix is assumed to be symmetric, and the elements are provided by a vector, in which the entries of the upper-right triangular part of the matrix are given column by column (or equivalently, the entries of the lower-left triangular part are given row by row).

In `XXXConeSquare` sets, the entries of the matrix are given column by column (or equivalently, row by row), and the matrix is constrained to be symmetric. As an example, given a 2-by-2 matrix of variables `X` and a one-dimensional variable `t`, we can specify a root-det constraint as `[t, X11, X12, X22] ∈ RootDetConeTriangle` or `[t, X11, X12, X21, X22] ∈ RootDetConeSquare`.

We provide both forms to enable flexibility for solvers who may natively support one or the other. Transformations between `XXXConeTriangle` and `XXXConeSquare` are handled by bridges, which removes the chance of conversion mistakes by users or solver developers.

## 9.5 Multi-dimensional sets with combinatorial structure

Other sets are vector-valued, with a particular combinatorial structure. Read their docstrings for more information on how to interpret them.

| Set | Description |
|---|---|
| SOS1 | A Special Ordered Set (SOS) of Type I |
| SOS2 | A Special Ordered Set (SOS) of Type II |
| Indicator | A set to specify an indicator constraint |
| Complements | A set to specify a mixed complementarity constraint |
| AllDifferent | The `all_different` global constraint |
| BinPacking | The `bin_packing` global constraint |
| Circuit | The `circuit` global constraint |
| CountAtLeast | The `at_least` global constraint |
| CountBelongs | The `nvalue` global constraint |
| CountDistinct | The `distinct` global constraint |
| CountGreaterThan | The `count_gt` global constraint |
| Cumulative | The `cumulative` global constraint |
| Path | The `path` global constraint |
| Table | The `table` global constraint |

# Chapter 10

# Models

The most significant part of MOI is the definition of the **model API** that is used to specify an instance of an optimization problem (for example, by adding variables and constraints). Objects that implement the model API must inherit from the `ModelLike` abstract type.

Notably missing from the model API is the method to solve an optimization problem. `ModelLike` objects may store an instance (for example, in memory or backed by a file format) without being linked to a particular solver. In addition to the model API, MOI defines `AbstractOptimizer` and provides methods to solve the model and interact with solutions. See the Solutions section for more details.

**Info**

Throughout the rest of the manual, `model` is used as a generic `ModelLike`, and `optimizer` is used as a generic `AbstractOptimizer`.

**Tip**

MOI does not export functions, but for brevity we often omit qualifying names with the MOI module. Best practice is to have

```
import MathOptInterface as MOI
```

and prefix all MOI methods with `MOI.` in user code. If a name is also available in base Julia, we always explicitly use the module prefix, for example, with `MOI.get`.

## 10.1 Attributes

Attributes are properties of the model that can be queried and modified. These include constants such as the number of variables in a model `NumberOfVariables`), and properties of variables and constraints such as the name of a variable (`VariableName`).

There are four types of attributes:

- Model attributes (subtypes of `AbstractModelAttribute`) refer to properties of a model.
- Optimizer attributes (subtypes of `AbstractOptimizerAttribute`) refer to properties of an optimizer.

- Constraint attributes (subtypes of `AbstractConstraintAttribute`) refer to properties of an individual constraint.
- Variable attributes (subtypes of `AbstractVariableAttribute`) refer to properties of an individual variable.

Some attributes are values that can be queried by the user but not modified, while other attributes can be modified by the user.

All interactions with attributes occur through the `get` and `set` functions.

Consult the docstrings of each attribute for information on what it represents.

## 10.2 ModelLike API

The following attributes are available:

- `ListOfConstraintAttributesSet`
- `ListOfConstraintIndices`
- `ListOfConstraintTypesPresent`
- `ListOfConstraintsWithAttributeSet`
- `ListOfModelAttributesSet`
- `ListOfVariableAttributesSet`
- `ListOfVariableIndices`
- `ListOfVariablesWithAttributeSet`
- `NumberOfConstraints`
- `NumberOfVariables`
- `Name`
- `ObjectiveFunction`
- `ObjectiveFunctionType`
- `ObjectiveSense`

## 10.3 AbstractOptimizer API

The following attributes are available:

- `DualStatus`
- `PrimalStatus`
- `RawStatusString`
- `ResultCount`
- `TerminationStatus`

- BarrierIterations
- DualObjectiveValue
- NodeCount
- NumberOfThreads
- ObjectiveBound
- ObjectiveValue
- RelativeGap
- RawOptimizerAttribute
- RawSolver
- Silent
- SimplexIterations
- SolverName
- SolverVersion
- SolveTimeSec
- TimeLimitSec
- ObjectiveLimit
- SolutionLimit
- NodeLimit
- AutomaticDifferentiationBackend

# Chapter 11

# Variables

## 11.1 Add a variable

Use `add_variable` to add a single variable.

```
julia> x = MOI.add_variable(model)
MOI.VariableIndex(1)
```

`add_variable` returns a `VariableIndex` type, which is used to refer to the added variable in other calls.

Check if a `VariableIndex` is valid using `is_valid`.

```
julia> MOI.is_valid(model, x)
true
```

Use `add_variables` to add a number of variables.

```
julia> y = MOI.add_variables(model, 2)
2-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(2)
 MOI.VariableIndex(3)
```

**Warning**

The integer does not necessarily correspond to the column inside an optimizer.

## 11.2 Delete a variable

Delete a variable using `delete`.

```
julia> MOI.delete(model, x)

julia> MOI.is_valid(model, x)
false
```

**Warning**

Not all `ModelLike` models support deleting variables. A `DeleteNotAllowed` error is thrown if this is not supported.

## 11.3 Variable attributes

The following attributes are available for variables:

- `VariableName`
- `VariablePrimalStart`
- `VariablePrimal`

Get and set these attributes using `get` and `set`.

```
julia> MOI.set(model, MOI.VariableName(), x, "var_x")

julia> MOI.get(model, MOI.VariableName(), x)
"var_x"
```

# Chapter 12

# Constraints

## 12.1 Add a constraint

Use `add_constraint` to add a single constraint.

```
julia> c = MOI.add_constraint(model, MOI.VectorOfVariables(x), MOI.Nonnegatives(2))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.Nonnegatives}(1)
```

`add_constraint` returns a `ConstraintIndex` type, which is used to refer to the added constraint in other calls.

Check if a `ConstraintIndex` is valid using `is_valid`.

```
julia> MOI.is_valid(model, c)
true
```

Use `add_constraints` to add a number of constraints of the same type.

```
julia> c = MOI.add_constraints(
           model,
           [x[1], x[2]],
           [MOI.GreaterThan(0.0), MOI.GreaterThan(1.0)]
       )
2-element Vector{MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.GreaterThan{Float64}}}:
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.GreaterThan{Float64}}(1)
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.GreaterThan{Float64}}(2)
```

This time, a vector of `ConstraintIndex` are returned.

Use `supports_constraint` to check if the model supports adding a constraint type.

```
julia> MOI.supports_constraint(
           model,
           MOI.VariableIndex,
           MOI.GreaterThan{Float64},
```

```
       )
true
```

## 12.2 Delete a constraint

Use `delete` to delete a constraint.

```
julia> MOI.delete(model, c)

julia> MOI.is_valid(model, c)
false
```

## 12.3 Constraint attributes

The following attributes are available for constraints:

- `ConstraintName`
- `ConstraintPrimalStart`
- `ConstraintDualStart`
- `ConstraintPrimal`
- `ConstraintDual`
- `ConstraintBasisStatus`
- `ConstraintFunction`
- `CanonicalConstraintFunction`
- `ConstraintSet`

Get and set these attributes using `get` and `set`.

```
julia> MOI.set(model, MOI.ConstraintName(), c, "con_c")

julia> MOI.get(model, MOI.ConstraintName(), c)
"con_c"
```

## 12.4 Constraints by function-set pairs

Below is a list of common constraint types and how they are represented as function-set pairs in MOI. In the notation below, $x$ is a vector of decision variables, $x_i$ is a scalar decision variable, $\alpha, \beta$ are scalar constants, $a, b$ are constant vectors, A is a constant matrix and $\mathbb{R}_+$ (resp. $\mathbb{R}_-$) is the set of non-negative (resp. non-positive) real numbers.

| Mathematical Constraint | MOI Function | MOI Set |
|---:|---:|---:|
| $a^T x \leq \beta$ | `ScalarAffineFunction` | `LessThan` |
| $a^T x \geq \alpha$ | `ScalarAffineFunction` | `GreaterThan` |
| $a^T x = \beta$ | `ScalarAffineFunction` | `EqualTo` |
| $\alpha \leq a^T x \leq \beta$ | `ScalarAffineFunction` | `Interval` |
| $x_i \leq \beta$ | `VariableIndex` | `LessThan` |
| $x_i \geq \alpha$ | `VariableIndex` | `GreaterThan` |
| $x_i = \beta$ | `VariableIndex` | `EqualTo` |
| $\alpha \leq x_i \leq \beta$ | `VariableIndex` | `Interval` |
| $Ax + b \in \mathbb{R}^n_+$ | `VectorAffineFunction` | `Nonnegatives` |
| $Ax + b \in \mathbb{R}^n_-$ | `VectorAffineFunction` | `Nonpositives` |
| $Ax + b = 0$ | `VectorAffineFunction` | `Zeros` |

### Linear constraints

By convention, solvers are not expected to support nonzero constant terms in the `ScalarAffineFunction`s the first four rows of the preceding table because they are redundant with the parameters of the sets. For example, encode $2x + 1 \leq 2$ as $2x \leq 1$.

Constraints with `VariableIndex` in `LessThan`, `GreaterThan`, `EqualTo`, or `Interval` sets have a natural interpretation as variable bounds. As such, it is typically not natural to impose multiple lower- or upper-bounds on the same variable, and the solver interfaces will throw respectively `LowerBoundAlreadySet` or `UpperBoundAlreadySet`.

Moreover, adding two `VariableIndex` constraints on the same variable with the same set is impossible because they share the same index as it is the index of the variable, see `ConstraintIndex`.

It is natural, however, to impose upper- and lower-bounds separately as two different constraints on a single variable. The difference between imposing bounds by using a single `Interval` constraint and by using separate `LessThan` and `GreaterThan` constraints is that the latter will allow the solver to return separate dual multipliers for the two bounds, while the former will allow the solver to return only a single dual for the interval constraint.

### Conic constraints

| Mathematical Constraint | MOI Function | MOI Set |
|---:|---:|---:|
| $\|Ax + b\|_2 \leq c^T x + d$ | `VectorAffineFunction` | `SecondOrderCone` |
| $y \geq \|x\|_2$ | `VectorOfVariables` | `SecondOrderCone` |
| $2yz \geq \|x\|_2^2, y, z \geq 0$ | `VectorOfVariables` | `RotatedSecondOrderCone` |
| $(a_1^T x + b_1, a_2^T x + b_2, a_3^T x + b_3) \in \mathcal{E}$ | `VectorAffineFunction` | `ExponentialCone` |
| $A(x) \in \mathcal{S}_+$ | `VectorAffineFunction` | `PositiveSemidefiniteConeTriangle` |
| $B(x) \in \mathcal{S}_+$ | `VectorAffineFunction` | `PositiveSemidefiniteConeSquare` |
| $x \in \mathcal{S}_+$ | `VectorOfVariables` | `PositiveSemidefiniteConeTriangle` |
| $x \in \mathcal{S}_+$ | `VectorOfVariables` | `PositiveSemidefiniteConeSquare` |

where $\mathcal{E}$ is the exponential cone (see `ExponentialCone`), $\mathcal{S}_+$ is the set of positive semidefinite symmetric matrices, $A$ is an affine map that outputs symmetric matrices and $B$ is an affine map that outputs square matrices.

| Mathematical Constraint | MOI Function | MOI Set |
|---|---|---|
| $\frac{1}{2}x^TQx + a^Tx + b \geq 0$ | `ScalarQuadraticFunction` | `GreaterThan` |
| $\frac{1}{2}x^TQx + a^Tx + b \leq 0$ | `ScalarQuadraticFunction` | `LessThan` |
| $\frac{1}{2}x^TQx + a^Tx + b = 0$ | `ScalarQuadraticFunction` | `EqualTo` |
| Bilinear matrix inequality | `VectorQuadraticFunction` | `PositiveSemidefiniteCone...` |

### Quadratic constraints

**Note**

For more details on the internal format of the quadratic functions see `ScalarQuadraticFunction` or `VectorQuadraticFunction`.

### Discrete and logical constraints

| Mathematical Constraint | MOI Function | MOI Set |
|---|---|---|
| $x_i \in \mathbb{Z}$ | `VariableIndex` | `Integer` |
| $x_i \in \{0, 1\}$ | `VariableIndex` | `ZeroOne` |
| $x_i \in \{0\} \cup [l, u]$ | `VariableIndex` | `Semicontinuous` |
| $x_i \in \{0\} \cup \{l, l+1, \ldots, u-1, u\}$ | `VariableIndex` | `Semiinteger` |
| At most one component of $x$ can be nonzero | `VectorOfVariables` | `SOS1` |
| At most two components of $x$ can be nonzero, and if so they must be adjacent components | `VectorOfVariables` | `SOS2` |
| $y = 1 \implies a^Tx \in S$ | `VectorAffineFunction` | `Indicator` |

## 12.5 JuMP mapping

The following bullet points show examples of how JuMP constraints are translated into MOI function-set pairs:

- `@constraint(m, 2x + y <= 10)` becomes `ScalarAffineFunction-in-LessThan`
- `@constraint(m, 2x + y >= 10)` becomes `ScalarAffineFunction-in-GreaterThan`
- `@constraint(m, 2x + y == 10)` becomes `ScalarAffineFunction-in-EqualTo`
- `@constraint(m, 0 <= 2x + y <= 10)` becomes `ScalarAffineFunction-in-Interval`
- `@constraint(m, 2x + y in ArbitrarySet())` becomes `ScalarAffineFunction-in-ArbitrarySet`.

Variable bounds are handled in a similar fashion:

- `@variable(m, x <= 1)` becomes `VariableIndex-in-LessThan`
- `@variable(m, x >= 1)` becomes `VariableIndex-in-GreaterThan`

One notable difference is that a variable with an upper and lower bound is translated into two constraints, rather than an interval, that is:

- `@variable(m, 0 <= x <= 1)` becomes `VariableIndex-in-LessThan` and `VariableIndex-in-GreaterThan`.

# Chapter 13

# Solutions

## 13.1 Solving and retrieving the results

Once an optimizer is loaded with the objective function and all of the constraints, we can ask the solver to solve the model by calling `optimize!`.

```
MOI.optimize!(optimizer)
```

## 13.2 Why did the solver stop?

The optimization procedure may stop for a number of reasons. The `TerminationStatus` attribute of the optimizer returns a `TerminationStatusCode` object which explains why the solver stopped.

The termination statuses distinguish between proofs of optimality, infeasibility, local convergence, limits, and termination because of something unexpected like invalid problem data or failure to converge.

A typical usage of the `TerminationStatus` attribute is as follows:

```
status = MOI.get(optimizer, TerminationStatus())
if status == MOI.OPTIMAL
    # Ok, we solved the problem!
else
    # Handle other cases.
end
```

After checking the `TerminationStatus`, check `ResultCount`. This attribute returns the number of results that the solver has available to return. A result is defined as a primal-dual pair, but either the primal or the dual may be missing from the result. While the `OPTIMAL` termination status normally implies that at least one result is available, other statuses do not. For example, in the case of infeasibility, a solver may return no result or a proof of infeasibility. The `ResultCount` attribute distinguishes between these two cases.

## 13.3 Primal solutions

Use the `PrimalStatus` optimizer attribute to return a `ResultStatusCode` describing the status of the primal solution.

Common returns are described below in the Common status situations section.

Query the primal solution using the `VariablePrimal` and `ConstraintPrimal` attributes.

Query the objective function value using the `ObjectiveValue` attribute.

## 13.4 Dual solutions

**Warning**

See Duality for a discussion of the MOI conventions for primal-dual pairs and certificates.

Use the `DualStatus` optimizer attribute to return a `ResultStatusCode` describing the status of the dual solution.

Query the dual solution using the `ConstraintDual` attribute.

Query the dual objective function value using the `DualObjectiveValue` attribute.

## 13.5 Common status situations

The sections below describe how to interpret typical or interesting status cases for three common classes of solvers. The example cases are illustrative, not comprehensive. Solver wrappers may provide additional information on how the solver's statuses map to MOI statuses.

**Info**

* in the tables indicate that multiple different values are possible.

### Primal-dual convex solver

Linear programming and conic optimization solvers fall into this category.

| What happened? | TerminationStatus | ResultCount | PrimalStatus | DualStatus |
|---|---|---|---|---|
| Proved optimality | OPTIMAL | 1 | FEASIBLE_POINT | FEASIBLE_POINT |
| Proved infeasible | INFEASIBLE | 1 | NO_SOLUTION | INFEASIBILITY_CERTIFICATE |
| Optimal within relaxed tolerances | ALMOST_OPTIMAL | 1 | FEASIBLE_POINT | FEASIBLE_POINT |
| Optimal within relaxed tolerances | ALMOST_OPTIMAL | 1 | ALMOST_FEASIBLE_POINT | ALMOST_FEASIBLE_POINT |
| Detected an unbounded ray of the primal | DUAL_INFEASIBLE | 1 | INFEASIBILITY_CERTIFICATE | NO_SOLUTION |
| Stall | SLOW_PROGRESS | 1 | * | * |

### Global branch-and-bound solvers

Mixed-integer programming solvers fall into this category.

**Info**

`CPXMIP_OPTIMAL_INFEAS` is a CPLEX status that indicates that a preprocessed problem was solved to optimality, but the solver was unable to recover a feasible solution to the original problem. Handling this status was one of the motivating drivers behind the design of MOI.

| What happened? | TerminationStatus | ResultCount | PrimalStatus | DualStatus |
|---|---|---|---|---|
| Proved optimality | OPTIMAL | 1 | FEASIBLE_POINT | NO_SOLUTION |
| Presolve detected infeasibility or unboundedness | INFEASIBLE_OR_UNBOUNDED | 0 | NO_SOLUTION | NO_SOLUTION |
| Proved infeasibility | INFEASIBLE | 0 | NO_SOLUTION | NO_SOLUTION |
| Timed out (no solution) | TIME_LIMIT | 0 | NO_SOLUTION | NO_SOLUTION |
| Timed out (with a solution) | TIME_LIMIT | 1 | FEASIBLE_POINT | NO_SOLUTION |
| CPXMIP_OPTIMAL_INFEAS | ALMOST_OPTIMAL | 1 | INFEASIBLE_POINT | NO_SOLUTION |

### Local search solvers

Nonlinear programming solvers fall into this category. It also includes non-global tree search solvers like Juniper.

| What happened? | TerminationStatus | ResultCount | PrimalStatus | DualStatus |
|---|---|---|---|---|
| Converged to a stationary point | LOCALLY_SOLVED | 1 | FEASIBLE_POINT | FEASIBLE_POINT |
| Completed a non-global tree search (with a solution) | LOCALLY_SOLVED | 1 | FEASIBLE_POINT | FEASIBLE_POINT |
| Converged to an infeasible point | LOCALLY_INFEASIBLE | 1 | INFEASIBLE_POINT | * |
| Completed a non-global tree search (no solution found) | LOCALLY_INFEASIBLE | 0 | NO_SOLUTION | NO_SOLUTION |
| Iteration limit | ITERATION_LIMIT | 1 | * | * |
| Diverging iterates | NORM_LIMIT or OBJECTIVE_LIMIT | 1 | * | * |

## 13.6 Querying solution attributes

Some solvers will not implement every solution attribute. Therefore, a call like `MOI.get(model, MOI.SolveTimeSec())` may throw an `UnsupportedAttribute` or `GetAttributeNotAllowed` error.

If you need to write code that is agnostic to the solver (for example, you are writing a library that an end-user passes their choice of solver to), you can work-around this problem using a `try-catch`:

```
function get_solve_time(model)
    try
        return MOI.get(model, MOI.SolveTimeSec())
    catch err
        if err isa Union{MOI.UnsupportedAttribute,MOI.GetAttributeNotAllowed}
            return NaN  # Solver doesn't support. Return a placeholder value.
        end
        rethrow(err)  # Something else went wrong. Rethrow the error
    end
end
```

If, after careful profiling, you find that the `try-catch` is taking a significant portion of your runtime, you can improve performance by caching the result of the `try-catch`:

```
mutable struct CachedSolveTime{M}
    model::M
    supports_solve_time::Bool
    CachedSolveTime(model::M) where {M} = new(model, true)
```

```
end

function get_solve_time(model::CachedSolveTime)
    if !model.supports_solve_time
        return NaN
    end
    try
        return MOI.get(model, MOI.SolveTimeSec())
    catch err
        if Union{MOI.UnsupportedAttribute,MOI.GetAttributeNotAllowed}
            model.supports_solve_time = false
            return NaN
        end
        rethrow(err)  # Something else went wrong. Rethrow the error
    end
end
```

# Chapter 14

# Problem modification

In addition to adding and deleting constraints and variables, MathOptInterface supports modifying, in-place, coefficients in the constraints and the objective function of a model.

These modifications can be grouped into two categories:

- modifications which replace the set of function of a constraint with a new set or function
- modifications which change, in-place, a component of a function

**Warning**

Some `ModelLike` objects do not support problem modification.

## 14.1 Modify the set of a constraint

Use `set` and `ConstraintSet` to modify the set of a constraint by replacing it with a new instance of the same type.

```
julia> c = MOI.add_constraint(
           model,
           MOI.ScalarAffineFunction([MOI.ScalarAffineTerm(1.0, x)], 0.0),
           MOI.EqualTo(1.0),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64},
↪  MathOptInterface.EqualTo{Float64}}(1)

julia> MOI.set(model, MOI.ConstraintSet(), c, MOI.EqualTo(2.0));

julia> MOI.get(model, MOI.ConstraintSet(), c) == MOI.EqualTo(2.0)
true
```

However, the following will fail as the new set is of a different type to the original set:

```
julia> MOI.set(model, MOI.ConstraintSet(), c, MOI.GreaterThan(2.0))
ERROR: [...]
```

### Special cases: set transforms

If our constraint is an affine inequality, then this corresponds to modifying the right-hand side of a constraint in linear programming.

In some special cases, solvers may support efficiently changing the set of a constraint (for example, from `LessThan` to `GreaterThan`). For these cases, MathOptInterface provides the `transform` method.

The `transform` function returns a new constraint index, and the old constraint index (that is, `c`) is no longer valid.

```
julia> c = MOI.add_constraint(
           model,
           MOI.ScalarAffineFunction([MOI.ScalarAffineTerm(1.0, x)], 0.0),
           MOI.LessThan(1.0),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64},
↪  MathOptInterface.LessThan{Float64}}(1)

julia> new_c = MOI.transform(model, c, MOI.GreaterThan(2.0));

julia> MOI.is_valid(model, c)
false

julia> MOI.is_valid(model, new_c)
true
```

**Note**

`transform` cannot be called with a set of the same type. Use `set` instead.

## 14.2 Modify the function of a constraint

Use `set` and `ConstraintFunction` to modify the function of a constraint by replacing it with a new instance of the same type.

```
julia> c = MOI.add_constraint(
           model,
           MOI.ScalarAffineFunction([MOI.ScalarAffineTerm(1.0, x)], 0.0),
           MOI.EqualTo(1.0),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64},
↪  MathOptInterface.EqualTo{Float64}}(1)

julia> new_f = MOI.ScalarAffineFunction([MOI.ScalarAffineTerm(2.0, x)], 1.0);

julia> MOI.set(model, MOI.ConstraintFunction(), c, new_f);

julia> MOI.get(model, MOI.ConstraintFunction(), c) ≈ new_f
true
```

However, the following will fail as the new function is of a different type to the original function:

```
julia> MOI.set(model, MOI.ConstraintFunction(), c, x)
ERROR: [...]
```

## 14.3 Modify constant term in a scalar function

Use `modify` and `ScalarConstantChange` to modify the constant term in a `ScalarAffineFunction` or `ScalarQuadraticFunction`.

```
julia> c = MOI.add_constraint(
           model,
           MOI.ScalarAffineFunction([MOI.ScalarAffineTerm(1.0, x)], 0.0),
           MOI.EqualTo(1.0),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64},
↪  MathOptInterface.EqualTo{Float64}}(1)

julia> MOI.modify(model, c, MOI.ScalarConstantChange(1.0));

julia> new_f = MOI.ScalarAffineFunction([MOI.ScalarAffineTerm(1.0, x)], 1.0);

julia> MOI.get(model, MOI.ConstraintFunction(), c) ≈ new_f
true
```

`ScalarConstantChange` can also be used to modify the objective function by passing an instance of `ObjectiveFunction`:

```
julia> MOI.set(
           model,
           MOI.ObjectiveFunction{MOI.ScalarAffineFunction{Float64}}(),
           new_f,
       );

julia> MOI.modify(
           model,
           MOI.ObjectiveFunction{MOI.ScalarAffineFunction{Float64}}(),
           MOI.ScalarConstantChange(-1.0)
       );

julia> MOI.get(
           model,
           MOI.ObjectiveFunction{MOI.ScalarAffineFunction{Float64}}(),
       ) ≈ MOI.ScalarAffineFunction([MOI.ScalarAffineTerm(1.0, x)], -1.0)
true
```

## 14.4 Modify constant terms in a vector function

Use `modify` and `VectorConstantChange` to modify the constant vector in a `VectorAffineFunction` or `VectorQuadraticFunction`

```
julia> c = MOI.add_constraint(
           model,
           MOI.VectorAffineFunction([
                   MOI.VectorAffineTerm(1, MOI.ScalarAffineTerm(1.0, x)),
                   MOI.VectorAffineTerm(2, MOI.ScalarAffineTerm(2.0, x)),
```

```
            ],
            [0.0, 0.0],
        ),
        MOI.Nonnegatives(2),
    )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorAffineFunction{Float64},
↪  MathOptInterface.Nonnegatives}(1)

julia> MOI.modify(model, c, MOI.VectorConstantChange([3.0, 4.0]));

julia> new_f = MOI.VectorAffineFunction(
            [
         MOI.VectorAffineTerm(1, MOI.ScalarAffineTerm(1.0, x)),
         MOI.VectorAffineTerm(2, MOI.ScalarAffineTerm(2.0, x)),
            ],
            [3.0, 4.0],
        );

julia> MOI.get(model, MOI.ConstraintFunction(), c) ≈ new_f
true
```

## 14.5 Modify affine coefficients in a scalar function

Use `modify` and `ScalarCoefficientChange` to modify the affine coefficient of a `ScalarAffineFunction` or `ScalarQuadraticFunction`.

```
julia> c = MOI.add_constraint(
            model,
            MOI.ScalarAffineFunction([MOI.ScalarAffineTerm(1.0, x)], 0.0),
            MOI.EqualTo(1.0),
        )
MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64},
↪  MathOptInterface.EqualTo{Float64}}(1)

julia> MOI.modify(model, c, MOI.ScalarCoefficientChange(x, 2.0));

julia> new_f = MOI.ScalarAffineFunction([MOI.ScalarAffineTerm(2.0, x)], 0.0);

julia> MOI.get(model, MOI.ConstraintFunction(), c) ≈ new_f
true
```

`ScalarCoefficientChange` can also be used to modify the objective function by passing an instance of `ObjectiveFunction`.

## 14.6 Modify quadratic coefficients in a scalar function

Use `modify` and `ScalarQuadraticCoefficientChange` to modify the quadratic coefficient of a `ScalarQuadraticFunction`.

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variables(model, 2);

julia> c = MOI.add_constraint(
```

```
            model,
            1.0 * x[1] * x[1] + 2.0 * x[1] * x[2],
            MOI.EqualTo(1.0),
        )
MathOptInterface.ConstraintIndex{MathOptInterface.ScalarQuadraticFunction{Float64},
↪  MathOptInterface.EqualTo{Float64}}(1)

julia> MOI.modify(
            model,
            c,
            MOI.ScalarQuadraticCoefficientChange(x[1], x[1], 3.0),
        );

julia> MOI.modify(
            model,
            c,
            MOI.ScalarQuadraticCoefficientChange(x[1], x[2], 4.0),
        );

julia> new_f = 1.5 * x[1] * x[1] + 4.0 * x[1] * x[2];

julia> MOI.get(model, MOI.ConstraintFunction(), c) ≈ new_f
true
```

`ScalarQuadraticCoefficientChange` can also be used to modify the objective function by passing an instance of `ObjectiveFunction`.

## 14.7 Modify affine coefficients in a vector function

Use `modify` and `MultirowChange` to modify a vector of affine coefficients in a `VectorAffineFunction` or a `VectorQuadraticFunction`.

```
julia> c = MOI.add_constraint(
            model,
            MOI.VectorAffineFunction([
                    MOI.VectorAffineTerm(1, MOI.ScalarAffineTerm(1.0, x)),
                    MOI.VectorAffineTerm(2, MOI.ScalarAffineTerm(2.0, x)),
                ],
                [0.0, 0.0],
            ),
            MOI.Nonnegatives(2),
        )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorAffineFunction{Float64},
↪  MathOptInterface.Nonnegatives}(1)

julia> MOI.modify(model, c, MOI.MultirowChange(x, [(1, 3.0), (2, 4.0)]));

julia> new_f = MOI.VectorAffineFunction(
            [
         MOI.VectorAffineTerm(1, MOI.ScalarAffineTerm(3.0, x)),
         MOI.VectorAffineTerm(2, MOI.ScalarAffineTerm(4.0, x)),
            ],
            [0.0, 0.0],
        );
```

```
julia> MOI.get(model, MOI.ConstraintFunction(), c) ≈ new_f
true
```

# Part IV

# Background

# Chapter 15

# Duality

Conic duality is the starting point for MOI's duality conventions. When all functions are affine (or coordinate projections), and all constraint sets are closed convex cones, the model may be called a conic optimization problem.

For a minimization problem in geometric conic form, the primal is:

$$\min_{x \in \mathbb{R}^n} \quad a_0^T x + b_0 \tag{15.1}$$

$$\text{s.t.} \quad A_i x + b_i \in \mathcal{C}_i \qquad i = 1 \dots m \tag{15.2}$$

and the dual is a maximization problem in standard conic form:

$$\max_{y_1, \dots, y_m} \quad -\sum_{i=1}^{m} b_i^T y_i + b_0 \tag{15.3}$$

$$\text{s.t.} \quad a_0 - \sum_{i=1}^{m} A_i^T y_i = 0 \tag{15.4}$$

$$y_i \in \mathcal{C}_i^* \qquad i = 1 \dots m \tag{15.5}$$

where each $\mathcal{C}_i$ is a closed convex cone and $\mathcal{C}_i^*$ is its dual cone.

For a maximization problem in geometric conic form, the primal is:

$$\max_{x \in \mathbb{R}^n} \quad a_0^T x + b_0 \tag{15.6}$$

$$\text{s.t.} \quad A_i x + b_i \in \mathcal{C}_i \qquad i = 1 \dots m \tag{15.7}$$

and the dual is a minimization problem in standard conic form:

$$\min_{y_1, \dots, y_m} \quad \sum_{i=1}^{m} b_i^T y_i + b_0 \tag{15.8}$$

$$\text{s.t.} \quad a_0 + \sum_{i=1}^{m} A_i^T y_i = 0 \tag{15.9}$$

$$y_i \in \mathcal{C}_i^* \qquad i = 1 \dots m \tag{15.10}$$

A linear inequality constraint $a^T x + b \geq c$ is equivalent to $a^T x + b - c \in \mathbb{R}_+$, and $a^T x + b \leq c$ is equivalent to $a^T x + b - c \in \mathbb{R}_-$. Variable-wise constraints are affine constraints with the appropriate identity mapping in place of $A_i$.

For the special case of minimization LPs, the MOI primal form can be stated as:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & a_0^T x + b_0 && (15.11) \\ \text{s.t.} \quad & A_1 x \geq b_1 && (15.12) \\ & A_2 x \leq b_2 && (15.13) \\ & A_3 x = b_3 && (15.14) \end{aligned}$$

By applying the stated transformations to conic form, taking the dual, and transforming back into linear inequality form, one obtains the following dual:

$$\begin{aligned} \max_{y_1, y_2, y_3} \quad & b_1^T y_1 + b_2^T y_2 + b_3^T y_3 + b_0 && (15.15) \\ \text{s.t.} \quad & A_1^T y_1 + A_2^T y_2 + A_3^T y_3 = a_0 && (15.16) \\ & y_1 \geq 0 && (15.17) \\ & y_2 \leq 0 && (15.18) \end{aligned}$$

For maximization LPs, the MOI primal form can be stated as:

$$\begin{aligned} \max_{x \in \mathbb{R}^n} \quad & a_0^T x + b_0 && (15.19) \\ \text{s.t.} \quad & A_1 x \geq b_1 && (15.20) \\ & A_2 x \leq b_2 && (15.21) \\ & A_3 x = b_3 && (15.22) \end{aligned}$$

and similarly, the dual is:

$$\begin{aligned} \min_{y_1, y_2, y_3} \quad & -b_1^T y_1 - b_2^T y_2 - b_3^T y_3 + b_0 && (15.23) \\ \text{s.t.} \quad & A_1^T y_1 + A_2^T y_2 + A_3^T y_3 = -a_0 && (15.24) \\ & y_1 \geq 0 && (15.25) \\ & y_2 \leq 0 && (15.26) \end{aligned}$$

**Warning**

For the LP case, the signs of the feasible dual variables depend only on the sense of the corresponding primal inequality and not on the objective sense.

## 15.1 Duality and scalar product

The scalar product is different from the canonical one for the sets PositiveSemidefiniteConeTriangle, LogDetConeTriangle, RootDetConeTriangle.

If the set $C_i$ of the section Duality is one of these three cones, then the rows of the matrix $A_i$ corresponding to off-diagonal entries are twice the value of the `coefficients` field in the VectorAffineFunction for the corresponding rows. See PositiveSemidefiniteConeTriangle for details.

## 15.2 Dual for problems with quadratic functions

### Quadratic Programs (QPs)

For quadratic programs with only affine conic constraints,

$$\begin{array}{lll} \min\limits_{x\in\mathbb{R}^n} & \frac{1}{2}x^TQ_0x + a_0^Tx + b_0 & \\ \text{s.t.} & A_ix + b_i \in \mathcal{C}_i & i = 1\dots m. \end{array}$$

with cones $\mathcal{C}_i \subseteq \mathbb{R}^{m_i}$ for $i = 1, \dots, m$, consider the Lagrangian function

$$L(x, y) = \frac{1}{2}x^TQ_0x + a_0^Tx + b_0 - \sum_{i=1}^{m} y_i^T(A_ix + b_i).$$

Let $z(y)$ denote $\sum_{i=1}^{m} A_i^Ty_i - a_0$, the Lagrangian can be rewritten as

$$L(x, y) = \frac{1}{2}x^TQ_0x - z(y)^Tx + b_0 - \sum_{i=1}^{m} y_i^Tb_i.$$

The condition $\nabla_x L(x, y) = 0$ gives

$$0 = \nabla_x L(x, y) = Q_0x + a_0 - \sum_{i=1}^{m} y_i^Tb_i$$

which gives $Q_0x = z(y)$. This allows to obtain that

$$\min_{x\in\mathbb{R}^n} L(x, y) = -\frac{1}{2}x^TQ_0x + b_0 - \sum_{i=1}^{m} y_i^Tb_i$$

so the dual problem is

$$\max_{y_i\in\mathcal{C}_i^*} \min_{x\in\mathbb{R}^n} -\frac{1}{2}x^TQ_0x + b_0 - \sum_{i=1}^{m} y_i^Tb_i.$$

If $Q_0$ is invertible, we have $x = Q_0^{-1} z(y)$ hence

$$\min_{x \in \mathbb{R}^n} L(x, y) = -\frac{1}{2} z(y)^T Q_0^{-1} z(y) + b_0 - \sum_{i=1}^m y_i^T b_i$$

so the dual problem is

$$\max_{y_i \in \mathcal{C}_i^*} -\frac{1}{2} z(y)^T Q_0^{-1} z(y) + b_0 - \sum_{i=1}^m y_i^T b_i.$$

### Quadratically Constrained Quadratic Programs (QCQPs)

Given a problem with both quadratic function and quadratic objectives:

$$\begin{array}{lll} \min\limits_{x \in \mathbb{R}^n} & \frac{1}{2} x^T Q_0 x + a_0^T x + b_0 & \\ \text{s.t.} & \frac{1}{2} x^T Q_i x + a_i^T x + b_i \in \mathcal{C}_i & i = 1 \dots m. \end{array}$$

with cones $\mathcal{C}_i \subseteq \mathbb{R}$ for $i = 1 \dots m$, consider the Lagrangian function

$$L(x, y) = \frac{1}{2} x^T Q_0 x + a_0^T x + b_0 - \sum_{i=1}^m y_i (\frac{1}{2} x^T Q_i x + a_i^T x + b_i)$$

A pair of primal-dual variables $(x^\star, y^\star)$ is optimal if

- $x^\star$ is a minimizer of

  $$\min_{x \in \mathbb{R}^n} L(x, y^\star).$$

  That is,

  $$0 = \nabla_x L(x, y^\star) = Q_0 x + a_0 - \sum_{i=1}^m y_i^\star (Q_i x + a_i).$$

- and $y^\star$ is a maximizer of

  $$\max_{y_i \in \mathcal{C}_i^*} L(x^\star, y).$$

  That is, for all $i = 1, \dots, m$, $\frac{1}{2} x^T Q_i x + a_i^T x + b_i$ is either zero or in the normal cone of $\mathcal{C}_i^*$ at $y^\star$. For instance, if $\mathcal{C}_i$ is $\{z \in \mathbb{R} : z \leq 0\}$, this means that if $\frac{1}{2} x^T Q_i x + a_i^T x + b_i$ is nonzero at $x^\star$ then $y_i^\star = 0$. This is the classical complementary slackness condition.

If $\mathcal{C}_i$ is a vector set, the discussion remains valid with $y_i (\frac{1}{2} x^T Q_i x + a_i^T x + b_i)$ replaced with the scalar product between $y_i$ and the vector of scalar-valued quadratic functions.

## 15.3 Dual for square semidefinite matrices

The set PositiveSemidefiniteConeTriangle is a self-dual. That is, querying ConstraintDual of a PositiveSemidefiniteConeT constraint returns a vector that is itself a member of PositiveSemidefiniteConeTriangle.

However, the dual of PositiveSemidefiniteConeSquare is not so straight forward. This section explains the duality convention we use, and how it is derived.

**Info**

If you have a PositiveSemidefiniteConeSquare constraint, the result matrix $A$ from ConstraintDual is not positive semidefinite. However, $A + A^\top$ is positive semidefinite.

Let $\mathcal{S}_+$ be the cone of symmetric semidefinite matrices in the $\frac{n(n+1)}{2}$ dimensional space of symmetric $\mathbb{R}^{n\times n}$ matrices. That is, $\mathcal{S}_+$ is the set PositiveSemidefiniteConeTriangle. It is well known that $\mathcal{S}_+$ is a self-dual proper cone.

Let $\mathcal{P}_+$ be the cone of symmetric semidefinite matrices in the $n^2$ dimensional space of $\mathbb{R}^{n\times n}$ matrices. That is $\mathcal{P}_+$ is the set PositiveSemidefiniteConeSquare.

In addition, let $\mathcal{D}_+$ be the cone of matrices $A$ such that $A + A^\top \in \mathcal{P}_+$.

$\mathcal{P}_+$ is not proper because it is not solid (it is not $n^2$ dimensional), so it is not necessarily true that $\mathcal{P}_+^{**} = \mathcal{P}_+$.

However, this is the case, because we will show that $\mathcal{P}_+^* = \mathcal{D}_+$ and $\mathcal{D}_+^* = \mathcal{P}_+$.

First, let us see why $\mathcal{P}_+^* = \mathcal{D}_+$.

If $B$ is symmetric, then

$$\langle A, B\rangle = \langle A^\top, B^\top\rangle = \langle A^\top, B\rangle$$

so

$$2\langle A, B\rangle = \langle A, B\rangle + \langle A^\top, B\rangle = \langle A + A^\top, B\rangle.$$

Therefore, $\langle A, B\rangle \geq 0$ for all $B \in \mathcal{P}_+$ if and only if $\langle A + A^\top, B\rangle \geq 0$ for all $B \in \mathcal{P}_+$. Since $A + A^\top$ is symmetric, and we know that $\mathcal{S}_+$ is self-dual, we have shown that $\mathcal{P}_+^*$ is the set of matrices $A$ such that $A + A^\top \in \mathcal{P}_+$.

Second, let us see why $\mathcal{D}_+^* = \mathcal{P}_+$.

Since $A \in \mathcal{D}_+$ implies that $A^\top \in \mathcal{D}_+$, $B \in \mathcal{D}_+^*$ means that $\langle A + A^\top, B\rangle \geq 0$ for all $A \in \mathcal{D}_+$, and hence $B \in \mathcal{P}_+$.

To see why it should be symmetric, simply notice that if $B_{i,j} < B_{j,i}$, then $\langle A, B\rangle$ can be made arbitrarily small by setting $A_{i,j} = A_{i,j} + s$ and $A_{j,i} = A_{j,i} - s$, with $s$ arbitrarily large, and $A$ stays in $\mathcal{D}_+$ because $A + A^\top$ does not change.

Typically, the primal/dual pair for semidefinite programs is presented as:

$$\min\langle C, X\rangle \tag{15.27}$$
$$\text{s.t. } \langle A_k, X\rangle = b_k \forall k \tag{15.28}$$
$$X \in \mathcal{S}_+ \tag{15.29}$$

with the dual

$$\max \sum_k b_k y_k \tag{15.30}$$
$$\text{s.t. } C - \sum A_k y_k \in \mathcal{S}_+ \tag{15.31}$$

If we allow $A_k$ to be non-symmetric, we should instead use:

$$\min \langle C, X \rangle \tag{15.32}$$
$$\text{s.t. } \langle A_k, X \rangle = b_k \forall k \tag{15.33}$$
$$X \in \mathcal{D}_+ \tag{15.34}$$

with the dual

$$\max \sum b_k y_k \tag{15.35}$$
$$\text{s.t. } C - \sum A_k y_k \in \mathcal{P}_+ \tag{15.36}$$

This is implemented as:

$$\min \langle C, Z \rangle + \langle C - C^\top, S \rangle \tag{15.37}$$
$$\text{s.t. } \langle A_k, Z \rangle + \langle A_k - A_k^\top, S \rangle = b_k \forall k \tag{15.38}$$
$$Z \in \mathcal{S}_+ \tag{15.39}$$

with the dual

$$\max \sum b_k y_k \tag{15.40}$$
$$\text{s.t. } C + C^\top - \sum (A_k + A_k^\top) y_k \in \mathcal{S}_+ \tag{15.41}$$
$$C - C^\top - \sum (A_k - A_k^\top) y_k = 0 \tag{15.42}$$

and we recover $Z = X + X^\top$.

# Chapter 16

# Infeasibility certificates

When given a conic problem that is infeasible or unbounded, some solvers can produce a certificate of infeasibility. This page explains what a certificate of infeasibility is, and the related conventions that MathOptInterface adopts.

## 16.1 Conic duality

MathOptInterface uses conic duality to define infeasibility certificates. A full explanation is given in the section Duality, but here is a brief overview.

### Minimization problems

For a minimization problem in geometric conic form, the primal is:

$$\min_{x \in \mathbb{R}^n} \quad a_0^\top x + b_0 \tag{16.1}$$

$$\text{s.t.} \quad A_i x + b_i \in \mathcal{C}_i \qquad i = 1 \dots m, \tag{16.2}$$

and the dual is a maximization problem in standard conic form:

$$\max_{y_1, \dots, y_m} \quad -\sum_{i=1}^{m} b_i^\top y_i + b_0 \tag{16.3}$$

$$\text{s.t.} \quad a_0 - \sum_{i=1}^{m} A_i^\top y_i = 0 \tag{16.4}$$

$$y_i \in \mathcal{C}_i^* \qquad i = 1 \dots m, \tag{16.5}$$

where each $\mathcal{C}_i$ is a closed convex cone and $\mathcal{C}_i^*$ is its dual cone.

### Maximization problems

For a maximization problem in geometric conic form, the primal is:

$$\max_{x \in \mathbb{R}^n} \quad a_0^\top x + b_0 \tag{16.6}$$

$$\text{s.t.} \quad A_i x + b_i \in \mathcal{C}_i \qquad i = 1 \dots m, \tag{16.7}$$

and the dual is a minimization problem in standard conic form:

$$\min_{y_1,\ldots,y_m} \quad \sum_{i=1}^{m} b_i^\top y_i + b_0 \tag{16.8}$$

$$\text{s.t.} \quad a_0 + \sum_{i=1}^{m} A_i^\top y_i = 0 \tag{16.9}$$

$$y_i \in \mathcal{C}_i^* \qquad i = 1 \ldots m. \tag{16.10}$$

## 16.2 Unbounded problems

A problem is unbounded if and only if:

1. there exists a feasible primal solution
2. the dual is infeasible.

A feasible primal solution—if one exists—can be obtained by setting `ObjectiveSense` to `FEASIBILITY_SENSE` before optimizing. Therefore, most solvers stop after they prove the dual is infeasible via a certificate of dual infeasibility, but before they have found a feasible primal solution. This is also the reason that MathOptInterface defines the `DUAL_INFEASIBLE` status instead of `UNBOUNDED`.

A certificate of dual infeasibility is an improving ray of the primal problem. That is, there exists some vector $d$ such that for all $\eta > 0$:

$$A_i(x + \eta d) + b_i \in \mathcal{C}_i, \;\; i = 1 \ldots m,$$

and (for minimization problems):

$$a_0^\top (x + \eta d) + b_0 < a_0^\top x + b_0,$$

for any feasible point $x$. The latter simplifies to $a_0^\top d < 0$. For maximization problems, the inequality is reversed, so that $a_0^\top d > 0$.

If the solver has found a certificate of dual infeasibility:

- `TerminationStatus` must be `DUAL_INFEASIBLE`
- `PrimalStatus` must be `INFEASIBILITY_CERTIFICATE`
- `VariablePrimal` must be the corresponding value of $d$
- `ConstraintPrimal` must be the corresponding value of $A_i d$
- `ObjectiveValue` must be the value $a_0^\top d$. Note that this is the value of the objective function at `d`, ignoring the constant `b_0`.

**Note**

The choice of whether to scale the ray $d$ to have magnitude 1 is left to the solver.

## 16.3 Infeasible problems

A certificate of primal infeasibility is an improving ray of the dual problem. However, because infeasibility is independent of the objective function, we first homogenize the primal problem by removing its objective.

For a minimization problem, a dual improving ray is some vector $d$ such that for all $\eta > 0$:

$$-\sum_{i=1}^{m} A_i^\top (y_i + \eta d_i) = 0 \tag{16.11}$$

$$(y_i + \eta d_i) \in \mathcal{C}_i^* \qquad i = 1 \dots m, \tag{16.12}$$

and:

$$-\sum_{i=1}^{m} b_i^\top (y_i + \eta d_i) > -\sum_{i=1}^{m} b_i^\top y_i,$$

for any feasible dual solution $y$. The latter simplifies to $-\sum_{i=1}^{m} b_i^\top d_i > 0$. For a maximization problem, the inequality is $\sum_{i=1}^{m} b_i^\top d_i < 0$. (Note that these are the same inequality, modulo a `-` sign.)

If the solver has found a certificate of primal infeasibility:

- `TerminationStatus` must be `INFEASIBLE`
- `DualStatus` must be `INFEASIBILITY_CERTIFICATE`
- `ConstraintDual` must be the corresponding value of $d$
- `DualObjectiveValue` must be the value $-\sum_{i=1}^{m} b_i^\top d_i$ for minimization problems and $\sum_{i=1}^{m} b_i^\top d_i$ for maximization problems.

**Note**

The choice of whether to scale the ray $d$ to have magnitude 1 is left to the solver.

### Infeasibility certificates of variable bounds

Many linear solvers (for example, Gurobi) do not provide explicit access to the primal infeasibility certificate of a variable bound. However, given a set of linear constraints:

$$l_A \leq Ax \leq u_A \tag{16.13}$$

$$l_x \leq x \leq u_x, \tag{16.14}$$

the primal certificate of the variable bounds can be computed using the primal certificate associated with the affine constraints, $d$. (Note that $d$ will have one element for each row of the $A$ matrix, and that some or all of the elements in the vectors $l_A$ and $u_A$ may be $\pm\infty$. If both $l_A$ and $u_A$ are finite for some row, the corresponding element in $d$ must be `0`.)

Given $d$, compute $\bar{d} = d^\top A$. If the bound is finite, a certificate for the lower variable bound of $x_i$ is $\max\{\bar{d}_i, 0\}$, and a certificate for the upper variable bound is $\min\{\bar{d}_i, 0\}$.

# Chapter 17

# Naming conventions

MOI follows several conventions for naming functions and structures. These should also be followed by packages extending MOI.

## 17.1 Sets

Sets encode the structure of constraints. Their names should follow the following conventions:

- Abstract types in the set hierarchy should begin with `Abstract` and end in `Set`, for example, `AbstractScalarSet`, `AbstractVectorSet`.
- Vector-valued conic sets should end with `Cone`, for example, `NormInfinityCone`, `SecondOrderCone`.
- Vector-valued Cartesian products should be plural and not end in `Cone`, for example, `Nonnegatives`, not `NonnegativeCone`.
- Matrix-valued conic sets should provide two representations: `ConeSquare` and `ConeTriangle`, for example, `RootDetConeTriangle` and `RootDetConeSquare`. See Matrix cones for more details.
- Scalar sets should be singular, not plural, for example, `Integer`, not `Integers`.
- As much as possible, the names should follow established conventions in the domain where this set is used: for instance, convex sets should have names close to those of CVX, and constraint-programming sets should follow MiniZinc's constraints.

# Part V

# API Reference

# Chapter 18

# Standard form

## 18.1 Functions

`MathOptInterface.AbstractFunction` – Type.

```
AbstractFunction
```

Abstract supertype for function objects.

**Required methods**

All functions must implement:

- `Base.copy`
- `Base.isapprox`
- `constant`

Abstract subtypes of `AbstractFunction` may require additional methods to be implemented.

source

`MathOptInterface.output_dimension` – Function.

```
output_dimension(f::AbstractFunction)
```

Return 1 if `f` is an `AbstractScalarFunction`, or the number of output components if `f` is an `AbstractVectorFunction`.

source

`MathOptInterface.constant` – Function.

```
constant(f::AbstractFunction[, ::Type{T}]) where {T}
```

Returns the constant term of a scalar-valued function, or the constant vector of a vector-valued function.

If `f` is untyped and `T` is provided, returns `zero(T)`.

source

```
constant(set::Union{EqualTo,GreaterThan,LessThan,Parameter})
```

Returns the constant term of the set `set`.

**Example**

```
julia> MOI.constant(MOI.GreaterThan(1.0))
1.0

julia> MOI.constant(MOI.LessThan(2.5))
2.5

julia> MOI.constant(MOI.EqualTo(3))
3

julia> MOI.constant(MOI.Parameter(4.5))
4.5
```

source

## 18.2 Scalar functions

`MathOptInterface.AbstractScalarFunction` – Type.

```
abstract type AbstractScalarFunction <: AbstractFunction
```

Abstract supertype for scalar-valued `AbstractFunction`s.

source

`MathOptInterface.VariableIndex` – Type.

```
VariableIndex
```

A type-safe wrapper for `Int64` for use in referencing variables in a model. To allow for deletion, indices need not be consecutive.

source

`MathOptInterface.ScalarAffineTerm` – Type.

```
ScalarAffineTerm{T}(coefficient::T, variable::VariableIndex) where {T}
```

Represents the scalar-valued term `coefficient * variable`.

**Example**

```
julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> MOI.ScalarAffineTerm(2.0, x)
MathOptInterface.ScalarAffineTerm{Float64}(2.0, MOI.VariableIndex(1))
```

source

MathOptInterface.ScalarAffineFunction – Type.

```
ScalarAffineFunction{T}(
    terms::Vector{ScalarAffineTerm{T}},
    constant::T,
) where {T}
```

Represents the scalar-valued affine function $a^\top x + b$, where:

- $a^\top x$ is represented by the vector of ScalarAffineTerms
- $b$ is a scalar constant::T

**Duplicates**

Duplicate variable indices in terms are accepted, and the corresponding coefficients are summed together.

**Example**

```
julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> terms = [MOI.ScalarAffineTerm(2.0, x), MOI.ScalarAffineTerm(3.0, x)]
2-element Vector{MathOptInterface.ScalarAffineTerm{Float64}}:
 MathOptInterface.ScalarAffineTerm{Float64}(2.0, MOI.VariableIndex(1))
 MathOptInterface.ScalarAffineTerm{Float64}(3.0, MOI.VariableIndex(1))

julia> f = MOI.ScalarAffineFunction(terms, 4.0)
4.0 + 2.0 MOI.VariableIndex(1) + 3.0 MOI.VariableIndex(1)
```

source

MathOptInterface.ScalarQuadraticTerm – Type.

```
ScalarQuadraticTerm{T}(
    coefficient::T,
    variable_1::VariableIndex,
    variable_2::VariableIndex,
) where {T}
```

Represents the scalar-valued term $cx_ix_j$ where $c$ is coefficient, $x_i$ is variable_1 and $x_j$ is variable_2.

**Example**

```
julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> MOI.ScalarQuadraticTerm(2.0, x, x)
MathOptInterface.ScalarQuadraticTerm{Float64}(2.0, MOI.VariableIndex(1), MOI.VariableIndex(1))
```

source

`MathOptInterface.ScalarQuadraticFunction` – Type.

```
ScalarQuadraticFunction{T}(
    quadratic_terms::Vector{ScalarQuadraticTerm{T}},
    affine_terms::Vector{ScalarAffineTerm{T}},
    constant::T,
) where {T}
```

The scalar-valued quadratic function $\frac{1}{2}x^\top Q x + a^\top x + b$, where:

- $Q$ is the symmetric matrix given by the vector of `ScalarQuadraticTerm`s
- $a^\top x$ is a sparse vector given by the vector of `ScalarAffineTerm`s
- $b$ is the scalar `constant::T`.

**Duplicates**

Duplicate indices in `quadratic_terms` or `affine_terms` are accepted, and the corresponding coefficients are summed together.

In `quadratic_terms`, "mirrored" indices, `(q, r)` and `(r, q)` where `r` and `q` are `VariableIndex`es, are considered duplicates; only one needs to be specified.

**The 0.5 factor**

Coupled with the interpretation of mirrored indices, the `0.5` factor in front of the $Q$ matrix is a common source of bugs.

As a rule, to represent $a * x^2 + b * x * y$:

- The coefficient $a$ in front of squared variables (diagonal elements in $Q$) must be doubled when creating a `ScalarQuadraticTerm`
- The coefficient $b$ in front of off-diagonal elements in $Q$ should be left as $b$, be cause the mirrored index $b * y * x$ will be implicitly added.

**Example**

To represent the function $f(x, y) = 2 * x^2 + 3 * x * y + 4 * x + 5$, do:

```
julia> x = MOI.VariableIndex(1);

julia> y = MOI.VariableIndex(2);

julia> constant = 5.0;
```

```
julia> affine_terms = [MOI.ScalarAffineTerm(4.0, x)];

julia> quadratic_terms = [
           MOI.ScalarQuadraticTerm(4.0, x, x),  # Note the changed coefficient
           MOI.ScalarQuadraticTerm(3.0, x, y),
       ]
2-element Vector{MathOptInterface.ScalarQuadraticTerm{Float64}}:
 MathOptInterface.ScalarQuadraticTerm{Float64}(4.0, MOI.VariableIndex(1), MOI.VariableIndex(1))
 MathOptInterface.ScalarQuadraticTerm{Float64}(3.0, MOI.VariableIndex(1), MOI.VariableIndex(2))

julia> f = MOI.ScalarQuadraticFunction(quadratic_terms, affine_terms, constant)
5.0 + 4.0 MOI.VariableIndex(1) + 2.0 MOI.VariableIndex(1)² + 3.0
↪  MOI.VariableIndex(1)*MOI.VariableIndex(2)
```

source

`MathOptInterface.ScalarNonlinearFunction` – Type.

```
ScalarNonlinearFunction(head::Symbol, args::Vector{Any})
```

The scalar-valued nonlinear function `head(args...)`, represented as a symbolic expression tree, with the call operator `head` and ordered arguments in `args`.

**head**

The `head::Symbol` must be an operator supported by the model.

The default list of supported univariate operators is given by:

- `Nonlinear.DEFAULT_UNIVARIATE_OPERATORS`

and the default list of supported multivariate operators is given by:

- `Nonlinear.DEFAULT_MULTIVARIATE_OPERATORS`

Additional operators can be registered by setting a `UserDefinedFunction` attribute.

See the full list of operators supported by a `ModelLike` by querying `ListOfSupportedNonlinearOperators`.

**args**

The vector `args` contains the arguments to the nonlinear function. If the operator is univariate, it must contain one element. Otherwise, it may contain multiple elements.

Each element must be one of the following:

- A constant value of type `T<:Real`
- A `VariableIndex`
- A `ScalarAffineFunction`
- A `ScalarQuadraticFunction`
- A `ScalarNonlinearFunction`

**Unsupported operators**

If the optimizer does not support head, an `UnsupportedNonlinearOperator` error will be thrown.

There is no guarantee about when this error will be thrown; it may be thrown when the function is first added to the model, or it may be thrown when `optimize!` is called.

**Example**

To represent the function $f(x) = sin(x)^2$, do:

```
julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> MOI.ScalarNonlinearFunction(
           :^,
           Any[MOI.ScalarNonlinearFunction(:sin, Any[x]), 2],
       )
^(sin(MOI.VariableIndex(1)), (2))
```

source

## 18.3 Vector functions

`MathOptInterface.AbstractVectorFunction` – Type.

```
abstract type AbstractVectorFunction <: AbstractFunction
```

Abstract supertype for vector-valued `AbstractFunction`s.

**Required methods**

All subtypes of `AbstractVectorFunction` must implement:

- `output_dimension`

source

`MathOptInterface.VectorOfVariables` – Type.

```
VectorOfVariables(variables::Vector{VariableIndex}) <: AbstractVectorFunction
```

The vector-valued function `f(x) = variables`, where `variables` is a subset of `VariableIndex`es in the model.

The list of `variables` may contain duplicates.

**Example**

```
julia> x = MOI.VariableIndex.(1:2)
2-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(1)
```

```
 MOI.VariableIndex(2)

julia> f = MOI.VectorOfVariables([x[1], x[2], x[1]])
┌                   ┐
│MOI.VariableIndex(1)│
│MOI.VariableIndex(2)│
│MOI.VariableIndex(1)│
└                   ┘

julia> MOI.output_dimension(f)
3
```

source

MathOptInterface.VectorAffineTerm – Type.

```
VectorAffineTerm{T}(
    output_index::Int64,
    scalar_term::ScalarAffineTerm{T},
) where {T}
```

A `VectorAffineTerm` is a `scalar_term` that appears in the `output_index` row of the vector-valued `VectorAffineFunction` or `VectorQuadraticFunction`.

**Example**

```
julia> x = MOI.VariableIndex(1);

julia> MOI.VectorAffineTerm(Int64(2), MOI.ScalarAffineTerm(3.0, x))
MathOptInterface.VectorAffineTerm{Float64}(2, MathOptInterface.ScalarAffineTerm{Float64}(3.0,
↪  MOI.VariableIndex(1)))
```

source

MathOptInterface.VectorAffineFunction – Type.

```
VectorAffineFunction{T}(
    terms::Vector{VectorAffineTerm{T}},
    constants::Vector{T},
) where {T}
```

The vector-valued affine function $Ax + b$, where:

- $Ax$ is the sparse matrix given by the vector of `VectorAffineTerm`s
- $b$ is the vector `constants`

**Duplicates**

Duplicate indices in the $A$ are accepted, and the corresponding coefficients are summed together.

**Example**

```
julia> x = MOI.VariableIndex(1);

julia> terms = [
           MOI.VectorAffineTerm(Int64(1), MOI.ScalarAffineTerm(2.0, x)),
           MOI.VectorAffineTerm(Int64(2), MOI.ScalarAffineTerm(3.0, x)),
       ];

julia> f = MOI.VectorAffineFunction(terms, [4.0, 5.0])
┌                             ┐
│4.0 + 2.0 MOI.VariableIndex(1)│
│5.0 + 3.0 MOI.VariableIndex(1)│
└                             ┘

julia> MOI.output_dimension(f)
2
```

source

MathOptInterface.VectorQuadraticTerm – Type.

```
VectorQuadraticTerm{T}(
    output_index::Int64,
    scalar_term::ScalarQuadraticTerm{T},
) where {T}
```

A VectorQuadraticTerm is a ScalarQuadraticTerm scalar_term that appears in the output_index row of the vector-valued VectorQuadraticFunction.

**Example**

```
julia> x = MOI.VariableIndex(1);

julia> MOI.VectorQuadraticTerm(Int64(2), MOI.ScalarQuadraticTerm(3.0, x, x))
MathOptInterface.VectorQuadraticTerm{Float64}(2,
↪  MathOptInterface.ScalarQuadraticTerm{Float64}(3.0, MOI.VariableIndex(1),
↪  MOI.VariableIndex(1)))
```

source

MathOptInterface.VectorQuadraticFunction – Type.

```
VectorQuadraticFunction{T}(
    quadratic_terms::Vector{VectorQuadraticTerm{T}},
    affine_terms::Vector{VectorAffineTerm{T}},
    constants::Vector{T},
) where {T}
```

The vector-valued quadratic function with ith component ("output index") defined as $\frac{1}{2}x^\top Q_i x + a_i^\top x + b_i$, where:

- $\frac{1}{2}x^\top Q_i x$ is the symmetric matrix given by the `VectorQuadraticTerm` elements in `quadratic_terms` with `output_index == i`
- $a_i^\top x$ is the sparse vector given by the `VectorAffineTerm` elements in `affine_terms` with `output_index == i`
- $b_i$ is a scalar given by `constants[i]`

**Duplicates**

Duplicate indices in `quadratic_terms` and `affine_terms` with the same `output_index` are handled in the same manner as duplicates in `ScalarQuadraticFunction`.

**Example**

```
julia> x = MOI.VariableIndex(1);

julia> y = MOI.VariableIndex(2);

julia> constants = [4.0, 5.0];

julia> affine_terms = [
           MOI.VectorAffineTerm(Int64(1), MOI.ScalarAffineTerm(2.0, x)),
           MOI.VectorAffineTerm(Int64(2), MOI.ScalarAffineTerm(3.0, x)),
       ];

julia> quad_terms = [
        MOI.VectorQuadraticTerm(Int64(1), MOI.ScalarQuadraticTerm(2.0, x, x)),
        MOI.VectorQuadraticTerm(Int64(2), MOI.ScalarQuadraticTerm(3.0, x, y)),
           ];

julia> f = MOI.VectorQuadraticFunction(quad_terms, affine_terms, constants)
┌                                                                          ┐
│4.0 + 2.0 MOI.VariableIndex(1) + 1.0 MOI.VariableIndex(1)²                │
│5.0 + 3.0 MOI.VariableIndex(1) + 3.0 MOI.VariableIndex(1)*MOI.VariableIndex(2)│
└                                                                          ┘

julia> MOI.output_dimension(f)
2
```

source

`MathOptInterface.VectorNonlinearFunction` – Type.

```
VectorNonlinearFunction(args::Vector{ScalarNonlinearFunction})
```

The vector-valued nonlinear function composed of a vector of `ScalarNonlinearFunction`.

**args**

The vector `args` contains the scalar elements of the nonlinear function. Each element must be a `ScalarNonlinearFunction`, but if you pass a `Vector{Any}`, the elements can be automatically converted from one of the following:

- A constant value of type `T<:Real`
- A `VariableIndex`

- A ScalarAffineFunction
- A ScalarQuadraticFunction
- A ScalarNonlinearFunction

**Example**

To represent the function $f(x) = [sin(x)^2, x]$, do:

```
julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> g = MOI.ScalarNonlinearFunction(
           :^,
           Any[MOI.ScalarNonlinearFunction(:sin, Any[x]), 2.0],
       )
^(sin(MOI.VariableIndex(1)), 2.0)

julia> MOI.VectorNonlinearFunction([g, x])
┌                                 ┐
│^(sin(MOI.VariableIndex(1)), 2.0)│
│+(MOI.VariableIndex(1))          │
└                                 ┘
```

Note the automatic conversion from `x` to `+(x)`.

source

## 18.4 Sets

`MathOptInterface.AbstractSet` – Type.

```
AbstractSet
```

Abstract supertype for set objects used to encode constraints.

**Required methods**

For sets of type `S` with `isbitstype(S) == false`, you must implement:

- `Base.copy(set::S)`
- `Base.:(==)(x::S, y::S)`

Subtypes of `AbstractSet` such as AbstractScalarSet and AbstractVectorSet may prescribe additional required methods.

**Optional methods**

You may optionally implement:

- dual_set
- dual_set_type

**Note for developers**

When creating a new set, the set struct must not contain any `VariableIndex` or `ConstraintIndex` objects.

source

`MathOptInterface.AbstractScalarSet` – Type.

```
AbstractScalarSet
```

Abstract supertype for subsets of $\mathbb{R}$.

source

`MathOptInterface.AbstractVectorSet` – Type.

```
AbstractVectorSet
```

Abstract supertype for subsets of $\mathbb{R}^n$ for some $n$.

**Required methods**

All `AbstractVectorSet`s of type `S` must implement:

- `dimension`, unless the dimension is stored in the `set.dimension` field
- `Utilities.set_dot`, unless the dot product between two vectors in the set is equivalent to `LinearAlgebra.dot`.

source

### Utilities

`MathOptInterface.dimension` – Function.

```
dimension(set::AbstractSet)
```

Return the `output_dimension` that an `AbstractFunction` should have to be used with the set `set`.

**Example**

```
julia> MOI.dimension(MOI.Reals(4))
4

julia> MOI.dimension(MOI.LessThan(3.0))
1

julia> MOI.dimension(MOI.PositiveSemidefiniteConeTriangle(2))
3
```

source

`MathOptInterface.dual_set` – Function.

```
dual_set(set::AbstractSet)
```

Return the dual set of `set`, that is the dual cone of the set. This follows the definition of duality discussed in Duality.

See Dual cone for more information.

If the dual cone is not defined it returns an error.

**Example**

```
julia> MOI.dual_set(MOI.Reals(4))
MathOptInterface.Zeros(4)

julia> MOI.dual_set(MOI.SecondOrderCone(5))
MathOptInterface.SecondOrderCone(5)

julia> MOI.dual_set(MOI.ExponentialCone())
MathOptInterface.DualExponentialCone()
```

source

`MathOptInterface.dual_set_type` – Function.

```
dual_set_type(S::Type{<:AbstractSet})
```

Return the type of dual set of sets of type `S`, as returned by `dual_set`. If the dual cone is not defined it returns an error.

**Example**

```
julia> MOI.dual_set_type(MOI.Reals)
MathOptInterface.Zeros

julia> MOI.dual_set_type(MOI.SecondOrderCone)
MathOptInterface.SecondOrderCone

julia> MOI.dual_set_type(MOI.ExponentialCone)
MathOptInterface.DualExponentialCone
```

source

`MathOptInterface.constant` – Method.

```
constant(set::Union{EqualTo,GreaterThan,LessThan,Parameter})
```

Returns the constant term of the set `set`.

**Example**

```
julia> MOI.constant(MOI.GreaterThan(1.0))
1.0

julia> MOI.constant(MOI.LessThan(2.5))
2.5

julia> MOI.constant(MOI.EqualTo(3))
3

julia> MOI.constant(MOI.Parameter(4.5))
4.5
```

source

MathOptInterface.supports_dimension_update – Function.

```
supports_dimension_update(S::Type{<:MOI.AbstractVectorSet})
```

Return a Bool indicating whether the elimination of any dimension of n-dimensional sets of type S give an n-1-dimensional set S. By default, this function returns false so it should only be implemented for sets that supports dimension update.

For instance, supports_dimension_update(MOI.Nonnegatives) is true because the elimination of any dimension of the n-dimensional nonnegative orthant gives the n-1-dimensional nonnegative orthant. However supports_dimension_update(MOI.ExponentialCone) is false.

source

MathOptInterface.update_dimension – Function.

```
update_dimension(s::AbstractVectorSet, new_dim::Int)
```

Returns a set with the dimension modified to new_dim.

source

## 18.5 Scalar sets

List of recognized scalar sets.

MathOptInterface.GreaterThan – Type.

```
GreaterThan{T<:Real}(lower::T)
```

The set $[lower, \infty) \subseteq \mathbb{R}$.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> MOI.add_constraint(model, x, MOI.GreaterThan(0.0))
MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.GreaterThan{Float64}}(1)
```

source

MathOptInterface.LessThan – Type.

```
LessThan{T<:Real}(upper::T)
```

The set $(-\infty, upper] \subseteq \mathbb{R}$.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> MOI.add_constraint(model, x, MOI.LessThan(2.0))
MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.LessThan{Float64}}(1)
```

source

MathOptInterface.EqualTo – Type.

```
EqualTo{T<:Number}(value::T)
```

The set containing the single point $\{value\} \subseteq \mathbb{R}$.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> MOI.add_constraint(model, x, MOI.EqualTo(2.0))
MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.EqualTo{Float64}}(1)
```

source

MathOptInterface.Interval – Type.

```
Interval{T<:Real}(lower::T, upper::T)
```

The interval $[lower, upper] \subseteq \mathbb{R} \cup \{-\infty, +\infty\}$.

If `lower` or `upper` is `-Inf` or `Inf`, respectively, the set is interpreted as a one-sided interval.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> MOI.add_constraint(model, x, MOI.Interval(1.0, 2.0))
MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Interval{Float64}}(1)
```

source

`MathOptInterface.Integer` – Type.

```
Integer()
```

The set of integers, $\mathbb{Z}$.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> MOI.add_constraint(model, x, MOI.Integer())
MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex, MathOptInterface.Integer}(1)
```

source

`MathOptInterface.ZeroOne` – Type.

```
ZeroOne()
```

The set $\{0, 1\}$.

Variables belonging to the `ZeroOne` set are also known as "binary" variables.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> MOI.add_constraint(model, x, MOI.ZeroOne())
MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex, MathOptInterface.ZeroOne}(1)
```

source

`MathOptInterface.Semicontinuous` – Type.

```
Semicontinuous{T<:Real}(lower::T, upper::T)
```

The set $\{0\} \cup [lower, upper]$.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> MOI.add_constraint(model, x, MOI.Semicontinuous(2.0, 3.0))
MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Semicontinuous{Float64}}(1)
```

source

`MathOptInterface.Semiinteger` – Type.

```
Semiinteger{T<:Real}(lower::T, upper::T)
```

The set $\{0\} \cup \{lower, lower + 1, \ldots, upper - 1, upper\}$.

Note that if `lower` and `upper` are not equivalent to an integer, then the solver may throw an error, or it may round up `lower` and round down `upper` to the nearest integers.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> MOI.add_constraint(model, x, MOI.Semiinteger(2.0, 3.0))
MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Semiinteger{Float64}}(1)
```

source

`MathOptInterface.Parameter` – Type.

```
Parameter{T<:Number}(value::T)
```

The set containing the single point $\{value\} \subseteq \mathbb{R}$.

The `Parameter` set is conceptually similar to the `EqualTo` set, except that a variable constrained to the `Parameter` set cannot have other constraints added to it, and the `Parameter` set can never be deleted. Thus, solvers are free to treat the variable as a constant, and they need not add it as a decision variable to the model.

Because of this behavior, you must add parameters using `add_constrained_variable`, and solvers should declare `supports_add_constrained_variable` and not `supports_constraint` for the `Parameter` set.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> p, ci = MOI.add_constrained_variable(model, MOI.Parameter(2.5))
(MOI.VariableIndex(1), MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Parameter{Float64}}(1))

julia> MOI.set(model, MOI.ConstraintSet(), ci, MOI.Parameter(3.0))

julia> MOI.get(model, MOI.ConstraintSet(), ci)
MathOptInterface.Parameter{Float64}(3.0)
```

source

## 18.6 Vector sets

List of recognized vector sets.

`MathOptInterface.Reals` – Type.

```
Reals(dimension::Int)
```

The set $\mathbb{R}^{dimension}$ (containing all points) of non-negative dimension `dimension`.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variables(model, 3);

julia> MOI.add_constraint(model, MOI.VectorOfVariables(x), MOI.Reals(3))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables, MathOptInterface.Reals}(1)
```

source

`MathOptInterface.Zeros` – Type.

```
Zeros(dimension::Int)
```

The set $\{0\}^{dimension}$ (containing only the origin) of non-negative dimension `dimension`.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variables(model, 3);

julia> MOI.add_constraint(model, MOI.VectorOfVariables(x), MOI.Zeros(3))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables, MathOptInterface.Zeros}(1)
```

source

`MathOptInterface.Nonnegatives` – Type.

```
Nonnegatives(dimension::Int)
```

The nonnegative orthant $\{x \in \mathbb{R}^{dimension} : x \geq 0\}$ of non-negative dimension `dimension`.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variables(model, 3);

julia> MOI.add_constraint(model, MOI.VectorOfVariables(x), MOI.Nonnegatives(3))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.Nonnegatives}(1)
```

source

`MathOptInterface.Nonpositives` – Type.

```
Nonpositives(dimension::Int)
```

The nonpositive orthant $\{x \in \mathbb{R}^{dimension} : x \leq 0\}$ of non-negative dimension `dimension`.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variables(model, 3);

julia> MOI.add_constraint(model, MOI.VectorOfVariables(x), MOI.Nonpositives(3))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.Nonpositives}(1)
```

source

MathOptInterface.NormInfinityCone – Type.

```
NormInfinityCone(dimension::Int)
```

The $\ell_\infty$-norm cone $\{(t, x) \in \mathbb{R}^{dimension} : t \geq \|x\|_\infty = \max_i |x_i|\}$ of dimension `dimension`.

The `dimension` must be at least 1.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> t = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> x = MOI.add_variables(model, 3);

julia> MOI.add_constraint(model, MOI.VectorOfVariables([t; x]), MOI.NormInfinityCone(4))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.NormInfinityCone}(1)
```

source

MathOptInterface.NormOneCone – Type.

```
NormOneCone(dimension::Int)
```

The $\ell_1$-norm cone $\{(t, x) \in \mathbb{R}^{dimension} : t \geq \|x\|_1 = \sum_i |x_i|\}$ of dimension `dimension`.

The `dimension` must be at least 1.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> t = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> x = MOI.add_variables(model, 3);

julia> MOI.add_constraint(model, MOI.VectorOfVariables([t; x]), MOI.NormOneCone(4))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.NormOneCone}(1)
```

source

MathOptInterface.NormCone – Type.

```
NormCone(p::Float64, dimension::Int)
```

The $\ell_p$-norm cone $\{(t, x) \in \mathbb{R}^{dimension} : t \geq \left(\sum_i |x_i|^p\right)^{\frac{1}{p}}\}$ of dimension `dimension`.

The `dimension` must be at least 1.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> t = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> x = MOI.add_variables(model, 3);

julia> MOI.add_constraint(model, MOI.VectorOfVariables([t; x]), MOI.NormCone(3, 4))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.NormCone}(1)
```

source

`MathOptInterface.SecondOrderCone` – Type.

```
SecondOrderCone(dimension::Int)
```

The second-order cone (or Lorenz cone or $\ell_2$-norm cone) $\{(t, x) \in \mathbb{R}^{dimension} : t \geq \|x\|_2\}$ of dimension `dimension`.

The `dimension` must be at least 1.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> t = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> x = MOI.add_variables(model, 3);

julia> MOI.add_constraint(model, MOI.VectorOfVariables([t; x]), MOI.SecondOrderCone(4))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.SecondOrderCone}(1)
```

source

`MathOptInterface.RotatedSecondOrderCone` – Type.

```
RotatedSecondOrderCone(dimension::Int)
```

The rotated second-order cone $\{(t, u, x) \in \mathbb{R}^{dimension} : 2tu \geq \|x\|_2^2, t, u \geq 0\}$ of dimension `dimension`.

The `dimension` must be at least 2.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> t = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> u = MOI.add_variable(model)
MOI.VariableIndex(2)

julia> x = MOI.add_variables(model, 3);

julia> MOI.add_constraint(
           model,
           MOI.VectorOfVariables([t; u; x]),
           MOI.RotatedSecondOrderCone(5),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.RotatedSecondOrderCone}(1)
```

source

`MathOptInterface.GeometricMeanCone` – Type.

```
GeometricMeanCone(dimension::Int)
```

The geometric mean cone $\{(t, x) \in \mathbb{R}^{n+1} : x \geq 0, t \leq \sqrt[n]{x_1 x_2 \cdots x_n}\}$, where `dimension = n + 1 >= 2`.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> t = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> x = MOI.add_variables(model, 3);

julia> MOI.add_constraint(
           model,
           MOI.VectorOfVariables([t; x]),
           MOI.GeometricMeanCone(4),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.GeometricMeanCone}(1)
```

source

`MathOptInterface.DualGeometricMeanCone` – Type.

```
DualGeometricMeanCone(dimension::Int)
```

The dual geometric mean cone $\{(u, v) \in \mathbb{R}^{n+1} : v \geq 0, 0 \geq u \geq -n \sqrt[n]{\prod_i v_i}\}$, where `dimension = n + 1 >= 2`.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> t = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> x = MOI.add_variables(model, 3);

julia> MOI.add_constraint(
           model,
           MOI.VectorOfVariables([t; x]),
           MOI.DualGeometricMeanCone(4),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.DualGeometricMeanCone}(1)
```

source

`MathOptInterface.ExponentialCone` – Type.

```
ExponentialCone()
```

The 3-dimensional exponential cone $\{(x, y, z) \in \mathbb{R}^3 : y \exp(x/y) \leq z, y > 0\}$.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variables(model, 3);

julia> MOI.add_constraint(model, MOI.VectorOfVariables(x), MOI.ExponentialCone())
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.ExponentialCone}(1)
```

source

`MathOptInterface.DualExponentialCone` – Type.

```
DualExponentialCone()
```

The 3-dimensional dual exponential cone $\{(u, v, w) \in \mathbb{R}^3 : -u \exp(v/u) \leq \exp(1)w, u < 0\}$.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variables(model, 3);

julia> MOI.add_constraint(model, MOI.VectorOfVariables(x), MOI.DualExponentialCone())
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.DualExponentialCone}(1)
```

source

MathOptInterface.PowerCone – Type.

```
PowerCone{T<:Real}(exponent::T)
```

The 3-dimensional power cone $\{(x, y, z) \in \mathbb{R}^3 : x^{exponent} y^{1-exponent} \geq |z|, x \geq 0, y \geq 0\}$ with parameter exponent.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variables(model, 3);

julia> MOI.add_constraint(model, MOI.VectorOfVariables(x), MOI.PowerCone(0.5))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.PowerCone{Float64}}(1)
```

source

MathOptInterface.DualPowerCone – Type.

```
DualPowerCone{T<:Real}(exponent::T)
```

The 3-dimensional power cone $\{(u, v, w) \in \mathbb{R}^3 : (\frac{u}{exponent})^{exponent} (\frac{v}{1-exponent})^{1-exponent} \geq |w|, u \geq 0, v \geq 0\}$ with parameter exponent.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variables(model, 3);

julia> MOI.add_constraint(model, MOI.VectorOfVariables(x), MOI.DualPowerCone(0.5))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.DualPowerCone{Float64}}(1)
```

source

MathOptInterface.RelativeEntropyCone – Type.

```
RelativeEntropyCone(dimension::Int)
```

The relative entropy cone $\{(u, v, w) \in \mathbb{R}^{1+2n} : u \geq \sum_{i=1}^{n} w_i \log(\frac{w_i}{v_i}), v_i > 0, w_i > 0\}$, where `dimension = 2n + 1 >= 1`.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> u = MOI.add_variable(model);

julia> v = MOI.add_variables(model, 3);

julia> w = MOI.add_variables(model, 3);

julia> MOI.add_constraint(
           model,
           MOI.VectorOfVariables([u; v; w]),
           MOI.RelativeEntropyCone(7),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.RelativeEntropyCone}(1)
```

source

`MathOptInterface.DualRelativeEntropyCone` – Type.

```
DualRelativeEntropyCone(dimension::Int)
```

The dual relative entropy cone $\{(u, v, w) \in \mathbb{R}^{1+2n} : \forall i, w_i \geq u(\log(\frac{u}{v_i}) - 1), v_i > 0, u > 0\}$ where `dimension = 2n + 1 >= 1`.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> u = MOI.add_variable(model);

julia> v = MOI.add_variables(model, 3);

julia> w = MOI.add_variables(model, 3);

julia> MOI.add_constraint(
           model,
           MOI.VectorOfVariables([u; v; w]),
           MOI.DualRelativeEntropyCone(7),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.DualRelativeEntropyCone}(1)
```

source

`MathOptInterface.NormSpectralCone` – Type.

```
NormSpectralCone(row_dim::Int, column_dim::Int)
```

The epigraph of the matrix spectral norm (maximum singular value function) $\{(t, X) \in \mathbb{R}^{1+row_dim \times column_dim} : t \geq \sigma_1(X)\}$, where $\sigma_i$ is the $i$th singular value of the matrix $X$ of non-negative row dimension `row_dim` and column dimension `column_dim`.

The matrix X is vectorized by stacking the columns, matching the behavior of Julia's vec function.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> t = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> X = reshape(MOI.add_variables(model, 6), 2, 3)
2×3 Matrix{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(2)  MOI.VariableIndex(4)  MOI.VariableIndex(6)
 MOI.VariableIndex(3)  MOI.VariableIndex(5)  MOI.VariableIndex(7)

julia> MOI.add_constraint(
           model,
           MOI.VectorOfVariables([t; vec(X)]),
           MOI.NormSpectralCone(2, 3),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.NormSpectralCone}(1)
```

source

`MathOptInterface.NormNuclearCone` – Type.

```
NormNuclearCone(row_dim::Int, column_dim::Int)
```

The epigraph of the matrix nuclear norm (sum of singular values function) $\{(t, X) \in \mathbb{R}^{1+row_dim \times column_dim} : t \geq \sum_i \sigma_i(X)\}$, where $\sigma_i$ is the $i$th singular value of the matrix $X$ of non-negative row dimension `row_dim` and column dimension `column_dim`.

The matrix X is vectorized by stacking the columns, matching the behavior of Julia's vec function.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> t = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> X = reshape(MOI.add_variables(model, 6), 2, 3)
2×3 Matrix{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(2)  MOI.VariableIndex(4)  MOI.VariableIndex(6)
```

```
 MOI.VariableIndex(3)  MOI.VariableIndex(5)  MOI.VariableIndex(7)

julia> MOI.add_constraint(
           model,
           MOI.VectorOfVariables([t; vec(X)]),
           MOI.NormNuclearCone(2, 3),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.NormNuclearCone}(1)
```

source

`MathOptInterface.SOS1` – Type.

```
SOS1{T<:Real}(weights::Vector{T})
```

The set corresponding to the Special Ordered Set (SOS) constraint of Type I.

Of the variables in the set, at most one can be nonzero.

The `weights` induce an ordering of the variables such that the kth element in the set corresponds to the kth weight in `weights`. Solvers may use these weights to improve the efficiency of the solution process, but the ordering does not change the set of feasible solutions.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variables(model, 3);

julia> MOI.add_constraint(
           model,
           MOI.VectorOfVariables(x),
           MOI.SOS1([1.0, 3.0, 2.5]),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.SOS1{Float64}}(1)
```

source

`MathOptInterface.SOS2` – Type.

```
SOS2{T<:Real}(weights::Vector{T})
```

The set corresponding to the Special Ordered Set (SOS) constraint of Type II.

The `weights` induce an ordering of the variables such that the kth element in the set corresponds to the kth weight in `weights`. Therefore, the weights must be unique.

Of the variables in the set, at most two can be nonzero, and if two are nonzero, they must be adjacent in the ordering of the set.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variables(model, 3);

julia> MOI.add_constraint(
           model,
           MOI.VectorOfVariables(x),
           MOI.SOS2([1.0, 3.0, 2.5]),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.SOS2{Float64}}(1)
```

source

MathOptInterface.Indicator – Type.

```
Indicator{ACTIVATE_ON_ZERO}(set::AbstractScalarSet)
Indicator{ACTIVATE_ON_ONE}(set::AbstractScalarSet)
```

The set corresponding to an indicator constraint.

The type parameter must be an ActivationCondition.

When the type parameter is ACTIVATE_ON_ZERO, this means:

$$\{(y, x) \in \{0, 1\} \times \mathbb{R} : y = 0 \implies x \in set\}$$

When the type parameter is ACTIVATE_ON_ONE, this means:

$$\{(y, x) \in \{0, 1\} \times \mathbb{R} : y = 1 \implies x \in set\}$$

**Notes**

Most solvers expect that the first row of the function is interpretable as a VariableIndex (for example, `1.0 * x + 0.0`), and that the variable is constrained to the ZeroOne set. An error will be thrown if this is not the case.

**Example**

The constraint $\{(y, x) \in \{0, 1\} \times \mathbb{R}^2 : y = 1 \implies x_1 + x_2 \leq 9\}$ is defined as

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variables(model, 2);

julia> y, _ = MOI.add_constrained_variable(model, MOI.ZeroOne());

julia> f = MOI.Utilities.vectorize([y, 1.0 * x[1] + 1.0 * x[2]])
┌                                                    ┐
│0.0 + 1.0 MOI.VariableIndex(3)                      │
│0.0 + 1.0 MOI.VariableIndex(1) + 1.0 MOI.VariableIndex(2)│
└                                                    ┘

julia> s = MOI.Indicator{MOI.ACTIVATE_ON_ONE}(MOI.LessThan(9.0))
MathOptInterface.Indicator{MathOptInterface.ACTIVATE_ON_ONE,
↪  MathOptInterface.LessThan{Float64}}(MathOptInterface.LessThan{Float64}(9.0))
```

```
julia> MOI.add_constraint(model, f, s)
MathOptInterface.ConstraintIndex{MathOptInterface.VectorAffineFunction{Float64},
↪  MathOptInterface.Indicator{MathOptInterface.ACTIVATE_ON_ONE,
↪  MathOptInterface.LessThan{Float64}}}(1)
```

The constraint $\{(y, x) \in \{0, 1\} \times \mathbb{R} : y = 0 \implies x = 0\}$ is defined as

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model);

julia> y, _ = MOI.add_constrained_variable(model, MOI.ZeroOne());

julia> f = MOI.VectorOfVariables([y, x]);

julia> s = MOI.Indicator{MOI.ACTIVATE_ON_ZERO}(MOI.EqualTo(0.0))
MathOptInterface.Indicator{MathOptInterface.ACTIVATE_ON_ZERO,
↪  MathOptInterface.EqualTo{Float64}}(MathOptInterface.EqualTo{Float64}(0.0))

julia> MOI.add_constraint(model, f, s)
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.Indicator{MathOptInterface.ACTIVATE_ON_ZERO,
↪  MathOptInterface.EqualTo{Float64}}}(1)
```

source

`MathOptInterface.ActivationCondition` – Type.

```
ActivationCondition
```

Activation condition for an indicator constraint.

The enum value is used as first type parameter of `Indicator{A,S}`.

**Values**

**`ACTIVATE_ON_ZERO`**

The indicator constraint holds when the binary variable is zero.

**`ACTIVATE_ON_ONE`**

The indicator constraint holds when the binary variable is one.

source

`MathOptInterface.ACTIVATE_ON_ZERO` – Constant.

```
ACTIVATE_ON_ZERO::ActivationCondition
```

An instance of the `ActivationCondition` enum.

**About**

The indicator constraint holds when the binary variable is zero.

source

`MathOptInterface.ACTIVATE_ON_ONE` – Constant.

```
ACTIVATE_ON_ONE::ActivationCondition
```

An instance of the `ActivationCondition` enum.

**About**

The indicator constraint holds when the binary variable is one.

source

`MathOptInterface.Complements` – Type.

```
Complements(dimension::Base.Integer)
```

The set corresponding to a mixed complementarity constraint.

Complementarity constraints should be specified with an `AbstractVectorFunction`-in-`Complements(dimension)` constraint.

The dimension of the vector-valued function `F` must be `dimension`. This defines a complementarity constraint between the scalar function `F[i]` and the variable in `F[i + dimension/2]`. Thus, `F[i + dimension/2]` must be interpretable as a single variable `x_i` (for example, `1.0 * x + 0.0`), and `dimension` must be even.

The mixed complementarity problem consists of finding `x_i` in the interval `[lb, ub]` (that is, in the set `Interval(lb, ub)`), such that the following holds:

1. `F_i(x) == 0` if `lb_i < x_i < ub_i`
2. `F_i(x) >= 0` if `lb_i == x_i`
3. `F_i(x) <= 0` if `x_i == ub_i`

Classically, the bounding set for `x_i` is `Interval(0, Inf)`, which recovers: `0 <= F_i(x) ⊥ x_i >= 0`, where the `⊥` operator implies `F_i(x) * x_i = 0`.

**Example**

The problem:

```
x -in- Interval(-1, 1)
[-4 * x - 3, x] -in- Complements(2)
```

defines the mixed complementarity problem where the following holds:

1. `-4 * x - 3 == 0` if `-1 < x < 1`
2. `-4 * x - 3 >= 0` if `x == -1`

3. `-4 * x - 3 <= 0` if `x == 1`

There are three solutions:

1. `x = -3/4` with `F(x) = 0`
2. `x = -1` with `F(x) = 1`
3. `x = 1` with `F(x) = -7`

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x, _ = MOI.add_constrained_variable(model, MOI.Interval(-1.0, 1.0));

julia> MOI.add_constraint(
            model,
            MOI.Utilities.vectorize([-4.0 * x - 3.0, x]),
            MOI.Complements(2),
        )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorAffineFunction{Float64},
↪  MathOptInterface.Complements}(1)
```

The function F can also be defined in terms of single variables. For example, the problem:

```
[x_3, x_4] -in- Nonnegatives(2)
[x_1, x_2, x_3, x_4] -in- Complements(4)
```

defines the complementarity problem where `0 <= x_1 ⊥ x_3 >= 0` and `0 <= x_2 ⊥ x_4 >= 0`.

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variables(model, 4);

julia> MOI.add_constraint(model, MOI.VectorOfVariables(x[3:4]), MOI.Nonnegatives(2))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.Nonnegatives}(1)

julia> MOI.add_constraint(
            model,
            MOI.VectorOfVariables(x),
            MOI.Complements(4),
        )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.Complements}(1)
```

source

`MathOptInterface.HyperRectangle` – Type.

```
HyperRectangle(lower::Vector{T}, upper::Vector{T}) where {T}
```

The set $\{x \in \bar{\mathbb{R}}^d : x_i \in [lower_i, upper_i] \forall i = 1, \ldots, d\}$.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variables(model, 3)
3-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(1)
 MOI.VariableIndex(2)
 MOI.VariableIndex(3)

julia> MOI.add_constraint(
           model,
           MOI.VectorOfVariables(x),
           MOI.HyperRectangle(zeros(3), ones(3)),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.HyperRectangle{Float64}}(1)
```

source

`MathOptInterface.Scaled` – Type.

```
struct Scaled{S<:AbstractVectorSet} <: AbstractVectorSet
    set::S
end
```

Given a vector $a \in \mathbb{R}^d$ and a set representing the set $\mathcal{S} \in \mathbb{R}^d$ such that `Utilities.set_dot` for $x \in \mathcal{S}$ and $y \in \mathcal{S}^*$ is

$$\sum_{i=1}^{d} a_i x_i y_i$$

the set `Scaled(set)` is defined as

$$\{(\sqrt{a_1}x_1, \sqrt{a_2}x_2, \ldots, \sqrt{a_d}x_d) : x \in S\}$$

**Example**

This can be used to scale a vector of numbers

```
julia> set = MOI.PositiveSemidefiniteConeTriangle(2)
MathOptInterface.PositiveSemidefiniteConeTriangle(2)

julia> a = MOI.Utilities.SetDotScalingVector{Float64}(set)
3-element MathOptInterface.Utilities.SetDotScalingVector{Float64,
↪  MathOptInterface.PositiveSemidefiniteConeTriangle}:
 1.0
 1.4142135623730951
```

```
 1.0

julia> using LinearAlgebra

julia> MOI.Utilities.operate(*, Float64, Diagonal(a), ones(3))
3-element Vector{Float64}:
 1.0
 1.4142135623730951
 1.0
```

It can be also used to scale a vector of function

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variables(model, 3);

julia> func = MOI.VectorOfVariables(x)
┌                   ┐
│MOI.VariableIndex(1)│
│MOI.VariableIndex(2)│
│MOI.VariableIndex(3)│
└                   ┘

julia> set = MOI.PositiveSemidefiniteConeTriangle(2)
MathOptInterface.PositiveSemidefiniteConeTriangle(2)

julia> MOI.Utilities.operate(*, Float64, Diagonal(a), func)
┌                                          ┐
│0.0 + 1.0 MOI.VariableIndex(1)            │
│0.0 + 1.4142135623730951 MOI.VariableIndex(2)│
│0.0 + 1.0 MOI.VariableIndex(3)            │
└                                          ┘
```

source

`MathOptInterface.VectorNonlinearOracle` – Type.

```
VectorNonlinearOracle(;
    dimension::Int,
    l::Vector{Float64},
    u::Vector{Float64},
    eval_f::Function,
    jacobian_structure::Vector{Tuple{Int,Int}},
    eval_jacobian::Function,
    hessian_lagrangian_structure::Vector{Tuple{Int,Int}} = Tuple{Int,Int}[],
    eval_hessian_lagrangian::Union{Nothing,Function} = nothing,
) <: AbstractVectorSet
```

The set:

$$S = \{x \in \mathbb{R}^{dimension} : l \leq f(x) \leq u\}$$

where $f$ is defined by the vectors `l` and `u`, and the callback oracles `eval_f`, `eval_jacobian`, and `eval_hessian_lagrangian`.

**f**

The `eval_f` function must have the signature

```
eval_f(ret::AbstractVector, x::AbstractVector)::Nothing
```

which fills $f(x)$ into the dense vector `ret`.

**Jacobian**

The `eval_jacobian` function must have the signature

```
eval_jacobian(ret::AbstractVector, x::AbstractVector)::Nothing
```

which fills the sparse Jacobian $\nabla f(x)$ into `ret`.

The one-indexed sparsity structure must be provided in the `jacobian_structure` argument and it must be independent of the point $x$.

`jacobian_structure` is not required to be sorted and it may contain duplicates, in which case the solver will combine the corresponding non-zero values in `ret` by adding them together.

**Hessian**

The `eval_hessian_lagrangian` function is optional.

If `eval_hessian_lagrangian === nothing`, Ipopt will use a Hessian approximation instead of the exact Hessian.

If `eval_hessian_lagrangian` is a function, it must have the signature

```
eval_hessian_lagrangian(
    ret::AbstractVector,
    x::AbstractVector,
    μ::AbstractVector,
)::Nothing
```

which fills the sparse Hessian of the Lagrangian $\sum \mu_i \nabla^2 f_i(x)$ into `ret`.

The one-indexed sparsity structure must be provided in the `hessian_lagrangian_structure` argument and it must be independent of the point $x$.

`hessian_lagrangian_structure` is not required to be sorted and it may contain duplicates, in which case the solver will combine the corresponding non-zero values in `ret` by adding them together.

Any mix of lower and upper-triangular indices is valid. Elements `(i, j)` and `(j, i)`, if both present, are treated as duplicates.

**Example**

To model the set:

$$\begin{aligned} 0 \le x^2 & \le 1 \\ 0 \le y^2 + x \cdot z^3 - w & \le 0 \end{aligned}$$

do

```
julia> import MathOptInterface as MOI

julia> set = MOI.VectorNonlinearOracle(;
           dimension = 4,
           l = [0.0, 0.0],
           u = [1.0, 0.0],
           eval_f = (ret, x) -> begin
               ret[1] = x[2]^2
               ret[2] = x[3]^2 + x[2] * x[4]^3 - x[1]
               return
           end,
           jacobian_structure = [(1, 2), (2, 1), (2, 2), (2, 3), (2, 4)],
           eval_jacobian = (ret, x) -> begin
               ret[1] = 2.0 * x[2]
               ret[2] = -1.0
               ret[3] = x[4]^3
               ret[4] = 2.0 * x[3]
               ret[5] = 3.0 * x[2] * x[4]^2
               return
           end,
           hessian_lagrangian_structure = [(2, 2), (3, 3), (4, 4), (2, 4)],
           eval_hessian_lagrangian = (ret, x, u) -> begin
               ret[1] = 2.0 * u[1]
               ret[2] = 2.0 * u[2]
               ret[3] = 6.0 * x[4] * u[2]
               ret[4] = 3.0 * x[4]^2 * u[2]
               return
           end,
       );

julia> set
VectorNonlinearOracle{Float64}(;
    dimension = 4,
    l = [0.0, 0.0],
    u = [1.0, 0.0],
    ...,
)
```

source

## 18.7 Constraint programming sets

`MathOptInterface.AllDifferent` – Type.

```
AllDifferent(dimension::Int)
```

The set $\{x \in \mathbb{Z}^d\}$ such that no two elements in $x$ take the same value and `dimension = d`.

**Also known as**

This constraint is called `all_different` in MiniZinc, and is sometimes also called `distinct`.

**Example**

To enforce `x[1] != x[2] AND x[1] != x[3] AND x[2] != x[3]`:

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = [MOI.add_constrained_variable(model, MOI.Integer())[1] for _ in 1:3]
3-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(1)
 MOI.VariableIndex(2)
 MOI.VariableIndex(3)

julia> MOI.add_constraint(model, MOI.VectorOfVariables(x), MOI.AllDifferent(3))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.AllDifferent}(1)
```

source

`MathOptInterface.BinPacking` – Type.

```
BinPacking(c::T, w::Vector{T}) where {T}
```

The set $\{x \in \mathbb{Z}^d\}$ where `d = length(w)`, such that each item `i` in `1:d` of weight `w[i]` is put into bin `x[i]`, and the total weight of each bin does not exceed `c`.

There are additional assumptions that the capacity, `c`, and the weights, `w`, must all be non-negative.

The bin numbers depend on the bounds of `x`, so they may be something other than the integers `1:d`.

**Also known as**

This constraint is called `bin_packing` in MiniZinc.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> bins = MOI.add_variables(model, 5)
5-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(1)
 MOI.VariableIndex(2)
 MOI.VariableIndex(3)
 MOI.VariableIndex(4)
 MOI.VariableIndex(5)

julia> weights = Float64[1, 1, 2, 2, 3]
5-element Vector{Float64}:
 1.0
 1.0
 2.0
 2.0
 3.0

julia> MOI.add_constraint.(model, bins, MOI.Integer())
5-element Vector{MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Integer}}:
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex, MathOptInterface.Integer}(1)
```

```
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex, MathOptInterface.Integer}(2)
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex, MathOptInterface.Integer}(3)
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex, MathOptInterface.Integer}(4)
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex, MathOptInterface.Integer}(5)

julia> MOI.add_constraint.(model, bins, MOI.Interval(4.0, 6.0))
5-element Vector{MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Interval{Float64}}}:
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Interval{Float64}}(1)
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Interval{Float64}}(2)
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Interval{Float64}}(3)
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Interval{Float64}}(4)
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Interval{Float64}}(5)

julia> MOI.add_constraint(model, MOI.VectorOfVariables(bins), MOI.BinPacking(3.0, weights))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.BinPacking{Float64}}(1)
```

source

`MathOptInterface.Circuit` – Type.

```
Circuit(dimension::Int)
```

The set $\{x \in \{1..d\}^d\}$ that constraints $x$ to be a circuit, such that $x_i = j$ means that $j$ is the successor of $i$, and `dimension = d`.

Graphs with multiple independent circuits, such as `[2, 1, 3]` and `[2, 1, 4, 3]`, are not valid.

**Also known as**

This constraint is called `circuit` in MiniZinc, and it is equivalent to forming a (potentially sub-optimal) tour in the travelling salesperson problem.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = [MOI.add_constrained_variable(model, MOI.Integer())[1] for _ in 1:3]
3-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(1)
 MOI.VariableIndex(2)
 MOI.VariableIndex(3)

julia> MOI.add_constraint(model, MOI.VectorOfVariables(x), MOI.Circuit(3))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.Circuit}(1)
```

source

`MathOptInterface.CountAtLeast` – Type.

```
CountAtLeast(n::Int, d::Vector{Int}, set::Set{Int})
```

The set $\{x \in \mathbb{Z}^{d_1+d_2+\dots d_N}\}$, where `x` is partitioned into `N` subsets ($\{x_1, \dots, x_{d_1}\}$, $\{x_{d_1+1}, \dots, x_{d_1+d_2}\}$ and so on), and at least $n$ elements of each subset take one of the values in `set`.

**Also known as**

This constraint is called `at_least` in MiniZinc.

**Example**

To ensure that 3 appears at least once in each of the subsets `{a, b}` and `{b, c}`:

```
julia> model = MOI.Utilities.Model{Float64}();

julia> a, _ = MOI.add_constrained_variable(model, MOI.Integer())
(MOI.VariableIndex(1), MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Integer}(1))

julia> b, _ = MOI.add_constrained_variable(model, MOI.Integer())
(MOI.VariableIndex(2), MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Integer}(2))

julia> c, _ = MOI.add_constrained_variable(model, MOI.Integer())
(MOI.VariableIndex(3), MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Integer}(3))

julia> x, d, set = [a, b, b, c], [2, 2], [3]
(MathOptInterface.VariableIndex[MOI.VariableIndex(1), MOI.VariableIndex(2),
↪  MOI.VariableIndex(2), MOI.VariableIndex(3)], [2, 2], [3])

julia> MOI.add_constraint(model, MOI.VectorOfVariables(x), MOI.CountAtLeast(1, d, Set(set)))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.CountAtLeast}(1)
```

source

`MathOptInterface.CountBelongs` – Type.

```
CountBelongs(dimension::Int, set::Set{Int})
```

The set $\{(n, x) \in \mathbb{Z}^{1+d}\}$, such that `n` elements of the vector `x` take on of the values in `set` and `dimension = 1 + d`.

**Also known as**

This constraint is called `among` by MiniZinc.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> n, _ = MOI.add_constrained_variable(model, MOI.Integer())
(MOI.VariableIndex(1), MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Integer}(1))

julia> x = [MOI.add_constrained_variable(model, MOI.Integer())[1] for _ in 1:3]
3-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(2)
 MOI.VariableIndex(3)
 MOI.VariableIndex(4)

julia> set = Set([3, 4, 5]);

julia> MOI.add_constraint(model, MOI.VectorOfVariables([n; x]), MOI.CountBelongs(4, set))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.CountBelongs}(1)
```

source

`MathOptInterface.CountDistinct` – Type.

```
CountDistinct(dimension::Int)
```

The set $\{(n, x) \in \mathbb{Z}^{1+d}\}$, such that the number of distinct values in `x` is `n` and `dimension = 1 + d`.

**Also known as**

This constraint is called `nvalues` in MiniZinc.

**Example**

To model:

- if `n == 1`, then `x[1] == x[2] == x[3]`
- if `n == 2`, then
  - `x[1] == x[2] != x[3]` or
  - `x[1] != x[2] == x[3]` or
  - `x[1] == x[3] != x[2]`
- if `n == 3`, then `x[1] != x[2]`, `x[2] != x[3]` and `x[3] != x[1]`

```
julia> model = MOI.Utilities.Model{Float64}();

julia> n, _ = MOI.add_constrained_variable(model, MOI.Integer())
(MOI.VariableIndex(1), MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Integer}(1))

julia> x = [MOI.add_constrained_variable(model, MOI.Integer())[1] for _ in 1:3]
3-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(2)
 MOI.VariableIndex(3)
```

```
 MOI.VariableIndex(4)

julia> MOI.add_constraint(model, MOI.VectorOfVariables(vcat(n, x)), MOI.CountDistinct(4))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.CountDistinct}(1)
```

**Relationship to AllDifferent**

When the first element is `d`, `CountDistinct` is equivalent to an `AllDifferent` constraint.

source

`MathOptInterface.CountGreaterThan` – Type.

```
CountGreaterThan(dimension::Int)
```

The set $\{(c, y, x) \in \mathbb{Z}^{1+1+d}\}$, such that `c` is strictly greater than the number of occurrences of `y` in `x` and `dimension = 1 + 1 + d`.

**Also known as**

This constraint is called `count_gt` in MiniZinc.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> c, _ = MOI.add_constrained_variable(model, MOI.Integer())
(MOI.VariableIndex(1), MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Integer}(1))

julia> y, _ = MOI.add_constrained_variable(model, MOI.Integer())
(MOI.VariableIndex(2), MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Integer}(2))

julia> x = [MOI.add_constrained_variable(model, MOI.Integer())[1] for _ in 1:3]
3-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(3)
 MOI.VariableIndex(4)
 MOI.VariableIndex(5)

julia> MOI.add_constraint(model, MOI.VectorOfVariables([c; y; x]), MOI.CountGreaterThan(5))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.CountGreaterThan}(1)
```

source

`MathOptInterface.Cumulative` – Type.

```
Cumulative(dimension::Int)
```

The set $\{(s, d, r, b) \in \mathbb{Z}^{3n+1}\}$, representing the `cumulative` global constraint, where `n == length(s) == length(r) == length(b)` and `dimension = 3n + 1`.

`Cumulative` requires that a set of tasks given by start times $s$, durations $d$, and resource requirements $r$, never requires more than the global resource bound $b$ at any one time.

**Also known as**

This constraint is called `cumulative` in MiniZinc.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> s = [MOI.add_constrained_variable(model, MOI.Integer())[1] for _ in 1:3]
3-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(1)
 MOI.VariableIndex(2)
 MOI.VariableIndex(3)

julia> d = [MOI.add_constrained_variable(model, MOI.Integer())[1] for _ in 1:3]
3-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(4)
 MOI.VariableIndex(5)
 MOI.VariableIndex(6)

julia> r = [MOI.add_constrained_variable(model, MOI.Integer())[1] for _ in 1:3]
3-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(7)
 MOI.VariableIndex(8)
 MOI.VariableIndex(9)

julia> b, _ = MOI.add_constrained_variable(model, MOI.Integer())
(MOI.VariableIndex(10), MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Integer}(10))

julia> MOI.add_constraint(model, MOI.VectorOfVariables([s; d; r; b]), MOI.Cumulative(10))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.Cumulative}(1)
```

source

`MathOptInterface.Path` – Type.

```
Path(from::Vector{Int}, to::Vector{Int})
```

Given a graph comprised of a set of nodes `1..N` and a set of arcs `1..E` represented by an edge from node `from[i]` to node `to[i]`, `Path` constrains the set $(s, t, ns, es) \in (1..N) \times (1..E) \times \{0, 1\}^N \times \{0, 1\}^E$, to form subgraph that is a path from node `s` to node `t`, where node `n` is in the path if `ns[n]` is 1, and edge `e` is in the path if `es[e]` is 1.

The path must be acyclic, and it must traverse all nodes `n` for which `ns[n]` is 1, and all edges `e` for which `es[e]` is 1.

**Also known as**

This constraint is called path in MiniZinc.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> N, E = 4, 5
(4, 5)

julia> from = [1, 1, 2, 2, 3]
5-element Vector{Int64}:
 1
 1
 2
 2
 3

julia> to = [2, 3, 3, 4, 4]
5-element Vector{Int64}:
 2
 3
 3
 4
 4

julia> s, _ = MOI.add_constrained_variable(model, MOI.Integer())
(MOI.VariableIndex(1), MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Integer}(1))

julia> t, _ = MOI.add_constrained_variable(model, MOI.Integer())
(MOI.VariableIndex(2), MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Integer}(2))

julia> ns = MOI.add_variables(model, N)
4-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(3)
 MOI.VariableIndex(4)
 MOI.VariableIndex(5)
 MOI.VariableIndex(6)

julia> MOI.add_constraint.(model, ns, MOI.ZeroOne())
4-element Vector{MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.ZeroOne}}:
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex, MathOptInterface.ZeroOne}(3)
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex, MathOptInterface.ZeroOne}(4)
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex, MathOptInterface.ZeroOne}(5)
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex, MathOptInterface.ZeroOne}(6)

julia> es = MOI.add_variables(model, E)
5-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(7)
 MOI.VariableIndex(8)
 MOI.VariableIndex(9)
 MOI.VariableIndex(10)
 MOI.VariableIndex(11)
```

```
julia> MOI.add_constraint.(model, es, MOI.ZeroOne())
5-element Vector{MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.ZeroOne}}:
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex, MathOptInterface.ZeroOne}(7)
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex, MathOptInterface.ZeroOne}(8)
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex, MathOptInterface.ZeroOne}(9)
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex, MathOptInterface.ZeroOne}(10)
 MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex, MathOptInterface.ZeroOne}(11)

julia> MOI.add_constraint(model, MOI.VectorOfVariables([s; t; ns; es]), MOI.Path(from, to))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables, MathOptInterface.Path}(1)
```

source

MathOptInterface.Reified – Type.

```
Reified(set::AbstractSet)
```

The constraint $[z; f(x)] \in Reified(S)$ ensures that $f(x) \in S$ if and only if $z == 1$, where $z \in \{0, 1\}$.

```
julia> model = MOI.Utilities.UniversalFallback(MOI.Utilities.Model{Float64}());

julia> z, _ = MOI.add_constrained_variable(model, MOI.ZeroOne())
(MOI.VariableIndex(1), MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.ZeroOne}(1))

julia> x = MOI.add_variable(model)
MOI.VariableIndex(2)

julia> MOI.add_constraint(
           model,
           MOI.Utilities.vectorize([z, 2.0 * x]),
           MOI.Reified(MOI.GreaterThan(1.0)),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorAffineFunction{Float64},
↪  MathOptInterface.Reified{MathOptInterface.GreaterThan{Float64}}}(1)
```

source

MathOptInterface.Table – Type.

```
Table(table::Matrix{T}) where {T}
```

The set $\{x \in \mathbb{R}^d\}$ where `d = size(table, 2)`, such that `x` belongs to one row of `table`. That is, there exists some `j` in `1:size(table, 1)`, such that `x[i] = table[j, i]` for all `i=1:size(table, 2)`.

**Also known as**

This constraint is called `table` in MiniZinc.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variables(model, 3)
3-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(1)
 MOI.VariableIndex(2)
 MOI.VariableIndex(3)

julia> table = Float64[1 1 0; 0 1 1; 1 0 1; 1 1 1]
4×3 Matrix{Float64}:
 1.0  1.0  0.0
 0.0  1.0  1.0
 1.0  0.0  1.0
 1.0  1.0  1.0

julia> MOI.add_constraint(model, MOI.VectorOfVariables(x), MOI.Table(table))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.Table{Float64}}(1)
```

source

## 18.8 Matrix sets

Matrix sets are vectorized to be subtypes of `AbstractVectorSet`.

For sets of symmetric matrices, storing both the `(i, j)` and `(j, i)` elements is redundant. Use the `AbstractSymmetricMatrixSe` set to represent only the vectorization of the upper triangular part of the matrix.

When the matrix of expressions constrained to be in the set is not symmetric, and hence additional constraints are needed to force the equality of the `(i, j)` and `(j, i)` elements, use the `AbstractSymmetricMatrixSetSquare` set.

The `Bridges.Constraint.SquareBridge` can transform a set from the square form to the `triangular_form` by adding appropriate constraints if the `(i, j)` and `(j, i)` expressions are different.

`MathOptInterface.AbstractSymmetricMatrixSetTriangle` – Type.

```
abstract type AbstractSymmetricMatrixSetTriangle <: AbstractVectorSet end
```

Abstract supertype for subsets of the (vectorized) cone of symmetric matrices, with `side_dimension` rows and columns. The entries of the upper-right triangular part of the matrix are given column by column (or equivalently, the entries of the lower-left triangular part are given row by row). A vectorized cone of `dimension` $n$ corresponds to a square matrix with side dimension $\sqrt{1/4 + 2n} - 1/2$. (Because a $d \times d$ matrix has $d(d+1)/2$ elements in the upper or lower triangle.)

**Example**

The matrix

$$\begin{bmatrix} 1 & 2 & 4 \\ 2 & 3 & 5 \\ 4 & 5 & 6 \end{bmatrix}$$

has side_dimension 3 and vectorization $(1, 2, 3, 4, 5, 6)$.

**Note**

Two packed storage formats exist for symmetric matrices, the respective orders of the entries are:

- upper triangular column by column (or lower triangular row by row);
- lower triangular column by column (or upper triangular row by row).

The advantage of the first format is the mapping between the `(i, j)` matrix indices and the k index of the vectorized form. It is simpler and does not depend on the side dimension of the matrix. Indeed,

- the entry of matrix indices `(i, j)` has vectorized index `k = div((j - 1) * j, 2) + i` if $i \leq j$ and `k = div((i - 1) * i, 2) + j` if $j \leq i$;
- and the entry with vectorized index k has matrix indices `i = div(1 + isqrt(8k - 7), 2)` and `j = k - div((i - 1) * i, 2)` or `j = div(1 + isqrt(8k - 7), 2)` and `i = k - div((j - 1) * j, 2)`.

**Duality note**

The scalar product for the symmetric matrix in its vectorized form is the sum of the pairwise product of the diagonal entries plus twice the sum of the pairwise product of the upper diagonal entries; see [p. 634, 1]. This has important consequence for duality.

Consider for example the following problem (PositiveSemidefiniteConeTriangle is a subtype of AbstractSymmetricMatrix

$$\begin{aligned} \max_{x \in \mathbb{R}} \quad & x \\ \text{s.t.} \quad & (1, -x, 1) \in \text{PositiveSemidefiniteConeTriangle}(2). \end{aligned}$$

The dual is the following problem

$$\begin{aligned} \min_{x \in \mathbb{R}^3} \quad & y_1 + y_3 \\ \text{s.t.} \quad & 2y_2 = 1 \\ & y \in \text{PositiveSemidefiniteConeTriangle}(2). \end{aligned}$$

Why do we use $2y_2$ in the dual constraint instead of $y_2$ ? The reason is that $2y_2$ is the scalar product between $y$ and the symmetric matrix whose vectorized form is $(0, 1, 0)$. Indeed, with our modified scalar products we have

$$\langle (0, 1, 0), (y_1, y_2, y_3) \rangle = \operatorname{trace} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} y_1 & y_2 \\ y_2 & y_3 \end{pmatrix} = 2y_2.$$

**References**

[1] Boyd, S. and Vandenberghe, L.. Convex optimization. Cambridge university press, 2004.

source

`MathOptInterface.AbstractSymmetricMatrixSetSquare` – Type.

```
abstract type AbstractSymmetricMatrixSetSquare <: AbstractVectorSet end
```

Abstract supertype for subsets of the (vectorized) cone of symmetric matrices, with `side_dimension` rows and columns. The entries of the matrix are given column by column (or equivalently, row by row). The matrix is both constrained to be symmetric and to have its `triangular_form` belong to the corresponding set. That is, if the functions in entries $(i, j)$ and $(j, i)$ are different, then a constraint will be added to make sure that the entries are equal.

**Example**

`PositiveSemidefiniteConeSquare` is a subtype of `AbstractSymmetricMatrixSetSquare` and constraining the matrix

$$\begin{bmatrix} 1 & -y \\ -z & 0 \end{bmatrix}$$

to be symmetric positive semidefinite can be achieved by constraining the vector $(1, -z, -y, 0)$ (or $(1, -y, -z, 0)$) to belong to the `PositiveSemidefiniteConeSquare(2)`. It both constrains $y = z$ and $(1, -y, 0)$ (or $(1, -z, 0)$) to be in `PositiveSemidefiniteConeTriangle(2)`, since `triangular_form(PositiveSemidefiniteConeSquare)` is `PositiveSemidefiniteConeTriangle`.

source

`MathOptInterface.side_dimension` – Function.

```
side_dimension(
    set::Union{
        AbstractSymmetricMatrixSetTriangle,
        AbstractSymmetricMatrixSetSquare,
        HermitianPositiveSemidefiniteConeTriangle,
    },
)
```

Side dimension of the matrices in `set`.

**Convention**

By convention, the side dimension should be stored in the `side_dimension` field. If this is not the case for a subtype of `AbstractSymmetricMatrixSetTriangle`, or `AbstractSymmetricMatrixSetSquare` you must implement this method.

source

`MathOptInterface.triangular_form` – Function.

```
triangular_form(S::Type{<:AbstractSymmetricMatrixSetSquare})
triangular_form(set::AbstractSymmetricMatrixSetSquare)
```

Return the `AbstractSymmetricMatrixSetTriangle` corresponding to the vectorization of the upper triangular part of matrices in the `AbstractSymmetricMatrixSetSquare` set.

source

List of recognized matrix sets.

`MathOptInterface.PositiveSemidefiniteConeTriangle` – Type.

```
PositiveSemidefiniteConeTriangle(side_dimension::Int) <: AbstractSymmetricMatrixSetTriangle
```

The (vectorized) cone of symmetric positive semidefinite matrices, with non-negative `side_dimension` rows and columns.

See `AbstractSymmetricMatrixSetTriangle` for more details on the vectorized form.

source

`MathOptInterface.PositiveSemidefiniteConeSquare` – Type.

```
PositiveSemidefiniteConeSquare(side_dimension::Int) <: AbstractSymmetricMatrixSetSquare
```

The cone of symmetric positive semidefinite matrices, with non-negative side length `side_dimension`.

See `AbstractSymmetricMatrixSetSquare` for more details on the vectorized form.

The entries of the matrix are given column by column (or equivalently, row by row).

The matrix is both constrained to be symmetric and to be positive semidefinite. That is, if the functions in entries $(i, j)$ and $(j, i)$ are different, then a constraint will be added to make sure that the entries are equal.

**Example**

Constraining the matrix

$$\begin{bmatrix} 1 & -y \\ -z & 0 \end{bmatrix}$$

to be symmetric positive semidefinite can be achieved by constraining the vector $(1, -z, -y, 0)$ (or $(1, -y, -z, 0)$) to belong to the `PositiveSemidefiniteConeSquare(2)`.

It both constrains $y = z$ and $(1, -y, 0)$ (or $(1, -z, 0)$) to be in `PositiveSemidefiniteConeTriangle(2)`.

source

`MathOptInterface.HermitianPositiveSemidefiniteConeTriangle` – Type.

```
HermitianPositiveSemidefiniteConeTriangle(side_dimension::Int) <: AbstractVectorSet
```

The (vectorized) cone of Hermitian positive semidefinite matrices, with non-negative `side_dimension` rows and columns.

Because the matrix is Hermitian, the diagonal elements are real, and the complex-valued lower triangular entries are obtained as the conjugate of corresponding upper triangular entries.

**Vectorization format**

The vectorized form starts with real part of the entries of the upper triangular part of the matrix, given column by column as explained in AbstractSymmetricMatrixSetSquare.

It is then followed by the imaginary part of the off-diagonal entries of the upper triangular part, also given column by column.

For example, the matrix

$$\begin{bmatrix} 1 & 2+7im & 4+8im \\ 2-7im & 3 & 5+9im \\ 4-8im & 5-9im & 6 \end{bmatrix}$$

has side_dimension 3 and is represented as the vector $[1, 2, 3, 4, 5, 6, 7, 8, 9]$.

source

MathOptInterface.LogDetConeTriangle – Type.

```
LogDetConeTriangle(side_dimension::Int)
```

The log-determinant cone $\{(t, u, X) \in \mathbb{R}^{2+d(d+1)/2} : t \leq u \log(\det(X/u)), u > 0\}$, where the matrix X is represented in the same symmetric packed format as in the PositiveSemidefiniteConeTriangle.

The non-negative argument side_dimension is the side dimension of the matrix X, that is, its number of rows or columns.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> t = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> X = MOI.add_variables(model, 3);

julia> MOI.add_constraint(
           model,
           MOI.VectorOfVariables([t; X]),
           MOI.LogDetConeTriangle(2),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.LogDetConeTriangle}(1)
```

source

MathOptInterface.LogDetConeSquare – Type.

```
LogDetConeSquare(side_dimension::Int)
```

The log-determinant cone $\{(t, u, X) \in \mathbb{R}^{2+d^2} : t \leq u \log(\det(X/u)), X \text{ symmetric}, u > 0\}$, where the matrix X is represented in the same format as in the PositiveSemidefiniteConeSquare.

Similarly to `PositiveSemidefiniteConeSquare`, constraints are added to ensure that X is symmetric.

The non-negative argument `side_dimension` is the side dimension of the matrix X, that is, its number of rows or columns.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> t = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> X = reshape(MOI.add_variables(model, 4), 2, 2)
2×2 Matrix{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(2)  MOI.VariableIndex(4)
 MOI.VariableIndex(3)  MOI.VariableIndex(5)

julia> MOI.add_constraint(
           model,
           MOI.VectorOfVariables([t; vec(X)]),
           MOI.LogDetConeSquare(2),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.LogDetConeSquare}(1)
```

source

`MathOptInterface.RootDetConeTriangle` – Type.

```
RootDetConeTriangle(side_dimension::Int)
```

The root-determinant cone $\{(t, X) \in \mathbb{R}^{1+d(d+1)/2} : t \leq \det(X)^{1/d}\}$, where the matrix X is represented in the same symmetric packed format as in the `PositiveSemidefiniteConeTriangle`.

The non-negative argument `side_dimension` is the side dimension of the matrix X, that is, its number of rows or columns.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> t = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> X = MOI.add_variables(model, 3);

julia> MOI.add_constraint(
           model,
           MOI.VectorOfVariables([t; X]),
           MOI.RootDetConeTriangle(2),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.RootDetConeTriangle}(1)
```

source

`MathOptInterface.RootDetConeSquare` – Type.

```
RootDetConeSquare(side_dimension::Int)
```

The root-determinant cone $\{(t, X) \in \mathbb{R}^{1+d^2} : t \leq \det(X)^{1/d}, X \text{ symmetric}\}$, where the matrix X is represented in the same format as `PositiveSemidefiniteConeSquare`.

Similarly to `PositiveSemidefiniteConeSquare`, constraints are added to ensure that X is symmetric.

The non-negative argument `side_dimension` is the side dimension of the matrix X, that is, its number of rows or columns.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> t = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> X = reshape(MOI.add_variables(model, 4), 2, 2)
2×2 Matrix{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(2)  MOI.VariableIndex(4)
 MOI.VariableIndex(3)  MOI.VariableIndex(5)

julia> MOI.add_constraint(
           model,
           MOI.VectorOfVariables([t; vec(X)]),
           MOI.RootDetConeSquare(2),
       )
MathOptInterface.ConstraintIndex{MathOptInterface.VectorOfVariables,
↪  MathOptInterface.RootDetConeSquare}(1)
```

source

# Chapter 19

# Models

## 19.1 Attribute interface

`MathOptInterface.is_set_by_optimize` – Function.

```
is_set_by_optimize(::AnyAttribute)
```

Return a `Bool` indicating whether the value of the attribute is set during an `optimize!` call, that is, the attribute is used to query the result of the optimization.

If an attribute can be set by the user, define `is_copyable` instead.

An attribute cannot be both `is_copyable` and `is_set_by_optimize`.

**Default fallback**

This function returns `false` by default so it should be implemented for attributes that are set by `optimize!`.

**Undefined behavior**

Querying the value of the attribute that `is_set_by_optimize` before a call to `optimize!` is undefined and depends on solver-specific behavior.

**Example**

```
julia> MOI.is_set_by_optimize(MOI.ObjectiveValue())
true

julia> MOI.is_set_by_optimize(MOI.VariableName())
false
```

source

`MathOptInterface.is_copyable` – Function.

```
is_copyable(::AnyAttribute)
```

Return a `Bool` indicating whether the value of the attribute may be copied during `copy_to` using `set`.

If an attribute `is_copyable`, then it cannot be modified by the optimizer, and `get` must always return the value that was `set` by the user.

If an attribute is the result of an optimization, define `is_set_by_optimize` instead.

An attribute cannot be both `is_set_by_optimize` and `is_copyable`.

**Default fallback**

By default `is_copyable(attr)` returns `!is_set_by_optimize(attr)`, which is most probably `true`.

If an attribute should not be copied, define `is_copyable(::MyAttribute) = false`.

source

`MathOptInterface.get` – Function.

```
MOI.get(b::AbstractBridge, ::MOI.NumberOfVariables)::Int64
```

Return the number of variables created by the bridge `b` in the model.

See also `MOI.NumberOfConstraints`.

**Implementation notes**

- There is a default fallback, so you need only implement this if the bridge adds new variables.

source

```
MOI.get(b::AbstractBridge, ::MOI.ListOfVariableIndices)
```

Return the list of variables created by the bridge `b`.

See also `MOI.ListOfVariableIndices`.

**Implementation notes**

- There is a default fallback, so you need only implement this if the bridge adds new variables.

source

```
MOI.get(b::AbstractBridge, ::MOI.NumberOfConstraints{F,S})::Int64 where {F,S}
```

Return the number of constraints of the type F-in-S created by the bridge `b`.

See also `MOI.NumberOfConstraints`.

**Implementation notes**

- There is a default fallback, so you need only implement this for the constraint types returned by `added_constraint_types`.

source

```
MOI.get(b::AbstractBridge, ::MOI.ListOfConstraintIndices{F,S}) where {F,S}
```

Return a `Vector{ConstraintIndex{F,S}}` with indices of all constraints of type F-in-S created by the bride `b`.

See also `MOI.ListOfConstraintIndices`.

**Implementation notes**

- There is a default fallback, so you need only implement this for the constraint types returned by `added_constraint_types`.

source

```
function MOI.get(
    model::MOI.ModelLike,
    attr::MOI.AbstractConstraintAttribute,
    bridge::AbstractBridge,
)
```

Return the value of the attribute `attr` of the model `model` for the constraint bridged by `bridge`.

source

```
get(optimizer::AbstractOptimizer, attr::AbstractOptimizerAttribute)
```

Return an attribute `attr` of the optimizer `optimizer`.

```
get(model::ModelLike, attr::AbstractModelAttribute)
```

Return an attribute `attr` of the model `model`.

```
get(model::ModelLike, attr::AbstractVariableAttribute, v::VariableIndex)
```

If the attribute `attr` is set for the variable `v` in the model `model`, return its value, return `nothing` otherwise. If the attribute `attr` is not supported by `model` then an error should be thrown instead of returning `nothing`.

```
get(model::ModelLike, attr::AbstractVariableAttribute, v::Vector{VariableIndex})
```

Return a vector of attributes corresponding to each variable in the collection `v` in the model `model`.

```
get(model::ModelLike, attr::AbstractConstraintAttribute, c::ConstraintIndex)
```

If the attribute `attr` is set for the constraint `c` in the model `model`, return its value, return `nothing` otherwise. If the attribute `attr` is not supported by `model` then an error should be thrown instead of returning `nothing`.

```
get(
    model::ModelLike,
    attr::AbstractConstraintAttribute,
    c::Vector{ConstraintIndex{F,S}},
) where {F,S}
```

Return a vector of attributes corresponding to each constraint in the collection `c` in the model `model`.

```
get(model::ModelLike, ::Type{VariableIndex}, name::String)
```

If a variable with name `name` exists in the model `model`, return the corresponding index, otherwise return `nothing`. Errors if two variables have the same name.

```
get(
    model::ModelLike,
    ::Type{ConstraintIndex{F,S}},
    name::String,
) where {F,S}
```

If an F-in-S constraint with name `name` exists in the model `model`, return the corresponding index, otherwise return `nothing`. Errors if two constraints have the same name.

```
get(model::ModelLike, ::Type{ConstraintIndex}, name::String)
```

If any constraint with name `name` exists in the model `model`, return the corresponding index, otherwise return `nothing`. This version is available for convenience but may incur a performance penalty because it is not type stable. Errors if two constraints have the same name.

source

`MathOptInterface.get!` – Function.

```
get!(output, model::ModelLike, args...)
```

An in-place version of `get`.

The signature matches that of `get` except that the result is placed in the vector `output`.

source

`MathOptInterface.set` – Function.

```
function MOI.set(
    model::MOI.ModelLike,
    attr::MOI.AbstractConstraintAttribute,
    bridge::AbstractBridge,
    value,
)
```

Set the value of the attribute `attr` of the model `model` for the constraint bridged by `bridge`.

source

```
set(optimizer::AbstractOptimizer, attr::AbstractOptimizerAttribute, value)
```

Assign `value` to the attribute `attr` of the optimizer `optimizer`.

```
set(model::ModelLike, attr::AbstractModelAttribute, value)
```

Assign `value` to the attribute `attr` of the model `model`.

```
set(model::ModelLike, attr::AbstractVariableAttribute, v::VariableIndex, value)
```

Assign `value` to the attribute `attr` of variable `v` in model `model`.

```
set(
    model::ModelLike,
    attr::AbstractVariableAttribute,
    v::Vector{VariableIndex},
    vector_of_values,
)
```

Assign a value respectively to the attribute `attr` of each variable in the collection `v` in model `model`.

```
set(
    model::ModelLike,
    attr::AbstractConstraintAttribute,
    c::ConstraintIndex,
    value,
)
```

Assign a value to the attribute `attr` of constraint `c` in model `model`.

```
set(
    model::ModelLike,
    attr::AbstractConstraintAttribute,
    c::Vector{ConstraintIndex{F,S}},
    vector_of_values,
) where {F,S}
```

Assign a value respectively to the attribute `attr` of each constraint in the collection `c` in model `model`.

An `UnsupportedAttribute` error is thrown if `model` does not support the attribute `attr` (see `supports`) and a `SetAttributeNotAllowed` error is thrown if it supports the attribute `attr` but it cannot be set.

```
set(
    model::ModelLike,
    ::ConstraintSet,
    c::ConstraintIndex{F,S},
    set::S,
) where {F,S}
```

Change the set of constraint `c` to the new set `set` which should be of the same type as the original set.

```
set(
    model::ModelLike,
    ::ConstraintFunction,
    c::ConstraintIndex{F,S},
    func::F,
) where {F,S}
```

Replace the function in constraint `c` with `func`. `F` must match the original function type used to define the constraint.

**Note**

Setting the constraint function is not allowed if `F` is `VariableIndex`; a `SettingVariableIndexNotAllowed` error is thrown instead. This is because, it would require changing the index `c` since the index of `VariableIndex` constraints must be the same as the index of the variable.

source

`MathOptInterface.supports` – Function.

```
MOI.supports(
    model::MOI.ModelLike,
    attr::MOI.AbstractConstraintAttribute,
    BT::Type{<:AbstractBridge},
)
```

Return a `Bool` indicating whether `BT` supports setting `attr` to `model`.

source

```
supports(model::ModelLike, sub::AbstractSubmittable)::Bool
```

Return a `Bool` indicating whether `model` supports the submittable `sub`.

```
supports(model::ModelLike, attr::AbstractOptimizerAttribute)::Bool
```

Return a `Bool` indicating whether `model` supports the optimizer attribute `attr`. That is, it returns `false` if `copy_to(model, src)` shows a warning in case `attr` is in the `ListOfOptimizerAttributesSet` of `src`; see `copy_to` for more details on how unsupported optimizer attributes are handled in copy.

```
supports(model::ModelLike, attr::AbstractModelAttribute)::Bool
```

Return a `Bool` indicating whether `model` supports the model attribute `attr`. That is, it returns `false` if `copy_to(model, src)` cannot be performed in case `attr` is in the `ListOfModelAttributesSet` of `src`.

```
supports(
    model::ModelLike,
    attr::AbstractVariableAttribute,
    ::Type{VariableIndex},
)::Bool
```

Return a `Bool` indicating whether `model` supports the variable attribute `attr`. That is, it returns `false` if `copy_to(model, src)` cannot be performed in case `attr` is in the `ListOfVariableAttributesSet` of `src`.

```
supports(
    model::ModelLike,
    attr::AbstractConstraintAttribute,
    ::Type{ConstraintIndex{F,S}},
)::Bool where {F,S}
```

Return a `Bool` indicating whether `model` supports the constraint attribute `attr` applied to an F-in-S constraint. That is, it returns `false` if `copy_to(model, src)` cannot be performed in case `attr` is in the `ListOfConstraintAttributesSet` of `src`.

For all five methods, if the attribute is only not supported in specific circumstances, it should still return `true`.

Note that `supports` is only defined for attributes for which `is_copyable` returns `true` as other attributes do not appear in the list of attributes set obtained by `ListOfXXXAttributesSet`.

source

`MathOptInterface.attribute_value_type` – Function.

```
attribute_value_type(attr::AnyAttribute)
```

Given an attribute `attr`, return the type of value expected by `get`, or returned by `set`.

**Notes**

- Only implement this if it make sense to do so. If un-implemented, the default is `Any`.

source

## 19.2 Model interface

`MathOptInterface.ModelLike` – Type.

```
ModelLike
```

Abstract supertype for objects that implement the "Model" interface for defining an optimization problem.

source

`MathOptInterface.is_empty` – Function.

```
is_empty(model::ModelLike)
```

Returns `false` if the `model` has any model attribute set or has any variables or constraints.

Note that an empty model can have optimizer attributes set.

source

`MathOptInterface.empty!` – Function.

```
empty!(model::ModelLike)
```

Empty the model, that is, remove all variables, constraints and model attributes but not optimizer attributes.

source

`MathOptInterface.write_to_file` – Function.

```
write_to_file(model::ModelLike, filename::String; kwargs...)
```

Write the current model to the file at `filename`.

Supported file types depend on the model type.

Additional keyword arguments are passed to the `Model` constructor of the relevant file format. See, for example, `FileFormats.LP.Model` and `FileFormats.MPS.Model`.

**Example**

```
julia> model = MOI.Utilities.Model{Int}();

julia> x, _ = MOI.add_constrained_variable(model, MOI.Interval(2, 3));

julia> MOI.set(model, MOI.VariableName(), x, "x");

julia> filename = joinpath(tempdir(), "model.lp");

julia> MOI.write_to_file(model, filename; coefficient_type = Int);

julia> print(read(filename, String))
minimize
obj: 0
```

```
subject to
Bounds
2 <= x <= 3
End
```

source

`MathOptInterface.read_from_file` – Function.

```
read_from_file(model::ModelLike, filename::String; kwargs...)
```

Read the file `filename` into the model `model`. If `model` is non-empty, this may throw an error.

Supported file types depend on the model type.

Additional keyword arguments are passed to the `Model` constructor of the relevant file format. See, for example, `FileFormats.LP.Model` and `FileFormats.MPS.Model`.

**Note**

Once the contents of the file are loaded into the model, you can query the variables via `MOI.get(model, MOI.ListOfVariableIndices())`. However, some filetypes, such as LP files, do not maintain an explicit ordering of the variables. Therefore, the returned list may be in an arbitrary order.

To avoid depending on the order of the indices, look up each variable index by name using `MOI.get(model, MOI.VariableIndex, "name")`.

**Example**

```
julia> model = MOI.Utilities.Model{Int}();

julia> x, _ = MOI.add_constrained_variable(model, MOI.GreaterThan(2));

julia> MOI.set(model, MOI.VariableName(), x, "x");

julia> filename = joinpath(tempdir(), "model.lp");

julia> MOI.write_to_file(model, filename; coefficient_type = Int);

julia> new_model = MOI.Utilities.Model{Int}();

julia> MOI.read_from_file(new_model, filename; coefficient_type = Int);

julia> print(new_model)
Minimize ScalarAffineFunction{Int64}:
 (0)

Subject to:

VariableIndex-in-GreaterThan{Int64}
 x >= (2)

julia> MOI.get(new_model, MOI.VariableIndex, "x")
MOI.VariableIndex(1)
```

source

MathOptInterface.supports_incremental_interface – Function.

```
supports_incremental_interface(model::ModelLike)
```

Return a Bool indicating whether model supports building incrementally via add_variable and add_constraint.

The main purpose of this function is to determine whether a model can be loaded into model incrementally or whether it should be cached and copied at once instead.

source

MathOptInterface.copy_to – Function.

```
copy_to(dest::ModelLike, src::ModelLike)::IndexMap
```

Copy the model from src into dest.

The target dest is emptied, and all previous indices to variables and constraints in dest are invalidated.

Returns an IndexMap object that translates variable and constraint indices from the src model to the corresponding indices in the dest model.

**Notes**

- If a constraint that in src is not supported by dest, then an UnsupportedConstraint error is thrown.
- If an AbstractModelAttribute, AbstractVariableAttribute, or AbstractConstraintAttribute is set in src but not supported by dest, then an UnsupportedAttribute error is thrown.

AbstractOptimizerAttributes are not copied to the dest model.

**IndexMap**

Implementations of copy_to must return an IndexMap. For technical reasons, this type is defined in the Utilities submodule as MOI.Utilities.IndexMap. However, since it is an integral part of the MOI API, we provide MOI.IndexMap as an alias.

**Example**

```
# Given empty `ModelLike` objects `src` and `dest`.

x = add_variable(src)

is_valid(src, x)   # true
is_valid(dest, x)  # false (`dest` has no variables)

index_map = copy_to(dest, src)
is_valid(dest, x) # false (unless index_map[x] == x)
is_valid(dest, index_map[x]) # true
```

source

MathOptInterface.IndexMap – Type.

```
IndexMap()
```

The dictionary-like object returned by `copy_to`.

**IndexMap**

Implementations of `copy_to` must return an `IndexMap`. For technical reasons, the `IndexMap` type is defined in the Utilities submodule as `MOI.Utilities.IndexMap`. However, since it is an integral part of the MOI API, we provide this `MOI.IndexMap` as an alias.

source

## 19.3 Model attributes

`MathOptInterface.AbstractModelAttribute` – Type.

```
AbstractModelAttribute
```

Abstract supertype for attribute objects that can be used to set or get attributes (properties) of the model.

source

`MathOptInterface.Name` – Type.

```
Name()
```

An `AbstractModelAttribute` for the string identifying the model.

It has a default value of `""` if not set.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.Name)::String
MOI.set(::Optimizer, ::MOI.Name, ::String)::Nothing
MOI.supports(::Optimizer, ::MOI.Name)::Bool
```

source

`MathOptInterface.ObjectiveFunction` – Type.

```
ObjectiveFunction{F<:AbstractScalarFunction}()
```

An `AbstractModelAttribute` for the objective function which has a type `F<:AbstractScalarFunction`.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(
    ::Optimizer,
    ::MOI.ObjectiveFunction{F},
)::F where {F<:MOI.AbstractFunction}
MOI.set(
    ::Optimizer,
    ::MOI.ObjectiveFunction{F},
    ::F,
)::F where {F<:MOI.AbstractFunction}
MOI.supports(::Optimizer, ::MOI.ObjectiveFunction{<:MOI.AbstractFunction})::Bool
```

When implementing get, F may to be equivalent but not necessarily identical to the function type set by the user. If the objective function cannot be converted to F, an `InexactError` must be thrown.

source

`MathOptInterface.ObjectiveFunctionType` – Type.

```
ObjectiveFunctionType()
```

An `AbstractModelAttribute` for the type F of the objective function set using the `ObjectiveFunction{F}` attribute.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(
    ::Optimizer,
    ::MOI.ObjectiveFunctionType,
)::Type{<:MOI.AbstractFunction}
```

They should not implement `set` or `supports`.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> MOI.set(model, MOI.ObjectiveFunction{MOI.VariableIndex}(), x)

julia> MOI.get(model, MOI.ObjectiveFunctionType())
MathOptInterface.VariableIndex
```

source

`MathOptInterface.ObjectiveSense` – Type.

```
ObjectiveSense()
```

An `AbstractModelAttribute` for the objective sense of the objective function, which must be an `OptimizationSense`.

The default is `FEASIBILITY_SENSE`.

**Interaction with `ObjectiveFunction`**

Setting the sense to `FEASIBILITY_SENSE` unsets the `ObjectiveFunction` attribute. That is, if you first set `ObjectiveFunction` and then set `ObjectiveSense` to be `FEASIBILITY_SENSE`, no objective function will be passed to the solver.

In addition, some reformulations of `ObjectiveFunction` via bridges rely on the value of `ObjectiveSense`. Therefore, you should set `ObjectiveSense` before setting `ObjectiveFunction`.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.ObjectiveSense)::MOI.OptimizationSense
MOI.set(::Optimizer, ::MOI.ObjectiveSense, ::MOI.OptimizationSense)::Nothing
MOI.supports(::Optimizer, ::MOI.ObjectiveSense)::Bool
```

source

`MathOptInterface.OptimizationSense` – Type.

```
OptimizationSense
```

An Enum for the value of the `ObjectiveSense` attribute.

**Values**

**`MIN_SENSE`**

The goal is to minimize the objective function.

**`MAX_SENSE`**

The goal is to maximize the objective function.

**`FEASIBILITY_SENSE`**

The model does not have an objective function.

source

`MathOptInterface.MIN_SENSE` – Constant.

```
MIN_SENSE::OptimizationSense
```

An instance of the `OptimizationSense` enum.

**About**

The goal is to minimize the objective function.

source

`MathOptInterface.MAX_SENSE` – Constant.

```
MAX_SENSE::OptimizationSense
```

An instance of the `OptimizationSense` enum.

**About**

The goal is to maximize the objective function.

source

`MathOptInterface.FEASIBILITY_SENSE` – Constant.

```
FEASIBILITY_SENSE::OptimizationSense
```

An instance of the `OptimizationSense` enum.

**About**

The model does not have an objective function.

source

`MathOptInterface.NumberOfVariables` – Type.

```
NumberOfVariables()
```

An `AbstractModelAttribute` for the number of variables in the model.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.NumberOfVariables)::Int64
```

They should not implement `set` or `supports`.

source

`MathOptInterface.ListOfVariableIndices` – Type.

```
ListOfVariableIndices()
```

An `AbstractModelAttribute` for querying the `Vector{MOI.VariableIndex}` of all `[MOI.VariableIndex]` present in the model.

**Order**

The variables must be returned in the order in which they were added to the model.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.ListOfVariableIndices)::Vector{MOI.VariableIndex}
```

They should not implement `set` or `supports`.

source

`MathOptInterface.ListOfConstraintTypesPresent` – Type.

```
ListOfConstraintTypesPresent()
```

An `AbstractModelAttribute` for the list of tuples of the form `(F, S)`, indicating that the attribute `NumberOfConstraints{F,S}` has a value greater than zero.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(
    ::Optimizer,
    ::MOI.ListOfConstraintTypesPresent,
)::Vector{Tuple{Type,Type}}
```

They should not implement `set` or `supports`.

source

`MathOptInterface.NumberOfConstraints` – Type.

```
NumberOfConstraints{F,S}()
```

An `AbstractModelAttribute` for querying the number of constraints of the type F-in-S present in the model.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(
    ::Optimizer,
    ::MOI.NumberOfConstraints{F,S},
)::Int64 where {F<:MOI.AbstractFunction,MOI.AbstractSet}
```

They should not implement `set` or `supports`.

source

`MathOptInterface.ListOfConstraintIndices` – Type.

```
ListOfConstraintIndices{F,S}()
```

An AbstractModelAttribute for the Vector{MOI.ConstraintIndex{F,S}} of all constraint indices of type F-in-S in the model.

**Order**

The constraints must be returned in the order in which they were added to the model.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(
    ::Optimizer,
    ::MOI.ListOfConstraintIndices{F,S},
)::Vector{MOI.ConstraintIndex{F,S}} where {F<:MOI.AbstractFunction,MOI.AbstractSet}
```

They should not implement set or supports.

source

MathOptInterface.ListOfOptimizerAttributesSet – Type.

```
ListOfOptimizerAttributesSet()
```

An AbstractOptimizerAttribute for the Vector{AbstractOptimizerAttribute} of all optimizer attributes that were set.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(
    ::Optimizer,
    ::MOI.ListOfOptimizerAttributesSet,
)::Vector{MOI.AbstractOptimizerAttribute}
```

They should not implement set or supports.

source

MathOptInterface.ListOfModelAttributesSet – Type.

```
ListOfModelAttributesSet()
```

An AbstractModelAttribute for the Vector{AbstractModelAttribute} of all model attributes attr such that:

1. is_copyable(attr) returns true, and
2. the attribute was set to the model

source

MathOptInterface.ListOfVariableAttributesSet – Type.

```
ListOfVariableAttributesSet()
```

An `AbstractModelAttribute` for the `Vector{AbstractVariableAttribute}` of all variable attributes `attr` such that:

1. `is_copyable(attr)` returns `true`
2. the attribute was set for at least one variable in the model

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(
    ::Optimizer,
    ::MOI.ListOfVariableAttributesSet,
)::Vector{MOI.AbstractVariableAttribute}
```

They should not implement `set` or `supports`.

source

`MathOptInterface.ListOfVariablesWithAttributeSet` – Type.

```
ListOfVariablesWithAttributeSet(attr::AbstractVariableAttribute)
```

An `AbstractModelAttribute` for the `Vector{MOI.VariableIndex}` of all variables with the attribute `attr` set.

The returned list may not be minimal, so some elements may have their default value set.

**Note**

This is an optional attribute to implement. The default fallback is to get `ListOfVariableIndices`.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(
    ::Optimizer,
    ::MOI.ListOfVariablesWithAttributeSet{<:MOI.AbstractVariableAttribute},
)::Vector{MOI.VariableIndex}
```

They should not implement `set` or `supports`.

source

`MathOptInterface.ListOfConstraintAttributesSet` – Type.

```
ListOfConstraintAttributesSet{F, S}()
```

An AbstractModelAttribute for the Vector{AbstractConstraintAttribute} of all constraint attributes attr such that:

1. is_copyable(attr) returns true and
2. the attribute was set to F-in-S constraints.

**Note**

The attributes ConstraintFunction and ConstraintSet should not be included in the list even if then have been set with set.

source

MathOptInterface.ListOfConstraintsWithAttributeSet – Type.

```
ListOfConstraintsWithAttributeSet{F,S}(attr:AbstractConstraintAttribute)
```

An AbstractModelAttribute for the Vector{ConstraintIndex{F,S}} of all constraints with the attribute attr set.

The returned list may not be minimal, so some elements may have their default value set.

**Note**

This is an optional attribute to implement. The default fallback is to get ListOfConstraintIndices.

source

MathOptInterface.UserDefinedFunction – Type.

```
UserDefinedFunction(name::Symbol, arity::Int) <: AbstractModelAttribute
```

Set this attribute to register a user-defined function by the name of name with arity arguments.

Once registered, name will appear in ListOfSupportedNonlinearOperators.

You cannot register multiple UserDefinedFunctions with the same name but different arity.

**Value type**

The value to be set is a tuple containing one, two, or three functions to evaluate the function, the first-order derivative, and the second-order derivative respectively. Both derivatives are optional, but if you pass the second-order derivative you must also pass the first-order derivative.

For univariate functions with arity == 1, the functions in the tuple must have the form:

- f(x::T)::T: returns the value of the function at x
- ∇f(x::T)::T: returns the first-order derivative of f with respect to x
- ∇²f(x::T)::T: returns the second-order derivative of f with respect to x.

For multivariate functions with arity > 1, the functions in the tuple must have the form:

- f(x::T...)::T: returns the value of the function at x

- `∇f(g::AbstractVector{T}, x::T...)::Nothing`: fills the components of `g`, with `g[i]` being the first-order partial derivative of `f` with respect to `x[i]`
- `∇²f(H::AbstractMatrix{T}, x::T...)::Nothing`: fills the non-zero components of `H`, with `H[i, j]` being the second-order partial derivative of `f` with respect to `x[i]` and then `x[j]`. `H` is initialized to the zero matrix, so you do not need to set any zero elements.

**Example**

```
julia> f(x, y) = x^2 + y^2
f (generic function with 1 method)

julia> function ∇f(g, x, y)
           g .= 2 * x, 2 * y
           return
       end
∇f (generic function with 1 method)

julia> function ∇²f(H, x...)
           H[1, 1] = H[2, 2] = 2.0
           return
       end
∇²f (generic function with 1 method)

julia> model = MOI.Utilities.UniversalFallback(MOI.Utilities.Model{Float64}());

julia> MOI.set(model, MOI.UserDefinedFunction(:f, 2), (f,))

julia> MOI.set(model, MOI.UserDefinedFunction(:g, 2), (f, ∇f))

julia> MOI.set(model, MOI.UserDefinedFunction(:h, 2), (f, ∇f, ∇²f))

julia> x = MOI.add_variables(model, 2)
2-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(1)
 MOI.VariableIndex(2)

julia> MOI.set(model, MOI.ObjectiveSense(), MOI.MIN_SENSE)

julia> obj_f = MOI.ScalarNonlinearFunction(:f, Any[x[1], x[2]])
f(MOI.VariableIndex(1), MOI.VariableIndex(2))

julia> MOI.set(model, MOI.ObjectiveFunction{typeof(obj_f)}(), obj_f)

julia> print(model)
Minimize ScalarNonlinearFunction:
 f(v[1], v[2])

Subject to:
```

source

`MathOptInterface.ListOfSupportedNonlinearOperators` – Type.

```
ListOfSupportedNonlinearOperators() <: AbstractModelAttribute
```

When queried with `get`, return a `Vector{Symbol}` listing the operators supported by the model. These operators may appear in the head field of `ScalarNonlinearFunction`.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(
    ::Optimizer,
    ::MOI.ListOfSupportedNonlinearOperators,
)::Vector{Symbol}
```

They should not implement `set` or `supports`.

source

`MathOptInterface.ConstraintBridgingCost` – Type.

```
ConstraintBridgingCost{F<:AbstractFunction,S<:AbstractSet}()::Float64
```

An `AbstractModelAttribute` for the cost or bridging a constraint of type `F` in `S`.

**Implementation**

There is a default fallback which returns `0.0` if `supports_constraint` is `true` and `Inf` if `false`.

Therefore, you should implement this method only if your optimizer does something unusual.

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.ConstraintBridgingCost{F,S})::Float64
```

They should not implement `set` or `supports`.

source

`MathOptInterface.VariableBridgingCost` – Type.

```
VariableBridgingCost{S<:AbstractSet}()::Float64
```

An `AbstractModelAttribute` for the cost or bridging a variable in a set of type `S`.

**Implementation**

There is a default fallback which returns `0.0` if `supports_add_constrained_variable` or `supports_add_constrained_variab` is `true` and `Inf` if `false`.

Therefore, you should implement this method only if your optimizer does something unusual.

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.VariableBridgingCost{S})::Float64
```

They should not implement `set` or `supports`.

source

## 19.4 Optimizer interface

`MathOptInterface.AbstractOptimizer` – Type.

```
AbstractOptimizer <: ModelLike
```

Abstract supertype for objects representing an instance of an optimization problem tied to a particular solver. This is typically a solver's in-memory representation. In addition to `ModelLike`, `AbstractOptimizer` objects let you solve the model and query the solution.

source

`MathOptInterface.OptimizerWithAttributes` – Type.

```
struct OptimizerWithAttributes
    optimizer_constructor
    params::Vector{Pair{AbstractOptimizerAttribute,<:Any}}
end
```

Object grouping an optimizer constructor and a list of optimizer attributes. Instances are created with `instantiate`.

source

`MathOptInterface.optimize!` – Function.

```
optimize!(optimizer::AbstractOptimizer)
```

Optimize the problem contained in `optimizer`.

Before calling `optimize!`, the problem should first be constructed using the incremental interface (see `supports_incremental_interface`) or `copy_to`.

source

`MathOptInterface.optimize!` – Method.

```
optimize!(dest::AbstractOptimizer, src::ModelLike)::Tuple{IndexMap,Bool}
```

A "one-shot" call that copies the problem from `src` into `dest` and then uses `dest` to optimize the problem.

Returns a tuple of an `IndexMap` and a `Bool` `copied`.

- The IndexMap object translates variable and constraint indices from the src model to the corresponding indices in the dest optimizer. See copy_to for details.
- If copied == true, src was copied to dest and then cached, allowing incremental modification if supported by the solver.
- If copied == false, a cache of the model was not kept in dest. Therefore, only the solution information (attributes for which is_set_by_optimize is true) is available to query.

**Note**

The main purpose of optimize! method with two arguments is for use in Utilities.CachingOptimizer.

**Relationship to the single-argument optimize!**

The default fallback of optimize!(dest::AbstractOptimizer, src::ModelLike) is

```
function optimize!(dest::AbstractOptimizer, src::ModelLike)
    index_map = copy_to(dest, src)
    optimize!(dest)
    return index_map, true
end
```

Therefore, subtypes of AbstractOptimizer should either implement this two-argument method, or implement both copy_to(::Optimizer, ::ModelLike) and optimize!(::Optimizer).

source

MathOptInterface.instantiate – Function.

```
instantiate(
    optimizer_constructor,
    with_cache_type::Union{Nothing,Type} = nothing,
    with_bridge_type::Union{Nothing,Type} = nothing,
)
```

Create an instance of an optimizer by either:

- calling optimizer_constructor.optimizer_constructor() and setting the parameters in optimizer_constructor.p if optimizer_constructor is a OptimizerWithAttributes
- calling optimizer_constructor() if optimizer_constructor is callable.

**withcachetype**

If with_cache_type is not nothing, then the optimizer is wrapped in a Utilities.CachingOptimizer to store a cache of the model. This is most useful if the optimizer you are constructing does not support the incremental interface (see supports_incremental_interface).

**withbridgetype**

If with_bridge_type is not nothing, the optimizer is wrapped in a Bridges.full_bridge_optimizer, enabling all the bridges defined in the MOI.Bridges submodule with coefficient type with_bridge_type.

In addition, if the optimizer created by `optimizer_constructor` does not support the incremental interface (see `supports_incremental_interface`), then, irrespective of `with_cache_type`, the optimizer is wrapped in a `Utilities.CachingOptimizer` to store a cache of the bridged model.

If `with_cache_type` and `with_bridge_type` are both not `nothing`, then they must be the same type.

source

`MathOptInterface.default_cache` – Function.

```
default_cache(optimizer::ModelLike, ::Type{T}) where {T}
```

Return a new instance of the default model type to be used as cache for `optimizer` in a `Utilities.CachingOptimizer` for holding constraints of coefficient type `T`. By default, this returns `Utilities.UniversalFallback(Utilities.Model{T}())`. If copying from a instance of a given model type is faster for `optimizer` then a new method returning an instance of this model type should be defined.

source

## 19.5 Optimizer attributes

`MathOptInterface.AbstractOptimizerAttribute` – Type.

```
AbstractOptimizerAttribute
```

Abstract supertype for attribute objects that can be used to set or get attributes (properties) of the optimizer.

**Notes**

The difference between `AbstractOptimizerAttribute` and `AbstractModelAttribute` lies in the behavior of `is_empty`, `empty!` and `copy_to`. Typically optimizer attributes affect only how the model is solved.

source

`MathOptInterface.SolverName` – Type.

```
SolverName()
```

An `AbstractOptimizerAttribute` for the string identifying the solver.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.SolverName)::String
```

They should not implement `set` or `supports`.

source

`MathOptInterface.SolverVersion` – Type.

```
SolverVersion()
```

An `AbstractOptimizerAttribute` for the string identifying the version of the solver.

**Versioning systems**

For solvers supporting semantic versioning, the `SolverVersion` should be a string of the form "vMAJOR.MINOR.PATCH", so that it can be converted to a Julia `VersionNumber` (for example, 'VersionNumber("v1.2.3")).

We do not require Semantic Versioning because some solvers use alternate versioning systems. For example, CPLEX uses Calendar Versioning, so `SolverVersion` will return a string like `"202001"`.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.SolverVersion)::String
```

They should not implement `set` or `supports`.

source

`MathOptInterface.Silent` – Type.

```
Silent()
```

An `AbstractOptimizerAttribute` for silencing the output of an optimizer.

When `set` to `true`, this attribute takes precedence over any other attribute controlling verbosity and requires the optimizer to produce no output.

The default value is `false` which has no effect. In this case the verbosity is controlled by other optimizer-specific attributes.

**Value and default**

The provided value must be a `Bool`.

The default value is `false`.

**Note**

Every optimizer should have verbosity on by default. For instance, if a solver has a solver-specific log level attribute, the MOI implementation should set it to 1 by default. If the user sets `Silent` to `true`, then the log level should be set to `0`, even if the user specifically sets a value of log level. If the value of `Silent` is `false` then the log level set to the solver is the value given by the user for this solver-specific parameter or 1 if none is given.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.Silent)::Bool
MOI.set(::Optimizer, ::MOI.Silent, ::Bool)::Nothing
MOI.supports(::Optimizer, ::MOI.Silent)::Bool
```

source

MathOptInterface.TimeLimitSec – Type.

```
TimeLimitSec()
```

An AbstractOptimizerAttribute for setting a time limit (in seconds) for a call to optimize!.

**Value and default**

The provided limit must be a Union{Nothing,Real}.

When set to nothing, it deactivates the time limit.

The default value is nothing.

**TerminationStatus**

The optimizer may stop when the SolveTimeSec is larger than the TimeLimitSec. If stopped because of this limit, the TerminationStatus must be TIME_LIMIT.

Note that most optimizers do not strictly respect a time limit. Instead, they terminate at the first convenient time after the time limit has been exceeded. Thus, you may find that the SolveTimeSec exceeds the TimeLimitSec by a few seconds.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.TimeLimitSec)::Union{Nothing,Float64}
MOI.set(::Optimizer, ::MOI.TimeLimitSec, ::Union{Nothing,Real})::Nothing
MOI.supports(::Optimizer, ::MOI.TimeLimitSec)::Bool
```

source

MathOptInterface.ObjectiveLimit – Type.

```
ObjectiveLimit()
```

An AbstractOptimizerAttribute for setting a limit on the objective value.

**Value and default**

The provided limit must be a Union{Nothing,Real}.

When set to nothing, the limit reverts to the solver's default.

The default value is nothing.

**TerminationStatus**

The solver may stop when the ObjectiveValue is better (lower for minimization, higher for maximization) than the ObjectiveLimit. If stopped, because of this limit, the TerminationStatus should be OBJECTIVE_LIMIT.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.ObjectiveLimit)::Union{Nothing,Float64}
MOI.set(::Optimizer, ::MOI.ObjectiveLimit, ::Union{Nothing,Real})::Nothing
MOI.supports(::Optimizer, ::MOI.ObjectiveLimit)::Bool
```

source

`MathOptInterface.SolutionLimit` – Type.

```
SolutionLimit()
```

An `AbstractOptimizerAttribute` for setting a limit on the number of available feasible solutions.

**Value and default**

The provided limit must be a `Union{Nothing,Int}`.

When set to `nothing`, the limit reverts to the solver's default.

The default value is `nothing`.

**Termination criteria**

The solver may stop when the `ResultCount` is larger than or equal to the `SolutionLimit`. If stopped because of this attribute, the `TerminationStatus` must be `SOLUTION_LIMIT`.

**Solution quality**

The quality of the available solutions is solver-dependent. The set of resulting solutions is not guaranteed to contain an optimal solution.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.SolutionLimit)::Union{Nothing,Int}
MOI.set(::Optimizer, ::MOI.SolutionLimit, ::Union{Nothing,Int})::Nothing
MOI.supports(::Optimizer, ::MOI.SolutionLimit)::Bool
```

source

`MathOptInterface.NodeLimit` – Type.

```
NodeLimit()
```

An `AbstractOptimizerAttribute` for setting a limit on the number of branch-and-bound nodes explored by a mixed-integer program (MIP) solver.

**Value and default**

The provided limit must be a `Union{Nothing,Int}`.

When set to `nothing`, the limit reverts to the solver's default.

The default value is `nothing`.

**Termination criteria**

The solver may stop when the NodeCount is larger than or equal to the NodeLimit. If stopped because of this attribute, the TerminationStatus must be NODE_LIMIT.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.NodeLimit)::Union{Nothing,Int}
MOI.set(::Optimizer, ::MOI.NodeLimit, ::Union{Nothing,Int})::Nothing
MOI.supports(::Optimizer, ::MOI.NodeLimit)::Bool
```

source

MathOptInterface.RawOptimizerAttribute – Type.

```
RawOptimizerAttribute(name::String)
```

An AbstractOptimizerAttribute for the solver-specific parameter identified by name.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.RawOptimizerAttribute)::Any
MOI.set(::Optimizer, ::MOI.RawOptimizerAttribute, ::Any)::Nothing
MOI.supports(::Optimizer, ::MOI.RawOptimizerAttribute)::Bool
```

source

MathOptInterface.NumberOfThreads – Type.

```
NumberOfThreads()
```

An AbstractOptimizerAttribute for setting the number of threads used for an optimization.

**Value and default**

The provided value must be nothing or a positive Int.

When set to nothing, the value reverts to the solver's default.

The default value is nothing.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.NumberOfThreads)::Union{Nothing,Int}
MOI.set(::Optimizer, ::MOI.NumberOfThreads, ::Union{Nothing,Int})::Nothing
MOI.supports(::Optimizer, ::MOI.NumberOfThreads)::Bool
```

source

`MathOptInterface.RawSolver` – Type.

```
RawSolver()
```

An `AbstractModelAttribute` for the object that may be used to access a solver-specific API for this optimizer.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.RawSolver)::Any
```

They should not implement `set` or `supports`.

source

`MathOptInterface.AbsoluteGapTolerance` – Type.

```
AbsoluteGapTolerance()
```

An `AbstractOptimizerAttribute` for setting the absolute gap tolerance for an optimization.

**Definition**

The mathematical definition of "absolute gap" and its allowed range are solver-dependent. However, most solvers that implement this attribute will stop once $|f - b| g_{abs}$, where $b$ is the best bound, $f$ is the best feasible objective value, and $g_{abs}$ is the absolute gap.

**Value and default**

The provided value must be a `Union{Nothing,Float64}`.

When set to `nothing`, the limit reverts to the solver's default.

**TerminationStatus**

The optimizer may stop when the absolute difference between `ObjectiveValue` and `ObjectiveBound` is smaller than the `AbsoluteGapTolerance`. If stopped because of this limit, the `TerminationStatus` may be `OPTIMAL`.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.AbsoluteGapTolerance)::Union{Nothing,Float64}
MOI.set(
    ::Optimizer,
    ::MOI.AbsoluteGapTolerance,
    ::Union{Nothing,Float64},
)::Nothing
MOI.supports(::Optimizer, ::MOI.AbsoluteGapTolerance)::Bool
```

source

`MathOptInterface.RelativeGapTolerance` – Type.

```
RelativeGapTolerance()
```

An `AbstractOptimizerAttribute` for setting the relative gap tolerance for an optimization.

**Definition**

The mathematical definition of "relative gap" and its allowed range are solver-dependent. Typically, solvers expect a value between `0.0` and `1.0`.

**Value and default**

The provided value must be a `Union{Nothing,Float64}`.

When set to `nothing`, the limit reverts to the solver's default.

**TerminationStatus**

The optimizer may stop when the `RelativeGap` is smaller than the `RelativeGapTolerance`. If stopped because of this limit, the `TerminationStatus` may be `OPTIMAL`.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.RelativeGapTolerance)::Union{Nothing,Float64}
MOI.set(
    ::Optimizer,
    ::MOI.RelativeGapTolerance,
    ::Union{Nothing,Float64},
)::Nothing
MOI.supports(::Optimizer, ::MOI.RelativeGapTolerance)::Bool
```

source

`MathOptInterface.AutomaticDifferentiationBackend` – Type.

```
AutomaticDifferentiationBackend() <: AbstractOptimizerAttribute
```

An `AbstractOptimizerAttribute` for setting the automatic differentiation backend used by the solver.

The value must be a subtype of `Nonlinear.AbstractAutomaticDifferentiation`.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(
    ::Optimizer,
    ::MOI.AutomaticDifferentiationBackend,
)::MOI.Nonlinear.AbstractAutomaticDifferentiation
MOI.set(
    ::Optimizer,
    ::MOI.AutomaticDifferentiationBackend,
```

```
    ::MOI.Nonlinear.AbstractAutomaticDifferentiation,
)::Nothing
MOI.supports(::Optimizer, ::MOI.AutomaticDifferentiationBackend)::Bool
```

source

List of attributes useful for optimizers

`MathOptInterface.TerminationStatus` – Type.

```
TerminationStatus()
```

An `AbstractModelAttribute` for the `TerminationStatusCode` explaining why the optimizer stopped.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.TerminationStatus)::MOI.TerminationStatusCode
```

They should not implement `set` or `supports`.

source

`MathOptInterface.TerminationStatusCode` – Type.

```
TerminationStatusCode
```

An Enum of possible values for the `TerminationStatus` attribute.

This attribute explains why the optimizer stopped executing in the most recent call to `optimize!`.

**Values**

**`OPTIMIZE_NOT_CALLED`**

The algorithm has not started.

**`OPTIMAL`**

The algorithm found a globally optimal solution.

**`INFEASIBLE`**

The algorithm proved that no primal feasible solution exists.

**`DUAL_INFEASIBLE`**

The algorithm proved that no dual feasible solution exists.

To check if the primal problem is feasible, set the objective sense to `FEASIBILITY_SENSE` and re-solve the problem.

If a primal feasible point does not exist, the original problem is both primal and dual infeasible.

If a primal feasible solution exists, this status typically implies that the problem is unbounded, with some technical exceptions (for example, if the problem is a conic optimization problem in which strong duality does not hold).

The technical exceptions do not apply to linear programs. The combination of `DUAL_INFEASIBLE` and a primal feasible point means that the primal linear program is unbounded.

**`LOCALLY_SOLVED`**

The algorithm converged to a stationary point, local optimal solution, could not find directions for improvement, or otherwise completed its search without global guarantees.

**`LOCALLY_INFEASIBLE`**

The algorithm converged to an infeasible point or otherwise completed its search without finding a feasible solution, without guarantees that no feasible solution exists.

If you know a primal feasible solution exists, use `VariablePrimalStart` to provide a feasible starting point to the solver.

**`INFEASIBLE_OR_UNBOUNDED`**

The algorithm stopped because it proved that the problem is infeasible or unbounded, without distinguishing between the two cases.

To distinguish between the two cases, set the objective sense to `FEASIBILITY_SENSE` and re-solve the problem. If a primal feasible point exists, the original problem is unbounded. If a primal feasible point does not exist, the original problem is infeasible.

**`ALMOST_OPTIMAL`**

The algorithm found a globally optimal solution to relaxed tolerances.

**`ALMOST_INFEASIBLE`**

The algorithm concluded that no feasible solution exists within relaxed tolerances.

**`ALMOST_DUAL_INFEASIBLE`**

The algorithm concluded that no dual bound exists for the problem within relaxed tolerances.

**`ALMOST_LOCALLY_SOLVED`**

The algorithm converged to a stationary point, local optimal solution, or could not find directions for improvement within relaxed tolerances.

**`ITERATION_LIMIT`**

An iterative algorithm stopped after conducting the maximum number of iterations.

**`TIME_LIMIT`**

The algorithm stopped after a user-specified computation time.

This status may be returned in relation to the `TimeLimitSec` attribute, or some other solver-specific attribute.

**`NODE_LIMIT`**

A branch-and-bound algorithm stopped because it explored a maximum number of nodes in the branch-and-bound tree.

This status may be returned in relation to the `NodeLimit` attribute, or some other solver-specific attribute.

**`SOLUTION_LIMIT`**

The algorithm stopped because it found the required number of solutions. This is often used in MIPs to get the solver to return the first feasible solution it encounters.

This status may be returned in relation to the `SolutionLimit` attribute, or some other solver-specific attribute.

**`MEMORY_LIMIT`**

The algorithm stopped because it ran out of memory.

**`OBJECTIVE_LIMIT`**

The algorithm stopped because it found a solution better than a minimum limit set by the user.

This status may be returned in relation to the `ObjectiveLimit` attribute, or some other solver-specific attribute.

**`NORM_LIMIT`**

The algorithm stopped because the norm of an iterate became too large.

This typically means that the primal problem is unbounded, but that the solver could not prove so.

**`OTHER_LIMIT`**

The algorithm stopped due to a limit not covered by one of the `_LIMIT_` statuses above.

**`SLOW_PROGRESS`**

The algorithm stopped because it was unable to continue making progress towards the solution.

**`NUMERICAL_ERROR`**

The algorithm stopped because it encountered unrecoverable numerical error.

**`INVALID_MODEL`**

The algorithm stopped because the model is invalid.

The reason for this return code is solver-specific, but common causes are that the problem has zero variables or constraints, or that the problem data contains an invalid number such as `NaN`.

**`INVALID_OPTION`**

The algorithm stopped because it was provided an invalid option.

**`INTERRUPTED`**

The algorithm stopped because of an interrupt signal.

This typically means that the solver was interrupted by the user with `CTRL+C`.

**`OTHER_ERROR`**

The algorithm stopped because of an error not covered by one of the statuses defined above. Check the solver log for further details.

source

`MathOptInterface.OPTIMIZE_NOT_CALLED` – Constant.

```
OPTIMIZE_NOT_CALLED::TerminationStatusCode
```

An instance of the `TerminationStatusCode` enum.

**About**

The algorithm has not started.

source

`MathOptInterface.OPTIMAL` – Constant.

```
OPTIMAL::TerminationStatusCode
```

An instance of the `TerminationStatusCode` enum.

**About**

The algorithm found a globally optimal solution.

source

`MathOptInterface.INFEASIBLE` – Constant.

```
INFEASIBLE::TerminationStatusCode
```

An instance of the `TerminationStatusCode` enum.

**About**

The algorithm proved that no primal feasible solution exists.

source

`MathOptInterface.DUAL_INFEASIBLE` – Constant.

```
DUAL_INFEASIBLE::TerminationStatusCode
```

An instance of the `TerminationStatusCode` enum.

**About**

The algorithm proved that no dual feasible solution exists.

To check if the primal problem is feasible, set the objective sense to `FEASIBILITY_SENSE` and re-solve the problem.

If a primal feasible point does not exist, the original problem is both primal and dual infeasible.

If a primal feasible solution exists, this status typically implies that the problem is unbounded, with some technical exceptions (for example, if the problem is a conic optimization problem in which strong duality does not hold).

The technical exceptions do not apply to linear programs. The combination of `DUAL_INFEASIBLE` and a primal feasible point means that the primal linear program is unbounded.

source

`MathOptInterface.LOCALLY_SOLVED` – Constant.

```
LOCALLY_SOLVED::TerminationStatusCode
```

An instance of the TerminationStatusCode enum.

**About**

The algorithm converged to a stationary point, local optimal solution, could not find directions for improvement, or otherwise completed its search without global guarantees.

source

MathOptInterface.LOCALLY_INFEASIBLE – Constant.

```
LOCALLY_INFEASIBLE::TerminationStatusCode
```

An instance of the TerminationStatusCode enum.

**About**

The algorithm converged to an infeasible point or otherwise completed its search without finding a feasible solution, without guarantees that no feasible solution exists.

If you know a primal feasible solution exists, use VariablePrimalStart to provide a feasible starting point to the solver.

source

MathOptInterface.INFEASIBLE_OR_UNBOUNDED – Constant.

```
INFEASIBLE_OR_UNBOUNDED::TerminationStatusCode
```

An instance of the TerminationStatusCode enum.

**About**

The algorithm stopped because it proved that the problem is infeasible or unbounded, without distinguishing between the two cases.

To distinguish between the two cases, set the objective sense to FEASIBILITY_SENSE and re-solve the problem. If a primal feasible point exists, the original problem is unbounded. If a primal feasible point does not exist, the original problem is infeasible.

source

MathOptInterface.ALMOST_OPTIMAL – Constant.

```
ALMOST_OPTIMAL::TerminationStatusCode
```

An instance of the TerminationStatusCode enum.

**About**

The algorithm found a globally optimal solution to relaxed tolerances.

source

`MathOptInterface.ALMOST_INFEASIBLE` – Constant.

```
ALMOST_INFEASIBLE::TerminationStatusCode
```

An instance of the `TerminationStatusCode` enum.

**About**

The algorithm concluded that no feasible solution exists within relaxed tolerances.

source

`MathOptInterface.ALMOST_DUAL_INFEASIBLE` – Constant.

```
ALMOST_DUAL_INFEASIBLE::TerminationStatusCode
```

An instance of the `TerminationStatusCode` enum.

**About**

The algorithm concluded that no dual bound exists for the problem within relaxed tolerances.

source

`MathOptInterface.ALMOST_LOCALLY_SOLVED` – Constant.

```
ALMOST_LOCALLY_SOLVED::TerminationStatusCode
```

An instance of the `TerminationStatusCode` enum.

**About**

The algorithm converged to a stationary point, local optimal solution, or could not find directions for improvement within relaxed tolerances.

source

`MathOptInterface.ITERATION_LIMIT` – Constant.

```
ITERATION_LIMIT::TerminationStatusCode
```

An instance of the `TerminationStatusCode` enum.

**About**

An iterative algorithm stopped after conducting the maximum number of iterations.

source

`MathOptInterface.TIME_LIMIT` – Constant.

```
TIME_LIMIT::TerminationStatusCode
```

An instance of the TerminationStatusCode enum.

**About**

The algorithm stopped after a user-specified computation time.

This status may be returned in relation to the TimeLimitSec attribute, or some other solver-specific attribute.

source

MathOptInterface.NODE_LIMIT – Constant.

```
NODE_LIMIT::TerminationStatusCode
```

An instance of the TerminationStatusCode enum.

**About**

A branch-and-bound algorithm stopped because it explored a maximum number of nodes in the branch-and-bound tree.

This status may be returned in relation to the NodeLimit attribute, or some other solver-specific attribute.

source

MathOptInterface.SOLUTION_LIMIT – Constant.

```
SOLUTION_LIMIT::TerminationStatusCode
```

An instance of the TerminationStatusCode enum.

**About**

The algorithm stopped because it found the required number of solutions. This is often used in MIPs to get the solver to return the first feasible solution it encounters.

This status may be returned in relation to the SolutionLimit attribute, or some other solver-specific attribute.

source

MathOptInterface.MEMORY_LIMIT – Constant.

```
MEMORY_LIMIT::TerminationStatusCode
```

An instance of the TerminationStatusCode enum.

**About**

The algorithm stopped because it ran out of memory.

source

`MathOptInterface.OBJECTIVE_LIMIT` – Constant.

```
OBJECTIVE_LIMIT::TerminationStatusCode
```

An instance of the `TerminationStatusCode` enum.

**About**

The algorithm stopped because it found a solution better than a minimum limit set by the user.

This status may be returned in relation to the `ObjectiveLimit` attribute, or some other solver-specific attribute.

source

`MathOptInterface.NORM_LIMIT` – Constant.

```
NORM_LIMIT::TerminationStatusCode
```

An instance of the `TerminationStatusCode` enum.

**About**

The algorithm stopped because the norm of an iterate became too large.

This typically means that the primal problem is unbounded, but that the solver could not prove so.

source

`MathOptInterface.OTHER_LIMIT` – Constant.

```
OTHER_LIMIT::TerminationStatusCode
```

An instance of the `TerminationStatusCode` enum.

**About**

The algorithm stopped due to a limit not covered by one of the _LIMIT_ statuses above.

source

`MathOptInterface.SLOW_PROGRESS` – Constant.

```
SLOW_PROGRESS::TerminationStatusCode
```

An instance of the `TerminationStatusCode` enum.

**About**

The algorithm stopped because it was unable to continue making progress towards the solution.

source

`MathOptInterface.NUMERICAL_ERROR` – Constant.

```
NUMERICAL_ERROR::TerminationStatusCode
```

An instance of the TerminationStatusCode enum.

**About**

The algorithm stopped because it encountered unrecoverable numerical error.

source

MathOptInterface.INVALID_MODEL – Constant.

```
INVALID_MODEL::TerminationStatusCode
```

An instance of the TerminationStatusCode enum.

**About**

The algorithm stopped because the model is invalid.

The reason for this return code is solver-specific, but common causes are that the problem has zero variables or constraints, or that the problem data contains an invalid number such as NaN.

source

MathOptInterface.INVALID_OPTION – Constant.

```
INVALID_OPTION::TerminationStatusCode
```

An instance of the TerminationStatusCode enum.

**About**

The algorithm stopped because it was provided an invalid option.

source

MathOptInterface.INTERRUPTED – Constant.

```
INTERRUPTED::TerminationStatusCode
```

An instance of the TerminationStatusCode enum.

**About**

The algorithm stopped because of an interrupt signal.

This typically means that the solver was interrupted by the user with CTRL+C.

source

MathOptInterface.OTHER_ERROR – Constant.

```
OTHER_ERROR::TerminationStatusCode
```

An instance of the TerminationStatusCode enum.

**About**

The algorithm stopped because of an error not covered by one of the statuses defined above. Check the solver log for further details.

source

MathOptInterface.PrimalStatus – Type.

```
PrimalStatus(result_index::Int = 1)
```

An AbstractModelAttribute for the ResultStatusCode of the primal result result_index.

**result_index**

If result_index is omitted, it defaults to 1.

See ResultCount for information on how the results are ordered.

If result_index is larger than the value of ResultCount then NO_SOLUTION is returned.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.PrimalStatus)::MOI.ResultStatusCode
```

They should not implement set or supports.

source

MathOptInterface.DualStatus – Type.

```
DualStatus(result_index::Int = 1)
```

An AbstractModelAttribute for the ResultStatusCode of the dual result result_index.

**result_index**

See ResultCount for information on how the results are ordered.

If result_index is larger than the value of ResultCount then NO_SOLUTION is returned.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.PrimalStatus)::MOI.ResultStatusCode
```

They should not implement set or supports.

source

`MathOptInterface.RawStatusString` – Type.

```
RawStatusString()
```

An `AbstractModelAttribute` for a solver specific string explaining why the optimizer stopped.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.RawStatusString)::MOI.String
```

They should not implement `set` or `supports`.

source

`MathOptInterface.ResultCount` – Type.

```
ResultCount()
```

An `AbstractModelAttribute` for the number of results available.

**Order of solutions**

A number of attributes contain an index, `result_index`, which is used to refer to one of the available results. Thus, `result_index` must be an integer between 1 and the number of available results.

As a general rule, the first result (`result_index = 1`) is the most important result (for example, an optimal solution or an infeasibility certificate). Other results will typically be alternate solutions that the solver found during the search for the first result.

If a (local) optimal solution is available, that is, `TerminationStatus` is `OPTIMAL` or `LOCALLY_SOLVED`, the first result must correspond to the (locally) optimal solution. Other results may be alternative optimal solutions, or they may be other suboptimal solutions; use `ObjectiveValue` to distinguish between them.

If a primal or dual infeasibility certificate is available, that is, `TerminationStatus` is `INFEASIBLE` or `DUAL_INFEASIBLE` and the corresponding `PrimalStatus` or `DualStatus` is `INFEASIBILITY_CERTIFICATE`, then the first result must be a certificate. Other results may be alternate certificates, or infeasible points.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.ResultCount)::Int
```

They should not implement `set` or `supports`.

source

`MathOptInterface.ObjectiveValue` – Type.

```
ObjectiveValue(result_index::Int = 1)
```

An AbstractModelAttribute for the objective value of the primal solution result_index.

**PrimalStatus**

Before querying this attribute you should first check PrimalStatus to confirm that a primal solution is available.

If the PrimalStatus is NO_SOLUTION the result of querying this attribute is undefined.

**result_index**

The optimizer may return multiple primal solutions. See ResultCount for information on how the results are ordered.

If the solver does not have a primal value for the objective because the result_index is beyond the available solutions (whose number is indicated by the ResultCount attribute), getting this attribute must throw a ResultIndexBoundsError.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(
    ::Optimizer,
    ::MOI.ObjectiveValue,
)::Union{T,Vector{T}} where {T<:Real}
```

They should not implement set or supports.

source

MathOptInterface.DualObjectiveValue – Type.

```
DualObjectiveValue(result_index::Int = 1)
```

An AbstractModelAttribute for the value of the objective function of the dual solution result_index.

**DualStatus**

Before querying this attribute you should first check DualStatus to confirm that a dual solution is available.

If the DualStatus is NO_SOLUTION the result of querying this attribute is undefined.

**result_index**

The optimizer may return multiple dual solutions. See ResultCount for information on how the results are ordered.

If the solver does not have a dual value for the objective because the result_index is beyond the available solutions (whose number is indicated by the ResultCount attribute), getting this attribute must throw a ResultIndexBoundsError.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(
    ::Optimizer,
    ::MOI.DualObjectiveValue,
)::Union{T,Vector{T}} where {T<:Real}
```

They should not implement `set` or `supports`.

source

MathOptInterface.ObjectiveBound – Type.

```
ObjectiveBound()
```

An `AbstractModelAttribute` for the best known bound on the optimal objective value.

source

MathOptInterface.RelativeGap – Type.

```
RelativeGap()
```

An `AbstractModelAttribute` for the final relative optimality gap.

**Warning**

The definition of this gap is solver-dependent. However, most solvers implementing this attribute define the relative gap as some variation of $\frac{|b-f|}{|f|}$, where $b$ is the best bound and $f$ is the best feasible objective value.

source

MathOptInterface.SolveTimeSec – Type.

```
SolveTimeSec()
```

An `AbstractModelAttribute` for the total elapsed solution time (in seconds) as reported by the optimizer.

source

MathOptInterface.SimplexIterations – Type.

```
SimplexIterations()
```

An `AbstractModelAttribute` for the cumulative number of simplex iterations while solving a problem.

For a mixed-integer program (MIP), the return value is the total simplex iterations for all nodes.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.SimplexIterations)::Int64
```

They should not implement `set` or `supports`.

source

`MathOptInterface.BarrierIterations` – Type.

```
BarrierIterations()
```

An `AbstractModelAttribute` for the cumulative number of barrier iterations while solving a problem.

For a mixed-integer program (MIP), the return value is the total barrier iterations for all nodes.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.BarrierIterations)::Int64
```

They should not implement `set` or `supports`.

source

`MathOptInterface.NodeCount` – Type.

```
NodeCount()
```

An `AbstractModelAttribute` for the total number of branch-and-bound nodes explored while solving a mixed-integer program (MIP).

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.NodeCount)::Int64
```

They should not implement `set` or `supports`.

source

### Result Status

`MathOptInterface.ResultStatusCode` – Type.

```
ResultStatusCode
```

An Enum of possible values for the `PrimalStatus` and `DualStatus` attributes.

The values indicate how to interpret the result vector.

**Values**

**NO_SOLUTION**

The result vector is empty.

**FEASIBLE_POINT**

The result vector is a feasible point.

**NEARLY_FEASIBLE_POINT**

The result vector is feasible if some constraint tolerances are relaxed.

**INFEASIBLE_POINT**

The result vector is an infeasible point.

**INFEASIBILITY_CERTIFICATE**

The result vector is an infeasibility certificate.

If the PrimalStatus is INFEASIBILITY_CERTIFICATE, then the primal result vector is a certificate of dual infeasibility.

If the DualStatus is INFEASIBILITY_CERTIFICATE, then the dual result vector is a proof of primal infeasibility.

**NEARLY_INFEASIBILITY_CERTIFICATE**

The result satisfies a relaxed criterion for a certificate of infeasibility.

If the PrimalStatus is NEARLY_INFEASIBILITY_CERTIFICATE, then the primal result vector is a certificate of dual infeasibility.

If the DualStatus is NEARLY_INFEASIBILITY_CERTIFICATE, then the dual result vector is a proof of primal infeasibility.

**REDUCTION_CERTIFICATE**

The result vector is an ill-posed certificate; see this article for details.

If the PrimalStatus is REDUCTION_CERTIFICATE, then the primal result vector is a proof that the dual problem is ill-posed.

If the DualStatus is REDUCTION_CERTIFICATE, then the dual result vector is a proof that the primal is ill-posed.

**NEARLY_REDUCTION_CERTIFICATE**

The result satisfies a relaxed criterion for an ill-posed certificate.

**UNKNOWN_RESULT_STATUS**

The result vector contains a solution with an unknown interpretation. Check the solver log for more details.

**OTHER_RESULT_STATUS**

The result vector contains a solution with an interpretation not covered by one of the statuses defined above. Check the solver log for more details.

source

`MathOptInterface.NO_SOLUTION` – Constant.

```
NO_SOLUTION::ResultStatusCode
```

An instance of the ResultStatusCode enum.

**About**

The result vector is empty.

source

MathOptInterface.FEASIBLE_POINT – Constant.

```
FEASIBLE_POINT::ResultStatusCode
```

An instance of the ResultStatusCode enum.

**About**

The result vector is a feasible point.

source

MathOptInterface.NEARLY_FEASIBLE_POINT – Constant.

```
NEARLY_FEASIBLE_POINT::ResultStatusCode
```

An instance of the ResultStatusCode enum.

**About**

The result vector is feasible if some constraint tolerances are relaxed.

source

MathOptInterface.INFEASIBLE_POINT – Constant.

```
INFEASIBLE_POINT::ResultStatusCode
```

An instance of the ResultStatusCode enum.

**About**

The result vector is an infeasible point.

source

MathOptInterface.INFEASIBILITY_CERTIFICATE – Constant.

```
INFEASIBILITY_CERTIFICATE::ResultStatusCode
```

An instance of the ResultStatusCode enum.

**About**

The result vector is an infeasibility certificate.

If the PrimalStatus is INFEASIBILITY_CERTIFICATE, then the primal result vector is a certificate of dual infeasibility.

If the DualStatus is INFEASIBILITY_CERTIFICATE, then the dual result vector is a proof of primal infeasibility.

source

MathOptInterface.NEARLY_INFEASIBILITY_CERTIFICATE – Constant.

```
NEARLY_INFEASIBILITY_CERTIFICATE::ResultStatusCode
```

An instance of the ResultStatusCode enum.

**About**

The result satisfies a relaxed criterion for a certificate of infeasibility.

If the PrimalStatus is NEARLY_INFEASIBILITY_CERTIFICATE, then the primal result vector is a certificate of dual infeasibility.

If the DualStatus is NEARLY_INFEASIBILITY_CERTIFICATE, then the dual result vector is a proof of primal infeasibility.

source

MathOptInterface.REDUCTION_CERTIFICATE – Constant.

```
REDUCTION_CERTIFICATE::ResultStatusCode
```

An instance of the ResultStatusCode enum.

**About**

The result vector is an ill-posed certificate; see this article for details.

If the PrimalStatus is REDUCTION_CERTIFICATE, then the primal result vector is a proof that the dual problem is ill-posed.

If the DualStatus is REDUCTION_CERTIFICATE, then the dual result vector is a proof that the primal is ill-posed.

source

MathOptInterface.NEARLY_REDUCTION_CERTIFICATE – Constant.

```
NEARLY_REDUCTION_CERTIFICATE::ResultStatusCode
```

An instance of the ResultStatusCode enum.

**About**

The result satisfies a relaxed criterion for an ill-posed certificate.

source

`MathOptInterface.UNKNOWN_RESULT_STATUS` – Constant.

```
UNKNOWN_RESULT_STATUS::ResultStatusCode
```

An instance of the `ResultStatusCode` enum.

**About**

The result vector contains a solution with an unknown interpretation. Check the solver log for more details.

source

`MathOptInterface.OTHER_RESULT_STATUS` – Constant.

```
OTHER_RESULT_STATUS::ResultStatusCode
```

An instance of the `ResultStatusCode` enum.

**About**

The result vector contains a solution with an interpretation not covered by one of the statuses defined above. Check the solver log for more details.

source

### Conflict Status

`MathOptInterface.compute_conflict!` – Function.

```
compute_conflict!(optimizer::AbstractOptimizer)
```

Computes a minimal subset of constraints such that the model with the other constraint removed is still infeasible.

Some solvers call a set of conflicting constraints an Irreducible Inconsistent Subsystem (IIS).

See also `ConflictStatus` and `ConstraintConflictStatus`.

**Note**

If the model is modified after a call to `compute_conflict!`, the implementor is not obliged to purge the conflict. Any calls to the above attributes may return values for the original conflict without a warning. Similarly, when modifying the model, the conflict can be discarded.

source

`MathOptInterface.ConflictStatus` – Type.

```
ConflictStatus()
```

An AbstractModelAttribute for the ConflictStatusCode explaining why [compute_conflict!] stopped when computing the conflict.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.ConflictStatus)::MOI.ConflictStatusCode
```

They should not implement set or supports.

source

MathOptInterface.ConflictStatusCode – Type.

```
ConflictStatusCode
```

An Enum of possible values for the ConflictStatus attribute.

This attribute is meant to explain the reason why the conflict finder stopped executing in the most recent call to compute_conflict!.

**Values**

**COMPUTE_CONFLICT_NOT_CALLED**

The function compute_conflict! has not yet been called.

**NO_CONFLICT_EXISTS**

There is no conflict because the problem is feasible.

**NO_CONFLICT_FOUND**

The solver could not find a conflict.

**CONFLICT_FOUND**

The solver found a conflict.

source

MathOptInterface.COMPUTE_CONFLICT_NOT_CALLED – Constant.

```
COMPUTE_CONFLICT_NOT_CALLED::ConflictStatusCode
```

An instance of the ConflictStatusCode enum.

**About**

The function compute_conflict! has not yet been called.

source

`MathOptInterface.NO_CONFLICT_EXISTS` – Constant.

```
NO_CONFLICT_EXISTS::ConflictStatusCode
```

An instance of the `ConflictStatusCode` enum.

**About**

There is no conflict because the problem is feasible.

source

`MathOptInterface.NO_CONFLICT_FOUND` – Constant.

```
NO_CONFLICT_FOUND::ConflictStatusCode
```

An instance of the `ConflictStatusCode` enum.

**About**

The solver could not find a conflict.

source

`MathOptInterface.CONFLICT_FOUND` – Constant.

```
CONFLICT_FOUND::ConflictStatusCode
```

An instance of the `ConflictStatusCode` enum.

**About**

The solver found a conflict.

source

`MathOptInterface.ConflictCount` – Type.

```
ConflictCount()
```

An `AbstractModelAttribute` for the number of conflicts found by the solver in the most recent call to `compute_conflict!`.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.ConflictCount)::Int
```

They should not implement `set` or `supports`.

source

`MathOptInterface.ConstraintConflictStatus` – Type.

```
ConstraintConflictStatus(conflict_index = 1)
```

A constraint attribute to query the `ConflictParticipationStatusCode` indicating whether the constraint participates in the conflict.

**`conflict_index`**

The optimizer may return multiple conflicts. See `ConflictCount` for querying the number of conflicts found.

If the solver does not have a conflict because the `conflict_index` is beyond the available solutions (whose number is indicated by the `ConflictCount` attribute), then `MOI.check_result_index_bounds(model, ConstraintConflictStatus(conflict_index))` will throw a `ResultIndexBoundsError`.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(
    ::Optimizer,
    ::MOI.ConstraintConflictStatus,
    ::MOI.ConstraintIndex,
)::ConflictParticipationStatusCode
```

They should not implement `set` or `supports`.

source

`MathOptInterface.ConflictParticipationStatusCode` – Type.

```
ConflictParticipationStatusCode
```

An Enum for the value of the `ConstraintConflictStatus` attribute.

This attribute is meant to indicate whether a given constraint participates or not in the last computed conflict.

**Values**

**`NOT_IN_CONFLICT`**

The constraint does not participate in the conflict.

**`IN_CONFLICT`**

The constraint participates in the conflict.

**`MAYBE_IN_CONFLICT`**

The solver was not able to prove whether the constraint is required to participate in the conflict.

source

`MathOptInterface.NOT_IN_CONFLICT` – Constant.

```
NOT_IN_CONFLICT::ConflictParticipationStatusCode
```

An instance of the ConflictParticipationStatusCode enum.

**About**

The constraint does not participate in the conflict.

source

MathOptInterface.IN_CONFLICT – Constant.

```
IN_CONFLICT::ConflictParticipationStatusCode
```

An instance of the ConflictParticipationStatusCode enum.

**About**

The constraint participates in the conflict.

source

MathOptInterface.MAYBE_IN_CONFLICT – Constant.

```
MAYBE_IN_CONFLICT::ConflictParticipationStatusCode
```

An instance of the ConflictParticipationStatusCode enum.

**About**

The solver was not able to prove whether the constraint is required to participate in the conflict.

source

# Chapter 20

# Variables

## 20.1 Functions

`MathOptInterface.add_variable` – Function.

```
add_variable(model::ModelLike)::VariableIndex
```

Add a scalar variable to the model, returning a variable index.

A `AddVariableNotAllowed` error is thrown if adding variables cannot be done in the current state of the model `model`.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model)
MOI.VariableIndex(1)
```

source

`MathOptInterface.add_variables` – Function.

```
add_variables(model::ModelLike, n::Int)::Vector{VariableIndex}
```

Add `n` scalar variables to the model, returning a vector of variable indices.

An `AddVariableNotAllowed` error is thrown if adding variables cannot be done in the current state of the model `model`.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> MOI.add_variables(model, 2)
2-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(1)
 MOI.VariableIndex(2)
```

source

`MathOptInterface.add_constrained_variable` – Function.

```
add_constrained_variable(
    model::ModelLike,
    set::AbstractScalarSet
)::Tuple{MOI.VariableIndex,
         MOI.ConstraintIndex{MOI.VariableIndex, typeof(set)}}
```

Add to `model` a scalar variable constrained to belong to `set`, returning the index of the variable created and the index of the constraint constraining the variable to belong to `set`.

By default, this function falls back to creating a free variable with `add_variable` and then constraining it to belong to `set` with `add_constraint`.

source

```
add_constrained_variable(
    model::ModelLike,
    set::Tuple{<:GreaterThan,<:LessThan},
)
```

A special-case method to add a scalar variable with a lower and upper bound.

This method should be implemented by optimizers which have native support for adding a variable with bounds and which cannot performantly modify the variable bounds after creation.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> set = (MOI.GreaterThan(1.0), MOI.LessThan(2.0));

julia> x, (c_l, c_u) = MOI.add_constrained_variable(model, set);

julia> c_l
MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.GreaterThan{Float64}}(1)

julia> c_u
MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.LessThan{Float64}}(1)

julia> print(model)
Feasibility

Subject to:

VariableIndex-in-GreaterThan{Float64}
 v[1] >= 1.0

VariableIndex-in-LessThan{Float64}
 v[1] <= 2.0
```

source

MathOptInterface.add_constrained_variables – Function.

```
add_constrained_variables(
    model::ModelLike,
    sets::AbstractVector{<:AbstractScalarSet}
)::Tuple{
    Vector{MOI.VariableIndex},
    Vector{MOI.ConstraintIndex{MOI.VariableIndex,eltype(sets)}},
}
```

Add to `model` scalar variables constrained to belong to `sets`, returning the indices of the variables created and the indices of the constraints constraining the variables to belong to each set in `sets`. That is, if it returns `variables` and `constraints`, `constraints[i]` is the index of the constraint constraining `variable[i]` to belong to `sets[i]`.

By default, this function falls back to calling `add_constrained_variable` on each set.

source

```
add_constrained_variables(
    model::ModelLike,
    set::AbstractVectorSet,
)::Tuple{
    Vector{MOI.VariableIndex},
    MOI.ConstraintIndex{MOI.VectorOfVariables,typeof(set)},
}
```

Add to `model` a vector of variables constrained to belong to `set`, returning the indices of the variables created and the index of the constraint constraining the vector of variables to belong to `set`.

By default, this function falls back to creating free variables with `add_variables` and then constraining it to belong to `set` with `add_constraint`.

source

MathOptInterface.supports_add_constrained_variable – Function.

```
supports_add_constrained_variable(
    model::ModelLike,
    S::Type{<:AbstractScalarSet}
)::Bool
```

Return a `Bool` indicating whether `model` supports constraining a variable to belong to a set of type `S` either on creation of the variable with `add_constrained_variable` or after the variable is created with `add_constraint`.

By default, this function falls back to `supports_add_constrained_variables(model, Reals) && supports_constraint(mo MOI.VariableIndex, S)` which is the correct definition for most models.

**Example**

Suppose that a solver supports only two kind of variables: binary variables and continuous variables with a lower bound. If the solver decides not to support `VariableIndex-in-Binary` and `VariableIndex-in-GreaterThan` constraints, it only has to implement `add_constrained_variable` for these two sets which prevents the user to add both a binary constraint and a lower bound on the same variable. Moreover, if the user adds a `VariableIndex-in-GreaterThan` constraint, implementing this interface (that is, `supports_add_constrained_variables`) enables the constraint to be transparently bridged into a supported constraint.

source

`MathOptInterface.supports_add_constrained_variables` – Function.

```
supports_add_constrained_variables(
    model::ModelLike,
    S::Type{<:AbstractVectorSet}
)::Bool
```

Return a `Bool` indicating whether `model` supports constraining a vector of variables to belong to a set of type `S` either on creation of the vector of variables with `add_constrained_variables` or after the variable is created with `add_constraint`.

By default, if `S` is `Reals` then this function returns `true` and otherwise, it falls back to `supports_add_constrained_variables(Reals) && supports_constraint(model, MOI.VectorOfVariables, S)` which is the correct definition for most models.

**Example**

In the standard conic form (see Duality), the variables are grouped into several cones and the constraints are affine equality constraints. If `Reals` is not one of the cones supported by the solvers then it needs to implement `supports_add_constrained_variables(::Optimizer, ::Type{Reals}) = false` as free variables are not supported. The solvers should then implement `supports_add_constrained_variables(::Optimizer, ::Type{<:SupportedCones}) = true` where `SupportedCones` is the union of all cone types that are supported; it does not have to implement the method `supports_constraint(::Type{VectorOfVariables}, Type{<:SupportedCones})` as it should return `false` and it's the default. This prevents the user to constrain the same variable in two different cones. When a `VectorOfVariables-in-S` is added, the variables of the vector have already been created so they already belong to given cones. If bridges are enabled, the constraint will therefore be bridged by adding slack variables in `S` and equality constraints ensuring that the slack variables are equal to the corresponding variables of the given constraint function.

Note that there may also be sets for which `!supports_add_constrained_variables(model, S)` and `supports_constraint(model, MOI.VectorOfVariables, S)`. For instance, suppose a solver supports positive semidefinite variable constraints and two types of variables: binary variables and nonnegative variables. Then the solver should support adding `VectorOfVariables-in-PositiveSemidefiniteConeTriangle` constraints, but it should not support creating variables constrained to belong to the `PositiveSemidefiniteConeTriangle` because the variables in `PositiveSemidefiniteConeTriangle` should first be created as either binary or non-negative.

source

`MathOptInterface.is_valid` – Method.

```
is_valid(model::ModelLike, index::Index)::Bool
```

Return a `Bool` indicating whether this index refers to a valid object in the model `model`.

source

`MathOptInterface.delete` – Method.

```
delete(model::ModelLike, index::Index)
```

Delete the referenced object from the model. Throw `DeleteNotAllowed` if if `index` cannot be deleted.

The following modifications also take effect if `Index` is `VariableIndex`:

- If `index` used in the objective function, it is removed from the function, that is, it is substituted for zero.
- For each `func`-in-`set` constraint of the model:
  - If `func isa VariableIndex` and `func == index` then the constraint is deleted.
  - If `func isa VectorOfVariables` and `index in func.variables` then
    - if `length(func.variables) == 1` is one, the constraint is deleted;
    - if `length(func.variables) > 1` and `supports_dimension_update(set)` then then the variable is removed from `func` and `set` is replaced by `update_dimension(set, MOI.dimension(set) - 1)`.
    - Otherwise, a `DeleteNotAllowed` error is thrown.
  - Otherwise, the variable is removed from `func`, that is, it is substituted for zero.

source

`MathOptInterface.delete` – Method.

```
delete(model::ModelLike, indices::Vector{R<:Index}) where {R}
```

Delete the referenced objects in the vector `indices` from the model. It may be assumed that `R` is a concrete type. The default fallback sequentially deletes the individual items in `indices`, although specialized implementations may be more efficient.

source

## 20.2 Attributes

`MathOptInterface.AbstractVariableAttribute` – Type.

```
AbstractVariableAttribute
```

Abstract supertype for attribute objects that can be used to set or get attributes (properties) of variables in the model.

source

`MathOptInterface.VariableName` – Type.

```
VariableName()
```

An `AbstractVariableAttribute` for a `String` identifying the variable.

The default name is `""` if not set by the user.

**Duplicate names**

Two variables may have the same name; however, variables with duplicate names cannot be looked up using `get`.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model);

julia> MOI.supports(model, MOI.VariableName(), MOI.VariableIndex)
true

julia> MOI.get(model, MOI.VariableName(), x)
""

julia> MOI.set(model, MOI.VariableName(), x, "x")

julia> MOI.get(model, MOI.VariableName(), x)
"x"
```

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.VariableName, ::MOI.VariableIndex)::String
MOI.set(::Optimizer, ::MOI.VariableName, ::MOI.VariableIndex, ::String)::Nothing
MOI.supports(::Optimizer, ::MOI.VariableName, ::Type{MOI.VariableIndex})::Bool
```

source

`MathOptInterface.VariablePrimalStart` – Type.

```
VariablePrimalStart()
```

An `AbstractVariableAttribute` for the initial assignment to the variable's primal value that the optimizer may use to warm-start the solve.

May be a number or `nothing` (unset).

**Example**

```
julia> model = MOI.Utilities.UniversalFallback(MOI.Utilities.Model{Float64}());

julia> x = MOI.add_variable(model);
```

```
julia> MOI.supports(model, MOI.VariablePrimalStart(), MOI.VariableIndex)
true

julia> MOI.get(model, MOI.VariablePrimalStart(), x)

julia> MOI.set(model, MOI.VariablePrimalStart(), x, 1.0)

julia> MOI.get(model, MOI.VariablePrimalStart(), x)
1.0
```

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.VariablePrimalStart, ::MOI.VariableIndex)::Union{Nothing,T}
MOI.set(::Optimizer, ::MOI.VariablePrimalStart, ::MOI.VariableIndex,
↪  ::Union{Nothing,T})::Nothing
MOI.supports(::Optimizer, ::MOI.VariablePrimalStart, ::Type{MOI.VariableIndex})::Bool
```

source

`MathOptInterface.VariablePrimal` – Type.

```
VariablePrimal(result_index::Int = 1)
```

An `AbstractVariableAttribute` for the variable's primal value in result `result_index`.

**PrimalStatus**

Before querying this attribute you should first check `PrimalStatus` to confirm that a primal solution is available.

If the `PrimalStatus` is `NO_SOLUTION` the result of querying this attribute is undefined.

**`result_index`**

The optimizer may return multiple primal solutions. See `ResultCount` for information on how the results are ordered.

If the solver does not have a primal value for the variable because the `result_index` is beyond the available solutions (whose number is indicated by the `ResultCount` attribute), getting this attribute must throw a `ResultIndexBoundsError`.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.VariablePrimal, ::MOI.VariableIndex)::T
```

They should not implement `set` or `supports`.

source

`MathOptInterface.VariableBasisStatus` – Type.

```
VariableBasisStatus(result_index::Int = 1)
```

An `AbstractVariableAttribute` for the `BasisStatusCode` of the variable in result `result_index`, with respect to a basic solution.

**PrimalStatus**

Before querying this attribute you should first check `PrimalStatus` to confirm that a primal solution is available.

If the `PrimalStatus` is `NO_SOLUTION` the result of querying this attribute is undefined.

**`result_index`**

The optimizer may return multiple primal solutions. See `ResultCount` for information on how the results are ordered.

If the solver does not have a primal value for the variable because the `result_index` is beyond the available solutions (whose number is indicated by the `ResultCount` attribute), getting this attribute must throw a `ResultIndexBoundsError`.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.VariableBasisStatus, ::MOI.VariableIndex)::MOI.BasisStatusCode
```

They should not implement `set` or `supports`.

source

# Chapter 21

# Constraints

## 21.1 Types

`MathOptInterface.ConstraintIndex` – Type.

```
ConstraintIndex{F,S}
```

A type-safe wrapper for `Int64` for use in referencing F-in-S constraints in a model.

The parameter `F` is the type of the function in the constraint, and the parameter `S` is the type of set in the constraint.

To allow for deletion, indices need not be consecutive.

Indices within a constraint type (that is, F-in-S) must be unique, but non-unique indices across different constraint types are allowed.

If `F` is `VariableIndex` then the index is equal to the index of the variable. That is for an `index::ConstraintIndex{VariableIn` we always have

```
index.value == MOI.get(model, MOI.ConstraintFunction(), index).value
```

source

## 21.2 Functions

`MathOptInterface.is_valid` – Method.

```
is_valid(model::ModelLike, index::Index)::Bool
```

Return a `Bool` indicating whether this index refers to a valid object in the model `model`.

source

`MathOptInterface.add_constraint` – Function.

```
MOI.add_constraint(map::Map, vi::MOI.VariableIndex, set::MOI.AbstractScalarSet)
```

Record that a constraint `vi`-in-`set` is added and throws if a lower or upper bound is set by this constraint and such bound has already been set for `vi`.

source

```
add_constraint(model::ModelLike, func::F, set::S)::ConstraintIndex{F,S} where {F,S}
```

Add the constraint $f(x) \in \mathcal{S}$ where $f$ is defined by `func`, and $\mathcal{S}$ is defined by `set`.

```
add_constraint(model::ModelLike, v::VariableIndex, set::S)::ConstraintIndex{VariableIndex,S}
↪  where {S}
add_constraint(model::ModelLike, vec::Vector{VariableIndex},
↪  set::S)::ConstraintIndex{VectorOfVariables,S} where {S}
```

Add the constraint $v \in \mathcal{S}$ where $v$ is the variable (or vector of variables) referenced by `v` and $\mathcal{S}$ is defined by `set`.

- An `UnsupportedConstraint` error is thrown if `model` does not support F-in-S constraints,
- a `AddConstraintNotAllowed` error is thrown if it supports F-in-S constraints but it cannot add the constraint in its current state and
- a `ScalarFunctionConstantNotZero` error may be thrown if `func` is an `AbstractScalarFunction` with nonzero constant and `set` is `EqualTo`, `GreaterThan`, `LessThan` or `Interval`.
- a `LowerBoundAlreadySet` error is thrown if F is a `VariableIndex` and a constraint was already added to this variable that sets a lower bound.
- a `UpperBoundAlreadySet` error is thrown if F is a `VariableIndex` and a constraint was already added to this variable that sets an upper bound.

source

`MathOptInterface.add_constraints` – Function.

```
add_constraints(model::ModelLike, funcs::Vector{F},
↪  sets::Vector{S})::Vector{ConstraintIndex{F,S}} where {F,S}
```

Add the set of constraints specified by each function-set pair in `funcs` and `sets`. `F` and `S` should be concrete types. This call is equivalent to `add_constraint.(model, funcs, sets)` but may be more efficient.

source

`MathOptInterface.transform` – Function.

```
transform(
    model::ModelLike,
    c::ConstraintIndex{F,S1},
    newset::S2,
)::ConstraintIndex{F,S2}
```

Replace the set in constraint `c` with `newset`.

The constraint index `c` will no longer be valid, and the function returns a new constraint index with the correct type.

Solvers may only support a subset of constraint transforms that they perform efficiently (for example, changing from a `LessThan` to `GreaterThan` set). In addition, set modification (where `S1 = S2`) should be performed via the `modify` function.

Typically, the user should delete the constraint and add a new one.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model);

julia> c = MOI.add_constraint(model, 1.0 * x, MOI.LessThan(2.0));

julia> print(model)
Feasibility

Subject to:

ScalarAffineFunction{Float64}-in-LessThan{Float64}
 0.0 + 1.0 v[1] <= 2.0

julia> c2 = MOI.transform(model, c, MOI.GreaterThan(0.0))
MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64},
↪  MathOptInterface.GreaterThan{Float64}}(1)

julia> print(model)
Feasibility

Subject to:

ScalarAffineFunction{Float64}-in-GreaterThan{Float64}
 0.0 + 1.0 v[1] >= 0.0

julia> MOI.is_valid(model, c)
false
```

source

`MathOptInterface.supports_constraint` – Function.

```
MOI.supports_constraint(
    BT::Type{<:AbstractBridge},
    F::Type{<:MOI.AbstractFunction},
    S::Type{<:MOI.AbstractSet},
)::Bool
```

Return a `Bool` indicating whether the bridges of type `BT` support bridging `F`-in-`S` constraints.

**Implementation notes**

- This method depends only on the type of the inputs, not the runtime values.
- There is a default fallback, so you need only implement this method for constraint types that the bridge implements.

source

```
supports_constraint(
    model::ModelLike,
    ::Type{F},
    ::Type{S},
)::Bool where {F<:AbstractFunction,S<:AbstractSet}
```

Return a `Bool` indicating whether `model` supports F-in-S constraints, that is, `copy_to(model, src)` does not throw `UnsupportedConstraint` when `src` contains F-in-S constraints. If F-in-S constraints are only not supported in specific circumstances, for example, F-in-S constraints cannot be combined with another type of constraint, it should still return `true`.

source

## 21.3 Attributes

`MathOptInterface.AbstractConstraintAttribute` – Type.

```
AbstractConstraintAttribute
```

Abstract supertype for attribute objects that can be used to set or get attributes (properties) of constraints in the model.

source

`MathOptInterface.ConstraintName` – Type.

```
ConstraintName()
```

An `AbstractConstraintAttribute` for a `String` identifying the constraint.

The default name is `""` if not set by the user.

**Duplicate names**

Two constraints may have the same name; however, constraints with duplicate names cannot be looked up using `get`, regardless of whether they have the same F-in-S type.

**VariableIndex constraints**

You should not implement `ConstraintName` for `VariableIndex` constraints.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model);

julia> c = MOI.add_constraint(model, 1.0 * x, MOI.EqualTo(1.0));

julia> MOI.supports(model, MOI.ConstraintName(), typeof(c))
true

julia> MOI.get(model, MOI.ConstraintName(), c)
""

julia> MOI.set(model, MOI.ConstraintName(), c, "c")

julia> MOI.get(model, MOI.ConstraintName(), c)
"c"

julia> MOI.get(model, MOI.ConstraintIndex, "c")
MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64},
↪  MathOptInterface.EqualTo{Float64}}(1)

julia> F, S = MOI.ScalarAffineFunction{Float64}, MOI.EqualTo{Float64};

julia> MOI.get(model, MOI.ConstraintIndex{F,S}, "c")
MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64},
↪  MathOptInterface.EqualTo{Float64}}(1)
```

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.ConstraintName, ::MOI.ConstraintIndex)::String
MOI.set(::Optimizer, ::MOI.ConstraintName, ::MOI.ConstraintIndex, ::String)::Nothing
MOI.supports(::Optimizer, ::MOI.ConstraintName, ::Type{<:MOI.ConstraintIndex})::Bool
MOI.get(::Optimizer, ::MOI.ConstraintIndex, ::MOI.ConstraintIndex,
↪  ::String)::MOI.ConstraintIndex
```

source

`MathOptInterface.ConstraintPrimalStart` – Type.

```
ConstraintPrimalStart()
```

An `AbstractConstraintAttribute` for the initial assignment to the constraint's `ConstraintPrimal` that the optimizer may use to warm-start the solve.

May be `nothing` (unset), a number for `AbstractScalarFunction`, or a vector for `AbstractVectorFunction`.

source

`MathOptInterface.ConstraintDualStart` – Type.

```
ConstraintDualStart()
```

An `AbstractConstraintAttribute` for the initial assignment to the constraint's `ConstraintDual` that the optimizer may use to warm-start the solve.

May be `nothing` (unset), a number for `AbstractScalarFunction`, or a vector for `AbstractVectorFunction`.

source

`MathOptInterface.ConstraintPrimal` – Type.

```
ConstraintPrimal(result_index::Int = 1)
```

An `AbstractConstraintAttribute` for the constraint's primal value in result `result_index`.

**Definition**

If the constraint is $f(x) \in S$, then in most cases the `ConstraintPrimal` is the value of $f$, evaluated at the corresponding `VariablePrimal` solution.

However, some conic solvers reformulate $b - Ax \in S$ to $s = b - Ax$ and $s \in S$. These solvers may return the value of s for `ConstraintPrimal`, rather than b - Ax. (Although these are constrained by an equality constraint, due to numerical tolerances they may not be identical.)

**PrimalStatus**

Before querying this attribute you should first check `PrimalStatus` to confirm that a primal solution is available.

If the `PrimalStatus` is `NO_SOLUTION` the result of querying this attribute is undefined.

**`result_index`**

The optimizer may return multiple primal solutions. See `ResultCount` for information on how the results are ordered.

If the solver does not have a primal value for the constraint because the `result_index` is beyond the available solutions (whose number is indicated by the `ResultCount` attribute), getting this attribute must throw a `ResultIndexBoundsError`.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(
    ::Optimizer,
    ::MOI.ConstraintPrimal,
    ::MOI.ConstraintIndex{<:MOI.AbstractScalarFunction}
)::T
MOI.get(
    ::Optimizer,
    ::MOI.ConstraintPrimal,
    ::MOI.ConstraintIndex{<:MOI.AbstractVectorFunction}
)::Vector{T}
```

They should not implement `set` or `supports`.

source

`MathOptInterface.ConstraintDual` – Type.

```
ConstraintDual(result_index::Int = 1)
```

An `AbstractConstraintAttribute` for the constraint's dual value in result `result_index`.

**DualStatus**

Before querying this attribute you should first check `DualStatus` to confirm that a dual solution is available.

If the `DualStatus` is `NO_SOLUTION` the result of querying this attribute is undefined.

**`result_index`**

The optimizer may return multiple dual solutions. See `ResultCount` for information on how the results are ordered.

If the solver does not have a dual value for the constraint because the `result_index` is beyond the available solutions (whose number is indicated by the `ResultCount` attribute), getting this attribute must throw a `ResultIndexBoundsError`.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(
    ::Optimizer,
    ::MOI.ConstraintDual,
    ::MOI.ConstraintIndex{<:MOI.AbstractScalarFunction}
)::T
MOI.get(
    ::Optimizer,
    ::MOI.ConstraintDual,
    ::MOI.ConstraintIndex{<:MOI.AbstractVectorFunction}
)::Vector{T}
```

They should not implement `set` or `supports`.

source

`MathOptInterface.ConstraintBasisStatus` – Type.

```
ConstraintBasisStatus(result_index::Int = 1)
```

An `AbstractConstraintAttribute` for the `BasisStatusCode` of the constraint in result `result_index`, with respect to a basic solution.

If `result_index` is omitted, it is 1 by default.

If the solver does not have a basis status for the constraint because the `result_index` is beyond the available solutions (whose number is indicated by the `ResultCount` attribute), getting this attribute must

throw a ResultIndexBoundsError. Otherwise, if the result is unavailable for another reason (for instance, only a dual solution is available), the result is undefined. Users should first check PrimalStatus before accessing the ConstraintBasisStatus attribute.

See ResultCount for information on how the results are ordered.

**Notes**

For the basis status of a variable, query VariableBasisStatus.

ConstraintBasisStatus does not apply to VariableIndex constraints. You can infer the basis status of a VariableIndex constraint by looking at the result of VariableBasisStatus.

source

MathOptInterface.ConstraintFunction – Type.

```
ConstraintFunction()
```

An AbstractConstraintAttribute for the AbstractFunction object used to define the constraint.

It is guaranteed to be equivalent but not necessarily identical to the function provided by the user.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model);

julia> c = MOI.add_constraint(model, 1.0 * x, MOI.GreaterThan(0.0));

julia> MOI.get(model, MOI.ConstraintFunction(), c)
0.0 + 1.0 MOI.VariableIndex(1)

julia> MOI.set(model, MOI.ConstraintFunction(), c, 2.0 * x)

julia> MOI.get(model, MOI.ConstraintFunction(), c)
0.0 + 2.0 MOI.VariableIndex(1)
```

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(
    ::Optimizer,
    ::MOI.ConstraintFunction,
    ::MOI.ConstraintIndex{F,S},
)::F where {F,S}
```

If the optimizer supports modifying an existing function, it should implement:

```
MOI.set(
    ::Optimizer,
    ::MOI.ConstraintFunction,
```

```
    ::MOI.ConstraintIndex{F,S},
    ::F,
)::Nothing where {F,S}
```

It should not implement `supports`.

source

`MathOptInterface.CanonicalConstraintFunction` – Type.

```
CanonicalConstraintFunction()
```

An `AbstractConstraintAttribute` for a canonical representation of the `AbstractFunction` object used to define the constraint.

Getting this attribute is guaranteed to return a function that is equivalent but not necessarily identical to the function provided by the user.

**Fallback**

By default, `MOI.get(model, MOI.CanonicalConstraintFunction(), ci)` falls back to `MOI.Utilities.canonical(MOI.get MOI.ConstraintFunction(), ci))`.

However, if `model` knows that the constraint function is canonical then it can implement a specialized method that directly return the function without calling `Utilities.canonical`. Therefore, the value returned **cannot** be assumed to be a copy of the function stored in `model`.

Moreover, `Utilities.Model` checks with `Utilities.is_canonical` whether the function stored internally is already canonical and if it's the case, then it returns the function stored internally instead of a copy.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model);

julia> c = MOI.add_constraint(model, 1.0 * x + 1.0 * x, MOI.GreaterThan(0.0));

julia> MOI.get(model, MOI.CanonicalConstraintFunction(), c)
0.0 + 2.0 MOI.VariableIndex(1)
```

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(
    ::Optimizer,
    ::MOI.CanonicalConstraintFunction,
    ::MOI.ConstraintIndex{F,S},
)::F where {F,S}
```

They should not implement `set` or `supports`.

source

`MathOptInterface.ConstraintSet` – Type.

```
ConstraintSet()
```

An `AbstractConstraintAttribute` for the `AbstractSet` object used to define the constraint.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model);

julia> c = MOI.add_constraint(model, x, MOI.GreaterThan(0.0));

julia> MOI.get(model, MOI.ConstraintSet(), c)
MathOptInterface.GreaterThan{Float64}(0.0)

julia> MOI.set(model, MOI.ConstraintSet(), c, MOI.GreaterThan(1.0))

julia> MOI.get(model, MOI.ConstraintSet(), c)
MathOptInterface.GreaterThan{Float64}(1.0)
```

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(
    ::Optimizer,
    ::MOI.ConstraintSet,
    ::MOI.ConstraintIndex{F,S},
)::S where {F,S}
```

If the optimizer supports modifying an existing set, it should implement:

```
MOI.set(
    ::Optimizer,
    ::MOI.ConstraintSet,
    ::MOI.ConstraintIndex{F,S},
    ::S,
)::Nothing where {F,S}
```

It should not implement `supports`.

source

`MathOptInterface.BasisStatusCode` – Type.

```
BasisStatusCode
```

An Enum for the value of the ConstraintBasisStatus and VariableBasisStatus attributes, explaining the status of a given element with respect to an optimal solution basis.

**Notes**

When queried as part of ConstraintBasisStatus, NONBASIC_AT_LOWER and NONBASIC_AT_UPPER should be returned only for constraints with the Interval set. In this case, they are necessary to distinguish which side of the constraint is active. One-sided constraints (for example, LessThan and GreaterThan) should use NONBASIC instead of the NONBASIC_AT_* values.

This restriction does not apply to VariableBasisStatus, which should return NONBASIC_AT_* regardless of whether the alternative bound exists.

**Values**

**BASIC**

The element is in the basis.

**NONBASIC**

The element is not in the basis.

**NONBASIC_AT_LOWER**

The element is not in the basis and is at its lower bound.

**NONBASIC_AT_UPPER**

The element is not in the basis and is at its upper bound.

**SUPER_BASIC**

The element is not in the basis but is also not at one of its bounds.

In a linear program, this status occurs when a variable with no bounds is not in the basis, for example, because it takes the value 0.0.

source

MathOptInterface.BASIC – Constant.

```
BASIC::BasisStatusCode
```

An instance of the BasisStatusCode enum.

**About**

The element is in the basis.

source

MathOptInterface.NONBASIC – Constant.

```
NONBASIC::BasisStatusCode
```

An instance of the BasisStatusCode enum.

**About**

The element is not in the basis.

source

`MathOptInterface.NONBASIC_AT_LOWER` – Constant.

```
NONBASIC_AT_LOWER::BasisStatusCode
```

An instance of the `BasisStatusCode` enum.

**About**

The element is not in the basis and is at its lower bound.

source

`MathOptInterface.NONBASIC_AT_UPPER` – Constant.

```
NONBASIC_AT_UPPER::BasisStatusCode
```

An instance of the `BasisStatusCode` enum.

**About**

The element is not in the basis and is at its upper bound.

source

`MathOptInterface.SUPER_BASIC` – Constant.

```
SUPER_BASIC::BasisStatusCode
```

An instance of the `BasisStatusCode` enum.

**About**

The element is not in the basis but is also not at one of its bounds.

In a linear program, this status occurs when a variable with no bounds is not in the basis, for example, because it takes the value `0.0`.

source

`MathOptInterface.LagrangeMultiplier` – Type.

```
LagrangeMultiplier(result_index::Int = 1)
```

An `AbstractConstraintAttribute` for the Lagrange multiplier associated with a constraint.

**Relationship to `ConstraintDual`**

This attribute differs from `ConstraintDual` in one important case. When there is a `VectorNonlinearOracle` constraint of the form:

$$x \in VectorNonlinearOracle$$

the associated `ConstraintDual` is $\mu^\top \nabla f(x)$, and the value of `LagrangeMultiplier` is the vector $\mu$ directly.

Both values are useful in different circumstances.

**DualStatus**

Before querying this attribute you should first check `DualStatus` to confirm that a dual solution is available.

If the `DualStatus` is `NO_SOLUTION` the result of querying this attribute is undefined.

**`result_index`**

The optimizer may return multiple dual solutions. See `ResultCount` for information on how the results are ordered.

If the solver does not have a dual value for the constraint because the `result_index` is beyond the available solutions (whose number is indicated by the `ResultCount` attribute), getting this attribute must throw a `ResultIndexBoundsError`.

**Implementation**

Optimizers should implement the following methods:

```
MOI.get(::Optimizer, ::MOI.LagrangeMultiplier, ::MOI.ConstraintIndex)
```

They should not implement `set` or `supports`.

Solvers should implement `LagrangeMultiplier` only if they also implement the `ConstraintDual`, and only if the two values are different.

source

`MathOptInterface.LagrangeMultiplierStart` – Type.

```
LagrangeMultiplierStart()
```

An `AbstractConstraintAttribute` for the initial assignment to the constraint's `LagrangeMultiplier` that the optimizer may use to warm-start the solve.

May be `nothing` (unset), a number for `AbstractScalarFunction`, or a vector for `AbstractVectorFunction`.

source

# Chapter 22

# Modifications

`MathOptInterface.modify` – Function.

```
modify(
    model::ModelLike,
    ci::ConstraintIndex,
    change::AbstractFunctionModification,
)
```

Apply the modification specified by `change` to the function of constraint `ci`.

An `ModifyConstraintNotAllowed` error is thrown if modifying constraints is not supported by the model `model`.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model);

julia> ci = MOI.add_constraint(model, 1.0 * x, MOI.EqualTo(1.0));

julia> MOI.modify(model, ci, MOI.ScalarConstantChange(10.0))

julia> print(model)
Feasibility

Subject to:

ScalarAffineFunction{Float64}-in-EqualTo{Float64}
 10.0 + 1.0 v[1] == 1.0
```

source

```
modify(
    model::ModelLike,
    cis::AbstractVector{<:ConstraintIndex},
    changes::AbstractVector{<:AbstractFunctionModification},
)
```

Apply multiple modifications specified by changes to the functions of constraints `cis`.

A `ModifyConstraintNotAllowed` error is thrown if modifying constraints is not supported by `model`.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variables(model, 2);

julia> ci = MOI.add_constraint.(model, 1.0 .* x, MOI.EqualTo(1.0));

julia> MOI.modify(model, ci, MOI.ScalarCoefficientChange.(x, [2.0, 0.5]))

julia> print(model)
Feasibility

Subject to:

ScalarAffineFunction{Float64}-in-EqualTo{Float64}
 0.0 + 2.0 v[1] == 1.0
 0.0 + 0.5 v[2] == 1.0
```

source

```
modify(model::ModelLike, ::ObjectiveFunction, change::AbstractFunctionModification)
```

Apply the modification specified by change to the objective function of `model`. To change the function completely, call set instead.

An `ModifyObjectiveNotAllowed` error is thrown if modifying objectives is not supported by the model `model`.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model);

julia> MOI.set(model, MOI.ObjectiveSense(), MOI.MIN_SENSE)

julia> f = 1.0 * x;

julia> attr = MOI.ObjectiveFunction{typeof(f)}()
MathOptInterface.ObjectiveFunction{MathOptInterface.ScalarAffineFunction{Float64}}()

julia> MOI.set(model, attr, f)

julia> MOI.modify(model, attr, MOI.ScalarConstantChange(10.0))

julia> print(model)
Minimize ScalarAffineFunction{Float64}:
 10.0 + 1.0 v[1]

Subject to:
```

source

```
modify(
    model::ModelLike,
    attr::ObjectiveFunction,
    changes::AbstractVector{<:AbstractFunctionModification},
)
```

Apply multiple modifications specified by changes to the functions of constraints cis.

A ModifyObjectiveNotAllowed error is thrown if modifying objective coefficients is not supported by model.

**Example**

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variables(model, 2);

julia> MOI.set(model, MOI.ObjectiveSense(), MOI.MIN_SENSE)

julia> f = 1.0 * x[1] + 1.0 * x[2];

julia> attr = MOI.ObjectiveFunction{typeof(f)}()
MathOptInterface.ObjectiveFunction{MathOptInterface.ScalarAffineFunction{Float64}}()

julia> MOI.set(model, attr, f)

julia> MOI.modify(model, attr, MOI.ScalarCoefficientChange.(x, [2.0, 0.5]))

julia> print(model)
Minimize ScalarAffineFunction{Float64}:
 0.0 + 2.0 v[1] + 0.5 v[2]

Subject to:
```

source

MathOptInterface.AbstractFunctionModification – Type.

```
AbstractFunctionModification
```

An abstract supertype for structs which specify partial modifications to functions, to be used for making small modifications instead of replacing the functions entirely.

source

MathOptInterface.ScalarConstantChange – Type.

```
ScalarConstantChange{T}(new_constant::T)
```

A struct used to request a change in the constant term of a scalar-valued function.

Applicable to ScalarAffineFunction and ScalarQuadraticFunction.

source

MathOptInterface.VectorConstantChange – Type.

```
VectorConstantChange{T}(new_constant::Vector{T})
```

A struct used to request a change in the constant vector of a vector-valued function.

Applicable to VectorAffineFunction and VectorQuadraticFunction.

source

MathOptInterface.ScalarCoefficientChange – Type.

```
ScalarCoefficientChange{T}(variable::VariableIndex, new_coefficient::T)
```

A struct used to request a change in the linear coefficient of a single variable in a scalar-valued function.

Applicable to ScalarAffineFunction and ScalarQuadraticFunction.

source

MathOptInterface.ScalarQuadraticCoefficientChange – Type.

```
ScalarQuadraticCoefficientChange{T}(
    variable_1::VariableIndex,
    variable_2::VariableIndex,
    new_coefficient::T,
)
```

A struct used to request a change in the quadratic coefficient of a ScalarQuadraticFunction.

**Scaling factors**

A ScalarQuadraticFunction has an implicit 0.5 scaling factor in front of the Q matrix. This modification applies to terms in the Q matrix.

If variable_1 == variable_2, this modification sets the corresponding diagonal element of the Q matrix to new_coefficient.

If variable_1 != variable_2, this modification is equivalent to setting both the corresponding upper- and lower-triangular elements of the Q matrix to new_coefficient.

As a consequence:

- to modify the term x^2 to become 2x^2, new_coefficient must be 4

- to modify the term xy to become 2xy, `new_coefficient` must be 2

source

`MathOptInterface.MultirowChange` – Type.

```
MultirowChange{T}(
    variable::VariableIndex,
    new_coefficients::Vector{Tuple{Int64,T}},
) where {T}
```

A struct used to request a change in the linear coefficients of a single variable in a vector-valued function.

New coefficients are specified by `(output_index, coefficient)` tuples.

Applicable to `VectorAffineFunction` and `VectorQuadraticFunction`.

source

# Chapter 23

# Nonlinear programming

## 23.1 Types

`MathOptInterface.AbstractNLPEvaluator` – Type.

```
AbstractNLPEvaluator
```

Abstract supertype for the callback object that is used to query function values, derivatives, and expression graphs.

It is used in `NLPBlockData`.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true);

julia> supertype(typeof(evaluator))
MathOptInterface.AbstractNLPEvaluator
```

source

`MathOptInterface.NLPBoundsPair` – Type.

```
NLPBoundsPair(lower::Float64, upper::Float64)
```

A struct holding a pair of lower and upper bounds.

`-Inf` and `Inf` can be used to indicate no lower or upper bound, respectively.

**Example**

```
julia> bounds = MOI.NLPBoundsPair.([25.0, 40.0], [Inf, 40.0])
2-element Vector{MathOptInterface.NLPBoundsPair}:
 MathOptInterface.NLPBoundsPair(25.0, Inf)
 MathOptInterface.NLPBoundsPair(40.0, 40.0)
```

source

MathOptInterface.NLPBlockData – Type.

```
struct NLPBlockData
    constraint_bounds::Vector{NLPBoundsPair}
    evaluator::AbstractNLPEvaluator
    has_objective::Bool
end
```

A struct encoding a set of nonlinear constraints of the form $lb \leq g(x) \leq ub$ and, if `has_objective == true`, a nonlinear objective function $f(x)$.

Nonlinear objectives override any objective set by using the `ObjectiveFunction` attribute.

The `evaluator` is a callback object that is used to query function values, derivatives, and expression graphs. If `has_objective == false`, then it is an error to query properties of the objective function, and in Hessian-of-the-Lagrangian queries, σ must be set to zero.

**Note**

Throughout the evaluator, all variables are ordered according to `ListOfVariableIndices`. Hence, MOI copies of nonlinear problems must not re-order variables.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> model = MOI.Utilities.UniversalFallback(MOI.Utilities.Model{Float64}());

julia> block = MOI.NLPBlockData(
           MOI.NLPBoundsPair.([25.0, 40.0], [Inf, 40.0]),
           MOI.Test.HS071(true),
           true,
       );

julia> MOI.set(model, MOI.NLPBlock(), block)
```

source

## 23.2 Attributes

MathOptInterface.NLPBlock – Type.

```
NLPBlock()
```

An `AbstractModelAttribute` that stores an `NLPBlockData`, representing a set of nonlinear constraints, and optionally a nonlinear objective.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> model = MOI.Utilities.UniversalFallback(MOI.Utilities.Model{Float64}());

julia> block = MOI.NLPBlockData(
           MOI.NLPBoundsPair.([25.0, 40.0], [Inf, 40.0]),
           MOI.Test.HS071(true),
           true,
       );

julia> MOI.set(model, MOI.NLPBlock(), block)
```

source

`MathOptInterface.NLPBlockDual` – Type.

```
NLPBlockDual(result_index::Int = 1)
```

An `AbstractModelAttribute` for the Lagrange multipliers on the constraints from the `NLPBlock` in result `result_index`.

If `result_index` is omitted, it is 1 by default.

**Example**

```
julia> MOI.NLPBlockDual()
MathOptInterface.NLPBlockDual(1)

julia> MOI.NLPBlockDual(2)
MathOptInterface.NLPBlockDual(2)
```

source

`MathOptInterface.NLPBlockDualStart` – Type.

```
NLPBlockDualStart()
```

An `AbstractModelAttribute` for the initial assignment of the Lagrange multipliers on the constraints from the `NLPBlock` that the solver may use to warm-start the solve.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> model = MOI.Utilities.UniversalFallback(MOI.Utilities.Model{Float64}());

julia> block = MOI.NLPBlockData(
           MOI.NLPBoundsPair.([25.0, 40.0], [Inf, 40.0]),
           MOI.Test.HS071(true),
           true,
       );
```

```
julia> MOI.set(model, MOI.NLPBlock(), block)

julia> MOI.set(model, MOI.NLPBlockDualStart(), [1.0, 2.0])
```

source

## 23.3 Functions

`MathOptInterface.initialize` – Function.

```
initialize(
    d::AbstractNLPEvaluator,
    requested_features::Vector{Symbol},
)::Nothing
```

Initialize `d` with the set of features in `requested_features`. Check `features_available` before calling `initialize` to see what features are supported by `d`.

> **Warning**
>
> This method must be called before any other methods.

**Features**

The following features are defined:

- `:Grad`: enables `eval_objective_gradient`
- `:Jac`: enables `eval_constraint_jacobian` and `eval_constraint_gradient`
- `:JacVec`: enables `eval_constraint_jacobian_product` and `eval_constraint_jacobian_transpose_product`
- `:Hess`: enables `eval_hessian_lagrangian`
- `:HessVec`: enables `eval_hessian_lagrangian_product`
- `:ExprGraph`: enables `objective_expr` and `constraint_expr`.

In all cases, including when `requested_features` is empty, `eval_objective` and `eval_constraint` are supported.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true);

julia> MOI.initialize(evaluator, [:Grad, :Jac])
```

source

`MathOptInterface.features_available` – Function.

```
features_available(d::AbstractNLPEvaluator)::Vector{Symbol}
```

Returns the subset of features available for this problem instance.

See `initialize` for the list of defined features.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true, true);

julia> MOI.features_available(evaluator)
6-element Vector{Symbol}:
 :Grad
 :Jac
 :JacVec
 :ExprGraph
 :Hess
 :HessVec
```

source

`MathOptInterface.eval_objective` – Function.

```
eval_objective(d::AbstractNLPEvaluator, x::AbstractVector{T})::T where {T}
```

Evaluate the objective $f(x)$, returning a scalar value.

**Initialize**

Before calling this function, you must call `initialize`, but you do not need to pass a value.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true);

julia> MOI.initialize(evaluator, Symbol[])

julia> MOI.eval_objective(evaluator, [1.0, 2.0, 3.0, 4.0])
27.0
```

source

`MathOptInterface.eval_constraint` – Function.

```
eval_constraint(
    d::AbstractNLPEvaluator,
    g::AbstractVector{T},
    x::AbstractVector{T},
)::Nothing where {T}
```

Given a set of vector-valued constraints $l \leq g(x) \leq u$, evaluate the constraint function $g(x)$, storing the result in the vector `g`.

**Initialize**

Before calling this function, you must call `initialize`, but you do not need to pass a value.

**Implementation notes**

When implementing this method, you must not assume that `g` is `Vector{Float64}`, but you may assume that it supports `setindex!` and `length`. For example, it may be the `view` of a vector.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true);

julia> MOI.initialize(evaluator, Symbol[])

julia> g = fill(NaN, 2);

julia> MOI.eval_constraint(evaluator, g, [1.0, 2.0, 3.0, 4.0])

julia> g
2-element Vector{Float64}:
 24.0
 30.0
```

source

`MathOptInterface.eval_objective_gradient` – Function.

```
eval_objective_gradient(
    d::AbstractNLPEvaluator,
    grad::AbstractVector{T},
    x::AbstractVector{T},
)::Nothing where {T}
```

Evaluate the gradient of the objective function $grad = \nabla f(x)$ as a dense vector, storing the result in the vector `grad`.

**Initialize**

Before calling this function, you must call `initialize` with `:Grad`.

**Implementation notes**

When implementing this method, you must not assume that `grad` is `Vector{Float64}`, but you may assume that it supports `setindex!` and `length`. For example, it may be the `view` of a vector.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true);

julia> MOI.initialize(evaluator, Symbol[:Grad])

julia> grad = fill(NaN, 4);

julia> MOI.eval_objective_gradient(evaluator, grad, [1.0, 2.0, 3.0, 4.0])

julia> grad
4-element Vector{Float64}:
 28.0
  4.0
  5.0
  6.0
```

source

`MathOptInterface.jacobian_structure` – Function.

```
jacobian_structure(d::AbstractNLPEvaluator)::Vector{Tuple{Int64,Int64}}
```

Returns a vector of tuples, `(row, column)`, where each indicates the position of a structurally nonzero element in the Jacobian matrix: $J_g(x) = \begin{bmatrix} \nabla g_1(x) \\ \nabla g_2(x) \\ \vdots \\ \nabla g_m(x) \end{bmatrix}$, where $g_i$ is the $i$th component of the nonlinear constraints $g(x)$.

The indices are not required to be sorted and can contain duplicates, in which case the solver should combine the corresponding elements by adding them together.

The sparsity structure is assumed to be independent of the point $x$.

**Initialize**

Before calling this function, you must call `initialize` with `:Jac`.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true);

julia> MOI.initialize(evaluator, Symbol[:Jac])

julia> MOI.jacobian_structure(evaluator)
8-element Vector{Tuple{Int64, Int64}}:
 (1, 1)
 (1, 2)
 (1, 3)
 (1, 4)
 (2, 1)
 (2, 2)
 (2, 3)
 (2, 4)
```

source

MathOptInterface.eval_constraint_gradient – Function.

```
eval_constraint_gradient(
    d::AbstractNLPEvaluator,
    ∇g::AbstractVector{T},
    x::AbstractVector{T},
    i::Int,
)::Nothing where {T}
```

Evaluate the gradient of constraint `i`, $\nabla g_i(x)$, and store the non-zero values in `∇g`, corresponding to the structure returned by `constraint_gradient_structure`.

**Implementation notes**

When implementing this method, you must not assume that `∇g` is `Vector{Float64}`, but you may assume that it supports `setindex!` and `length`. For example, it may be the `view` of a vector.

**Initialize**

Before calling this function, you must call `initialize` with `:Jac`.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true);

julia> MOI.initialize(evaluator, Symbol[:Jac])

julia> indices = MOI.constraint_gradient_structure(evaluator, 1);

julia> ∇g = zeros(length(indices));

julia> MOI.eval_constraint_gradient(evaluator, ∇g, [1.0, 2.0, 3.0, 4.0], 1)

julia> ∇g
4-element Vector{Float64}:
 24.0
 12.0
  8.0
  6.0
```

source

MathOptInterface.constraint_gradient_structure – Function.

```
constraint_gradient_structure(d::AbstractNLPEvaluator, i::Int)::Vector{Int64}
```

Returns a vector of indices, where each element indicates the position of a structurally nonzero element in the gradient of constraint $\nabla g_i(x)$.

The indices are not required to be sorted and can contain duplicates, in which case the solver should combine the corresponding elements by adding them together.

The sparsity structure is assumed to be independent of the point $x$.

**Initialize**

Before calling this function, you must call `initialize` with `:Jac`.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true);

julia> MOI.initialize(evaluator, Symbol[:Jac])

julia> indices = MOI.constraint_gradient_structure(evaluator, 1)
4-element Vector{Int64}:
 1
 2
 3
 4
```

source

`MathOptInterface.eval_constraint_jacobian` – Function.

```
eval_constraint_jacobian(
    d::AbstractNLPEvaluator,
    J::AbstractVector{T},
    x::AbstractVector{T},
)::Nothing where {T}
```

Evaluates the sparse Jacobian matrix $J_g(x) = \begin{bmatrix} \nabla g_1(x) \\ \nabla g_2(x) \\ \vdots \\ \nabla g_m(x) \end{bmatrix}$.

The result is stored in the vector `J` in the same order as the indices returned by `jacobian_structure`.

**Implementation notes**

When implementing this method, you must not assume that `J` is `Vector{Float64}`, but you may assume that it supports `setindex!` and `length`. For example, it may be the `view` of a vector.

**Initialize**

Before calling this function, you must call `initialize` with `:Hess`.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true);

julia> MOI.initialize(evaluator, Symbol[:Jac])
```

```
julia> J_indices = MOI.jacobian_structure(evaluator);

julia> J = zeros(length(J_indices));

julia> MOI.eval_constraint_jacobian(evaluator, J, [1.0, 2.0, 3.0, 4.0])

julia> J
8-element Vector{Float64}:
 24.0
 12.0
  8.0
  6.0
  2.0
  4.0
  6.0
  8.0
```

source

`MathOptInterface.eval_constraint_jacobian_product` – Function.

```
eval_constraint_jacobian_product(
    d::AbstractNLPEvaluator,
    y::AbstractVector{T},
    x::AbstractVector{T},
    w::AbstractVector{T},
)::Nothing where {T}
```

Computes the Jacobian-vector product $y = J_g(x)w$, storing the result in the vector `y`.

The vectors have dimensions such that `length(w) == length(x)`, and `length(y)` is the number of nonlinear constraints.

**Implementation notes**

When implementing this method, you must not assume that `y` is `Vector{Float64}`, but you may assume that it supports `setindex!` and `length`. For example, it may be the `view` of a vector.

**Initialize**

Before calling this function, you must call `initialize` with `:JacVec`.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true);

julia> MOI.initialize(evaluator, Symbol[:Jac, :JacVec])

julia> y = zeros(2);

julia> x = [1.0, 2.0, 3.0, 4.0];
```

```
julia> w = [1.5, 2.5, 3.5, 4.5];

julia> MOI.eval_constraint_jacobian_product(evaluator, y, x, w)

julia> y
2-element Vector{Float64}:
 121.0
  70.0
```

source

`MathOptInterface.eval_constraint_jacobian_transpose_product` – Function.

```
eval_constraint_jacobian_transpose_product(
    d::AbstractNLPEvaluator,
    y::AbstractVector{T},
    x::AbstractVector{T},
    w::AbstractVector{T},
)::Nothing where {T}
```

Computes the Jacobian-transpose-vector product $y = J_g(x)^T w$, storing the result in the vector `y`.

The vectors have dimensions such that `length(y) == length(x)`, and `length(w)` is the number of nonlinear constraints.

**Implementation notes**

When implementing this method, you must not assume that `y` is `Vector{Float64}`, but you may assume that it supports `setindex!` and `length`. For example, it may be the `view` of a vector.

**Initialize**

Before calling this function, you must call `initialize` with `:JacVec`.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true);

julia> MOI.initialize(evaluator, Symbol[:Jac, :JacVec])

julia> y = zeros(4);

julia> x = [1.0, 2.0, 3.0, 4.0];

julia> w = [1.5, 2.5];

julia> MOI.eval_constraint_jacobian_transpose_product(evaluator, y, x, w)

julia> y
4-element Vector{Float64}:
 41.0
 28.0
 27.0
 29.0
```

source

MathOptInterface.hessian_lagrangian_structure – Function.

```
hessian_lagrangian_structure(
    d::AbstractNLPEvaluator,
)::Vector{Tuple{Int64,Int64}}
```

Returns a vector of tuples, `(row, column)`, where each indicates the position of a structurally nonzero element in the Hessian-of-the-Lagrangian matrix: $\nabla^2 f(x) + \sum_{i=1}^m \nabla^2 g_i(x)$.

The indices are not required to be sorted and can contain duplicates, in which case the solver should combine the corresponding elements by adding them together.

Any mix of lower and upper-triangular indices is valid. Elements `(i, j)` and `(j, i)`, if both present, should be treated as duplicates.

The sparsity structure is assumed to be independent of the point $x$.

**Initialize**

Before calling this function, you must call `initialize` with `:Hess`.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true);

julia> MOI.initialize(evaluator, Symbol[:Hess])

julia> MOI.hessian_lagrangian_structure(evaluator)
10-element Vector{Tuple{Int64, Int64}}:
 (1, 1)
 (2, 1)
 (2, 2)
 (3, 1)
 (3, 2)
 (3, 3)
 (4, 1)
 (4, 2)
 (4, 3)
 (4, 4)
```

source

MathOptInterface.hessian_objective_structure – Function.

```
hessian_objective_structure(
    d::AbstractNLPEvaluator,
)::Vector{Tuple{Int64,Int64}}
```

Returns a vector of tuples, `(row, column)`, where each indicates the position of a structurally nonzero element in the Hessian matrix: $\nabla^2 f(x)$.

The indices are not required to be sorted and can contain duplicates, in which case the solver should combine the corresponding elements by adding them together.

Any mix of lower and upper-triangular indices is valid. Elements `(i, j)` and `(j, i)`, if both present, should be treated as duplicates.

The sparsity structure is assumed to be independent of the point $x$.

**Initialize**

Before calling this function, you must call `initialize` with `:Hess`.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true);

julia> MOI.initialize(evaluator, Symbol[:Hess])

julia> MOI.hessian_objective_structure(evaluator)
6-element Vector{Tuple{Int64, Int64}}:
 (1, 1)
 (2, 1)
 (3, 1)
 (4, 1)
 (4, 2)
 (4, 3)
```

source

`MathOptInterface.hessian_constraint_structure` – Function.

```
hessian_constraint_structure(
    d::AbstractNLPEvaluator,
    i::Int64,
)::Vector{Tuple{Int64,Int64}}
```

Returns a vector of tuples, `(row, column)`, where each indicates the position of a structurally nonzero element in the Hessian matrix: $\nabla^2 g_i(x)$.

The indices are not required to be sorted and can contain duplicates, in which case the solver should combine the corresponding elements by adding them together.

Any mix of lower and upper-triangular indices is valid. Elements `(i, j)` and `(j, i)`, if both present, should be treated as duplicates.

The sparsity structure is assumed to be independent of the point $x$.

**Initialize**

Before calling this function, you must call `initialize` with `:Hess`.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true);

julia> MOI.initialize(evaluator, Symbol[:Hess])

julia> MOI.hessian_constraint_structure(evaluator, 1)
6-element Vector{Tuple{Int64, Int64}}:
 (2, 1)
 (3, 1)
 (3, 2)
 (4, 1)
 (4, 2)
 (4, 3)

julia> MOI.hessian_constraint_structure(evaluator, 2)
4-element Vector{Tuple{Int64, Int64}}:
 (1, 1)
 (2, 2)
 (3, 3)
 (4, 4)
```

source

`MathOptInterface.eval_hessian_objective` – Function.

```
eval_hessian_objective(
    d::AbstractNLPEvaluator,
    H::AbstractVector{T},
    x::AbstractVector{T},
)::Nothing where {T}
```

This function computes the sparse Hessian matrix: $\nabla^2 f(x)$, storing the result in the vector `H` in the same order as the indices returned by `hessian_objective_structure`.

**Implementation notes**

When implementing this method, you must not assume that `H` is `Vector{Float64}`, but you may assume that it supports `setindex!` and `length`. For example, it may be the `view` of a vector.

**Initialize**

Before calling this function, you must call `initialize` with `:Hess`.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true, true);

julia> MOI.initialize(evaluator, Symbol[:Hess])

julia> indices = MOI.hessian_objective_structure(evaluator);

julia> H = zeros(length(indices));

julia> x = [1.0, 2.0, 3.0, 4.0];
```

```
julia> MOI.eval_hessian_objective(evaluator, H, x)

julia> H
6-element Vector{Float64}:
 8.0
 4.0
 4.0
 7.0
 1.0
 1.0
```

source

MathOptInterface.eval_hessian_constraint – Function.

```
eval_hessian_constraint(
    d::AbstractNLPEvaluator,
    H::AbstractVector{T},
    x::AbstractVector{T},
    i::Int64,
)::Nothing where {T}
```

This function computes the sparse Hessian matrix: $\nabla^2 g_i(x)$, storing the result in the vector H in the same order as the indices returned by hessian_constraint_structure.

**Implementation notes**

When implementing this method, you must not assume that H is Vector{Float64}, but you may assume that it supports setindex! and length. For example, it may be the view of a vector.

**Initialize**

Before calling this function, you must call initialize with :Hess.

**Example**

This example uses the Test.HS071 evaluator.

```
julia> evaluator = MOI.Test.HS071(true, true);

julia> MOI.initialize(evaluator, Symbol[:Hess])

julia> indices = MOI.hessian_constraint_structure(evaluator, 1);

julia> H = zeros(length(indices));

julia> x = [1.0, 2.0, 3.0, 4.0];

julia> MOI.eval_hessian_constraint(evaluator, H, x, 1)

julia> H
6-element Vector{Float64}:
 12.0
  8.0
```

```
 4.0
 6.0
 3.0
 2.0
```

source

`MathOptInterface.eval_hessian_lagrangian` – Function.

```
eval_hessian_lagrangian(
    d::AbstractNLPEvaluator,
    H::AbstractVector{T},
    x::AbstractVector{T},
    σ::T,
    μ::AbstractVector{T},
)::Nothing where {T}
```

Given scalar weight σ and vector of constraint weights μ, this function computes the sparse Hessian-of-the-Lagrangian matrix: $\sigma\nabla^2 f(x) + \sum_{i=1}^{m} \mu_i \nabla^2 g_i(x)$, storing the result in the vector H in the same order as the indices returned by `hessian_lagrangian_structure`.

**Implementation notes**

When implementing this method, you must not assume that H is `Vector{Float64}`, but you may assume that it supports `setindex!` and `length`. For example, it may be the `view` of a vector.

**Initialize**

Before calling this function, you must call `initialize` with `:Hess`.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true);

julia> MOI.initialize(evaluator, Symbol[:Hess])

julia> indices = MOI.hessian_lagrangian_structure(evaluator);

julia> H = zeros(length(indices));

julia> x = [1.0, 2.0, 3.0, 4.0];

julia> σ = 1.0;

julia> μ = [1.0, 1.0];

julia> MOI.eval_hessian_lagrangian(evaluator, H, x, σ, μ)

julia> H
10-element Vector{Float64}:
 10.0
 16.0
```

```
 2.0
12.0
 4.0
 2.0
13.0
 4.0
 3.0
 2.0
```

source

`MathOptInterface.eval_hessian_lagrangian_product` – Function.

```
eval_hessian_lagrangian_product(
    d::AbstractNLPEvaluator,
    h::AbstractVector{T},
    x::AbstractVector{T},
    v::AbstractVector{T},
    σ::T,
    μ::AbstractVector{T},
)::Nothing where {T}
```

Given scalar weight σ and vector of constraint weights μ, computes the Hessian-of-the-Lagrangian-vector product $h = \left(\sigma \nabla^2 f(x) + \sum_{i=1}^m \mu_i \nabla^2 g_i(x)\right) v$, storing the result in the vector `h`.

The vectors have dimensions such that `length(h) == length(x) == length(v)`.

**Implementation notes**

When implementing this method, you must not assume that `h` is `Vector{Float64}`, but you may assume that it supports `setindex!` and `length`. For example, it may be the `view` of a vector.

**Initialize**

Before calling this function, you must call `initialize` with `:HessVec`.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true, true);

julia> MOI.initialize(evaluator, Symbol[:HessVec])

julia> H = fill(NaN, 4);

julia> x = [1.0, 2.0, 3.0, 4.0];

julia> v = [1.5, 2.5, 3.5, 4.5];

julia> σ = 1.0;

julia> μ = [1.0, 1.0];

julia> MOI.eval_hessian_lagrangian_product(evaluator, H, x, v, σ, μ)
```

```
julia> H
4-element Vector{Float64}:
 155.5
  61.0
  48.5
  49.0
```

source

MathOptInterface.objective_expr – Function.

```
objective_expr(d::AbstractNLPEvaluator)::Expr
```

Returns a Julia `Expr` object representing the expression graph of the objective function.

**Format**

The expression has a number of limitations, compared with arbitrary Julia expressions:

- All sums and products are flattened out as simple `Expr(:+, ...)` and `Expr(:*, ...)` objects.
- All decision variables must be of the form `Expr(:ref, :x, MOI.VariableIndex(i))`, where `i` is the $i$th variable in `ListOfVariableIndices`.
- There are currently no restrictions on recognized functions; typically these will be built-in Julia functions like `^`, `exp`, `log`, `cos`, `tan`, `sqrt`, etc., but modeling interfaces may choose to extend these basic functions, or error if they encounter unsupported functions.

**Initialize**

Before calling this function, you must call `initialize` with `:ExprGraph`.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true);

julia> MOI.initialize(evaluator, [:ExprGraph])

julia> MOI.objective_expr(evaluator)
:(x[MOI.VariableIndex(1)] * x[MOI.VariableIndex(4)] * (x[MOI.VariableIndex(1)] +
↪  x[MOI.VariableIndex(2)] + x[MOI.VariableIndex(3)]) + x[MOI.VariableIndex(3)])
```

source

MathOptInterface.constraint_expr – Function.

```
constraint_expr(d::AbstractNLPEvaluator, i::Integer)::Expr
```

Returns a Julia `Expr` object representing the expression graph for the $i$th nonlinear constraint.

**Format**

The format is the same as `objective_expr`, with an additional comparison operator indicating the sense of and bounds on the constraint.

For single-sided comparisons, the body of the constraint must be on the left-hand side, and the right-hand side must be a constant.

For double-sided comparisons (that is, $l \leq g(x) \leq u$), the body of the constraint must be in the middle, and the left- and right-hand sides must be constants.

The bounds on the constraints must match the `NLPBoundsPair`s passed to `NLPBlockData`.

**Initialize**

Before calling this function, you must call `initialize` with `:ExprGraph`.

**Example**

This example uses the `Test.HS071` evaluator.

```
julia> evaluator = MOI.Test.HS071(true);

julia> MOI.initialize(evaluator, [:ExprGraph])

julia> MOI.constraint_expr(evaluator, 1)
:(x[MOI.VariableIndex(1)] * x[MOI.VariableIndex(2)] * x[MOI.VariableIndex(3)] *
↪  x[MOI.VariableIndex(4)] >= 25.0)

julia> MOI.constraint_expr(evaluator, 2)
:(x[MOI.VariableIndex(1)] ^ 2 + x[MOI.VariableIndex(2)] ^ 2 + x[MOI.VariableIndex(3)] ^ 2 +
↪  x[MOI.VariableIndex(4)] ^ 2 == 40.0)
```

source

# Chapter 24

# Callbacks

`MathOptInterface.AbstractCallback` – Type.

```
abstract type AbstractCallback <: AbstractModelAttribute end
```

Abstract type for a model attribute representing a callback function. The value set to subtypes of `AbstractCallback` is a function that may be called during `optimize!`. As `optimize!` is in progress, the result attributes (that is, the attributes `attr` such that `is_set_by_optimize(attr)`) may not be accessible from the callback, hence trying to get result attributes might throw a `OptimizeInProgress` error.

At most one callback of each type can be registered. If an optimizer already has a function for a callback type, and the user registers a new function, then the old one is replaced.

The value of the attribute should be a function taking only one argument, commonly called `callback_data`, that can be used for instance in `LazyConstraintCallback`, `HeuristicCallback` and `UserCutCallback`.

source

`MathOptInterface.AbstractSubmittable` – Type.

```
AbstractSubmittable
```

Abstract supertype for objects that can be submitted to the model.

source

`MathOptInterface.submit` – Function.

```
submit(
    optimizer::AbstractOptimizer,
    sub::AbstractSubmittable,
    values...,
)::Nothing
```

Submit `values` to the submittable `sub` of the optimizer `optimizer`.

An `UnsupportedSubmittable` error is thrown if `model` does not support the attribute `attr` (see `supports`) and a `SubmitNotAllowed` error is thrown if it supports the submittable `sub` but it cannot be submitted.

source

## 24.1 Attributes

`MathOptInterface.CallbackNodeStatus` – Type.

```
CallbackNodeStatus(callback_data)
```

An optimizer attribute describing the (in)feasibility of the primal solution available from `CallbackVariablePrimal` during a callback identified by `callback_data`.

Returns a `CallbackNodeStatusCode` Enum.

source

`MathOptInterface.CallbackVariablePrimal` – Type.

```
CallbackVariablePrimal(callback_data)
```

An `AbstractVariableAttribute` for the assignment to the variable's primal value during the callback identified by `callback_data`.

source

`MathOptInterface.CallbackNodeStatusCode` – Type.

```
CallbackNodeStatusCode
```

An Enum for the value of the `CallbackNodeStatus` attribute.

**Values**

**`CALLBACK_NODE_STATUS_INTEGER`**

The primal solution available from `CallbackVariablePrimal` is integer feasible.

**`CALLBACK_NODE_STATUS_FRACTIONAL`**

The primal solution available from `CallbackVariablePrimal` is integer infeasible.

**`CALLBACK_NODE_STATUS_UNKNOWN`**

The status of the primal solution available from `CallbackVariablePrimal` is unknown.

source

`MathOptInterface.CALLBACK_NODE_STATUS_INTEGER` – Constant.

```
CALLBACK_NODE_STATUS_INTEGER::CallbackNodeStatusCode
```

An instance of the `CallbackNodeStatusCode` enum.

**About**

The primal solution available from `CallbackVariablePrimal` is integer feasible.

source

MathOptInterface.CALLBACK_NODE_STATUS_FRACTIONAL – Constant.

```
CALLBACK_NODE_STATUS_FRACTIONAL::CallbackNodeStatusCode
```

An instance of the CallbackNodeStatusCode enum.

**About**

The primal solution available from CallbackVariablePrimal is integer infeasible.

source

MathOptInterface.CALLBACK_NODE_STATUS_UNKNOWN – Constant.

```
CALLBACK_NODE_STATUS_UNKNOWN::CallbackNodeStatusCode
```

An instance of the CallbackNodeStatusCode enum.

**About**

The status of the primal solution available from CallbackVariablePrimal is unknown.

source

## 24.2 Lazy constraints

MathOptInterface.LazyConstraintCallback – Type.

```
LazyConstraintCallback() <: AbstractCallback
```

The callback can be used to reduce the feasible set given the current primal solution by submitting a LazyConstraint. For instance, it may be called at an incumbent of a mixed-integer problem. Note that there is no guarantee that the callback is called at every feasible primal solution.

The current primal solution is accessed through CallbackVariablePrimal. Trying to access other result attributes will throw OptimizeInProgress as discussed in AbstractCallback.

**Example**

```
x = MOI.add_variables(optimizer, 8)
MOI.set(optimizer, MOI.LazyConstraintCallback(), callback_data -> begin
    sol = MOI.get(optimizer, MOI.CallbackVariablePrimal(callback_data), x)
    if # should add a lazy constraint
        func = # computes function
        set = # computes set
        MOI.submit(optimizer, MOI.LazyConstraint(callback_data), func, set)
    end
end)
```

source

MathOptInterface.LazyConstraint – Type.

```
LazyConstraint(callback_data)
```

Lazy constraint `func-in-set` submitted as `func, set`. The optimal solution returned by `VariablePrimal` will satisfy all lazy constraints that have been submitted.

This can be submitted only from the `LazyConstraintCallback`. The field `callback_data` is a solver-specific callback type that is passed as the argument to the feasible solution callback.

**Example**

Suppose `x` and `y` are `VariableIndex`s of `optimizer`. To add a `LazyConstraint` for `2x + 3y <= 1`, write

```
func = 2.0x + 3.0y
set = MOI.LessThan(1.0)
MOI.submit(optimizer, MOI.LazyConstraint(callback_data), func, set)
```

inside a `LazyConstraintCallback` of data `callback_data`.

source

## 24.3 User cuts

`MathOptInterface.UserCutCallback` – Type.

```
UserCutCallback() <: AbstractCallback
```

The callback can be used to submit `UserCut` given the current primal solution. For instance, it may be called at fractional (that is, non-integer) nodes in the branch and bound tree of a mixed-integer problem. Note that there is not guarantee that the callback is called everytime the solver has an infeasible solution.

The infeasible solution is accessed through `CallbackVariablePrimal`. Trying to access other result attributes will throw `OptimizeInProgress` as discussed in `AbstractCallback`.

**Example**

```
x = MOI.add_variables(optimizer, 8)
MOI.set(optimizer, MOI.UserCutCallback(), callback_data -> begin
    sol = MOI.get(optimizer, MOI.CallbackVariablePrimal(callback_data), x)
    if # can find a user cut
        func = # computes function
        set = # computes set
        MOI.submit(optimizer, MOI.UserCut(callback_data), func, set)
    end
end
```

source

`MathOptInterface.UserCut` – Type.

```
UserCut(callback_data)
```

Constraint func-to-set suggested to help the solver detect the solution given by `CallbackVariablePrimal` as infeasible. The cut is submitted as the tuple `(func, set)::Tuple{MOI.AbstractFunction,MOI.AbstractSet}`.

Typically `CallbackVariablePrimal` will violate integrality constraints, and a cut would be of the form `ScalarAffineFunction`-in-`LessThan` or `ScalarAffineFunction`-in-`GreaterThan`.

Note that, as opposed to `LazyConstraint`, the provided constraint must not modify the feasible set. The constraint should be redundant, for example, it may be a consequence of affine and integrality constraints.

This can be submitted only from the `UserCutCallback`. The field `callback_data` is a solver-specific callback type that is passed as the argument to the infeasible solution callback.

Note that the solver may silently ignore the provided constraint.

source

## 24.4 Heuristic solutions

`MathOptInterface.HeuristicCallback` – Type.

```
HeuristicCallback() <: AbstractCallback
```

The callback can be used to submit `HeuristicSolution` given the current primal solution. For example, it may be called at fractional (that is, non-integer) nodes in the branch and bound tree of a mixed-integer problem. Note that there is no guarantee that the callback is called every time the solver has an infeasible solution.

The current primal solution is accessed through `CallbackVariablePrimal`. Trying to access other result attributes will throw `OptimizeInProgress` as discussed in `AbstractCallback`.

**Example**

```
x = MOI.add_variables(optimizer, 8)
MOI.set(optimizer, MOI.HeuristicCallback(), callback_data -> begin
    sol = MOI.get(optimizer, MOI.CallbackVariablePrimal(callback_data), x)
    if # can find a heuristic solution
        values = # computes heuristic solution
        MOI.submit(optimizer, MOI.HeuristicSolution(callback_data), x,
                   values)
    end
end
```

source

`MathOptInterface.HeuristicSolution` – Type.

```
HeuristicSolution(callback_data)
```

Heuristically obtained feasible solution. The solution is submitted as `(variables, values)::Tuple{Vector{MOI.VariableI`
where `values[i]` gives the value of `variables[i]`.

The `submit` call returns a `HeuristicSolutionStatus` indicating whether the provided solution was accepted or rejected.

This can be submitted only from the `HeuristicCallback`. The field `callback_data` is a solver-specific callback type that is passed as the argument to the heuristic callback.

Some solvers require a complete solution, others only partial solutions.

source

`MathOptInterface.HeuristicSolutionStatus` – Type.

```
HeuristicSolutionStatus
```

An Enum of possible return values for `submit` with `HeuristicSolution`.

This status informs whether the heuristic solution was accepted or rejected.

**Values**

**`HEURISTIC_SOLUTION_ACCEPTED`**

The heuristic solution was accepted.

**`HEURISTIC_SOLUTION_REJECTED`**

The heuristic solution was rejected.

**`HEURISTIC_SOLUTION_UNKNOWN`**

No information available on the acceptance.

source

`MathOptInterface.HEURISTIC_SOLUTION_ACCEPTED` – Constant.

```
HEURISTIC_SOLUTION_ACCEPTED::HeuristicSolutionStatus
```

An instance of the `HeuristicSolutionStatus` enum.

**About**

The heuristic solution was accepted.

source

`MathOptInterface.HEURISTIC_SOLUTION_REJECTED` – Constant.

```
HEURISTIC_SOLUTION_REJECTED::HeuristicSolutionStatus
```

An instance of the `HeuristicSolutionStatus` enum.

**About**

The heuristic solution was rejected.

source

`MathOptInterface.HEURISTIC_SOLUTION_UNKNOWN` – Constant.

```
HEURISTIC_SOLUTION_UNKNOWN::HeuristicSolutionStatus
```

An instance of the `HeuristicSolutionStatus` enum.

**About**

No information available on the acceptance.

source

# Chapter 25

# Errors

When an MOI call fails on a model, precise errors should be thrown when possible instead of simply calling `error` with a message. The docstrings for the respective methods describe the errors that the implementation should throw in certain situations. This error-reporting system allows code to distinguish between internal errors (that should be shown to the user) and unsupported operations which may have automatic workarounds.

When an invalid index is used in an MOI call, an `InvalidIndex` is thrown:

`MathOptInterface.InvalidIndex` – Type.

```
struct InvalidIndex{IndexType<:Index} <: Exception
    index::IndexType
end
```

An error indicating that the index `index` is invalid.

source

When an invalid result index is used to retrieve an attribute, a `ResultIndexBoundsError` is thrown:

`MathOptInterface.ResultIndexBoundsError` – Type.

```
struct ResultIndexBoundsError{AttrType} <: Exception
    attr::AttrType
    result_count::Int
end
```

An error indicating that the requested attribute `attr` could not be retrieved, because the solver returned too few results compared to what was requested. For instance, the user tries to retrieve `VariablePrimal(2)` when only one solution is available, or when the model is infeasible and has no solution.

See also: `check_result_index_bounds`.

source

`MathOptInterface.check_result_index_bounds` – Function.

```
check_result_index_bounds(model::ModelLike, attr)
```

This function checks whether enough results are available in the `model` for the requested `attr`, using its `result_index` field. If the model does not have sufficient results to answer the query, it throws a `ResultIndexBoundsError`.

source

As discussed in JuMP mapping, for scalar constraint with a nonzero function constant, a `ScalarFunctionConstantNotZero` exception may be thrown:

`MathOptInterface.ScalarFunctionConstantNotZero` – Type.

```
struct ScalarFunctionConstantNotZero{T, F, S} <: Exception
    constant::T
end
```

An error indicating that the constant part of the function in the constraint F-in-S is nonzero.

source

Some `VariableIndex` constraints cannot be combined on the same variable:

`MathOptInterface.LowerBoundAlreadySet` – Type.

```
LowerBoundAlreadySet{S1, S2}
```

Error thrown when setting a `VariableIndex`-in-`S2` when a `VariableIndex`-in-`S1` has already been added and the sets `S1`, `S2` both set a lower bound, that is, they are `EqualTo`, `GreaterThan`, `Interval`, `Semicontinuous` or `Semiinteger`.

source

`MathOptInterface.UpperBoundAlreadySet` – Type.

```
UpperBoundAlreadySet{S1, S2}
```

Error thrown when setting a `VariableIndex`-in-`S2` when a `VariableIndex`-in-`S1` has already been added and the sets `S1`, `S2` both set an upper bound, that is, they are `EqualTo`, `LessThan`, `Interval`, `Semicontinuous` or `Semiinteger`.

source

As discussed in `AbstractCallback`, trying to `get` attributes inside a callback may throw:

`MathOptInterface.OptimizeInProgress` – Type.

```
struct OptimizeInProgress{AttrType<:AnyAttribute} <: Exception
    attr::AttrType
end
```

Error thrown from optimizer when `MOI.get(optimizer, attr)` is called inside an `AbstractCallback` while it is only defined once `optimize!` has completed. This can only happen when `is_set_by_optimize(attr)` is `true`.

source

Trying to submit the wrong type of `AbstractSubmittable` inside an `AbstractCallback` (for example, a `UserCut` inside a `LazyConstraintCallback`) will throw:

`MathOptInterface.InvalidCallbackUsage` – Type.

```
struct InvalidCallbackUsage{C, S} <: Exception
    callback::C
    submittable::S
end
```

An error indicating that `submittable` cannot be submitted inside `callback`.

For example, `UserCut` cannot be submitted inside `LazyConstraintCallback`.

source

The rest of the errors defined in MOI fall in two categories represented by the following two abstract types:

`MathOptInterface.UnsupportedError` – Type.

```
UnsupportedError <: Exception
```

Abstract type for error thrown when an element is not supported by the model.

source

`MathOptInterface.NotAllowedError` – Type.

```
NotAllowedError <: Exception
```

Abstract type for error thrown when an operation is supported but cannot be applied in the current state of the model.

source

The different `UnsupportedError` and `NotAllowedError` are the following errors:

`MathOptInterface.UnsupportedAttribute` – Type.

```
struct UnsupportedAttribute{AttrType} <: UnsupportedError
    attr::AttrType
    message::String
end
```

An error indicating that the attribute `attr` is not supported by the model, that is, that `supports` returns `false`.

source

`MathOptInterface.GetAttributeNotAllowed` – Type.

```
struct GetAttributeNotAllowed{AttrType} <: NotAllowedError
    attr::AttrType
    message::String
end
```

An error indicating that the attribute `attr` cannot be got for some reason (see the error string).

source

`MathOptInterface.SetAttributeNotAllowed` – Type.

```
struct SetAttributeNotAllowed{AttrType} <: NotAllowedError
    attr::AttrType
    message::String
end
```

An error indicating that the attribute `attr` is supported (see `supports`) but cannot be set for some reason (see the error string).

source

`MathOptInterface.AddVariableNotAllowed` – Type.

```
struct AddVariableNotAllowed <: NotAllowedError
    message::String # Human-friendly explanation why the attribute cannot be set
end
```

An error indicating that variables cannot be added to the model.

source

`MathOptInterface.UnsupportedConstraint` – Type.

```
struct UnsupportedConstraint{F<:AbstractFunction,S<:AbstractSet} <: UnsupportedError
    message::String
end
```

An error indicating that constraints of type F-in-S are not supported by the model, that is, that `supports_constraint` returns `false`.

```
julia> showerror(stdout, MOI.UnsupportedConstraint{MOI.VariableIndex,MOI.ZeroOne}())
UnsupportedConstraint{
    MathOptInterface.VariableIndex,
    MathOptInterface.ZeroOne,
}

This constraint type is not supported by the solver.

To fix this error you must choose a different solver.
```

source

MathOptInterface.AddConstraintNotAllowed – Type.

```
struct AddConstraintNotAllowed{F<:AbstractFunction, S<:AbstractSet} <: NotAllowedError
    message::String # Human-friendly explanation why the attribute cannot be set
end
```

An error indicating that constraints of type F-in-S are supported (see supports_constraint) but cannot be added.

source

MathOptInterface.ModifyConstraintNotAllowed – Type.

```
struct ModifyConstraintNotAllowed{F<:AbstractFunction, S<:AbstractSet,
                                  C<:AbstractFunctionModification} <: NotAllowedError
    constraint_index::ConstraintIndex{F, S}
    change::C
    message::String
end
```

An error indicating that the constraint modification change cannot be applied to the constraint of index ci.

source

MathOptInterface.ModifyObjectiveNotAllowed – Type.

```
struct ModifyObjectiveNotAllowed{C<:AbstractFunctionModification} <: NotAllowedError
    change::C
    message::String
end
```

An error indicating that the objective modification change cannot be applied to the objective.

source

MathOptInterface.DeleteNotAllowed – Type.

```
struct DeleteNotAllowed{IndexType <: Index} <: NotAllowedError
    index::IndexType
    message::String
end
```

An error indicating that the index index cannot be deleted.

source

MathOptInterface.UnsupportedSubmittable – Type.

```
struct UnsupportedSubmittable{SubmitType} <: UnsupportedError
    sub::SubmitType
    message::String
end
```

An error indicating that the submittable `sub` is not supported by the model, that is, that `supports` returns `false`.

source

`MathOptInterface.SubmitNotAllowed` – Type.

```
struct SubmitNotAllowed{SubmitTyp<:AbstractSubmittable} <: NotAllowedError
    sub::SubmitType
    message::String
end
```

An error indicating that the submittable `sub` is supported (see `supports`) but cannot be added for some reason (see the error string).

source

`MathOptInterface.UnsupportedNonlinearOperator` – Type.

```
UnsupportedNonlinearOperator(head::Symbol[, message::String]) <: UnsupportedError
```

An error thrown by optimizers if they do not support the operator head in a `ScalarNonlinearFunction`.

**Example**

```
julia> throw(MOI.UnsupportedNonlinearOperator(:black_box))
ERROR: MathOptInterface.UnsupportedNonlinearOperator: The nonlinear operator `:black_box` is not
↪  supported by the model.
Stacktrace:
[...]
```

source

Note that setting the `ConstraintFunction` of a `VariableIndex` constraint is not allowed:

`MathOptInterface.SettingVariableIndexNotAllowed` – Type.

```
SettingVariableIndexNotAllowed()
```

Error type that should be thrown when the user calls `set` to change the `ConstraintFunction` of a `VariableIndex` constraint.

source

# Part VI

# Submodules

# Chapter 26

# Benchmarks

## 26.1 Overview

### The `Benchmarks` submodule

To aid the development of efficient solver wrappers, MathOptInterface provides a suite of benchmarks in the `MOI.Benchmarks` submodule.

**Warning**

To use this submodule you must first install and load BenchmarkTools.jl.

```
import Pkg
Pkg.add("BenchmarkTools")
import BenchmarkTools
```

### Benchmarking a solver wrapper

Benchmarking a wrapper follows a two-step process.

First, prior to making changes, create a baseline for the benchmark results on a given benchmark suite as follows:

```
# You must load BenchmarkTools.jl to enable MOI.Benchmarks
import BenchmarkTools
# Replace `SolverPackage` with your choice of solver
using SolverPackage
import MathOptInterface as MOI

suite = MOI.Benchmarks.suite() do
    SolverPackage.Optimizer()
end

MOI.Benchmarks.create_baseline(
    suite, "current"; directory = "/tmp", verbose = true
)
```

Use the `exclude` argument to `Benchmarks.suite` to exclude benchmarks that the solver doesn't support.

Second, after making changes to the package, re-run the benchmark suite and compare to the prior saved results:

```
import BenchmarkTools
using SolverPackage
import MathOptInterface as MOI

suite = MOI.Benchmarks.suite() do
    SolverPackage.Optimizer()
end

MOI.Benchmarks.compare_against_baseline(
    suite, "current"; directory = "/tmp", verbose = true
)
```

This comparison will create a report detailing improvements and regressions.

## 26.2 API Reference

**Benchmarks**

Functions to help benchmark the performance of solver wrappers. See The Benchmarks submodule for more details.

`MathOptInterface.Benchmarks.suite` – Function.

```
suite(
    new_model::Function;
    exclude::Vector{Regex} = Regex[]
)
```

Create a suite of benchmarks. `new_model` should be a function that takes no arguments, and returns a new instance of the optimizer you wish to benchmark.

Use `exclude` to exclude a subset of benchmarks.

**BenchmarkTools**

To use this function you must first install and load the `BenchmarkTools.jl` package.

**Example**

```
julia> import BenchmarkTools, GLPK, Gurobi

julia> MOI.Benchmarks.suite(GLPK.Optimizer)

julia> MOI.Benchmarks.suite(; exclude = [r"delete"]) do
           return Gurobi.Optimizer()
       end
```

source

`MathOptInterface.Benchmarks.create_baseline` – Function.

```
create_baseline(suite, name::String; directory::String = ""; kwargs...)
```

Run all benchmarks in `suite` and save to files called `name` in `directory`.

Extra `kwargs` are based to `BenchmarkTools.run`.

**BenchmarkTools**

To use this function you must first install and load the `BenchmarkTools.jl` package.

**Example**

```
julia> import BenchmarkTools, GLPK

julia> my_suite = MOI.Benchmarks.suite(GLPK.Optimizer);

julia> MOI.Benchmarks.create_baseline(
           my_suite,
           "glpk_master";
           directory = "/tmp",
           verbose = true,
       )
```

source

`MathOptInterface.Benchmarks.compare_against_baseline` – Function.

```
compare_against_baseline(
    suite, name::String; directory::String = "",
    report_filename::String = "report.txt"
)
```

Run all benchmarks in `suite` and compare against files called `name` in `directory` that were created by a call to `create_baseline`.

A report summarizing the comparison is written to `report_filename` in `directory`.

Extra `kwargs` are based to `BenchmarkTools.run`.

**BenchmarkTools**

To use this function you must first install and load the `BenchmarkTools.jl` package.

**Example**

```
julia> import BenchmarkTools, GLPK

julia> my_suite = MOI.Benchmarks.suite(GLPK.Optimizer);

julia> MOI.Benchmarks.compare_against_baseline(
           my_suite,
           "glpk_master";
           directory = "/tmp",
           verbose = true,
       )
```

```
source
```

# Chapter 27

# Bridges

## 27.1 Overview

### The Bridges submodule

The `Bridges` module simplifies the process of converting models between equivalent formulations.

**Tip**

Read our paper for more details on how bridges are implemented.

#### Why bridges?

A constraint can often be written in a number of equivalent formulations. For example, the constraint $l \leq a^\top x \leq u$ (`ScalarAffineFunction`-in-`Interval`) could be re-formulated as two constraints: $a^\top x \geq l$ (`ScalarAffineFunction`-in-`GreaterThan`) and $a^\top x \leq u$ (`ScalarAffineFunction`-in-`LessThan`). An alternative re-formulation is to add a dummy variable y with the constraints $l \leq y \leq u$ (`VariableIndex`-in-`Interval`) and $a^\top x - y = 0$ (`ScalarAffineFunction`-in-`EqualTo`).

To avoid each solver having to code these transformations manually, MathOptInterface provides bridges.

A bridge is a small transformation from one constraint type to another (potentially collection of) constraint type.

Because these bridges are included in MathOptInterface, they can be re-used by any optimizer. Some bridges also implement constraint modifications and constraint primal and dual translations.

Several bridges can be used in combination to transform a single constraint into a form that the solver may understand. Choosing the bridges to use takes the form of finding a shortest path in the hyper-graph of bridges. The methodology is detailed in the MOI paper.

#### The three types of bridges

There are three types of bridges in MathOptInterface:

1. Constraint bridges
2. Variable bridges
3. Objective bridges

### Constraint bridges

Constraint bridges convert constraints formulated by the user into an equivalent form supported by the solver. Constraint bridges are subtypes of `Bridges.Constraint.AbstractBridge`.

The equivalent formulation may add constraints (and possibly also variables) in the underlying model.

In particular, constraint bridges can focus on rewriting the function of a constraint, and do not change the set. Function bridges are subtypes of `Bridges.Constraint.AbstractFunctionConversionBridge`.

Read the list of implemented constraint bridges for more details on the types of transformations that are available. Function bridges are `Bridges.Constraint.ScalarFunctionizeBridge` and `Bridges.Constraint.VectorFunctionizeBri`

### Variable bridges

Variable bridges convert variables added by the user, either free with `add_variable`/`add_variables`, or constrained with `add_constrained_variable`/`add_constrained_variables`, into an equivalent form supported by the solver. Variable bridges are subtypes of `Bridges.Variable.AbstractBridge`.

The equivalent formulation may add constraints (and possibly also variables) in the underlying model.

Read the list of implemented variable bridges for more details on the types of transformations that are available.

### Objective bridges

Objective bridges convert the `ObjectiveFunction` set by the user into an equivalent form supported by the solver. Objective bridges are subtypes of `Bridges.Objective.AbstractBridge`.

The equivalent formulation may add constraints (and possibly also variables) in the underlying model.

Read the list of implemented objective bridges for more details on the types of transformations that are available.

### Bridges.full_bridge_optimizer

**Tip**

Unless you have an advanced use-case, this is probably the only function you need to care about.

To enable the full power of MathOptInterface's bridges, wrap an `optimizer` in a `Bridges.full_bridge_optimizer`.

```
julia> inner_optimizer = MOI.Utilities.Model{Float64}()
MOIU.Model{Float64}
├ ObjectiveSense: FEASIBILITY_SENSE
├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
├ NumberOfVariables: 0
└ NumberOfConstraints: 0

julia> optimizer = MOI.Bridges.full_bridge_optimizer(inner_optimizer, Float64)
MOIB.LazyBridgeOptimizer{MOIU.Model{Float64}}
├ Variable bridges: none
├ Constraint bridges: none
├ Objective bridges: none
└ model: MOIU.Model{Float64}
  ├ ObjectiveSense: FEASIBILITY_SENSE
  ├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
  ├ NumberOfVariables: 0
  └ NumberOfConstraints: 0
```

Now, use `optimizer` as normal, and bridging will happen lazily behind the scenes. By lazily, we mean that bridging will happen if and only if the constraint is not supported by the `inner_optimizer`.

**Info**

Most bridges are added by default in `Bridges.full_bridge_optimizer`. However, for technical reasons, some bridges are not added by default. Three examples include `Bridges.Constraint.SOCtoPSDBridge`, `Bridges.Constraint.SOCtoNonConvexQuadBridge` and `Bridges.Constraint.RSOCtoNonConvexQuadBridge`. See the docs of those bridges for more information.

#### Add a single bridge

If you don't want to use `Bridges.full_bridge_optimizer`, you can wrap an optimizer in a single bridge.

However, this will force the constraint to be bridged, even if the `inner_optimizer` supports it.

```
julia> inner_optimizer = MOI.Utilities.Model{Float64}();

julia> optimizer = MOI.Bridges.Constraint.SplitInterval{Float64}(inner_optimizer);

julia> x = MOI.add_variable(optimizer)
MOI.VariableIndex(1)

julia> MOI.add_constraint(optimizer, x, MOI.Interval(0.0, 1.0))
MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Interval{Float64}}(1)

julia> MOI.get(optimizer, MOI.ListOfConstraintTypesPresent())
1-element Vector{Tuple{Type, Type}}:
 (MathOptInterface.VariableIndex, MathOptInterface.Interval{Float64})

julia> MOI.get(inner_optimizer, MOI.ListOfConstraintTypesPresent())
2-element Vector{Tuple{Type, Type}}:
 (MathOptInterface.VariableIndex, MathOptInterface.GreaterThan{Float64})
 (MathOptInterface.VariableIndex, MathOptInterface.LessThan{Float64})
```

#### Bridges.LazyBridgeOptimizer

If you don't want to use `Bridges.full_bridge_optimizer`, but you need more than a single bridge (or you want the bridging to happen lazily), you can manually construct a `Bridges.LazyBridgeOptimizer`.

First, wrap an inner optimizer:

```
julia> inner_optimizer = MOI.Utilities.Model{Float64}()
MOIU.Model{Float64}
├ ObjectiveSense: FEASIBILITY_SENSE
├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
├ NumberOfVariables: 0
└ NumberOfConstraints: 0

julia> optimizer = MOI.Bridges.LazyBridgeOptimizer(inner_optimizer)
MOIB.LazyBridgeOptimizer{MOIU.Model{Float64}}
├ Variable bridges: none
```

```
├ Constraint bridges: none
├ Objective bridges: none
└ model: MOIU.Model{Float64}
  ├ ObjectiveSense: FEASIBILITY_SENSE
  ├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
  ├ NumberOfVariables: 0
  └ NumberOfConstraints: 0
```

Then use `Bridges.add_bridge` to add individual bridges:

```
julia> MOI.Bridges.add_bridge(optimizer, MOI.Bridges.Constraint.SplitIntervalBridge{Float64})

julia> MOI.Bridges.add_bridge(optimizer, MOI.Bridges.Objective.FunctionizeBridge{Float64})
```

Now the constraints will be bridged only if needed:

```
julia> x = MOI.add_variable(optimizer)
MOI.VariableIndex(1)

julia> MOI.add_constraint(optimizer, x, MOI.Interval(0.0, 1.0))
MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.Interval{Float64}}(1)

julia> MOI.get(optimizer, MOI.ListOfConstraintTypesPresent())
1-element Vector{Tuple{Type, Type}}:
 (MathOptInterface.VariableIndex, MathOptInterface.Interval{Float64})

julia> MOI.get(inner_optimizer, MOI.ListOfConstraintTypesPresent())
1-element Vector{Tuple{Type, Type}}:
 (MathOptInterface.VariableIndex, MathOptInterface.Interval{Float64})
```

## 27.2 Implementation

### Implementing a bridge

The easiest way to implement a bridge is to follow an existing example. There are three locations of bridges in the source code:

- Constraint bridges are stored in `src/Bridges/Constraint/bridges`
- Objective bridges are stored in `src/Bridges/Objective/bridges`
- Variable bridges are stored in `src/Bridges/Variable/bridges`

The Implementing a constraint bridge tutorial has a more detailed guide on what is required to implement a bridge.

When opening a pull request that adds a new bridge, use the checklist Adding a new bridge.

If you need help or advice, please contact the Developer Chatroom.

### SetMap bridges

For constraint and variable bridges, a common reformulation is that $f(x) \in F$ is reformulated to $g(x) \in G$. In this case, no additional variables and constraints are added, and the bridge needs only a way to map between the functions `f` and `g` and the sets `F` and `G`.

To implementation a bridge of this form, subtype the abstract type `Bridges.Constraint.SetMapBridge` or `Bridges.Variable.SetMapBridge` and implement the API described in the docstring of each type.

#### `final_touch`

Some bridges require information from other parts of the model. One set of examples are the various combinatorial `ToMILP` bridges, such as `Bridges.Constraint.SOS1ToMILPBridge`, which require knowledge of the variable bounds.

Bridges requiring information from other parts of the model should implement `Bridges.final_touch` and `Bridges.needs_final_touch`.

During the bridge's construction, store the function and set and make no changes to the underlying model. Then, in `Bridges.final_touch`, query the additional information and add the reformulated problem to the `model`.

When implementing, you must consider that:

- `Bridges.final_touch` may be called multiple times, so that your reformulation should be applied only if necessary. Sometimes the additional data will be the same, and sometimes it may be different.
- We do not currently support `final_touch` bridges that introduce constraints which also require a `final_touch` bridge. Therefore, you should implement `final_touch` only if necessary, and we recommend that you contact the Developer Chatroom for advice before doing so.

### Testing

Use the `Bridges.runtests` function to test a bridge. It takes three arguments: the type of the bridge, the input model as a string, and the output model as a string.

Here is an example:

```
julia> MOI.Bridges.runtests(
           MOI.Bridges.Constraint.GreaterToLessBridge,
           """
           variables: x
           x >= 1.0
           """,
           """
           variables: x
           -1.0 * x <= -1.0
           """,
       )
Test Summary:    | Pass  Total  Time
Bridges.runtests |   29     29  0.0s
```

There are a number of other useful keyword arguments.

- `eltype` can be used to specify the element type of the model (and bridge). It defaults to `Float64`.

- `variable_start` and `constraint_start` are used as the values to set the `VariablePrimalStart` and `ConstraintPrimalStart` attributes to. They default to `1.2`. If you use a different `eltype`, you must set appropriate starting values of the same type. The default `1.2` was chosen to minimize the risk that the starting point is undefined, which could happen for common situations like `0.0` and `1.0`. The tests associated with the starting values do not necessarily check for correctness, only that they can be `set` and `get` to produce the same result.
- `print_inner_model` can be used to print the reformulated output model from the bridge. This is especially helpful during debugging to see what the bridge is doing, and to spot mistakes. It defaults to `false`.

Here is an example:

```
julia> MOI.Bridges.runtests(
           MOI.Bridges.Constraint.GreaterToLessBridge,
           """
           variables: x
           x >= 1
           """,
           """
           variables: x
           ::Int: -1 * x <= -1
           """;
           eltype = Int,
           print_inner_model = true,
           variable_start = 2,
           constraint_start = 2,
       )
Feasibility

Subject to:

ScalarAffineFunction{Int64}-in-LessThan{Int64}
 (0) - (1) x <= (-1)
Test Summary:    | Pass  Total  Time
Bridges.runtests |   29     29  0.0s
```

## 27.3 List of bridges

### List of bridges

This section describes the `Bridges.AbstractBridge`s that are implemented in MathOptInterface.

#### Constraint bridges

These bridges are subtypes of `Bridges.Constraint.AbstractBridge`.

`MathOptInterface.Bridges.Constraint.AbstractFunctionConversionBridge` – Type.

```
abstract type AbstractFunctionConversionBridge{F,S} <: AbstractBridge end
```

Abstract type to support writing bridges in which the function changes but the set does not.

By convention, the transformed function is stored in the `.constraint` field.

source

`MathOptInterface.Bridges.Constraint.AbstractToIntervalBridge` – Type.

```
AbstractToIntervalBridge{T<:AbstractFloat,S,F}
```

An abstract type that simplifies the creation of other bridges.

**Warning**

T must be a `AbstractFloat` type because otherwise `typemin` and `typemax` would either be not implemented (for example, `BigInt`), or would not give infinite value (for example, `Int`). For this reason, this bridge is only added to `MOI.Bridges.full_bridge_optimizer` when T is a subtype of `AbstractFloat`.

source

`MathOptInterface.Bridges.Constraint.AllDifferentToCountDistinctBridge` – Type.

```
AllDifferentToCountDistinctBridge{T,F} <: Bridges.Constraint.AbstractBridge
```

`AllDifferentToCountDistinctBridge` implements the following reformulations:

- $x \in$ AllDifferent$(d)$ to $(n, x) \in$ CountDistinct$(1 + d)$ and $n = d$
- $f(x) \in$ AllDifferent$(d)$ to $(d, f(x)) \in$ CountDistinct$(1 + d)$

**Source node**

`AllDifferentToCountDistinctBridge` supports:

- F in `MOI.AllDifferent`

where F is `MOI.VectorOfVariables` or `MOI.VectorAffineFunction{T}`.

**Target nodes**

`AllDifferentToCountDistinctBridge` creates:

- F in `MOI.CountDistinct`
- `MOI.VariableIndex` in `MOI.EqualTo{T}`

source

`MathOptInterface.Bridges.Constraint.BinPackingToMILPBridge` – Type.

```
BinPackingToMILPBridge{T,F} <: Bridges.Constraint.AbstractBridge
```

`BinPackingToMILPBridge` implements the following reformulation:

- $x \in BinPacking(c, w)$ into a mixed-integer linear program.

**Reformulation**

The reformulation is non-trivial, and it depends on the finite domain of each variable $x_i$, which we as define $S_i = \{l_i, \ldots, u_i\}$.

First, we introduce new binary variables $z_{ij}$, which are $1$ if variable $x_i$ takes the value $j$ in the optimal solution and $0$ otherwise:

$$\begin{aligned} z_{ij} &\in \{0,1\} \quad \forall i \in 1 \ldots d, j \in S_i \\ x_i - \sum_{j \in S_i} j \cdot z_{ij} &= 0 \quad \forall i \in 1 \ldots d \\ \sum_{j \in S_i} z_{ij} &= 1 \quad \forall i \in 1 \ldots d \end{aligned}$$

Then, we add the capacity constraint for all possible bins $j$:

$$\sum_i w_i z_{ij} \leq c \forall j \in \bigcup_{i=1,\ldots,d} S_i$$

**Source node**

`BinPackingToMILPBridge` supports:

- F in `MOI.BinPacking{T}`

**Target nodes**

`BinPackingToMILPBridge` creates:

- `MOI.VariableIndex` in `MOI.ZeroOne`
- `MOI.ScalarAffineFunction{T}` in `MOI.EqualTo{T}`
- `MOI.ScalarAffineFunction{T}` in `MOI.LessThan{T}`

source

`MathOptInterface.Bridges.Constraint.CircuitToMILPBridge` – Type.

```
CircuitToMILPBridge{T,F} <: Bridges.Constraint.AbstractBridge
```

`CircuitToMILPBridge` implements the following reformulation:

- $x \in$ Circuit$(d)$ to the Miller-Tucker-Zemlin formulation of the Traveling Salesperson Problem.

**Source node**

`CircuitToMILPBridge` supports:

- F in `MOI.Circuit`

where F is MOI.VectorOfVariables or MOI.VectorAffineFunction{T}.

**Target nodes**

CircuitToMILPBridge creates:

- MOI.VariableIndex in MOI.ZeroOne
- MOI.VariableIndex in MOI.Integer
- MOI.VariableIndex in MOI.Interval{T}
- MOI.ScalarAffineFunction{T} in MOI.EqualTo{T}
- MOI.ScalarAffineFunction{T} in MOI.LessThan{T}

source

MathOptInterface.Bridges.Constraint.ComplexNormInfinityToSecondOrderConeBridge – Type.

```
ComplexNormInfinityToSecondOrderConeBridge{T} <: Bridges.Constraint.AbstractBridge
```

ComplexNormInfinityToSecondOrderConeBridge implements the following reformulation:

- $(t, x) \in NormInfinity(1 + d)$ into $(t, real(x_i), imag(x_i)) \in SecondOrderCone()$ for all $i$.

**Source node**

ComplexNormInfinityToSecondOrderConeBridge supports:

- MOI.VectorAffineFunction{Complex{T}} in MOI.NormInfinityCone

**Target nodes**

ComplexNormInfinityToSecondOrderConeBridge creates:

- MOI.VectorAffineFunction{T} in MOI.SecondOrderCone

source

MathOptInterface.Bridges.Constraint.CountAtLeastToCountBelongsBridge – Type.

```
CountAtLeastToCountBelongsBridge{T,F} <: Bridges.Constraint.AbstractBridge
```

CountAtLeastToCountBelongsBridge implements the following reformulation:

- $x \in$ CountAtLeast$(n, d, \mathcal{S})$ to $(n_i, x_{d_i}) \in$ CountBelongs$(1 + d, \mathcal{S})$ and $n_i \geq n$ for all $i$.

**Source node**

CountAtLeastToCountBelongsBridge supports:

- F in MOI.CountAtLeast

where F is MOI.VectorOfVariables or MOI.VectorAffineFunction{T}.

**Target nodes**

CountAtLeastToCountBelongsBridge creates:

- F in MOI.CountBelongs
- MOI.VariableIndex in MOI.GreaterThan{T}

source

MathOptInterface.Bridges.Constraint.CountBelongsToMILPBridge – Type.

```
CountBelongsToMILPBridge{T,F} <: Bridges.Constraint.AbstractBridge
```

CountBelongsToMILPBridge implements the following reformulation:

- $(n, x) \in$ CountBelongs$(1 + d, \mathcal{S})$ into a mixed-integer linear program.

**Reformulation**

The reformulation is non-trivial, and it depends on the finite domain of each variable $x_i$, which we as define $S_i = \{l_i, \ldots, u_i\}$.

First, we introduce new binary variables $z_{ij}$, which are $1$ if variable $x_i$ takes the value $j$ in the optimal solution and $0$ otherwise:

$$\begin{aligned} z_{ij} &\in \{0, 1\} \quad \forall i \in 1 \ldots d, j \in S_i \\ x_i - \sum_{j \in S_i} j \cdot z_{ij} &= 0 \quad \forall i \in 1 \ldots d \\ \sum_{j \in S_i} z_{ij} &= 1 \quad \forall i \in 1 \ldots d \end{aligned}$$

Finally, $n$ is constrained to be the number of $z_{ij}$ elements that are in $\mathcal{S}$:

$$n - \sum_{i \in 1 \ldots d, j \in \mathcal{S}} z_{ij} = 0$$

**Source node**

CountBelongsToMILPBridge supports:

- F in MOI.CountBelongs

where F is MOI.VectorOfVariables or MOI.VectorAffineFunction{T}.

**Target nodes**

CountBelongsToMILPBridge creates:

- MOI.VariableIndex in MOI.ZeroOne

- MOI.ScalarAffineFunction{T} in MOI.EqualTo{T}

source

MathOptInterface.Bridges.Constraint.CountDistinctToMILPBridge – Type.

```
CountDistinctToMILPBridge{T,F} <: Bridges.Constraint.AbstractBridge
```

CountDistinctToMILPBridge implements the following reformulation:

- $(n, x) \in$ CountDistinct$(1 + d)$ into a mixed-integer linear program.

**Reformulation**

The reformulation is non-trivial, and it depends on the finite domain of each variable $x_i$, which we as define $S_i = \{l_i, \ldots, u_i\}$.

First, we introduce new binary variables $z_{ij}$, which are $1$ if variable $x_i$ takes the value $j$ in the optimal solution and $0$ otherwise:

$$\begin{aligned} z_{ij} \in \{0,1\} \quad \forall i \in 1 \ldots d, j \in S_i \\ x_i - \sum_{j \in S_i} j \cdot z_{ij} = 0 \quad \forall i \in 1 \ldots d \\ \sum_{j \in S_i} z_{ij} = 1 \quad \forall i \in 1 \ldots d \end{aligned}$$

Then, we introduce new binary variables $y_j$, which are $1$ if a variable takes the value $j$ in the optimal solution and $0$ otherwise.

$$\begin{aligned} y_j \in \{0,1\} \quad \forall j \in \bigcup_{i=1,\ldots,d} S_i \\ y_j \le \sum_{i \in 1 \ldots d : j \in S_i} z_{ij} \le M y_j \quad \forall j \in \bigcup_{i=1,\ldots,d} S_i \end{aligned}$$

Finally, $n$ is constrained to be the number of $y_j$ elements that are non-zero:

$$n - \sum_{j \in \bigcup_{i=1,\ldots,d} S_i} y_j = 0$$

**Formulation (special case)**

In the special case that the constraint is `[2, x, y] in CountDistinct(3)`, then the constraint is equivalent to `[x, y] in AllDifferent(2)`, which is equivalent to `x != y`.

$$(x - y \le -1) \vee (y - x \le -1)$$

which is equivalent to (for suitable M):

$$z \in \{0, 1\}$$
$$x - y - Mz \leq -1$$
$$y - x - M(1 - z) \leq -1$$

**Source node**

`CountDistinctToMILPBridge` supports:

- F in `MOI.CountDistinct`

where F is `MOI.VectorOfVariables` or `MOI.VectorAffineFunction{T}`.

**Target nodes**

`CountDistinctToMILPBridge` creates:

- `MOI.VariableIndex` in `MOI.ZeroOne`
- `MOI.ScalarAffineFunction{T}` in `MOI.EqualTo{T}`
- `MOI.ScalarAffineFunction{T}` in `MOI.LessThan{T}`

source

`MathOptInterface.Bridges.Constraint.CountGreaterThanToMILPBridge` – Type.

```
CountGreaterThanToMILPBridge{T,F} <: Bridges.Constraint.AbstractBridge
```

`CountGreaterThanToMILPBridge` implements the following reformulation:

- $(c, y, x) \in CountGreaterThan()$ into a mixed-integer linear program.

**Source node**

`CountGreaterThanToMILPBridge` supports:

- F in `MOI.CountGreaterThan`

**Target nodes**

`CountGreaterThanToMILPBridge` creates:

- `MOI.VariableIndex` in `MOI.ZeroOne`
- `MOI.ScalarAffineFunction{T}` in `MOI.EqualTo{T}`
- `MOI.ScalarAffineFunction{T}` in `MOI.GreaterThan{T}`

source

`MathOptInterface.Bridges.Constraint.DualGeoMeanBridge` – Type.

```
DualGeoMeanBridge{T,G,H} <: Bridges.Constraint.AbstractBridge
```

`DualGeoMeanBridge` implements the following reformulation:

- $(u, v) \in DualGeometricMeanCone$ into $(-u/length(v), v) \in GeometricMeanCone$ and $u \leq 0$

**Source node**

`DualGeoMeanBridge` supports:

- `H` in `MOI.DualGeometricMeanCone`

**Target nodes**

`DualGeoMeanBridge` creates:

- `G` in `MOI.GeometricMeanCone`
- `G` in `MOI.Nonnegatives`

source

`MathOptInterface.Bridges.Constraint.DualRelativeEntropyBridge` – Type.

```
DualRelativeEntropyBridge{T,F,G} <: Bridges.Constraint.AbstractBridge
```

`DualRelativeEntropyBridge` implements the following reformulation that converts a `MOI.DualRelativeEntropyCone` into an `MOI.DualExponentialCone`:

- $w_i \geq u \left( \log \left( \frac{u}{v_i} \right) - 1 \right)$ into $(-u, w_i, v_i) \in DualExponentialCone.$

**Source node**

`DualRelativeEntropyBridge` supports:

- `G` in `MOI.DualRelativeEntropyCone`

**Target nodes**

`DualRelativeEntropyBridge` creates:

- `F` in `MOI.DualExponentialCone`

source

`MathOptInterface.Bridges.Constraint.ExponentialConeToScalarNonlinearFunctionBridge` – Type.

```
ExponentialConeToScalarNonlinearFunctionBridge{T,F} <:
    Bridges.Constraint.AbstractBridge
```

`ExponentialConeToScalarNonlinearFunctionBridge` implements the following reformulation:

- $(x, y, z) \in$ ExponentialCone() to $y \cdot exp(x/y)) - z \leq 0, y \geq 0$.

**Source node**

ExponentialConeToScalarNonlinearFunctionBridge supports:

- F in MOI.ExponentialCone

**Target nodes**

ExponentialConeToScalarNonlinearFunctionBridge creates:

- MOI.ScalarNonlinearFunction in MOI.LessThan{T}
- MOI.ScalarAffineFunction in MOI.GreaterThan{T}

source

MathOptInterface.Bridges.Constraint.FlipSignBridge – Type.

```
FlipSignBridge{T,S1,S2,F,G}
```

An abstract type that simplifies the creation of other bridges.

source

MathOptInterface.Bridges.Constraint.FunctionConversionBridge – Type.

```
FunctionConversionBridge{T,F,G,S} <: AbstractFunctionConversionBridge{G,S}
```

FunctionConversionBridge implements the following reformulations:

- $g(x) \in S$ into $f(x) \in S$

for these pairs of functions:

- MOI.ScalarAffineFunctionto [MOI.ScalarQuadraticFunction'](@ref)
- MOI.ScalarQuadraticFunction to MOI.ScalarNonlinearFunction
- MOI.VectorAffineFunction to MOI.VectorQuadraticFunction

See also SetConversionBridge.

**Source node**

FunctionConversionBridge supports:

- G in S

**Target nodes**

FunctionConversionBridge creates:

- F in S

source

MathOptInterface.Bridges.Constraint.GeoMeanBridge – Type.

```
GeoMeanBridge{T,F,G,H} <: Bridges.Constraint.AbstractBridge
```

GeoMeanBridge implements a reformulation from MOI.GeometricMeanCone into MOI.RotatedSecondOrderCone.

The reformulation is best described in an example.

Consider the cone of dimension 4:

$$t \leq \sqrt[3]{x_1 x_2 x_3}$$

This can be rewritten as $\exists y_1 \geq 0$ such that:

$$\begin{aligned} t &\leq y_1, \\ y_1^4 &\leq x_1 x_2 x_3 y_1. \end{aligned}$$

Note that we need to create $y_1$ and not use $t^4$ directly because $t$ is allowed to be negative.

This is equivalent to:

$$\begin{aligned} t &\leq \frac{y_1}{\sqrt{4}}, \\ y_1^2 &\leq 2 y_2 y_3, \\ y_2^2 &\leq 2 x_1 x_2, \\ y_3^2 &\leq 2 x_3 (y_1/\sqrt{4}) \\ y_1 &\geq 0. \end{aligned}$$

More generally, you can show how the geometric mean code is recursively expanded into a set of new variables $y_i$ in MOI.Nonnegatives, a set of MOI.RotatedSecondOrderCone constraints, and a MOI.LessThan constraint between $t$ and $y_1$.

**Source node**

GeoMeanBridge supports:

- H in MOI.GeometricMeanCone

**Target nodes**

GeoMeanBridge creates:

- F in MOI.LessThan{T}
- G in MOI.RotatedSecondOrderCone
- G in MOI.Nonnegatives

source

MathOptInterface.Bridges.Constraint.GeoMeanToPowerBridge – Type.

```
GeoMeanToPowerBridge{T,F} <: Bridges.Constraint.AbstractBridge
```

GeoMeanToPowerBridge implements the following reformulation:

- $(y, x...) \in GeometricMeanCone(1+d)$ into $(x_1, t, y) \in PowerCone(1/d)$ and $(t, x_2, ..., x_d) in GeometricMeanC$ which is then recursively expanded into more PowerCone constraints.

**Source node**

GeoMeanToPowerBridge supports:

- F in MOI.GeometricMeanCone

**Target nodes**

GeoMeanToPowerBridge creates:

- F in MOI.PowerCone{T}
- MOI.VectorOfVariables in MOI.Nonnegatives

source

MathOptInterface.Bridges.Constraint.GeoMeantoRelEntrBridge – Type.

```
GeoMeantoRelEntrBridge{T,F,G,H} <: Bridges.Constraint.AbstractBridge
```

GeoMeantoRelEntrBridge implements the following reformulation:

- $(u, w) \in GeometricMeanCone$ into $(0, w, (u + y)\mathbf{1}) \in RelativeEntropyCone$ and $y \geq 0$

**Source node**

GeoMeantoRelEntrBridge supports:

- H in MOI.GeometricMeanCone

**Target nodes**

GeoMeantoRelEntrBridge creates:

- G in MOI.RelativeEntropyCone
- F in MOI.Nonnegatives

**Derivation**

The derivation of the bridge is as follows:

$$
\begin{aligned}
(u, w) \in GeometricMeanCone &\iff u \leq \left(\prod_{i=1}^{n} w_i\right)^{1/n} \\
&\iff 0 \leq u + y \leq \left(\prod_{i=1}^{n} w_i\right)^{1/n}, y \geq 0 \\
&\iff 1 \leq \frac{\left(\prod_{i=1}^{n} w_i\right)^{1/n}}{u + y}, y \geq 0 \\
&\iff 1 \leq \left(\prod_{i=1}^{n} \frac{w_i}{u + y}\right)^{1/n}, y \geq 0 \\
&\iff 0 \leq \sum_{i=1}^{n} \log\left(\frac{w_i}{u + y}\right), y \geq 0 \\
&\iff 0 \geq \sum_{i=1}^{n} \log\left(\frac{u + y}{w_i}\right), y \geq 0 \\
&\iff 0 \geq \sum_{i=1}^{n} (u + y) \log\left(\frac{u + y}{w_i}\right), y \geq 0 \\
&\iff (0, w, (u + y)\mathbf{1}) \in RelativeEntropyCone, y \geq 0
\end{aligned}
$$

This derivation assumes that $u + y > 0$, which is enforced by the relative entropy cone.

source

`MathOptInterface.Bridges.Constraint.GreaterToIntervalBridge` – Type.

```
GreaterToIntervalBridge{T,F} <: Bridges.Constraint.AbstractBridge
```

`GreaterToIntervalBridge` implements the following reformulations:

- $f(x) \geq l$ into $f(x) \in [l, \infty)$

**Source node**

`GreaterToIntervalBridge` supports:

- F in `MOI.GreaterThan{T}`

**Target nodes**

`GreaterToIntervalBridge` creates:

- F in `MOI.Interval{T}`

source

`MathOptInterface.Bridges.Constraint.GreaterToLessBridge` – Type.

```
GreaterToLessBridge{T,F,G} <: Bridges.Constraint.AbstractBridge
```

`GreaterToLessBridge` implements the following reformulation:

- $f(x) \geq l$ into $-f(x) \leq -l$

**Source node**

`GreaterToLessBridge` supports:

- `G` in `MOI.GreaterThan{T}`

**Target nodes**

`GreaterToLessBridge` creates:

- `F` in `MOI.LessThan{T}`

source

`MathOptInterface.Bridges.Constraint.HermitianToComplexSymmetricBridge` – Type.

```
HermitianToComplexSymmetricBridge{T,F,G} <: Bridges.Constraint.AbstractBridge
```

`HermitianToSymmetricBridge` implements the following reformulation:

- Hermitian positive semidefinite `n x n` represented as a vector of real entries with real and imaginary parts on different entries to a vector of complex entries.

See also `MOI.Bridges.Constraint.HermitianToSymmetricPSDBridge`.

**Source node**

`HermitianToComplexSymmetricBridge` supports:

- `G` in `MOI.HermitianPositiveSemidefiniteConeTriangle`

**Target node**

`HermitianToComplexSymmetricBridge` creates:

- `F` in `MOI.PositiveSemidefiniteConeTriangle`

Note that if `G` is `MOI.VectorAffineFunction{T}` then `F` will be `MOI.VectorAffineFunction{Complex{T}}`

source

`MathOptInterface.Bridges.Constraint.HermitianToSymmetricPSDBridge` – Type.

```
HermitianToSymmetricPSDBridge{T,F,G} <: Bridges.Constraint.AbstractBridge
```

`HermitianToSymmetricPSDBridge` implements the following reformulation:

- Hermitian positive semidefinite `n x n` complex matrix to a symmetric positive semidefinite `2n x 2n` real matrix.

See also `MOI.Bridges.Variable.HermitianToSymmetricPSDBridge`.

**Source node**

`HermitianToSymmetricPSDBridge` supports:

- `G` in `MOI.HermitianPositiveSemidefiniteConeTriangle`

**Target node**

`HermitianToSymmetricPSDBridge` creates:

- `F` in `MOI.PositiveSemidefiniteConeTriangle`

**Reformulation**

The reformulation is best described by example.

The Hermitian matrix:

$$\begin{bmatrix} x_{11} & x_{12} + y_{12}im & x_{13} + y_{13}im \\ x_{12} - y_{12}im & x_{22} & x_{23} + y_{23}im \\ x_{13} - y_{13}im & x_{23} - y_{23}im & x_{33} \end{bmatrix}$$

is positive semidefinite if and only if the symmetric matrix:

$$\begin{bmatrix} x_{11} & x_{12} & x_{13} & 0 & y_{12} & y_{13} \\ & x_{22} & x_{23} & -y_{12} & 0 & y_{23} \\ & & x_{33} & -y_{13} & -y_{23} & 0 \\ & & & x_{11} & x_{12} & x_{13} \\ & & & & x_{22} & x_{23} \\ & & & & & x_{33} \end{bmatrix}$$

is positive semidefinite.

The bridge achieves this reformulation by constraining the above matrix to belong to the `MOI.PositiveSemidefiniteConeTri`

source

`MathOptInterface.Bridges.Constraint.IndicatorActiveOnFalseBridge` – Type.

```
IndicatorActiveOnFalseBridge{T,F,S} <: Bridges.Constraint.AbstractBridge
```

`IndicatorActiveOnFalseBridge` implements the following reformulation:

- $\neg z \implies f(x) \in S$ into $y \implies f(x) \in S$, $z + y = 1$, and $y \in \{0, 1\}$

**Source node**

`IndicatorActiveOnFalseBridge` supports:

- MOI.VectorAffineFunction{T} in MOI.Indicator{MOI.ACTIVATE_ON_ZERO,S}

**Target nodes**

IndicatorActiveOnFalseBridge creates:

- MOI.VectorAffineFunction{T} in MOI.Indicator{MOI.ACTIVATE_ON_ONE,S}
- MOI.ScalarAffineFunction{T} in MOI.EqualTo
- MOI.VariableIndex in MOI.ZeroOne

source

MathOptInterface.Bridges.Constraint.IndicatorGreaterToLessThanBridge – Type.

```
IndicatorGreaterToLessThanBridge{T,A} <: Bridges.Constraint.AbstractBridge
```

IndicatorGreaterToLessThanBridge implements the following reformulation:

- $z \implies f(x) \geq l$ into $z \implies -f(x) \leq -l$

**Source node**

IndicatorGreaterToLessThanBridge supports:

- MOI.VectorAffineFunction{T} in MOI.Indicator{A,MOI.GreaterThan{T}}

**Target nodes**

IndicatorGreaterToLessThanBridge creates:

- MOI.VectorAffineFunction{T} in MOI.Indicator{A,MOI.LessThan{T}}

source

MathOptInterface.Bridges.Constraint.IndicatorLessToGreaterThanBridge – Type.

```
IndicatorLessToGreaterThanBridge{T,A} <: Bridges.Constraint.AbstractBridge
```

IndicatorLessToGreaterThanBridge implements the following reformulations:

- $z \implies f(x) \leq u$ into $z \implies -f(x) \geq -u$

**Source node**

IndicatorLessToGreaterThanBridge supports:

- MOI.VectorAffineFunction{T} in MOI.Indicator{A,MOI.LessThan{T}}

**Target nodes**

IndicatorLessToGreaterThanBridge creates:

- MOI.VectorAffineFunction{T} in MOI.Indicator{A,MOI.GreaterThan{T}}

source

MathOptInterface.Bridges.Constraint.IndicatorSOS1Bridge – Type.

```
IndicatorSOS1Bridge{T,S} <: Bridges.Constraint.AbstractBridge
```

IndicatorSOS1Bridge implements the following reformulation:

- $z \implies f(x) \in S$ into $f(x) + y \in S, SOS1(y, z)$

**Warning**

This bridge assumes that the solver supports MOI.SOS1{T} constraints in which one of the variables ($y$) is continuous.

**Source node**

IndicatorSOS1Bridge supports:

- MOI.VectorAffineFunction{T} in MOI.Indicator{MOI.ACTIVATE_ON_ONE,S}

**Target nodes**

IndicatorSOS1Bridge creates:

- MOI.ScalarAffineFunction{T} in S
- MOI.VectorOfVariables in MOI.SOS1{T}

source

MathOptInterface.Bridges.Constraint.IndicatorSetMapBridge – Type.

```
IndicatorSetMapBridge{T,A,S1,S2} <: Bridges.Constraint.AbstractBridge
```

IndicatorSetMapBridge implements the following reformulations:

- $z \implies f(x) \geq l$ into $z \implies -f(x) \leq -l$
- $z \implies f(x) \leq u$ into $z \implies -f(x) \geq -u$

**Source node**

IndicatorSetMapBridge supports:

- MOI.VectorAffineFunction{T} in MOI.Indicator{A,S1}

**Target nodes**

IndicatorSetMapBridge creates:

- MOI.VectorAffineFunction{T} in MOI.Indicator{A,S2}

source

MathOptInterface.Bridges.Constraint.IndicatorToMILPBridge – Type.

```
IndicatorToMILPBridge{T,F,A,S} <: Bridges.Constraint.AbstractBridge
```

IndicatorToMILPBridge implements the following reformulation:

- $x \in$ Indicator$(s)$ into a mixed-integer linear program.

**Source node**

IndicatorToMILPBridge supports:

- F in MOI.Indicator{A,S}

where F is MOI.VectorOfVariables or MOI.VectorAffineFunction{T}.

**Target nodes**

IndicatorToMILPBridge creates:

- MOI.VariableIndex in MOI.ZeroOne
- MOI.ScalarAffineFunction{T} in S

source

MathOptInterface.Bridges.Constraint.InequalityToComplementsBridge – Type.

```
InequalityToComplementsBridge{T,F,S,G} <: Bridges.Constraint.AbstractBridge
```

InequalityToComplementsBridge implements the following reformulations:

- $f(x) \geq b$ into $\exists y$ such that $f(x) - b \perp y \geq 0$
- $f(x) \leq b$ into $f(x) - b \perp y \leq 0$
- $f(x) = b$ into $f(x) - b \perp y$

**Source node**

InequalityToComplementsBridge supports:

- F in MOI.GreaterThan{T}
- F in MOI.LessThan{T}
- F in MOI.EqualTo

**Target nodes**

InequalityToComplementsBridge creates:

- MOI.VariableIndex in MOI.LessThan{T}
- MOI.VariableIndex in MOI.GreaterThan{T}
- G in MOI.Complements

source

MathOptInterface.Bridges.Constraint.IntegerToZeroOneBridge – Type.

```
IntegerToZeroOneBridge{T} <: Bridges.Constraint.AbstractBridge
```

IntegerToZeroOneBridge implements the following reformulation:

- $x \in \mathbf{Z}$ into $y_i \in \{0, 1\}$, $x == lb + \sum 2^{i-1} y_i$.

**Source node**

IntegerToZeroOneBridge supports:

- VariableIndex in MOI.Integer

**Target nodes**

IntegerToZeroOneBridge creates:

- MOI.VariableIndex in MOI.ZeroOne
- MOI.ScalarAffineFunction{T} in MOI.EqualTo{T}

**Developer note**

This bridge is implemented as a constraint bridge instead of a variable bridge because we don't want to substitute the linear combination of y for every instance of x. Doing so would be expensive and greatly reduce the sparsity of the constraints.

source

MathOptInterface.Bridges.Constraint.IntervalToHyperRectangleBridge – Type.

```
IntervalToHyperRectangleBridge{T,F,G} <: Bridges.Constraint.AbstractBridge
```

IntervalToHyperRectangleBridge implements the following reformulations:

- $l \leq g(x) \leq u$ into $[g(x)] \in$ MOI.HyperRectangle([l], [u])

where T is the coefficient type of g(x) and the type of l and u.

See also VectorizeBridge for equality and single-sided bound constraints.

**Source node**

IntervalToHyperRectangleBridge supports:

- G in MOI.Interval{T}

**Target nodes**

`IntervalToHyperRectangleBridge` creates:

- `F` in `MOI.HyperRectangle{T}`.

source

`MathOptInterface.Bridges.Constraint.LessToGreaterBridge` – Type.

```
LessToGreaterBridge{T,F,G} <: Bridges.Constraint.AbstractBridge
```

`LessToGreaterBridge` implements the following reformulation:

- $f(x) \leq u$ into $-f(x) \geq -u$

**Source node**

`LessToGreaterBridge` supports:

- `G` in `MOI.LessThan{T}`

**Target nodes**

`LessToGreaterBridge` creates:

- `F` in `MOI.GreaterThan{T}`

source

`MathOptInterface.Bridges.Constraint.LessToIntervalBridge` – Type.

```
LessToIntervalBridge{T,F} <: Bridges.Constraint.AbstractBridge
```

`LessToIntervalBridge` implements the following reformulations:

- $f(x) \leq u$ into $f(x) \in (-\infty, u]$

**Source node**

`LessToIntervalBridge` supports:

- `F` in `MOI.LessThan{T}`

**Target nodes**

`LessToIntervalBridge` creates:

- `F` in `MOI.Interval{T}`

source

`MathOptInterface.Bridges.Constraint.LogDetBridge` – Type.

```
LogDetBridge{T,F,G,H,I} <: Bridges.Constraint.AbstractBridge
```

The MOI.LogDetConeTriangle is representable by MOI.PositiveSemidefiniteConeTriangle and MOI.ExponentialCone constraints.

Indeed, $\log\det(X) = \sum_{i=1}^{n} \log(\delta_i)$ where $\delta_i$ are the eigenvalues of $X$.

Adapting the method from [1, p. 149], we see that $t \leq u\log(\det(X/u))$ for $u > 0$ if and only if there exists a lower triangular matrix  such that

$$\begin{pmatrix} X & \\ \top & \operatorname{Diag}() \end{pmatrix} \succeq 0$$
$$t - \sum_{i=1}^{n} u \log\left(\frac{ii}{u}\right) \leq 0$$

Which we reformulate further into

$$\begin{pmatrix} X & \\ \top & \operatorname{Diag}() \end{pmatrix} \succeq 0$$
$$(l_i, u, {}_{ii}) \in ExponentialCone \quad \forall i$$
$$t - \sum_{i=1}^{n} l_i \leq 0$$

**Source node**

LogDetBridge supports:

- I in MOI.LogDetConeTriangle

**Target nodes**

LogDetBridge creates:

- F in MOI.PositiveSemidefiniteConeTriangle
- G in MOI.ExponentialCone
- H in MOI.LessThan{T}

[1] Ben-Tal, Aharon, and Arkadi Nemirovski. Lectures on modern convex optimization: analysis, algorithms, and engineering applications. Society for Industrial and Applied Mathematics, 2001.

source

MathOptInterface.Bridges.Constraint.MultiSetMapBridge – Type.

```
abstract type MultiSetMapBridge{T,S1,G} <: AbstractBridge end
```

Same as SetMapBridge but the output constraint type does not only depend on the input constraint type.

When subtyping MultiSetMapBridge, added_constraint_types and supports should additionally be implemented by the bridge.

For example, if a bridge BridgeType may create either a constraint of type F2-in-S2 or F3-in-S3, these methods should be implemented as follows:

```
function MOI.Bridges.added_constraint_types(
    ::Type{<:BridgeType{T,F2,F3}},
) where {T,F2,F3}
    return Tuple{Type,Type}[(F2, S2), (F3, S3)]
end

function MOI.supports(
    model::MOI.ModelLike,
    attr::Union{MOI.ConstraintPrimalStart,MOI.ConstraintDualStart},
    ::Type{<:BridgeType{T,F2,F3}},
) where {T,F2,F3}
    return MOI.supports(model, attr, MOI.ConstraintIndex{F2,S2}) ||
           MOI.supports(model, attr, MOI.ConstraintIndex{F3,S3})
end
```

source

MathOptInterface.Bridges.Constraint.NonnegToNonposBridge – Type.

```
NonnegToNonposBridge{T,F,G} <: Bridges.Constraint.AbstractBridge
```

NonnegToNonposBridge implements the following reformulation:

- $f(x) \in \mathbb{R}_+$ into $-f(x) \in \mathbb{R}_-$

**Source node**

NonnegToNonposBridge supports:

- G in MOI.Nonnegatives

**Target nodes**

NonnegToNonposBridge creates:

- F in MOI.Nonpositives

source

MathOptInterface.Bridges.Constraint.NonposToNonnegBridge – Type.

```
NonposToNonnegBridge{T,F,G} <: Bridges.Constraint.AbstractBridge
```

NonposToNonnegBridge implements the following reformulation:

- $f(x) \in \mathbb{R}_-$ into $-f(x) \in \mathbb{R}_+$

**Source node**

NonposToNonnegBridge supports:

- G in MOI.Nonpositives

**Target nodes**

NonposToNonnegBridge creates:

- F in MOI.Nonnegatives

source

MathOptInterface.Bridges.Constraint.NormInfinityBridge – Type.

```
NormInfinityBridge{T,F,G} <: Bridges.Constraint.AbstractBridge
```

NormInfinityBridge implements the following reformulation:

- $|x|_\infty \leq t$ into $[t - x_i, t + x_i] \in \mathbb{R}_+$.

**Source node**

NormInfinityBridge supports:

- G in MOI.NormInfinityCone{T}

**Target nodes**

NormInfinityBridge creates:

- F in MOI.Nonnegatives

source

MathOptInterface.Bridges.Constraint.NormInfinityConeToNormConeBridge – Type.

```
NormInfinityConeToNormConeBridge{T,F} <: Bridges.Constraint.AbstractBridge
```

NormInfinityConeToNormConeBridge implements the following reformulations:

- $(t, x) in NormInfinityCone(d)$ into $(t, x) in NormCone(Inf, d)$

**Source node**

NormInfinityConeToNormConeBridge supports:

- F in MOI.NormInfinityCone

**Target nodes**

`NormInfinityConeToNormConeBridge` creates:

- `F` in `MOI.NormCone`

source

`MathOptInterface.Bridges.Constraint.NormNuclearBridge` – Type.

```
NormNuclearBridge{T,F,G,H} <: Bridges.Constraint.AbstractBridge
```

`NormNuclearBridge` implements the following reformulation:

- $t \geq \sum_i \sigma_i(X)$ into $\begin{bmatrix} U & X^\top \\ X & V \end{bmatrix} \succeq 0$ and $2t \geq tr(U) + tr(V)$.

**Source node**

`NormNuclearBridge` supports:

- `H` in `MOI.NormNuclearCone`

**Target nodes**

`NormNuclearBridge` creates:

- `F` in `MOI.GreaterThan{T}`
- `G` in `MOI.PositiveSemidefiniteConeTriangle`

source

`MathOptInterface.Bridges.Constraint.NormOneBridge` – Type.

```
NormOneBridge{T,F,G} <: Bridges.Constraint.AbstractBridge
```

`NormOneBridge` implements the following reformulation:

- $\sum |x_i| \leq t$ into $[t - \sum y_i, y_i - x_i, y_i + x_i] \in \mathbb{R}_+$.

**Source node**

`NormOneBridge` supports:

- `G` in `MOI.NormOneCone{T}`

**Target nodes**

`NormOneBridge` creates:

- `F` in `MOI.Nonnegatives`

source

MathOptInterface.Bridges.Constraint.NormOneConeToNormConeBridge – Type.

```
NormOneConeToNormConeBridge{T,F} <: Bridges.Constraint.AbstractBridge
```

NormOneConeToNormConeBridge implements the following reformulations:

- $(t, x) in NormOneCone(d)$ into $(t, x) in NormCone(1, d)$

**Source node**

NormOneConeToNormConeBridge supports:

- F in MOI.NormOneCone

**Target nodes**

NormOneConeToNormConeBridge creates:

- F in MOI.NormCone

source

MathOptInterface.Bridges.Constraint.NormSpectralBridge – Type.

```
NormSpectralBridge{T,F,G} <: Bridges.Constraint.AbstractBridge
```

NormSpectralBridge implements the following reformulation:

- $t \geq \sigma_1(X)$ into $\begin{bmatrix} t\mathbf{I} & X^\top \\ X & t\mathbf{I} \end{bmatrix} \succeq 0$

**Source node**

NormSpectralBridge supports:

- G in MOI.NormSpectralCone

**Target nodes**

NormSpectralBridge creates:

- F in MOI.PositiveSemidefiniteConeTriangle

source

MathOptInterface.Bridges.Constraint.NormToPowerBridge – Type.

```
NormToPowerBridge{T,F} <: Bridges.Constraint.AbstractBridge
```

NormToPowerBridge implements the following reformulation:

- $(t, x) \in NormCone(p, 1 + d)$ into $(r_i, t, x_i) \in PowerCone(1/p)$ for all $i$, and $\sum_i r_i == t$.

For details, see Alizadeh, F., and Goldfarb, D. (2001). "Second-order cone programming." Mathematical Programming, Series B, 95:3-51.

**Source node**

`NormToPowerBridge` supports:

- F in `MOI.NormCone`

**Target nodes**

`NormToPowerBridge` creates:

- F in `MOI.PowerCone{T}`
- `MOI.ScalarAffineFunction` in `MOI.EqualTo`

source

`MathOptInterface.Bridges.Constraint.NumberConversionBridge` – Type.

```
NumberConversionBridge{T,F1,S1,F2,S2} <: Bridges.Constraint.AbstractBridge
```

`NumberConversionBridge` implements the following reformulation:

- $f1(x) \in S1$ to $f2(x) \in S2$

where `f` and `S` are the same functional form, but differ in their coefficient type.

**Source node**

`NumberConversionBridge` supports:

- F1 in S1

**Target node**

`NumberConversionBridge` creates:

- F2 in S2

source

`MathOptInterface.Bridges.Constraint.QuadtoSOCBridge` – Type.

```
QuadtoSOCBridge{T} <: Bridges.Constraint.AbstractBridge
```

`QuadtoSOCBridge` converts quadratic inequalities

$$\frac{1}{2}x^T Q x + a^T x \leq ub$$

into MOI.RotatedSecondOrderCone constraints, but it only applies when $Q$ is positive definite.

This is because, if Q is positive definite, there exists U such that $Q = U^T U$, and so the inequality can then be rewritten as;

$$\|Ux\|_2^2 \leq 2(-a^T x + ub)$$

Therefore, QuadtoSOCBridge implements the following reformulations:

- $\frac{1}{2}x^T Q x + a^T x \leq ub$ into $(1, -a^T x + ub, Ux) \in RotatedSecondOrderCone$ where $Q = U^T U$
- $\frac{1}{2}x^T Q x + a^T x \geq lb$ into $(1, a^T x - lb, Ux) \in RotatedSecondOrderCone$ where $-Q = U^T U$

**Source node**

QuadtoSOCBridge supports:

- MOI.ScalarAffineFunction{T} in MOI.LessThan{T}
- MOI.ScalarAffineFunction{T} in MOI.GreaterThan{T}

**Target nodes**

RelativeEntropyBridge creates:

- MOI.VectorAffineFunction{T} in MOI.RotatedSecondOrderCone

**Errors**

This bridge errors if Q is not positive definite.

source

MathOptInterface.Bridges.Constraint.RSOCtoNonConvexQuadBridge – Type.

```
RSOCtoNonConvexQuadBridge{T} <: Bridges.Constraint.AbstractBridge
```

RSOCtoNonConvexQuadBridge implements the following reformulations:

- $||x||_2^2 \leq 2tu$ into $\sum x^2 - 2tu \leq 0$, $1t + 0 \geq 0$, and $1u + 0 \geq 0$.

The MOI.ScalarAffineFunctions $1t + 0$ and $1u + 0$ are used in case the variables have other bound constraints.

**Warning**

This transformation starts from a convex constraint and creates a non-convex constraint. Unless the solver has explicit support for detecting rotated second-order cones in quadratic form, this may (wrongly) be interpreted by the solver as being non-convex. Therefore, this bridge is not added automatically by MOI.Bridges.full_bridge_optimizer. Care is recommended when adding this bridge to a optimizer.

**Source node**

RSOCtoNonConvexQuadBridge supports:

- MOI.VectorOfVariables in MOI.RotatedSecondOrderCone

**Target nodes**

RSOCtoNonConvexQuadBridge creates:

- MOI.ScalarQuadraticFunction{T} in MOI.LessThan{T}
- MOI.ScalarAffineFunction{T} in MOI.GreaterThan{T}

source

MathOptInterface.Bridges.Constraint.RSOCtoPSDBridge – Type.

```
RSOCtoPSDBridge{T,F,G} <: Bridges.Constraint.AbstractBridge
```

RSOCtoPSDBridge implements the following reformulation:

- $||x||_2^2 \leq 2t \cdot u$ into $\begin{bmatrix} t & x^\top \\ x & 2tu\mathbf{I} \end{bmatrix} \succeq 0$

**Source node**

RSOCtoPSDBridge supports:

- G in MOI.RotatedSecondOrderCone

**Target nodes**

RSOCtoPSDBridge creates:

- F in MOI.PositiveSemidefiniteConeTriangle

source

MathOptInterface.Bridges.Constraint.RSOCtoSOCBridge – Type.

```
RSOCtoSOCBridge{T,F,G} <: Bridges.Constraint.AbstractBridge
```

RSOCtoSOCBridge implements the following reformulation:

- $||x||_2^2 \leq 2tu$ into $||\frac{t-u}{\sqrt{2}}, x||_2 \leq \frac{t+u}{\sqrt{2}}$

**Source node**

RSOCtoSOCBridge supports:

- G in MOI.RotatedSecondOrderCone

**Target node**

RSOCtoSOCBridge creates:

- F in MOI.SecondOrderCone

source

MathOptInterface.Bridges.Constraint.ReifiedAllDifferentToCountDistinctBridge – Type.

```
ReifiedAllDifferentToCountDistinctBridge{T,F} <:
Bridges.Constraint.AbstractBridge
```

ReifiedAllDifferentToCountDistinctBridge implements the following reformulations:

- $r \iff x \in$ AllDifferent$(d)$ to $r \iff (n, x) \in$ CountDistinct$(1 + d)$ and $n = d$
- $r \iff f(x) \in$ AllDifferent$(d)$ to $r \iff (d, f(x)) \in$ CountDistinct$(1 + d)$

**Source node**

ReifiedAllDifferentToCountDistinctBridge supports:

- F in MOI.Reified{MOI.AllDifferent}

where F is MOI.VectorOfVariables or MOI.VectorAffineFunction{T}.

**Target nodes**

ReifiedAllDifferentToCountDistinctBridge creates:

- F in MOI.Reified{MOI.CountDistinct}
- MOI.VariableIndex in MOI.EqualTo{T}

source

MathOptInterface.Bridges.Constraint.ReifiedCountDistinctToMILPBridge – Type.

```
ReifiedCountDistinctToMILPBridge{T,F} <: Bridges.Constraint.AbstractBridge
```

ReifiedCountDistinctToMILPBridge implements the following reformulation:

- $r \iff (n, x) \in$ CountDistinct$(1 + d)$ into a mixed-integer linear program.

**Reformulation**

The reformulation is non-trivial, and it depends on the finite domain of each variable $x_i$, which we as define $S_i = \{l_i, \ldots, u_i\}$.

First, we introduce new binary variables $z_{ij}$, which are $1$ if variable $x_i$ takes the value $j$ in the optimal solution and $0$ otherwise:

$$\begin{aligned} z_{ij} &\in \{0, 1\} \quad \forall i \in 1 \ldots d, j \in S_i \\ x_i - \sum_{j \in S_i} j \cdot z_{ij} &= 0 \quad \forall i \in 1 \ldots d \\ \sum_{j \in S_i} z_{ij} &= 1 \quad \forall i \in 1 \ldots d \end{aligned}$$

Then, we introduce new binary variables $y_j$, which are $1$ if a variable takes the value $j$ in the optimal solution and $0$ otherwise.

$$y_j \in \{0, 1\} \quad \forall j \in \bigcup_{i=1,\ldots,d} S_i$$
$$y_j \leq \sum_{i \in 1\ldots d : j \in S_i} z_{ij} \leq M y_j \quad \forall j \in \bigcup_{i=1,\ldots,d} S_i$$

Finally, $n$ is constrained to be the number of $y_j$ elements that are non-zero, with some slack:

$$n - \sum_{j \in \bigcup_{i=1,\ldots,d} S_i} y_j = \delta^+ - \delta^-$$

And then the slack is constrained to respect the reif variable $r$:

$$\begin{aligned} d_1 \leq \delta^+ \leq M d_1 \\ d_2 \leq \delta^- \leq M d_s \\ d_1 + d_2 + r = 1 \\ d_1, d_2 \in \{0, 1\} \end{aligned}$$

**Source node**

`ReifiedCountDistinctToMILPBridge` supports:

- F in `MOI.Reified{MOI.CountDistinct}`

where F is `MOI.VectorOfVariables` or `MOI.VectorAffineFunction{T}`.

**Target nodes**

`ReifiedCountDistinctToMILPBridge` creates:

- `MOI.VariableIndex` in `MOI.ZeroOne`
- `MOI.ScalarAffineFunction{T}` in `MOI.EqualTo{T}`
- `MOI.ScalarAffineFunction{T}` in `MOI.LessThan{T}`

source

`MathOptInterface.Bridges.Constraint.RelativeEntropyBridge` – Type.

```
RelativeEntropyBridge{T,F,G,H} <: Bridges.Constraint.AbstractBridge
```

`RelativeEntropyBridge` implements the following reformulation that converts a `MOI.RelativeEntropyCone` into an `MOI.ExponentialCone`:

- $u \geq \sum_{i=1}^{n} w_i \log\left(\frac{w_i}{v_i}\right)$ into $y_i \geq 0$, $u \geq \sum_{i=1}^{n} y_i$, and $(-y_i, w_i, v_i) \in ExponentialCone$.

**Source node**

RelativeEntropyBridge supports:

- H in MOI.RelativeEntropyCone

**Target nodes**

RelativeEntropyBridge creates:

- F in MOI.GreaterThan{T}
- G in MOI.ExponentialCone

source

MathOptInterface.Bridges.Constraint.RootDetBridge – Type.

```
RootDetBridge{T,F,G,H} <: Bridges.Constraint.AbstractBridge
```

The MOI.RootDetConeTriangle is representable by MOI.PositiveSemidefiniteConeTriangle and MOI.GeometricMeanCone constraints, see [1, p. 149].

Indeed, $t \leq \det(X)^{1/n}$ if and only if there exists a lower triangular matrix  such that:

$$\begin{pmatrix} X & \\ \top & \text{Diag}() \end{pmatrix} \succeq 0$$
$$(t, \text{Diag}()) \in GeometricMeanCone$$

**Source node**

RootDetBridge supports:

- I in MOI.RootDetConeTriangle

**Target nodes**

RootDetBridge creates:

- F in MOI.PositiveSemidefiniteConeTriangle
- G in MOI.GeometricMeanCone

[1] Ben-Tal, Aharon, and Arkadi Nemirovski. Lectures on modern convex optimization: analysis, algorithms, and engineering applications. Society for Industrial and Applied Mathematics, 2001.

source

MathOptInterface.Bridges.Constraint.SOCtoNonConvexQuadBridge – Type.

```
SOCtoNonConvexQuadBridge{T} <: Bridges.Constraint.AbstractBridge
```

SOCtoNonConvexQuadBridge implements the following reformulations:

- $||x||_2 \leq t$ into $\sum x^2 - t^2 \leq 0$ and $1t + 0 \geq 0$

The MOI.ScalarAffineFunction $1t + 0$ is used in case the variable has other bound constraints.

**Warning**

This transformation starts from a convex constraint and creates a non-convex constraint. Unless the solver has explicit support for detecting second-order cones in quadratic form, this may (wrongly) be interpreted by the solver as being non-convex. Therefore, this bridge is not added automatically by MOI.Bridges.full_bridge_optimizer. Care is recommended when adding this bridge to a optimizer.

**Source node**

SOCtoNonConvexQuadBridge supports:

- MOI.VectorOfVariables in MOI.SecondOrderCone

**Target nodes**

SOCtoNonConvexQuadBridge creates:

- MOI.ScalarQuadraticFunction{T} in MOI.LessThan{T}
- MOI.ScalarAffineFunction{T} in MOI.GreaterThan{T}

source

MathOptInterface.Bridges.Constraint.SOCtoPSDBridge – Type.

```
SOCtoPSDBridge{T,F,G} <: Bridges.Constraint.AbstractBridge
```

SOCtoPSDBridge implements the following reformulation:

- $||x||_2 \leq t$ into $\begin{bmatrix} t & x^\top \\ x & t\mathbf{I} \end{bmatrix} \succeq 0$

**Warning**

This bridge is not added by default by MOI.Bridges.full_bridge_optimizer because bridging second order cone constraints to semidefinite constraints can be achieved by the SOCtoRSOCBridge followed by the RSOCtoPSDBridge, while creating a smaller semidefinite constraint.

**Source node**

SOCtoPSDBridge supports:

- G in MOI.SecondOrderCone

**Target nodes**

SOCtoPSDBridge creates:

- F in MOI.PositiveSemidefiniteConeTriangle

source

MathOptInterface.Bridges.Constraint.SOCtoRSOCBridge – Type.

```
SOCtoRSOCBridge{T,F,G} <: Bridges.Constraint.AbstractBridge
```

SOCtoRSOCBridge implements the following reformulation:

- $||x||_2 \leq t$ into $(t + x_1)(t - x_1) \geq ||(x_2 \ldots, x_N)||_2^2$

**Assumptions**

- SOCtoRSOCBridge assumes that the length of x is at least one.

**Source node**

SOCtoRSOCBridge supports:

- G in MOI.SecondOrderCone

**Target node**

SOCtoRSOCBridge creates:

- F in MOI.RotatedSecondOrderCone

source

MathOptInterface.Bridges.Constraint.SOS1ToMILPBridge – Type.

```
SOS1ToMILPBridge{T,F} <: Bridges.Constraint.AbstractBridge
```

SOS1ToMILPBridge implements the following reformulation:

- $x \in$ SOS1$(d)$ into the following mixed-integer linear program.

Assuming $l_i \leq x_i \leq u_i$ with finite bounds $l_i$, $u_i$:

$$\begin{aligned} z_i \in \{0,1\} \quad \forall i \in 1 \ldots d \\ l_i z_i \leq x_i \leq u_i z_i \quad \forall i \in 1 \ldots d \\ \sum z_i = 1 \end{aligned}$$

**Source node**

SOS1ToMILPBridge supports:

- F in MOI.SOS1

where F is MOI.VectorOfVariables or MOI.VectorAffineFunction{T}.

**Target nodes**

SOS1ToMILPBridge creates:

- MOI.VariableIndex in MOI.ZeroOne
- MOI.ScalarAffineFunction{T} in MOI.EqualTo{T}
- MOI.ScalarAffineFunction{T} in MOI.LessThan{T}

source

MathOptInterface.Bridges.Constraint.SOS2ToMILPBridge – Type.

```
SOS2ToMILPBridge{T,F} <: Bridges.Constraint.AbstractBridge
```

SOS2ToMILPBridge implements the following reformulation:

- $x \in$ SOS2$(d)$ into a mixed-integer linear program.

**Source node**

SOS2ToMILPBridge supports:

- F in MOI.SOS2

where F is MOI.VectorOfVariables or MOI.VectorAffineFunction{T}.

**Target nodes**

SOS2ToMILPBridge creates:

- MOI.VariableIndex in MOI.ZeroOne
- MOI.ScalarAffineFunction{T} in MOI.EqualTo{T}
- MOI.ScalarAffineFunction{T} in MOI.LessThan{T}

source

MathOptInterface.Bridges.Constraint.ScalarFunctionizeBridge – Type.

```
ScalarFunctionizeBridge{T,S} =
↪  FunctionConversionBridge{T,MOI.ScalarAffineFunction{T},MOI.VariableIndex,S}
```

ScalarFunctionizeBridge implements the following reformulations:

- $x \in S$ into $1x + 0 \in S$

**Source node**

ScalarFunctionizeBridge supports:

- MOI.VariableIndex in S

**Target nodes**

ScalarFunctionizeBridge creates:

- MOI.ScalarAffineFunction{T} in S

source

MathOptInterface.Bridges.Constraint.ScalarSlackBridge – Type.

```
ScalarSlackBridge{T,F,S} <: Bridges.Constraint.AbstractBridge
```

ScalarSlackBridge implements the following reformulation:

- $f(x) \in S$ into $f(x) - y == 0$ and $y \in S$

**Source node**

ScalarSlackBridge supports:

- G in S, where G is not MOI.VariableIndex and S is not MOI.EqualTo

**Target nodes**

ScalarSlackBridge creates:

- F in MOI.EqualTo{T}
- MOI.VariableIndex in S

source

MathOptInterface.Bridges.Constraint.ScalarizeBridge – Type.

```
ScalarizeBridge{T,F,S}
```

ScalarizeBridge implements the following reformulations:

- $f(x) - a \in \mathbb{R}_+$ into $f_i(x) \geq a_i$ for all $i$
- $f(x) - a \in \mathbb{R}_-$ into $f_i(x) \leq a_i$ for all $i$
- $f(x) - a \in \{0\}$ into $f_i(x) == a_i$ for all $i$

**Source node**

ScalarizeBridge supports:

- G in MOI.Nonnegatives{T}
- G in MOI.Nonpositives{T}
- G in MOI.Zeros{T}

**Target nodes**

`ScalarizeBridge` creates:

- F in S, where S is one of `MOI.GreaterThan{T}`, `MOI.LessThan{T}`, and `MOI.EqualTo{T}`, depending on the type of the input set.

source

`MathOptInterface.Bridges.Constraint.SecondOrderConeToNormConeBridge` – Type.

```
SecondOrderConeToNormConeBridge{T,F} <: Bridges.Constraint.AbstractBridge
```

`SecondOrderConeToNormConeBridge` implements the following reformulations:

- $(t, x) in SecondOrderCone(d)$ into $(t, x) in NormCone(2, d)$

**Source node**

`SecondOrderConeToNormConeBridge` supports:

- F in `MOI.SecondOrderCone`

**Target nodes**

`SecondOrderConeToNormConeBridge` creates:

- F in `MOI.NormCone`

source

`MathOptInterface.Bridges.Constraint.SemiToBinaryBridge` – Type.

```
SemiToBinaryBridge{T,S} <: Bridges.Constraint.AbstractBridge
```

`SemiToBinaryBridge` implements the following reformulations:

- $x \in \{0\} \cup [l, u]$ into

$$\begin{aligned} x &\leq zu \\ x &\geq zl \\ z &\in \{0, 1\} \end{aligned}$$

- $x \in \{0\} \cup \{l, \ldots, u\}$ into

$$\begin{aligned} x &\leq zu \\ x &\geq zl \\ z &\in \{0, 1\} \\ x &\in \mathbb{Z} \end{aligned}$$

**Source node**

SemiToBinaryBridge supports:

- MOI.VariableIndex in MOI.Semicontinuous{T}
- MOI.VariableIndex in MOI.Semiinteger{T}

**Target nodes**

SemiToBinaryBridge creates:

- MOI.VariableIndex in MOI.ZeroOne
- MOI.ScalarAffineFunction{T} in MOI.LessThan{T}
- MOI.ScalarAffineFunction{T} in MOI.GreaterThan{T}
- MOI.VariableIndex{T} in MOI.Integer (if S is MOI.Semiinteger{T}

source

MathOptInterface.Bridges.Constraint.SetConversionBridge – Type.

```
SetConversionBridge{T,S2,S1,F} <:
    MOI.Bridges.Constraint.SetMapBridge{T,S2,S1,F,F}
```

SetConversionBridge implements the following reformulations:

- $f(x) \in S1$ into $f(x) \in S2$

In order to add this bridge, you need to create a bridge specific for a given type T and set S2:

```
MOI.Bridges.add_bridge(model, MOI.Bridges.Constraint.SetConversionBridge{T,S2})
```

In order to define a bridge with S2 specified but T unspecified, for example for JuMP.add_bridge, you can use

```
const MyBridge{T,S1,F} = MOI.Bridges.Constraint.SetConversionBridge{T,S2,S1,F}
```

See also FunctionConversionBridge.

**Source node**

SetConversionBridge supports:

- F in S1

**Target nodes**

SetConversionBridge creates:

- F in S2

source

MathOptInterface.Bridges.Constraint.SetDotInverseScalingBridge – Type.

```
SetDotInverseScalingBridge{T,S,F,G} <: Bridges.Constraint.AbstractBridge
```

SetDotInverseScalingBridge implements the reformulation from constraints in the MOI.Scaled{S} to constraints in the S.

**Source node**

SetDotInverseScalingBridge supports:

- G in MOI.Scaled{S}

**Target node**

SetDotInverseScalingBridge creates:

- F in S

source

MathOptInterface.Bridges.Constraint.SetDotScalingBridge – Type.

```
SetDotScalingBridge{T,S,F,G} <: Bridges.Constraint.AbstractBridge
```

SetDotScalingBridge implements the reformulation from constraints in S to constraints in MOI.Scaled{S}.

**Source node**

SetDotScalingBridge supports:

- G in S

**Target node**

SetDotScalingBridge creates:

- F in MOI.Scaled{S}

source

MathOptInterface.Bridges.Constraint.SetMapBridge – Type.

```
abstract type SetMapBridge{T,S2,S1,F,G} <: MultiSetMapBridge{T,S1,G} end
```

Consider two type of sets, S1 and S2, and a linear mapping A such that the image of a set of type S1 under A is a set of type S2.

A SetMapBridge{T,S2,S1,F,G} is a bridge that maps G-in-S1 constraints into F-in-S2 by mapping the function through A.

The linear map A is described by;

- MOI.Bridges.map_set

- MOI.Bridges.map_function.

Implementing a method for these two functions is sufficient to bridge constraints. However, in order for the getters and setters of attributes such as dual solutions and starting values to work as well, a method for the following functions must be implemented:

- MOI.Bridges.inverse_map_set
- MOI.Bridges.inverse_map_function
- MOI.Bridges.adjoint_map_function
- MOI.Bridges.inverse_adjoint_map_function

See the docstrings of each function to see which feature would be missing if it was not implemented for a given bridge.

source

MathOptInterface.Bridges.Constraint.SplitComplexEqualToBridge – Type.

```
SplitComplexEqualToBridge{T,F,G} <: Bridges.Constraint.AbstractBridge
```

SplitComplexEqualToBridge implements the following reformulation:

- $f(x) + g(x) * im = a + b * im$ into $f(x) = a$ and $g(x) = b$

**Source node**

SplitComplexEqualToBridge supports:

- G in MOI.EqualTo{Complex{T}}

where G is a function with Complex coefficients.

**Target nodes**

SplitComplexEqualToBridge creates:

- F in MOI.EqualTo{T}

where F is the type of the real/imaginary part of G.

source

MathOptInterface.Bridges.Constraint.SplitComplexIndicatorEqualToBridge – Type.

```
SplitComplexIndicatorEqualToBridge{T,F,G,A} <:
Bridges.Constraint.AbstractBridge
```

SplitComplexIndicatorEqualToBridge implements the following reformulation:

- $z \implies f(x) + g(x) * im = a + b * im$ into $z \implies f(x) = a$ and $z \implies g(x) = b$

**Source node**

`SplitComplexIndicatorEqualToBridge` supports:

- `G` in `MOI.Indicator{A,MOI.EqualTo{Complex{T}}`

where `G` is a function with `Complex` coefficients.

**Target nodes**

`SplitComplexIndicatorEqualToBridge` creates:

- `F` in `MOI.Indicator{A,MOI.EqualTo{T}}`

where `F` is the type of the real/imaginary part of `G`.

source

`MathOptInterface.Bridges.Constraint.SplitComplexZerosBridge` – Type.

```
SplitComplexZerosBridge{T,F,G} <: Bridges.Constraint.AbstractBridge
```

`SplitComplexZerosBridge` implements the following reformulation:

- $f(x) \in \{0\}^n$ into $\text{Re}(f(x)) \in \{0\}^n$ and $\text{Im}(f(x)) \in \{0\}^n$

**Source node**

`SplitComplexZerosBridge` supports:

- `G` in `MOI.Zeros`

where `G` is a function with `Complex` coefficients.

**Target nodes**

`SplitComplexZerosBridge` creates:

- `F` in `MOI.Zeros`

where `F` is the type of the real/imaginary part of `G`.

source

`MathOptInterface.Bridges.Constraint.SplitHyperRectangleBridge` – Type.

```
SplitHyperRectangleBridge{T,G,F} <: Bridges.Constraint.AbstractBridge
```

`SplitHyperRectangleBridge` implements the following reformulation:

- $f(x) \in \text{HyperRectangle}(l, u)$ to $[f(x) - l; u - f(x)] \in \mathbb{R}_+$.

**Source node**

`SplitHyperRectangleBridge` supports:

- F in MOI.HyperRectangle

**Target nodes**

SplitHyperRectangleBridge creates:

- G in MOI.Nonnegatives

source

MathOptInterface.Bridges.Constraint.SplitIntervalBridge – Type.

```
SplitIntervalBridge{T,F,S,LS,US} <: Bridges.Constraint.AbstractBridge
```

SplitIntervalBridge implements the following reformulations:

- $l \leq f(x) \leq u$ into $f(x) \geq l$ and $f(x) \leq u$
- $f(x) = b$ into $f(x) \geq b$ and $f(x) \leq b$
- $f(x) \in \{0\}$ into $f(x) \in \mathbb{R}_+$ and $f(x) \in \mathbb{R}_-$

**Source node**

SplitIntervalBridge supports:

- F in MOI.Interval{T}
- F in MOI.EqualTo{T}
- F in MOI.Zeros

**Target nodes**

SplitIntervalBridge creates:

- F in MOI.LessThan{T}
- F in MOI.GreaterThan{T}

or

- F in MOI.Nonnegatives
- F in MOI.Nonpositives

**Note**

If T<:AbstractFloat and S is MOI.Interval{T} then no lower (resp. upper) bound constraint is created if the lower (resp. upper) bound is typemin(T) (resp. typemax(T)). Similarly, when MOI.ConstraintSet is set, a lower or upper bound constraint may be deleted or created accordingly.

source

MathOptInterface.Bridges.Constraint.SquareBridge – Type.

```
SquareBridge{T,F,G,TT,ST} <: Bridges.Constraint.AbstractBridge
```

SquareBridge implements the following reformulations:

- $(t, u, X) \in LogDetConeSquare$ into $(t, u, Y) in LogDetConeTriangle$
- $(t, X) \in RootDetConeSquare$ into $(t, Y) in RootDetConeTriangle$
- $X \in AbstractSymmetricMatrixSetSquare$ into $Y in AbstractSymmetricMatrixSetTriangle$

where $Y$ is the upper triangluar component of $X$.

In addition, constraints are added as necessary to constrain the matrix $X$ to be symmetric. For example, the constraint for the matrix:

$$\begin{pmatrix} 1 & 1+x & 2-3x \\ 1+x & 2+x & 3-x \\ 2-3x & 2+x & 2x \end{pmatrix}$$

can be broken down to the constraint of the symmetric matrix

$$\begin{pmatrix} 1 & 1+x & 2-3x \\ \cdot & 2+x & 3-x \\ \cdot & \cdot & 2x \end{pmatrix}$$

and the equality constraint between the off-diagonal entries (2, 3) and (3, 2) $3-x == 2+x$. Note that no symmetrization constraint needs to be added between the off-diagonal entries (1, 2) and (2, 1) or between (1, 3) and (3, 1) because the expressions are the same.

**Source node**

SquareBridge supports:

- F in ST

**Target nodes**

SquareBridge creates:

- F in TT
- G in `MOI.EqualTo{T}`

source

MathOptInterface.Bridges.Constraint.TableToMILPBridge – Type.

```
TableToMILPBridge{T,F} <: Bridges.Constraint.AbstractBridge
```

TableToMILPBridge implements the following reformulation:

- $x \in Table(t)$ into

$$
\begin{aligned}
z_j &\in \{0, 1\} \quad \forall i, j \\
\sum_{j=1}^{n} z_j &= 1 \\
\sum_{j=1}^{n} t_{ij} z_j &= x_i \quad \forall i
\end{aligned}
$$

**Source node**

`TableToMILPBridge` supports:

- `F` in `MOI.Table{T}`

**Target nodes**

`TableToMILPBridge` creates:

- `MOI.VariableIndex` in `MOI.ZeroOne`
- `MOI.ScalarAffineFunction{T}` in `MOI.EqualTo{T}`

source

`MathOptInterface.Bridges.Constraint.ToScalarNonlinearBridge` – Type.

```
ToScalarNonlinearBridge{T,G,S} <: AbstractFunctionConversionBridge{G,S}
```

`ToScalarNonlinearBridge` implements the following reformulation:

- $g(x) \in S$ into $f(x) \in S$

where `g` is an abstract scalar function and `f` is a `MOI.ScalarNonlinearFunction`.

**Source node**

`ToScalarNonlinearBridge` supports:

- `G<:AbstractScalarFunction` in `S`

**Target nodes**

`ToScalarNonlinearBridge` creates:

- `MOI.ScalarNonlinearFunction` in `S`

source

`MathOptInterface.Bridges.Constraint.ToScalarQuadraticBridge` – Type.

```
ToScalarQuadraticBridge{T,G,S} <: AbstractFunctionConversionBridge{G,S}
```

`ToScalarQuadraticBridge` implements the following reformulation:

- $g(x) \in S$ into $f(x) \in S$

where `g` is an abstract scalar function and `f` is a `MOI.ScalarQuadraticFunction`.

**Source node**

`ToScalarQuadraticBridge` supports:

- `G<:AbstractScalarFunction` in `S`

**Target nodes**

`ToScalarQuadraticBridge` creates:

- `MOI.ScalarQuadraticFunction` in `S`

source

`MathOptInterface.Bridges.Constraint.ToVectorQuadraticBridge` – Type.

```
ToVectorQuadraticBridge{T,G,S} <: AbstractFunctionConversionBridge{G,S}
```

`ToVectorQuadraticBridge` implements the following reformulation:

- $g(x) \in S$ into $f(x) \in S$

where `g` is an abstract vector function and `f` is a `MOI.VectorQuadraticFunction`.

**Source node**

`ToVectorQuadraticBridge` supports:

- `G<:AbstractVectorFunction` in `S`

**Target nodes**

`ToVectorQuadraticBridge` creates:

- `MOI.VectorQuadraticFunction` in `S`

source

`MathOptInterface.Bridges.Constraint.VectorFunctionizeBridge` – Type.

```
VectorFunctionizeBridge{T,S} = FunctionConversionBridge{T,MOI.VectorAffineFunction{T},S}
```

`VectorFunctionizeBridge` implements the following reformulations:

- $x \in S$ into $Ix + 0 \in S$

**Source node**

VectorFunctionizeBridge supports:

- MOI.VectorOfVariables in S

**Target nodes**

VectorFunctionizeBridge creates:

- MOI.VectorAffineFunction{T} in S

source

MathOptInterface.Bridges.Constraint.VectorSlackBridge – Type.

```
VectorSlackBridge{T,F,S} <: Bridges.Constraint.AbstractBridge
```

VectorSlackBridge implements the following reformulation:

- $f(x) \in S$ into $f(x) - y \in \{0\}$ and $y \in S$

**Source node**

VectorSlackBridge supports:

- G in S, where G is not MOI.VectorOfVariables and S is not MOI.Zeros

**Target nodes**

VectorSlackBridge creates:

- F in MOI.Zeros
- MOI.VectorOfVariables in S

source

MathOptInterface.Bridges.Constraint.VectorizeBridge – Type.

```
VectorizeBridge{T,F,S,G} <: Bridges.Constraint.AbstractBridge
```

VectorizeBridge implements the following reformulations:

- $g(x) \geq a$ into $[g(x) - a] \in \mathbb{R}_+$
- $g(x) \leq a$ into $[g(x) - a] \in \mathbb{R}_-$
- $g(x) == a$ into $[g(x) - a] \in \{0\}$

where T is the coefficient type of `g(x) - a`.

See also `IntervalToHyperRectangleBridge` for double-sided bound constraints.

**Source node**

`VectorizeBridge` supports:

- `G` in `MOI.GreaterThan{T}`
- `G` in `MOI.LessThan{T}`
- `G` in `MOI.EqualTo{T}`

**Target nodes**

`VectorizeBridge` creates:

- `F` in `S`, where `S` is one of `MOI.Nonnegatives`, `MOI.Nonpositives`, `MOI.Zeros` depending on the type of the input set.

source

`MathOptInterface.Bridges.Constraint.ZeroOneBridge` – Type.

```
ZeroOneBridge{T} <: Bridges.Constraint.AbstractBridge
```

`ZeroOneBridge` implements the following reformulation:

- $x \in \{0, 1\}$ into $x \in \mathbb{Z}, 1x \in [0, 1]$.

**Note**

`ZeroOneBridge` adds a linear constraint instead of adding variable bounds to avoid conflicting with bounds set by the user.

**Source node**

`ZeroOneBridge` supports:

- `MOI.VariableIndex` in `MOI.ZeroOne`

**Target nodes**

`ZeroOneBridge` creates:

- `MOI.VariableIndex` in `MOI.Integer`
- `MOI.ScalarAffineFunction{T}` in `MOI.Interval{T}`

source

**Objective bridges**

These bridges are subtypes of `Bridges.Objective.AbstractBridge`.

`MathOptInterface.Bridges.Objective.FunctionConversionBridge` – Type.

```
FunctionConversionBridge{T,F,G} <: AbstractBridge
```

FunctionConversionBridge implements the following reformulations:

- $\min\{g(x)\}$ into $\min\{f(x)\}$
- $\max\{g(x)\}$ into $\max\{f(x)\}$

for these pairs of functions:

- MOI.ScalarAffineFunction to MOI.ScalarQuadraticFunction
- MOI.ScalarQuadraticFunction to MOI.ScalarNonlinearFunction
- MOI.VectorAffineFunction to MOI.VectorQuadraticFunction

**Source node**

FunctionConversionBridge supports:

- MOI.ObjectiveFunction{G}

**Target nodes**

FunctionConversionBridge creates:

- One objective node: MOI.ObjectiveFunction{F}

source

MathOptInterface.Bridges.Objective.FunctionizeBridge – Type.

```
FunctionizeBridge{T,G} <: FunctionConversionBridge{T,MOI.ScalarAffineFunction{T},G}
```

FunctionizeBridge implements the following reformulations:

- $\min\{x\}$ into $\min\{1x + 0\}$
- $\max\{x\}$ into $\max\{1x + 0\}$

where T is the coefficient type of 1 and 0.

**Source node**

FunctionizeBridge supports:

- MOI.ObjectiveFunction{G}

**Target nodes**

FunctionizeBridge creates:

- One objective node: MOI.ObjectiveFunction{MOI.ScalarAffineFunction{T}}

source

MathOptInterface.Bridges.Objective.QuadratizeBridge – Type.

```
QuadratizeBridge{T,G} <: FunctionConversionBridge{T,MOI.ScalarQuadraticFunction{T},G}
```

`QuadratizeBridge` implements the following reformulations:

- $\min\{a^\top x + b\}$ into $\min\{x^\top \mathbf{0} x + a^\top x + b\}$
- $\max\{a^\top x + b\}$ into $\max\{x^\top \mathbf{0} x + a^\top x + b\}$

where `T` is the coefficient type of `0`.

**Source node**

`QuadratizeBridge` supports:

- `MOI.ObjectiveFunction{G}`

**Target nodes**

`QuadratizeBridge` creates:

- One objective node: `MOI.ObjectiveFunction{MOI.ScalarQuadraticFunction{T}}`

source

`MathOptInterface.Bridges.Objective.SlackBridge` – Type.

```
SlackBridge{T,F,G}
```

`SlackBridge` implements the following reformulations:

- $\min\{f(x)\}$ into $\min\{y \mid f(x) - y \leq 0\}$
- $\max\{f(x)\}$ into $\max\{y \mid f(x) - y \geq 0\}$

where `F` is the type of `f(x) - y`, `G` is the type of `f(x)`, and `T` is the coefficient type of `f(x)`.

**Source node**

`SlackBridge` supports:

- `MOI.ObjectiveFunction{G}`

**Target nodes**

`SlackBridge` creates:

- One variable node: `MOI.VariableIndex` in `MOI.Reals`
- One objective node: `MOI.ObjectiveFunction{MOI.VariableIndex}`
- One constraint node, that depends on the `MOI.ObjectiveSense`:
  - F-in-`MOI.LessThan` if `MIN_SENSE`
  - F-in-`MOI.GreaterThan` if `MAX_SENSE`

**Warning**

When using this bridge, changing the optimization sense is not supported. Set the sense to `MOI.FEASIBILITY_SENSE` first to delete the bridge, then set `MOI.ObjectiveSense` and re-add the objective.

source

`MathOptInterface.Bridges.Objective.ToScalarNonlinearBridge` – Type.

```
ToScalarNonlinearBridge{T,G} <: FunctionConversionBridge{T,MOI.ScalarNonlinearFunction,G}
```

`ToScalarNonlinearBridge` implements the following reformulations:

- $\min\{x^\top x + a^\top x + b\}$ into $\min\{f(x)\}$
- $\max\{x^\top x + a^\top x + b\}$ into $\max\{f(x)\}$

where `f(x)` is a `MOI.ScalarNonlinearFunction`.

**Source node**

`ToScalarNonlinearBridge` supports:

- `MOI.ObjectiveFunction{G}`

**Target nodes**

`ToScalarNonlinearBridge` creates:

- One objective node: `MOI.ObjectiveFunction{MOI.ScalarNonlinearFunction}`

source

`MathOptInterface.Bridges.Objective.VectorFunctionizeBridge` – Type.

```
VectorFunctionizeBridge{T,G} <: FunctionConversionBridge{T,MOI.VectorAffineFunction{T},G}
```

`VectorFunctionizeBridge` implements the following reformulations:

- $\min\{x\}$ into $\min\{1x + 0\}$
- $\max\{x\}$ into $\max\{1x + 0\}$

where `T` is the coefficient type of `1` and `0`.

**Source node**

`VectorFunctionizeBridge` supports:

- `MOI.ObjectiveFunction{G}`

**Target nodes**

`VectorFunctionizeBridge` creates:

- One objective node: `MOI.ObjectiveFunction{MOI.VectorAffineFunction{T}}`

source

`MathOptInterface.Bridges.Objective.VectorSlackBridge` – Type.

```
VectorSlackBridge{T,F,G}
```

`VectorSlackBridge` implements the following reformulations:

- $\min\{f(x)\}$ into $\min\{y \mid y - f(x) \in \mathbb{R}_+\}$
- $\max\{f(x)\}$ into $\max\{y \mid f(x) - y \in \mathbb{R}_+\}$

where `F` is the type of `f(x) - y`, `G` is the type of `f(x)`, and `T` is the coefficient type of `f(x)`.

**Source node**

`VectorSlackBridge` supports:

- `MOI.ObjectiveFunction{G}`

**Target nodes**

`VectorSlackBridge` creates:

- One variable node: `MOI.VectorOfVariables` in `MOI.Reals`
- One objective node: `MOI.ObjectiveFunction{MOI.VectorOfVariables}`
- One constraint node: `F`-in-`MOI.Nonnegatives`

**Warning**

When using this bridge, changing the optimization sense is not supported. Set the sense to `MOI.FEASIBILITY_SENSE` first to delete the bridge, then set `MOI.ObjectiveSense` and re-add the objective.

source

### Variable bridges

These bridges are subtypes of `Bridges.Variable.AbstractBridge`.

`MathOptInterface.Bridges.Variable.FlipSignBridge` – Type.

```
abstract type FlipSignBridge{T,S1,S2} <: SetMapBridge{T,S2,S1} end
```

An abstract type that simplifies the creation of other bridges.

source

`MathOptInterface.Bridges.Variable.FreeBridge` – Type.

```
FreeBridge{T} <: Bridges.Variable.AbstractBridge
```

`FreeBridge` implements the following reformulation:

- $x \in \mathbb{R}$ into $y, z \geq 0$ with the substitution rule $x = y - z$,

where `T` is the coefficient type of `y - z`.

**Source node**

`FreeBridge` supports:

- `MOI.VectorOfVariables` in `MOI.Reals`

**Target nodes**

`FreeBridge` creates:

- One variable node: `MOI.VectorOfVariables` in `MOI.Nonnegatives`

source

`MathOptInterface.Bridges.Variable.HermitianToSymmetricPSDBridge` – Type.

```
HermitianToSymmetricPSDBridge{T} <: Bridges.Variable.AbstractBridge
```

`HermitianToSymmetricPSDBridge` implements the following reformulation:

- Hermitian positive semidefinite `n x n` complex matrix to a symmetric positive semidefinite `2n x 2n` real matrix satisfying equality constraints described below.

**Source node**

`HermitianToSymmetricPSDBridge` supports:

- `MOI.VectorOfVariables` in `MOI.HermitianPositiveSemidefiniteConeTriangle`

**Target node**

`HermitianToSymmetricPSDBridge` creates:

- `MOI.VectorOfVariables` in `MOI.PositiveSemidefiniteConeTriangle`
- `MOI.ScalarAffineFunction{T}` in `MOI.EqualTo{T}`

**Reformulation**

The reformulation is best described by example.

The Hermitian matrix:

$$\begin{bmatrix} x_{11} & x_{12} + y_{12}im & x_{13} + y_{13}im \\ x_{12} - y_{12}im & x_{22} & x_{23} + y_{23}im \\ x_{13} - y_{13}im & x_{23} - y_{23}im & x_{33} \end{bmatrix}$$

is positive semidefinite if and only if the symmetric matrix:

$$\begin{bmatrix} x_{11} & x_{12} & x_{13} & 0 & y_{12} & y_{13} \\ & x_{22} & x_{23} & -y_{12} & 0 & y_{23} \\ & & x_{33} & -y_{13} & -y_{23} & 0 \\ & & & x_{11} & x_{12} & x_{13} \\ & & & & x_{22} & x_{23} \\ & & & & & x_{33} \end{bmatrix}$$

is positive semidefinite.

The bridge achieves this reformulation by adding a new set of variables in `MOI.PositiveSemidefiniteConeTriangle(6)`, and then adding three groups of equality constraints to:

- constrain the two x blocks to be equal
- force the diagonal of the y blocks to be `0`
- force the lower triangular of the y block to be the negative of the upper triangle.

source

`MathOptInterface.Bridges.Variable.NonposToNonnegBridge` – Type.

```
NonposToNonnegBridge{T} <: Bridges.Variable.AbstractBridge
```

`NonposToNonnegBridge` implements the following reformulation:

- $x \in \mathbb{R}_-$ into $y \in \mathbb{R}_+$ with the substitution rule $x = -y$,

where `T` is the coefficient type of `-y`.

**Source node**

`NonposToNonnegBridge` supports:

- `MOI.VectorOfVariables` in `MOI.Nonpositives`

**Target nodes**

`NonposToNonnegBridge` creates:

- One variable node: `MOI.VectorOfVariables` in `MOI.Nonnegatives`,

source

`MathOptInterface.Bridges.Variable.ParameterToEqualToBridge` – Type.

```
ParameterToEqualToBridge{T} <: Bridges.Variable.AbstractBridge
```

`ParameterToEqualToBridge` implements the following reformulation:

- $x \in Parameter(v)$ into $x == v$

**Source node**

`ParameterToEqualToBridge` supports:

- `MOI.VariableIndex` in `MOI.Parameter`

**Target nodes**

`ParameterToEqualToBridge` creates:

- One variable node: `MOI.VariableIndex` in `MOI.EqualTo{T}`

source

`MathOptInterface.Bridges.Variable.RSOCtoPSDBridge` – Type.

```
RSOCtoPSDBridge{T} <: Bridges.Variable.AbstractBridge
```

`RSOCtoPSDBridge` implements the following reformulation:

- $||x||_2^2 \leq 2tu$ where $t, u \geq 0$ into $Y \succeq 0$, with the substitution rule: $Y = \begin{bmatrix} t & x^\top \\ x & 2u\mathbf{I} \end{bmatrix}$.

Additional bounds are added to ensure the off-diagonals of the $2uI$ submatrix are `0`, and linear constraints are added to ensure the diagonal of $2uI$ takes the same values.

As a special case, if $|x|| = 0$, then `RSOCtoPSDBridge` reformulates into $(t, u) \in \mathbb{R}_+$.

**Source node**

`RSOCtoPSDBridge` supports:

- `MOI.VectorOfVariables` in `MOI.RotatedSecondOrderCone`

**Target nodes**

`RSOCtoPSDBridge` creates:

- One variable node that depends on the input dimension:
  - `MOI.VectorOfVariables` in `MOI.Nonnegatives` if dimension is 1 or 2
  - `MOI.VectorOfVariables` in `MOI.PositiveSemidefiniteConeTriangle` otherwise
- The constraint node `MOI.VariableIndex` in `MOI.EqualTo`
- The constant node `MOI.ScalarAffineFunction` in `MOI.EqualTo`

source

MathOptInterface.Bridges.Variable.RSOCtoSOCBridge – Type.

```
RSOCtoSOCBridge{T} <: Bridges.Variable.AbstractBridge
```

RSOCtoSOCBridge implements the following reformulation:

- $||x||_2^2 \leq 2tu$ into $||v||_2 \leq w$, with the substitution rules $t = \frac{w}{\sqrt{2}} + \frac{v_1}{\sqrt{2}}$, $u = \frac{w}{\sqrt{2}} - \frac{v_1}{\sqrt{2}}$, and $x = (v_2, \ldots, v_N)$.

**Source node**

RSOCtoSOCBridge supports:

- MOI.VectorOfVariables in MOI.RotatedSecondOrderCone

**Target node**

RSOCtoSOCBridge creates:

- MOI.VectorOfVariables in MOI.SecondOrderCone

source

MathOptInterface.Bridges.Variable.SOCtoRSOCBridge – Type.

```
SOCtoRSOCBridge{T} <: Bridges.Variable.AbstractBridge
```

SOCtoRSOCBridge implements the following reformulation:

- $||x||_2 \leq t$ into $2uv \geq ||w||_2^2$, with the substitution rules $t = \frac{u}{\sqrt{2}} + \frac{v}{\sqrt{2}}$, $x = (\frac{u}{\sqrt{2}} - \frac{v}{\sqrt{2}}, w)$.

**Assumptions**

- SOCtoRSOCBridge assumes that $|x| \geq 1$.

**Source node**

SOCtoRSOCBridge supports:

- MOI.VectorOfVariables in MOI.SecondOrderCone

**Target node**

SOCtoRSOCBridge creates:

- MOI.VectorOfVariables in MOI.RotatedSecondOrderCone

source

MathOptInterface.Bridges.Variable.SetMapBridge – Type.

```
abstract type SetMapBridge{T,S1,S2} <: AbstractBridge end
```

Consider two type of sets, S1 and S2, and a linear mapping A such that the image of a set of type S1 under A is a set of type S2.

A SetMapBridge{T,S1,S2} is a bridge that substitutes constrained variables in S2 into the image through A of constrained variables in S1.

The linear map A is described by:

- MOI.Bridges.map_set
- MOI.Bridges.map_function

Implementing a method for these two functions is sufficient to bridge constrained variables. However, in order for the getters and setters of attributes such as dual solutions and starting values to work as well, a method for the following functions must be implemented:

- MOI.Bridges.inverse_map_set
- MOI.Bridges.inverse_map_function
- MOI.Bridges.adjoint_map_function
- MOI.Bridges.inverse_adjoint_map_function.

See the docstrings of each function to see which feature would be missing if it was not implemented for a given bridge.

**Fieldnames**

If S1 and S2 are [MOI.AbstractScalarSet], the struct must have fields:

- variable::MOI.VariableIndex
- constraint::MOI.ConstraintIndex{MOI.VariableIndex,S1}

If S1 and S2 are [MOI.AbstractVectorSet], the struct must have fields:

- variable::Vector{MOI.VariableIndex}
- constraint::MOI.ConstraintIndex{MOI.VectorOfVariables,S1}

source

MathOptInterface.Bridges.Variable.VectorizeBridge – Type.

```
VectorizeBridge{T,S} <: Bridges.Variable.AbstractBridge
```

VectorizeBridge implements the following reformulations:

- $x \geq a$ into $[y] \in \mathbb{R}_+$ with the substitution rule $x = a + y$
- $x \leq a$ into $[y] \in \mathbb{R}_-$ with the substitution rule $x = a + y$
- $x == a$ into $[y] \in \{0\}$ with the substitution rule $x = a + y$

where T is the coefficient type of a + y.

**Source node**

VectorizeBridge supports:

- MOI.VariableIndex in MOI.GreaterThan{T}
- MOI.VariableIndex in MOI.LessThan{T}
- MOI.VariableIndex in MOI.EqualTo{T}

**Target nodes**

VectorizeBridge creates:

- One variable node: MOI.VectorOfVariables in S, where S is one of MOI.Nonnegatives, MOI.Nonpositives, MOI.Zeros depending on the type of $S$.

source

MathOptInterface.Bridges.Variable.ZerosBridge – Type.

```
ZerosBridge{T} <: Bridges.Variable.AbstractBridge
```

ZerosBridge implements the following reformulation:

- $x \in \{0\}$ into the substitution rule $x = 0$,

where T is the coefficient type of 0.

**Source node**

ZerosBridge supports:

- MOI.VectorOfVariables in MOI.Zeros

**Target nodes**

ZerosBridge does not create target nodes. It replaces all instances of x with 0 via substitution. This means that no variables are created in the underlying model.

**Caveats**

The bridged variables are similar to parameters with zero values. Parameters with non-zero values can be created with constrained variables in MOI.EqualTo by combining a VectorizeBridge and this bridge.

However, functions modified by ZerosBridge cannot be unbridged. That is, for a given function, we cannot determine if the bridged variables were used.

A related implication is that this bridge does not support MOI.ConstraintDual. However, if a MOI.Utilities.CachingOptimi is used, the dual can be determined by the bridged optimizer using MOI.Utilities.get_fallback because the caching optimizer records the unbridged function.

source

## 27.4 API Reference

### Bridges

#### AbstractBridge API

`MathOptInterface.Bridges.AbstractBridge` – Type.

```
abstract type AbstractBridge end
```

An abstract type representing a bridged constraint or variable in a `MOI.Bridges.AbstractBridgeOptimizer`.

All bridges must implement:

- `added_constrained_variable_types`
- `added_constraint_types`
- `MOI.get(::AbstractBridge, ::MOI.NumberOfVariables)`
- `MOI.get(::AbstractBridge, ::MOI.ListOfVariableIndices)`
- `MOI.get(::AbstractBridge, ::MOI.NumberOfConstraints)`
- `MOI.get(::AbstractBridge, ::MOI.ListOfConstraintIndices)`

Subtypes of `AbstractBridge` may have additional requirements. Consult their docstrings for details.

In addition, all subtypes may optionally implement the following constraint attributes with the bridge in place of the constraint index:

- `MOI.ConstraintDual`
- `MOI.ConstraintPrimal`

source

`MathOptInterface.Bridges.added_constrained_variable_types` – Function.

```
added_constrained_variable_types(
    BT::Type{<:AbstractBridge},
)::Vector{Tuple{Type}}
```

Return a list of the types of constrained variables that bridges of concrete type `BT` add.

**Implementation notes**

- This method depends only on the type of the bridge, not the runtime value. If the bridge may add a constrained variable, the type must be included in the return vector.
- If the bridge adds a free variable via `MOI.add_variable` or `MOI.add_variables`, the return vector must include `(MOI.Reals,)`.

**Example**

```
julia> MOI.Bridges.added_constrained_variable_types(
           MOI.Bridges.Variable.NonposToNonnegBridge{Float64},
       )
1-element Vector{Tuple{Type}}:
 (MathOptInterface.Nonnegatives,)
```

source

MathOptInterface.Bridges.added_constraint_types – Function.

```
added_constraint_types(
    BT::Type{<:AbstractBridge},
)::Vector{Tuple{Type,Type}}
```

Return a list of the types of constraints that bridges of concrete type BT add.

**Implementation notes**

- This method depends only on the type of the bridge, not the runtime value. If the bridge may add a constraint, the type must be included in the return vector.

**Example**

```
julia> MOI.Bridges.added_constraint_types(
           MOI.Bridges.Constraint.ZeroOneBridge{Float64},
       )
2-element Vector{Tuple{Type, Type}}:
 (MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.Interval{Float64})
 (MathOptInterface.VariableIndex, MathOptInterface.Integer)
```

source

MathOptInterface.get – Method.

```
MOI.get(b::AbstractBridge, ::MOI.NumberOfVariables)::Int64
```

Return the number of variables created by the bridge b in the model.

See also MOI.NumberOfConstraints.

**Implementation notes**

- There is a default fallback, so you need only implement this if the bridge adds new variables.

source

MathOptInterface.get – Method.

```
MOI.get(b::AbstractBridge, ::MOI.ListOfVariableIndices)
```

Return the list of variables created by the bridge b.

See also MOI.ListOfVariableIndices.

**Implementation notes**

- There is a default fallback, so you need only implement this if the bridge adds new variables.

source

MathOptInterface.get – Method.

```
MOI.get(b::AbstractBridge, ::MOI.NumberOfConstraints{F,S})::Int64 where {F,S}
```

Return the number of constraints of the type F-in-S created by the bridge b.

See also MOI.NumberOfConstraints.

**Implementation notes**

- There is a default fallback, so you need only implement this for the constraint types returned by added_constraint_types.

source

MathOptInterface.get – Method.

```
MOI.get(b::AbstractBridge, ::MOI.ListOfConstraintIndices{F,S}) where {F,S}
```

Return a Vector{ConstraintIndex{F,S}} with indices of all constraints of type F-in-S created by the bride b.

See also MOI.ListOfConstraintIndices.

**Implementation notes**

- There is a default fallback, so you need only implement this for the constraint types returned by added_constraint_types.

source

MathOptInterface.Bridges.needs_final_touch – Function.

```
needs_final_touch(bridge::AbstractBridge)::Bool
```

Return whether final_touch is implemented by bridge.

source

MathOptInterface.Bridges.final_touch – Function.

```
final_touch(bridge::AbstractBridge, model::MOI.ModelLike)::Nothing
```

A function that is called immediately prior to `MOI.optimize!` to allow bridges to modify their reformulations with respect to other variables and constraints in `model`.

For example, if the correctness of `bridge` depends on the bounds of a variable or the fact that variables are integer, then the bridge can implement `final_touch` to check assumptions immediately before a call to `MOI.optimize!`.

If you implement this method, you must also implement `needs_final_touch`.

source

`MathOptInterface.Bridges.bridging_cost` – Function.

```
bridging_cost(b::AbstractBridgeOptimizer, S::Type{<:MOI.AbstractSet})
```

Return the cost of bridging variables constrained in `S` on creation, `is_bridged(b, S)` is assumed to be `true`.

```
bridging_cost(
    b::AbstractBridgeOptimizer,
    F::Type{<:MOI.AbstractFunction},
    S::Type{<:MOI.AbstractSet},
)
```

Return the cost of bridging F-in-S constraints.

`is_bridged(b, S)` is assumed to be `true`.

source

`MathOptInterface.Bridges.runtests` – Function.

```
runtests(
    Bridge::Type{<:AbstractBridge},
    input_fn::Function,
    output_fn::Function;
    variable_start = 1.2,
    constraint_start = 1.2,
    eltype = Float64,
    cannot_unbridge::Bool = false,
)
```

Run a series of tests that check the correctness of `Bridge`.

`input_fn` and `output_fn` are functions such that `input_fn(model)` and `output_fn(model)` load the corresponding model into `model`.

Set `cannot_unbridge` to `true` if the bridge transformation is not invertible. If `Bridge` is a variable bridge this allows `Variable.unbridged_map` to returns `nothing` so that the tests allow errors that can be raised

due to this. If `Bridge` is a constraint bridge this allows the getter of `MOI.ConstraintFunction` and `MOI.ConstraintPrimalStart` to throw `MOI.GetAttributeNotAllowed`.

**Example**

```
julia> MOI.Bridges.runtests(
           MOI.Bridges.Constraint.ZeroOneBridge,
           model -> MOI.add_constrained_variable(model, MOI.ZeroOne()),
           model -> begin
               x, _ = MOI.add_constrained_variable(model, MOI.Integer())
               MOI.add_constraint(model, 1.0 * x, MOI.Interval(0.0, 1.0))
           end,
       )
Test Summary:    | Pass  Total  Time
Bridges.runtests |   32     32  0.8s
```

source

```
runtests(
    Bridge::Type{<:AbstractBridge},
    input::String,
    output::String;
    variable_start = 1.2,
    constraint_start = 1.2,
    eltype = Float64,
)
```

Run a series of tests that check the correctness of `Bridge`.

`input` and `output` are models in the style of `MOI.Utilities.loadfromstring!`.

**Example**

```
julia> MOI.Bridges.runtests(
           MOI.Bridges.Constraint.ZeroOneBridge,
           """
           variables: x
           x in ZeroOne()
           """,
           """
           variables: x
           x in Integer()
           1.0 * x in Interval(0.0, 1.0)
           """,
       )
Test Summary:    | Pass  Total  Time
Bridges.runtests |   32     32  0.0s
```

source

### Constraint bridge API

`MathOptInterface.Bridges.Constraint.AbstractBridge` – Type.

```
abstract type AbstractBridge <: MOI.Bridges.AbstractType
```

Subtype of `MOI.Bridges.AbstractBridge` for constraint bridges.

In addition to the required implementation described in `MOI.Bridges.AbstractBridge`, subtypes of `AbstractBridge` must additionally implement:

- `MOI.supports_constraint(::Type{<:AbstractBridge}, ::Type{<:MOI.AbstractFunction}, ::Type{<:MOI.Abst`
- `concrete_bridge_type`
- `bridge_constraint`

source

`MathOptInterface.Bridges.Constraint.SingleBridgeOptimizer` – Type.

```
SingleBridgeOptimizer{BT<:AbstractBridge}(model::MOI.ModelLike)
```

Return `AbstractBridgeOptimizer` that always bridges any objective function supported by the bridge `BT`.

This is in contrast with the `MOI.Bridges.LazyBridgeOptimizer`, which only bridges the objective function if it is supported by the bridge `BT` and unsupported by `model`.

**Example**

```
julia> struct MyNewBridge{T} <: MOI.Bridges.Constraint.AbstractBridge end

julia> bridge = MOI.Bridges.Constraint.SingleBridgeOptimizer{MyNewBridge{Float64}}(
           MOI.Utilities.Model{Float64}(),
       )
MOIB.Constraint.SingleBridgeOptimizer{MyNewBridge{Float64}, MOIU.Model{Float64}}
├ ObjectiveSense: FEASIBILITY_SENSE
├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
├ NumberOfVariables: 0
└ NumberOfConstraints: 0
```

**Implementation notes**

All bridges should simplify the creation of `SingleBridgeOptimizers` by defining a constant that wraps the bridge in a `SingleBridgeOptimizer`.

```
julia> const MyNewBridgeModel{T,OT<:MOI.ModelLike} =
           MOI.Bridges.Constraint.SingleBridgeOptimizer{MyNewBridge{T},OT};
```

This enables users to create bridged models as follows:

```
julia> MyNewBridgeModel{Float64}(MOI.Utilities.Model{Float64}());
```

source

`MathOptInterface.supports_constraint` – Method.

```
MOI.supports_constraint(
    BT::Type{<:AbstractBridge},
    F::Type{<:MOI.AbstractFunction},
    S::Type{<:MOI.AbstractSet},
)::Bool
```

Return a `Bool` indicating whether the bridges of type `BT` support bridging F-in-S constraints.

**Implementation notes**

- This method depends only on the type of the inputs, not the runtime values.
- There is a default fallback, so you need only implement this method for constraint types that the bridge implements.

source

`MathOptInterface.Bridges.Constraint.concrete_bridge_type` – Function.

```
concrete_bridge_type(
    BT::Type{<:AbstractBridge},
    F::Type{<:MOI.AbstractFunction},
    S::Type{<:MOI.AbstractSet}
)::Type
```

Return the concrete type of the bridge supporting F-in-S constraints.

This function can only be called if `MOI.supports_constraint(BT, F, S)` is `true`.

**Example**

The `SplitIntervalBridge` bridges a `MOI.VariableIndex`-in-`MOI.Interval` constraint into a `MOI.VariableIndex`-in-`MOI.GreaterThan` and a `MOI.VariableIndex`-in-`MOI.LessThan` constraint.

```
julia> MOI.Bridges.Constraint.concrete_bridge_type(
           MOI.Bridges.Constraint.SplitIntervalBridge{Float64},
           MOI.VariableIndex,
           MOI.Interval{Float64},
       )
MathOptInterface.Bridges.Constraint.SplitIntervalBridge{Float64, MathOptInterface.VariableIndex,
↪  MathOptInterface.Interval{Float64}, MathOptInterface.GreaterThan{Float64},
↪  MathOptInterface.LessThan{Float64}}
```

source

`MathOptInterface.Bridges.Constraint.bridge_constraint` – Function.

```
bridge_constraint(
    BT::Type{<:AbstractBridge},
    model::MOI.ModelLike,
```

```
    func::AbstractFunction,
    set::MOI.AbstractSet,
)::BT
```

Bridge the constraint `func-in-set` using bridge `BT` to `model` and returns a bridge object of type `BT`.

**Implementation notes**

- The bridge type `BT` should be a concrete type, that is, all the type parameters of the bridge must be set.

source

`MathOptInterface.Bridges.Constraint.add_all_bridges` – Function.

```
add_all_bridges(model, ::Type{T}) where {T}
```

Add all bridges defined in the `Bridges.Constraint` submodule to `model`.

The coefficient type used is `T`.

source

`MathOptInterface.Bridges.Constraint.conversion_cost` – Function.

```
conversion_cost(
    F::Type{<:MOI.AbstractFunction},
    G::Type{<:MOI.AbstractFunction},
)::Float64
```

Return a `Float64` returning the cost of converting any function of type `G` to a function of type `F` with `convert`.

This cost is used to compute `MOI.Bridges.bridging_cost`.

The default cost is `Inf`, which means that `MOI.Bridges.Constraint.FunctionConversionBridge` should not attempt the conversion.

source

**Objective bridge API**

`MathOptInterface.Bridges.Objective.AbstractBridge` – Type.

```
abstract type AbstractBridge <: MOI.Bridges.AbstractBridge end
```

Subtype of `MOI.Bridges.AbstractBridge` for objective bridges.

In addition to the required implementation described in `MOI.Bridges.AbstractBridge`, subtypes of `AbstractBridge` must additionally implement:

- supports_objective_function
- concrete_bridge_type
- bridge_objective
- MOI.Bridges.set_objective_function_type

source

MathOptInterface.Bridges.Objective.SingleBridgeOptimizer – Type.

```
SingleBridgeOptimizer{BT<:AbstractBridge}(model::MOI.ModelLike)
```

Return AbstractBridgeOptimizer that always bridges any objective function supported by the bridge BT.

This is in contrast with the MOI.Bridges.LazyBridgeOptimizer, which only bridges the objective function if it is supported by the bridge BT and unsupported by model.

**Example**

```
julia> struct MyNewBridge{T} <: MOI.Bridges.Objective.AbstractBridge end

julia> bridge = MOI.Bridges.Objective.SingleBridgeOptimizer{MyNewBridge{Float64}}(
           MOI.Utilities.Model{Float64}(),
       );
```

**Implementation notes**

All bridges should simplify the creation of SingleBridgeOptimizers by defining a constant that wraps the bridge in a SingleBridgeOptimizer.

```
julia> const MyNewBridgeModel{T,OT<:MOI.ModelLike} =
           MOI.Bridges.Objective.SingleBridgeOptimizer{MyNewBridge{T},OT};
```

This enables users to create bridged models as follows:

```
julia> MyNewBridgeModel{Float64}(MOI.Utilities.Model{Float64}());
```

source

MathOptInterface.Bridges.Objective.supports_objective_function – Function.

```
supports_objective_function(
    BT::Type{<:MOI.Bridges.Objective.AbstractBridge},
    F::Type{<:MOI.AbstractFunction},
)::Bool
```

Return a Bool indicating whether the bridges of type BT support bridging objective functions of type F.

**Implementation notes**

- This method depends only on the type of the inputs, not the runtime values.
- There is a default fallback, so you need only implement this method For objective functions that the bridge implements.

source

`MathOptInterface.Bridges.set_objective_function_type` – Function.

```
set_objective_function_type(
    BT::Type{<:Objective.AbstractBridge},
)::Type{<:MOI.AbstractFunction}
```

Return the type of objective function that bridges of concrete type BT set.

**Implementation notes**

- This method depends only on the type of the bridge, not the runtime value.

**Example**

```
julia> MOI.Bridges.set_objective_function_type(
           MOI.Bridges.Objective.FunctionizeBridge{Float64},
       )
MathOptInterface.ScalarAffineFunction{Float64}
```

source

`MathOptInterface.Bridges.Objective.concrete_bridge_type` – Function.

```
concrete_bridge_type(
    BT::Type{<:MOI.Bridges.Objective.AbstractBridge},
    F::Type{<:MOI.AbstractFunction},
)::Type
```

Return the concrete type of the bridge supporting objective functions of type `F`.

This function can only be called if `MOI.supports_objective_function(BT, F)` is `true`.

source

`MathOptInterface.Bridges.Objective.bridge_objective` – Function.

```
bridge_objective(
    BT::Type{<:MOI.Bridges.Objective.AbstractBridge},
    model::MOI.ModelLike,
    func::MOI.AbstractFunction,
)::BT
```

Bridge the objective function `func` using bridge `BT` to `model` and returns a bridge object of type `BT`.

**Implementation notes**

- The bridge type BT must be a concrete type, that is, all the type parameters of the bridge must be set.

source

`MathOptInterface.Bridges.Objective.add_all_bridges` – Function.

```
add_all_bridges(model, ::Type{T}) where {T}
```

Add all bridges defined in the `Bridges.Objective` submodule to `model`.

The coefficient type used is T.

source

### Variable bridge API

`MathOptInterface.Bridges.Variable.AbstractBridge` – Type.

```
abstract type AbstractBridge <: MOI.Bridges.AbstractBridge end
```

Subtype of `MOI.Bridges.AbstractBridge` for variable bridges.

In addition to the required implementation described in `MOI.Bridges.AbstractBridge`, subtypes of `AbstractBridge` must additionally implement:

- `supports_constrained_variable`
- `concrete_bridge_type`
- `bridge_constrained_variable`

source

`MathOptInterface.Bridges.Variable.SingleBridgeOptimizer` – Type.

```
SingleBridgeOptimizer{BT<:AbstractBridge}(model::MOI.ModelLike)
```

Return `MOI.Bridges.AbstractBridgeOptimizer` that always bridges any variables constrained on creation supported by the bridge BT.

This is in contrast with the `MOI.Bridges.LazyBridgeOptimizer`, which only bridges the variables constrained on creation if they are supported by the bridge BT and unsupported by `model`.

**Warning**

Two `SingleBridgeOptimizers` cannot be used together as both of them assume that the underlying model only returns variable indices with nonnegative values. Use `MOI.Bridges.LazyBridgeOptimizer` instead.

**Example**

```
julia> struct MyNewBridge{T} <: MOI.Bridges.Variable.AbstractBridge end

julia> bridge = MOI.Bridges.Variable.SingleBridgeOptimizer{MyNewBridge{Float64}}(
           MOI.Utilities.Model{Float64}(),
       );
```

**Implementation notes**

All bridges should simplify the creation of `SingleBridgeOptimizers` by defining a constant that wraps the bridge in a `SingleBridgeOptimizer`.

```
julia> const MyNewBridgeModel{T,OT<:MOI.ModelLike} =
           MOI.Bridges.Variable.SingleBridgeOptimizer{MyNewBridge{T},OT};
```

This enables users to create bridged models as follows:

```
julia> MyNewBridgeModel{Float64}(MOI.Utilities.Model{Float64}());
```

source

`MathOptInterface.Bridges.Variable.supports_constrained_variable` – Function.

```
supports_constrained_variable(
    BT::Type{<:AbstractBridge},
    S::Type{<:MOI.AbstractSet},
)::Bool
```

Return a `Bool` indicating whether the bridges of type `BT` support bridging constrained variables in `S`. That is, it returns `true` if the bridge of type `BT` converts constrained variables of type `S` into a form supported by the solver.

**Implementation notes**

- This method depends only on the type of the bridge and set, not the runtime values.
- There is a default fallback, so you need only implement this method for sets that the bridge implements.

**Example**

```
julia> MOI.Bridges.Variable.supports_constrained_variable(
           MOI.Bridges.Variable.NonposToNonnegBridge{Float64},
           MOI.Nonpositives,
       )
true

julia> MOI.Bridges.Variable.supports_constrained_variable(
           MOI.Bridges.Variable.NonposToNonnegBridge{Float64},
           MOI.Nonnegatives,
       )
false
```

source

MathOptInterface.Bridges.Variable.concrete_bridge_type – Function.

```
concrete_bridge_type(
    BT::Type{<:AbstractBridge},
    S::Type{<:MOI.AbstractSet},
)::Type
```

Return the concrete type of the bridge supporting variables in S constraints.

This function can only be called if `MOI.supports_constrained_variable(BT, S)` is `true`.

**Example**

As a variable in `MOI.GreaterThan` is bridged into variables in `MOI.Nonnegatives` by the `VectorizeBridge`:

```
julia> MOI.Bridges.Variable.concrete_bridge_type(
           MOI.Bridges.Variable.VectorizeBridge{Float64},
           MOI.GreaterThan{Float64},
       )
MathOptInterface.Bridges.Variable.VectorizeBridge{Float64, MathOptInterface.Nonnegatives}
```

source

MathOptInterface.Bridges.Variable.bridge_constrained_variable – Function.

```
bridge_constrained_variable(
    BT::Type{<:AbstractBridge},
    model::MOI.ModelLike,
    set::MOI.AbstractSet,
)::BT
```

Bridge the constrained variable in `set` using bridge `BT` to `model` and returns a bridge object of type `BT`.

**Implementation notes**

- The bridge type `BT` must be a concrete type, that is, all the type parameters of the bridge must be set.

source

MathOptInterface.Bridges.Variable.add_all_bridges – Function.

```
add_all_bridges(model, ::Type{T}) where {T}
```

Add all bridges defined in the `Bridges.Variable` submodule to `model`.

The coefficient type used is `T`.

source

`MathOptInterface.Bridges.Variable.unbridged_map` – Function.

```
unbridged_map(
   bridge::MOI.Bridges.Variable.AbstractBridge,
    vi::MOI.VariableIndex,
)
```

For a bridged variable in a scalar set, return a tuple of pairs mapping the variables created by the bridge to an affine expression in terms of the bridged variable `vi`.

```
unbridged_map(
    bridge::MOI.Bridges.Variable.AbstractBridge,
    vis::Vector{MOI.VariableIndex},
)
```

For a bridged variable in a vector set, return a tuple of pairs mapping the variables created by the bridge to an affine expression in terms of the bridged variable `vis`. If this method is not implemented, it falls back to calling the following method for every variable of `vis`.

```
unbridged_map(
    bridge::MOI.Bridges.Variable.AbstractBridge,
    vi::MOI.VariableIndex,
    i::MOI.Bridges.IndexInVector,
)
```

For a bridged variable in a vector set, return a tuple of pairs mapping the variables created by the bridge to an affine expression in terms of the bridged variable `vi` corresponding to the `i`th variable of the vector.

If there is no way to recover the expression in terms of the bridged variable(s) `vi(s)`, return `nothing`. See `ZerosBridge` for an example of bridge returning `nothing`.

source

#### AbstractBridgeOptimizer API

`MathOptInterface.Bridges.AbstractBridgeOptimizer` – Type.

```
abstract type AbstractBridgeOptimizer <: MOI.AbstractOptimizer end
```

An abstract type that implements generic functions for bridges.

**Implementation notes**

By convention, the inner optimizer should be stored in a `model` field. If not, the optimizer must implement `MOI.optimize!`.

source

`MathOptInterface.Bridges.bridged_variable_function` – Function.

```
bridged_variable_function(
    b::AbstractBridgeOptimizer,
    vi::MOI.VariableIndex,
)
```

Return a `MOI.AbstractScalarFunction` of variables of `b.model` that equals `vi`. That is, if the variable `vi` is bridged, it returns its expression in terms of the variables of `b.model`. Otherwise, it returns `vi`.

source

`MathOptInterface.Bridges.unbridged_variable_function` – Function.

```
unbridged_variable_function(
    b::AbstractBridgeOptimizer,
    vi::MOI.VariableIndex,
)
```

Return a `MOI.AbstractScalarFunction` of variables of `b` that equals `vi`. That is, if the variable `vi` is an internal variable of `b.model` created by a bridge but not visible to the user, it returns its expression in terms of the variables of bridged variables. Otherwise, it returns `vi`.

source

`MathOptInterface.Bridges.bridged_function` – Function.

```
bridged_function(b::AbstractBridgeOptimizer, value)::typeof(value)
```

Substitute any bridged `MOI.VariableIndex` in `value` by an equivalent expression in terms of variables of `b.model`.

source

`MathOptInterface.Bridges.supports_constraint_bridges` – Function.

```
supports_constraint_bridges(b::AbstractBridgeOptimizer)::Bool
```

Return a `Bool` indicating if `b` supports `MOI.Bridges.Constraint.AbstractBridge`.

source

`MathOptInterface.Bridges.recursive_model` – Function.

```
recursive_model(b::AbstractBridgeOptimizer)
```

If a variable, constraint, or objective is bridged, return the context of the inner variables. For most optimizers, this should be `b.model`.

source

`MathOptInterface.Bridges.FirstBridge` – Type.

```
struct FirstBridge <: MOI.AbstractConstraintAttribute end
```

Returns the first bridge used to bridge the constraint.

**Warning**

The indices of the bridge correspond to internal indices and may not correspond to indices of the model this attribute is got from.

source

#### LazyBridgeOptimizer API

`MathOptInterface.Bridges.LazyBridgeOptimizer` – Type.

```
LazyBridgeOptimizer(model::MOI.ModelLike)
```

The `LazyBridgeOptimizer` is a bridge optimizer that supports multiple bridges, and only bridges things which are not supported by the internal model.

Internally, the `LazyBridgeOptimizer` solves a shortest hyper-path problem to determine which bridges to use.

In general, you should use `full_bridge_optimizer` instead of this constructor because `full_bridge_optimizer` automatically adds a large number of supported bridges.

See also: `add_bridge`, `remove_bridge`, `has_bridge` and `full_bridge_optimizer`.

**Example**

```
julia> model = MOI.Bridges.LazyBridgeOptimizer(MOI.Utilities.Model{Float64}());

julia> MOI.Bridges.add_bridge(model, MOI.Bridges.Variable.FreeBridge{Float64})

julia> MOI.Bridges.has_bridge(model, MOI.Bridges.Variable.FreeBridge{Float64})
true
```

source

`MathOptInterface.Bridges.full_bridge_optimizer` – Function.

```
full_bridge_optimizer(model::MOI.ModelLike, ::Type{T}) where {T}
```

Returns a `LazyBridgeOptimizer` bridging `model` for every bridge defined in this package (see below for the few exceptions) and for the coefficient type `T`, as well as the bridges in the list returned by the `ListOfNonstandardBridges` attribute.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> bridged_model = MOI.Bridges.full_bridge_optimizer(model, Float64);
```

**Exceptions**

The following bridges are not added by `full_bridge_optimizer`, except if they are in the list returned by the `ListOfNonstandardBridges` attribute:

- `Constraint.SOCtoNonConvexQuadBridge`
- `Constraint.RSOCtoNonConvexQuadBridge](@ref)`
- `Constraint.SOCtoPSDBridge`
- If T is not a subtype of `AbstractFloat`, subtypes of `Constraint.AbstractToIntervalBridge`
  - `Constraint.GreaterToIntervalBridge`
  - `Constraint.LessToIntervalBridge`)

See the docstring of the each bridge for the reason they are not added.

source

`MathOptInterface.Bridges.ListOfNonstandardBridges` – Type.

```
ListOfNonstandardBridges{T}() <: MOI.AbstractOptimizerAttribute
```

Any optimizer can be wrapped in a `LazyBridgeOptimizer` using `full_bridge_optimizer`. However, by default `LazyBridgeOptimizer` uses a limited set of bridges that are:

1. implemented in `MOI.Bridges`
2. generally applicable for all optimizers.

For some optimizers however, it is useful to add additional bridges, such as those that are implemented in external packages (for example, within the solver package itself) or only apply in certain circumstances (for example, `Constraint.SOCtoNonConvexQuadBridge`).

Such optimizers should implement the `ListOfNonstandardBridges` attribute to return a vector of bridge types that are added by `full_bridge_optimizer` in addition to the list of default bridges.

Note that optimizers implementing `ListOfNonstandardBridges` may require package-specific functions or sets to be used if the non-standard bridges are not added. Therefore, you are recommended to use `model = MOI.instantiate(Package.Optimizer; with_bridge_type = T)` instead of `model = MOI.instantiate(Package.Optimiz`
See `MOI.instantiate`.

**Example**

**An optimizer using a non-default bridge in `MOI.Bridges`**

Solvers supporting `MOI.ScalarQuadraticFunction` can support `MOI.SecondOrderCone` and `MOI.RotatedSecondOrderCone` by defining:

```
function MOI.get(::MyQuadraticOptimizer, ::ListOfNonstandardBridges{Float64})
    return Type[
        MOI.Bridges.Constraint.SOCtoNonConvexQuadBridge{Float64},
        MOI.Bridges.Constraint.RSOCtoNonConvexQuadBridge{Float64},
    ]
end
```

**An optimizer defining an internal bridge**

Suppose an optimizer can exploit specific structure of a constraint, for example, it can exploit the structure of the matrix `A` in the linear system of equations `A * x = b`.

The optimizer can define the function:

```
struct MatrixAffineFunction{T} <: MOI.AbstractVectorFunction
    A::SomeStructuredMatrixType{T}
    b::Vector{T}
end
```

and then a bridge

```
struct MatrixAffineFunctionBridge{T} <: MOI.Constraint.AbstractBridge
    # ...
end
# ...
```

from `VectorAffineFunction{T}` to the `MatrixAffineFunction`. Finally, it defines:

```
function MOI.get(::Optimizer{T}, ::ListOfNonstandardBridges{T}) where {T}
    return Type[MatrixAffineFunctionBridge{T}]
end
```

source

`MathOptInterface.Bridges.add_bridge` – Function.

```
add_bridge(b::LazyBridgeOptimizer, BT::Type{<:AbstractBridge})
```

Enable the use of the bridges of type `BT` by `b`.

source

`MathOptInterface.Bridges.remove_bridge` – Function.

```
remove_bridge(b::LazyBridgeOptimizer, BT::Type{<:AbstractBridge})
```

Disable the use of the bridges of type `BT` by `b`.

source

`MathOptInterface.Bridges.has_bridge` – Function.

```
has_bridge(b::LazyBridgeOptimizer, BT::Type{<:AbstractBridge})
```

Return a `Bool` indicating whether the bridges of type `BT` are used by `b`.

source

`MathOptInterface.Bridges.print_active_bridges` – Function.

```
print_active_bridges([io::IO=stdout,] b::MOI.Bridges.LazyBridgeOptimizer)
```

Print the set of bridges that are active in the model `b`.

source

```
print_active_bridges(
    [io::IO=stdout,]
    b::MOI.Bridges.LazyBridgeOptimizer,
    F::Type{<:MOI.AbstractFunction}
)
```

Print the set of bridges required for an objective function of type `F`.

source

```
print_active_bridges(
    [io::IO=stdout,]
    b::MOI.Bridges.LazyBridgeOptimizer,
    F::Type{<:MOI.AbstractFunction},
    S::Type{<:MOI.AbstractSet},
)
```

Print the set of bridges required for a constraint of type `F`-in-`S`.

source

```
print_active_bridges(
    [io::IO=stdout,]
    b::MOI.Bridges.LazyBridgeOptimizer,
    S::Type{<:MOI.AbstractSet}
)
```

Print the set of bridges required for a variable constrained to set `S`.

source

`MathOptInterface.Bridges.print_graph` – Function.

```
print_graph([io::IO = stdout,] b::LazyBridgeOptimizer)
```

Print the hyper-graph containing all variable, constraint, and objective types that could be obtained by bridging the variables, constraints, and objectives that are present in the model by all the bridges added to b.

Each node in the hyper-graph corresponds to a variable, constraint, or objective type.

- Variable nodes are indicated by [ ]
- Constraint nodes are indicated by ( )
- Objective nodes are indicated by | |

The number inside each pair of brackets is an index of the node in the hyper-graph.

Note that this hyper-graph is the full list of possible transformations. When the bridged model is created, we select the shortest hyper-path from this graph, so many nodes may be un-used.

To see which nodes are used, call print_active_bridges.

For more information, see Legat, B., Dowson, O., Garcia, J., and Lubin, M. (2020). "MathOptInterface: a data structure for mathematical optimization problems." URL

source

MathOptInterface.Bridges.debug_supports_constraint – Function.

```
debug_supports_constraint(
    b::LazyBridgeOptimizer,
    F::Type{<:MOI.AbstractFunction},
    S::Type{<:MOI.AbstractSet};
    io::IO = Base.stdout,
)
```

Prints to io explanations for the value of MOI.supports_constraint with the same arguments.

source

MathOptInterface.Bridges.debug_supports – Function.

```
debug_supports(
    b::LazyBridgeOptimizer,
    ::MOI.ObjectiveFunction{F};
    io::IO = Base.stdout,
) where F
```

Prints to io explanations for the value of MOI.supports with the same arguments.

source

### SetMap API

`MathOptInterface.Bridges.MapNotInvertible` – Type.

```
struct MapNotInvertible <: Exception
    message::String
end
```

An error thrown by `inverse_map_function` or `inverse_adjoint_map_function` indicating that the linear map A defined in `Variable.SetMapBridge` and `Constraint.SetMapBridge` is not invertible.

source

`MathOptInterface.Bridges.map_set` – Function.

```
map_set(bridge::MOI.Bridges.AbstractBridge, set)
map_set(::Type{BT}, set) where {BT}
```

Return the image of `set` through the linear map A defined in `Variable.SetMapBridge` and `Constraint.SetMapBridge`.

This function is used for bridging the constraint and setting the `MOI.ConstraintSet`.

source

`MathOptInterface.Bridges.inverse_map_set` – Function.

```
inverse_map_set(bridge::MOI.Bridges.AbstractBridge, set)
inverse_map_set(::Type{BT}, set) where {BT}
```

Return the preimage of `set` through the linear map A defined in `Variable.SetMapBridge` and `Constraint.SetMapBridge`.

This function is used for getting the `MOI.ConstraintSet`.

The method can alternatively be defined on the bridge type. This legacy interface is kept for backward compatibility.

source

`MathOptInterface.Bridges.map_function` – Function.

```
map_function(bridge::MOI.Bridges.AbstractBridge, func)
map_function(::Type{BT}, func) where {BT}
```

Return the image of `func` through the linear map A defined in `Variable.SetMapBridge` and `Constraint.SetMapBridge`.

This function is used for getting the `MOI.ConstraintPrimal` of variable bridges. For constraint bridges, this is used for bridging the constraint, setting the `MOI.ConstraintFunction` and `MOI.ConstraintPrimalStart` and modifying the function with `MOI.modify`.

The default implementation of `Constraint.bridge_constraint` uses `map_function` with the bridge type so if this function is defined on the bridge type, `Constraint.bridge_constraint` does not need to be implemented.

source

```
map_function(::Type{BT}, func, i::IndexInVector) where {BT}
```

Return the scalar function at the `i`th index of the vector function that would be returned by `map_function(BT, func)` except that it may compute the `i`th element. This is used by `bridged_function` and for getting the `MOI.VariablePrimal` and `MOI.VariablePrimalStart` of variable bridges.

source

`MathOptInterface.Bridges.inverse_map_function` – Function.

```
inverse_map_function(bridge::MOI.Bridges.AbstractBridge, func)
inverse_map_function(::Type{BT}, func) where {BT}
```

Return the image of `func` through the inverse of the linear map `A` defined in `Variable.SetMapBridge` and `Constraint.SetMapBridge`.

This function is used by `Variable.unbridged_map` and for setting the `MOI.VariablePrimalStart` of variable bridges and for getting the `MOI.ConstraintFunction`, the `MOI.ConstraintPrimal` and the `MOI.ConstraintPrimalStart` of constraint bridges.

If the linear map `A` is not invertible, the error `MapNotInvertible` is thrown.

The method can alternatively be defined on the bridge type. This legacy interface is kept for backward compatibility.

source

`MathOptInterface.Bridges.adjoint_map_function` – Function.

```
adjoint_map_function(bridge::MOI.Bridges.AbstractBridge, func)
adjoint_map_function(::Type{BT}, func) where {BT}
```

Return the image of `func` through the adjoint of the linear map `A` defined in `Variable.SetMapBridge` and `Constraint.SetMapBridge`.

This function is used for getting the `MOI.ConstraintDual` and `MOI.ConstraintDualStart` of constraint bridges.

The method can alternatively be defined on the bridge type. This legacy interface is kept for backward compatibility.

source

`MathOptInterface.Bridges.inverse_adjoint_map_function` – Function.

```
inverse_adjoint_map_function(bridge::MOI.Bridges.AbstractBridge, func)
inverse_adjoint_map_function(::Type{BT}, func) where {BT}
```

Return the image of `func` through the inverse of the adjoint of the linear map `A` defined in `Variable.SetMapBridge` and `Constraint.SetMapBridge`.

This function is used for getting the MOI.ConstraintDual of variable bridges and setting the MOI.ConstraintDualStart of constraint bridges.

If the linear map A is not invertible, the error MapNotInvertible is thrown.

The method can alternatively be defined on the bridge type. This legacy interface is kept for backward compatibility.

source

### Bridging graph API

MathOptInterface.Bridges.Graph – Type.

```
Graph()
```

A type-stable datastructure for computing the shortest hyperpath problem.

**Nodes**

There are three types of nodes in the graph:

- VariableNode
- ConstraintNode
- ObjectiveNode

Add nodes to the graph using add_node.

**Edges**

There are two types of edges in the graph:

- Edge
- ObjectiveEdge

Add edges to the graph using add_edge.

For the ability to add a variable constrained on creation as a free variable followed by a constraint, use set_variable_constraint_node.

**Optimal hyper-edges**

Use bridge_index to compute the minimum-cost bridge leaving a node.

Note that bridge_index lazy runs a Bellman-Ford algorithm to compute the set of minimum cost edges. Thus, the first call to bridge_index after adding new nodes or edges will take longer than subsequent calls.

source

MathOptInterface.Bridges.VariableNode – Type.

```
VariableNode(index::Int)
```

A node in Graph representing a variable constrained on creation.

source

`MathOptInterface.Bridges.ConstraintNode` – Type.

```
ConstraintNode(index::Int)
```

A node in `Graph` representing a constraint.

source

`MathOptInterface.Bridges.ObjectiveNode` – Type.

```
ObjectiveNode(index::Int)
```

A node in `Graph` representing an objective function.

source

`MathOptInterface.Bridges.Edge` – Type.

```
Edge(
    bridge_index::Int,
    added_variables::Vector{VariableNode},
    added_constraints::Vector{ConstraintNode},
    cost::Float64 = 1.0,
)
```

Return a new datastructure representing an edge in `Graph` that starts at a `VariableNode` or a `ConstraintNode`.

source

`MathOptInterface.Bridges.ObjectiveEdge` – Type.

```
ObjectiveEdge(
    bridge_index::Int,
    added_variables::Vector{VariableNode},
    added_constraints::Vector{ConstraintNode},
)
```

Return a new datastructure representing an edge in `Graph` that starts at an `ObjectiveNode`.

source

`MathOptInterface.Bridges.add_node` – Function.

```
add_node(graph::Graph, ::Type{VariableNode})::VariableNode
add_node(graph::Graph, ::Type{ConstraintNode})::ConstraintNode
add_node(graph::Graph, ::Type{ObjectiveNode})::ObjectiveNode
```

Add a new node to `graph`.

source

`MathOptInterface.Bridges.add_edge` – Function.

```
add_edge(graph::Graph, node::VariableNode, edge::Edge)::Nothing
add_edge(graph::Graph, node::ConstraintNode, edge::Edge)::Nothing
add_edge(graph::Graph, node::ObjectiveNode, edge::ObjectiveEdge)::Nothing
```

Add edge to `graph`, where edge starts at `node` and connects to the nodes defined in `edge`.

source

`MathOptInterface.Bridges.set_variable_constraint_node` – Function.

```
set_variable_constraint_node(
    graph::Graph,
    variable_node::VariableNode,
    constraint_node::ConstraintNode,
    cost::Int,
)
```

As an alternative to `variable_node`, add a virtual edge to `graph` that represents adding a free variable, followed by a constraint of type `constraint_node`, with bridging cost `cost`.

**Why is this needed?**

Variables can either be added as a variable constrained on creation, or as a free variable which then has a constraint added to it.

source

`MathOptInterface.Bridges.bridge_index` – Function.

```
bridge_index(graph::Graph, node::VariableNode)::Int
bridge_index(graph::Graph, node::ConstraintNode)::Int
bridge_index(graph::Graph, node::ObjectiveNode)::Int
```

Return the optimal index of the bridge to chose from `node`.

source

`MathOptInterface.Bridges.is_variable_edge_best` – Function.

```
is_variable_edge_best(graph::Graph, node::VariableNode)::Bool
```

Return a `Bool` indicating whether `node` should be added as a variable constrained on creation, or as a free variable followed by a constraint.

source

# Chapter 28

# FileFormats

## 28.1 Overview

### The FileFormats submodule

The `FileFormats` module provides functions for reading and writing MOI models using `write_to_file` and `read_from_file`.

#### Supported file types

You must read and write files to a `FileFormats.Model` object. Specific the file-type by passing a `FileFormats.FileFormat` enum. For example:

#### The Conic Benchmark Format

```
julia> model = MOI.FileFormats.Model(format = MOI.FileFormats.FORMAT_CBF)
MOI.FileFormats.CBF.Model
├ ObjectiveSense: FEASIBILITY_SENSE
├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
├ NumberOfVariables: 0
└ NumberOfConstraints: 0
```

#### The LP file format

```
julia> model = MOI.FileFormats.Model(format = MOI.FileFormats.FORMAT_LP)
MOI.FileFormats.LP.Model
├ ObjectiveSense: FEASIBILITY_SENSE
├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
├ NumberOfVariables: 0
└ NumberOfConstraints: 0
```

#### The MathOptFormat file format

```
julia> model = MOI.FileFormats.Model(format = MOI.FileFormats.FORMAT_MOF)
MOI.FileFormats.MOF.Model
├ ObjectiveSense: FEASIBILITY_SENSE
├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
```

```
├ NumberOfVariables: 0
└ NumberOfConstraints: 0
```

#### The MPS file format

```
julia> model = MOI.FileFormats.Model(format = MOI.FileFormats.FORMAT_MPS)
MOI.FileFormats.MPS.Model
├ ObjectiveSense: FEASIBILITY_SENSE
├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
├ NumberOfVariables: 0
└ NumberOfConstraints: 0
```

#### The NL file format

```
julia> model = MOI.FileFormats.Model(format = MOI.FileFormats.FORMAT_NL)
MOI.FileFormats.NL.Model
├ ObjectiveSense: unknown
├ ObjectiveFunctionType: unknown
├ NumberOfVariables: unknown
└ NumberOfConstraints: unknown
```

#### The REW file format

```
julia> model = MOI.FileFormats.Model(format = MOI.FileFormats.FORMAT_REW)
MOI.FileFormats.MPS.Model
├ ObjectiveSense: FEASIBILITY_SENSE
├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
├ NumberOfVariables: 0
└ NumberOfConstraints: 0
```

Note that the REW format is identical to the MPS file format, except that all names are replaced with generic identifiers.

#### The SDPA file format

```
julia> model = MOI.FileFormats.Model(format = MOI.FileFormats.FORMAT_SDPA)
MOI.FileFormats.SDPA.Model
├ ObjectiveSense: FEASIBILITY_SENSE
├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
├ NumberOfVariables: 0
└ NumberOfConstraints: 0
```

#### Write to file

To write a model `src` to a MathOptFormat file, use:

```
julia> src = MOI.Utilities.Model{Float64}();

julia> MOI.add_variable(src);

julia> dest = MOI.FileFormats.Model(format = MOI.FileFormats.FORMAT_MOF);

julia> MOI.copy_to(dest, src)
MathOptInterface.Utilities.IndexMap with 1 entry:
  MOI.VariableIndex(1) => MOI.VariableIndex(1)

julia> MOI.write_to_file(dest, "file.mof.json")

julia> print(read("file.mof.json", String))
{"name":"MathOptFormat
↪  Model","version":{"major":1,"minor":9},"variables":[{"name":"x1"}],"objective":{"sense":"feasibility"},"constraint
```

#### Read from file

To read a MathOptFormat file, use:

```
julia> dest = MOI.FileFormats.Model(format = MOI.FileFormats.FORMAT_MOF);

julia> MOI.read_from_file(dest, "file.mof.json")

julia> MOI.get(dest, MOI.ListOfVariableIndices())
1-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(1)

julia> rm("file.mof.json")  # Clean up after ourselves.
```

#### Detecting the file-type automatically

Instead of the `format` keyword, you can also use the `filename` keyword argument to `FileFormats.Model`. This will attempt to automatically guess the format from the file extension. For example:

```
julia> src = MOI.Utilities.Model{Float64}();

julia> dest = MOI.FileFormats.Model(filename = "file.cbf.gz");

julia> MOI.copy_to(dest, src)
MathOptInterface.Utilities.IndexMap()

julia> MOI.write_to_file(dest, "file.cbf.gz")

julia> src_2 = MOI.FileFormats.Model(filename = "file.cbf.gz");

julia> MOI.read_from_file(src_2, "file.cbf.gz")

julia> rm("file.cbf.gz")  # Clean up after ourselves.
```

Note how the compression format (GZip) is also automatically detected from the filename.

### Unsupported constraints

In some cases `src` may contain constraints that are not supported by the file format (for example, the CBF format supports integer variables but not binary). If so, copy `src` to a bridged model using `Bridges.full_bridge_optimizer`:

```
julia> src = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(src);

julia> MOI.add_constraint(src, x, MOI.ZeroOne());

julia> dest = MOI.FileFormats.Model(format = MOI.FileFormats.FORMAT_CBF);

julia> bridged = MOI.Bridges.full_bridge_optimizer(dest, Float64);

julia> MOI.copy_to(bridged, src);

julia> MOI.write_to_file(dest, "my_model.cbf")

julia> rm("my_model.cbf")  # Clean up after ourselves.
```

**Note**

Even after bridging, it may still not be possible to write the model to file because of unsupported constraints (for example, PSD variables in the LP file format).

### Read and write to `io`

In addition to `write_to_file` and `read_from_file`, you can read and write directly from `IO` streams using `Base.write` and `Base.read!`:

```
julia> src = MOI.Utilities.Model{Float64}();

julia> dest = MOI.FileFormats.Model(format = MOI.FileFormats.FORMAT_MPS);

julia> MOI.copy_to(dest, src)
MathOptInterface.Utilities.IndexMap()

julia> io = IOBuffer();

julia> write(io, dest)

julia> seekstart(io);

julia> src_2 = MOI.FileFormats.Model(format = MOI.FileFormats.FORMAT_MPS);

julia> read!(io, src_2);
```

### ScalarNonlinearFunction

By default, reading a `.nl` or `.mof.json` that contains nonlinear expressions will create an `NLPBlock`.

To instead read nonlinear expressions as `ScalarNonlinearFunction`, pass the `use_nlp_block = false` keyword argument to the `Model` constructor:

```
julia> model = MOI.FileFormats.Model(;
           format = MOI.FileFormats.FORMAT_MOF,
           use_nlp_block = false,
       );

julia> model = MOI.FileFormats.Model(;
           format = MOI.FileFormats.FORMAT_NL,
           use_nlp_block = false,
       );
```

#### Validating MOF files

MathOptFormat files are governed by a schema. Use JSONSchema.jl to check if a `.mof.json` file satisfies the schema.

First, construct the schema object as follows:

```
julia> import JSON, JSONSchema

julia> schema = JSONSchema.Schema(JSON.parsefile(MOI.FileFormats.MOF.SCHEMA_PATH))
A JSONSchema
```

Then, check if a model file is valid using `isvalid`:

```
julia> good_model = JSON.parse("""
       {
         "version": {"major": 1, "minor": 9},
         "variables": [{"name": "x"}],
         "objective": {"sense": "feasibility"},
         "constraints": []
       }
       """);

julia> isvalid(schema, good_model)
true
```

If we construct an invalid file, for example by mis-typing `name` as `NaMe`, the validation fails:

```
julia> bad_model = JSON.parse("""
       {
         "version": {"major": 1, "minor": 9},
         "variables": [{"NaMe": "x"}],
         "objective": {"sense": "feasibility"},
         "constraints": []
       }
       """);

julia> isvalid(schema, bad_model)
false
```

Use `JSONSchema.validate` to obtain more insight into why the validation failed:

```
julia> JSONSchema.validate(schema, bad_model)
Validation failed:
path:         [variables][1]
instance:     JSON.Object{String, Any}("NaMe" => "x")
schema key:   required
schema value: Any["name"]
```

## 28.2 API Reference

### File Formats

Functions to help read and write MOI models to/from various file formats. See The FileFormats submodule for more details.

`MathOptInterface.FileFormats.Model` – Function.

```
Model(;
    format::FileFormat = FORMAT_AUTOMATIC,
    filename::Union{Nothing, String} = nothing,
    kwargs...
)
```

Return model corresponding to the `FileFormats.FileFormat` `format`, or, if `format == FORMAT_AUTOMATIC`, guess the format from `filename`.

The `filename` argument is only needed if `format == FORMAT_AUTOMATIC`.

`kwargs` are passed to the underlying model constructor.

source

`MathOptInterface.FileFormats.FileFormat` – Type.

```
FileFormat
```

List of accepted export formats.

**Values**

**FORMAT_AUTOMATIC**

Try to detect the file format based on the file name.

**FORMAT_CBF**

The Conic Benchmark format.

See `FileFormats.CBF.Model` for more details.

**FORMAT_LP**

The LP file format.

See `FileFormats.LP.Model` for more details.

**FORMAT_MOF**

The MathOptFormat file format.

See FileFormats.MOF.Model for more details.

**FORMAT_MPS**

The MPS file format.

See FileFormats.MPS.Model for more details.

**FORMAT_NL**

The AMPL .nl file format.

See FileFormats.NL.Model for more details.

**FORMAT_REW**

The .rew file format, which is equivalent to the MPS format (FileFormats.FORMAT_MPS) with the generic_names = true keyword argument set by default.

See FileFormats.MPS.Model for more details.

**FORMAT_SDPA**

The SemiDefinite Programming Algorithm format.

See FileFormats.SDPA.Model for more details.

source

MathOptInterface.FileFormats.FORMAT_AUTOMATIC – Constant.

```
FORMAT_AUTOMATIC::FileFormat
```

An instance of the FileFormat enum.

**About**

Try to detect the file format based on the file name.

source

MathOptInterface.FileFormats.FORMAT_CBF – Constant.

```
FORMAT_CBF::FileFormat
```

An instance of the FileFormat enum.

**About**

The Conic Benchmark format.

See FileFormats.CBF.Model for more details.

source

MathOptInterface.FileFormats.CBF.Model – Type.

```
Model()
```

Create an empty instance of `FileFormats.CBF.Model`.

source

`MathOptInterface.FileFormats.FORMAT_LP` – Constant.

```
FORMAT_LP::FileFormat
```

An instance of the `FileFormat` enum.

**About**

The LP file format.

See `FileFormats.LP.Model` for more details.

source

`MathOptInterface.FileFormats.LP.Model` – Type.

```
Model(;
    maximum_length::Int = 255,
    warn::Bool = false,
    coefficient_type::Type{T} = Float64,
    generic_names::Bool = false,
) where {T}
```

Create an empty instance of FileFormats.LP.Model.

Keyword arguments are:

- `maximum_length::Int=255`: the maximum length for the name of a variable. lp_solve 5.0 allows only 16 characters, while CPLEX 12.5+ allow 255.
- `warn::Bool=false`: print a warning when variables or constraints are renamed.
- `coefficient_type::Type{T} = Float64`: the supported type to use when reading and writing files.
- `generic_names::Bool=false`: strip all names in the model and replace them with the generic names `C$i` and `R$i` for the i'th column and row respectively.

source

`MathOptInterface.FileFormats.FORMAT_MOF` – Constant.

```
FORMAT_MOF::FileFormat
```

An instance of the `FileFormat` enum.

**About**

The MathOptFormat file format.

See FileFormats.MOF.Model for more details.

source

MathOptInterface.FileFormats.MOF.Model – Type.

```
Model(;
    print_compact::Bool = false,
    warn::Bool = false,
    differentiation_backend::MOI.Nonlinear.AbstractAutomaticDifferentiation =
        MOI.Nonlinear.SparseReverseMode(),
    use_nlp_block::Union{Bool,Nothing} = nothing,
    coefficient_type::Type{T} = Float64,
    generic_names::Bool = false,
) where {T}
```

Create an empty instance of FileFormats.MOF.Model.

Keyword arguments are:

- print_compact::Bool=false: print the JSON file in a compact format without spaces or newlines.
- warn::Bool=false: print a warning when variables or constraints are renamed
- differentiation_backend::MOI.Nonlinear.AbstractAutomaticDifferentiation = MOI.Nonlinear.SparseRever automatic differentiation backend to use when reading models with nonlinear constraints and objectives.

use_nlp_block::Bool=true: if true parse "ScalarNonlinearFunction" into an MOI.NLPBlock. If false, "ScalarNonlinearFunction" are parsed as MOI.ScalarNonlinearFunction functions.

- coefficient_type::Type{T} = Float64: the supported type to use when reading and writing files.
- generic_names::Bool=false: strip all names in the model and replace them with the generic names C$i and R$i for the i'th column and row respectively.

source

MathOptInterface.FileFormats.FORMAT_MPS – Constant.

```
FORMAT_MPS::FileFormat
```

An instance of the FileFormat enum.

**About**

The MPS file format.

See FileFormats.MPS.Model for more details.

source

MathOptInterface.FileFormats.MPS.Model – Type.

```
Model(;
    warn::Bool = false,
    print_objsense::Bool = false,
    generic_names::Bool = false,
    quadratic_format::QuadraticFormat = kQuadraticFormatGurobi,
    coefficient_type::Type{T} = Float64,
) where {T}
```

Create an empty instance of FileFormats.MPS.Model.

Keyword arguments are:

- `warn::Bool=false`: print a warning when variables or constraints are renamed.
- `print_objsense::Bool=false`: print the OBJSENSE section when writing
- `generic_names::Bool=false`: strip all names in the model and replace them with the generic names `C$i` and `R$i` for the i'th column and row respectively.
- `quadratic_format::QuadraticFormat = kQuadraticFormatGurobi`: specify the solver-specific extension used when writing the quadratic components of the model. Options are `kQuadraticFormatGurobi`, `kQuadraticFormatCPLEX`, and `kQuadraticFormatMosek`.
- `coefficient_type::Type{T}=Float64`: the supported type to use when reading and writing files.

source

`MathOptInterface.FileFormats.FORMAT_NL` – Constant.

```
FORMAT_NL::FileFormat
```

An instance of the `FileFormat` enum.

**About**

The AMPL .nl file format.

See `FileFormats.NL.Model` for more details.

source

`MathOptInterface.FileFormats.NL.Model` – Type.

```
Model(; use_nlp_block::Bool = true)
```

Create a new Optimizer object.

source

`MathOptInterface.FileFormats.FORMAT_REW` – Constant.

```
FORMAT_REW::FileFormat
```

An instance of the `FileFormat` enum.

**About**

The .rew file format, which is equivalent to the MPS format (`FileFormats.FORMAT_MPS`) with the `generic_names = true` keyword argument set by default.

See `FileFormats.MPS.Model` for more details.

source

`MathOptInterface.FileFormats.FORMAT_SDPA` – Constant.

```
FORMAT_SDPA::FileFormat
```

An instance of the `FileFormat` enum.

**About**

The SemiDefinite Programming Algorithm format.

See `FileFormats.SDPA.Model` for more details.

source

`MathOptInterface.FileFormats.SDPA.Model` – Type.

```
Model(; coefficient_type::Type{T} = Float64) where {T}
```

Create an empty instance of `FileFormats.SDPA.Model{T}`.

It is important to be aware that the SDPA file format is interpreted in geometric form and not standard conic form. The standard conic form and geometric conic form are two dual standard forms for semidefinite programs (SDPs). The geometric conic form of an SDP is as follows:

$$\min_{y \in \mathbb{R}^m} \quad b^T y \tag{28.1}$$

$$\text{s.t.} \quad \sum_{i=1}^{m} A_i y_i - C \in \mathbb{K} \tag{28.2}$$

where $\mathcal{K}$ is a cartesian product of nonnegative orthant and positive semidefinite matrices that align with a block diagonal structure shared with the matrices `A_i` and `C`.

In other words, the geometric conic form contains free variables and affine constraints in either the nonnegative orthant or the positive semidefinite cone. That is, in the MathOptInterface's terminology, `MOI.VectorAffineFunction`-in-`MOI.Nonnegatives` and `MOI.VectorAffineFunction`-in-`MOI.PositiveSemidefiniteConeTriangle` constraints.

The corresponding standard conic form of the dual SDP is as follows:

$$\max_{X \in \mathbb{K}} \quad \text{tr}(CX) \tag{28.3}$$

$$\text{s.t.} \quad \text{tr}(A_i X) = b_i \qquad i = 1, \ldots, m. \tag{28.4}$$

In other words, the standard conic form contains nonnegative and positive semidefinite variables with equality constraints. That is, in the MathOptInterface's terminology, `MOI.VectorOfVariables`-in-`MOI.Nonnegatives`, `MOI.VectorOfVariables`-in-`MOI.PositiveSemidefiniteConeTriangle` and `MOI.ScalarAffineFunction`-in-`MOI.EqualTo` constraints.

If a model is in standard conic form, use `Dualization.jl` to transform it into the geometric conic form before writting it. Otherwise, the nonnegative (resp. positive semidefinite) variables will be bridged into free variables with affine constraints constraining them to belong to the nonnegative orthant (resp. positive semidefinite cone) by the `MOI.Bridges.Constraint.VectorFunctionizeBridge`. Moreover, equality constraints will be bridged into pairs of affine constraints in the nonnegative orthant by the `MOI.Bridges.Constraint.SplitInte` and then the `MOI.Bridges.Constraint.VectorizeBridge`.

If a solver is in standard conic form, use `Dualization.jl` to transform the model read into standard conic form before copying it to the solver. Otherwise, the free variables will be bridged into pairs of variables in the nonnegative orthant by the `MOI.Bridges.Variable.FreeBridge` and affine constraints will be bridged into equality constraints by creating a slack variable by the `MOI.Bridges.Constraint.VectorSlackBridge`.

source

### Other helpers

`MathOptInterface.FileFormats.NL.SolFileResults` – Type.

```
SolFileResults(
    filename::String,
    model::Model;
    suffix_lower_bound_duals::Vector{String} =
        ["ipopt_zL_out", "lower_bound_duals"],
    suffix_uuper_bound_duals::Vector{String} =
        ["ipopt_zU_out", "upper_bound_duals"],
)
```

Parse the `.sol` file `filename` created by solving `model` and return a `SolFileResults` struct.

The returned struct supports the `MOI.get` API for querying result attributes such as `MOI.TerminationStatus`, `MOI.VariablePrimal`, and `MOI.ConstraintDual`.

source

```
SolFileResults(
    raw_status::String,
    termination_status::MOI.TerminationStatusCode,
)
```

Return a `SolFileResults` struct with `MOI.RawStatusString` set to `raw_status`, `MOI.TerminationStatus` set to `termination_status`, and `MOI.PrimalStatus` and `MOI.DualStatus` set to `NO_SOLUTION`.

All other attributes are un-set.

source

## 28.3 The LP file format

### The LP file format

The purpose of this page is to document the LP file format, and the various differences that occur between solvers.

#### Resources

There are a bunch of different descriptions of the LP file format on the internet.

- CPLEX
- FICO
- Gurobi
- lpsolve
- Mosek
- QSopt

#### Grammar

This section gives the grammar of an LP file as we implement it. This grammar may be different to that of particular solvers.

The syntax rules for the grammar are:

- `<name>`: the name of a symbol in the grammar
- `A :==` B: A is equivalent to B
- `A | B`: either A or B
- `i""`: case insensitive string
- `[A]`: A is optional
- `(A)*`: there are 0 or more repeats of A
- `(A)+`: there is at least one or more repeats of A

In addition to the grammar, there are the following rules:

- Comments begin with \, and run until the next \n character
- Whitespace is ignored
- Newlines are ignored, except where explicitly described

```
<lp-file> :==
    <keyword-objective>\n
    [<section-objective>\n]
    <keyword-constraints>\n
    (<constraint>)*
    [<keyword-bounds>\n (<bound-expression>)+]
    [<keyword-general>\n (<identifier>)+]
    [<keyword-binary>\n (<identifier>)+]
    [<keyword-sos>\n (<constraint-sos>)+]
    [<keyword-end>]

<keyword-objective> :==
    i"min" | i"minimum" | i"minimize" | i"minimise"
  | i"max" | i"maximum" | i"maximize" | i"maximise"

<keyword-constraints> :==
    (i"subject to" | i"st" | i"st." | i"s.t." | i"such that")[":"]

<keyword-bounds> :== i"bound" | i"bounds"

<keyword-general> :== i"gen" | i"general" | i"generals"

<keyword-binary> :== i"bin" | i"binary" | i"binaries"

<keyword-sos> :== i"sos"

<keyword-end> :== i"end"

<digit> :== "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

<char> :== a-z | A-Z | !"#$%&()/,;?@_'`|~

<identifier> :== <char> (<char> | <digit> | ".")*

<number> :==
    "+" <number>
  | "-" <number>
  | <digit>+[.(<digit>)*][("e" | "E")("+" | "-")(<digit>)+]
  | i"inf"
  | i"infinity"

<quadratic-term> :==
    "+" <quadratic-term>
  | "-" <quadratic-term>
  | [<number> ["*"]] <identifier> "^" "2"
  | [<number> ["*"]] <identifier> "*" <identifier>

<quadratic-expression> :==
    "[" <quadratic-term> (("+" | "-") <quadratic-term>)* "]" ["/" "2"]

<term> :==
    "+" <term>
  | "-" <term>
  | <identifier>
  | <number>
```

```
  | <number> <identifier>
  | <number> "*" <identifier>
  | <quadratic-expression>

<expression> :== <term> (("+" | "-") <term>)*

<name> :== [<identifier>":"]

<objective> :== <name> [<expression>]

<inequality> :== "<" | "<=" | "=<" | ">" | ">=" | "=>" | "=" | "=="

<set-suffix> := <inequality> <number>

<set-prefix> := <number> <inequality>

<constraint-indicator> :== <identifier> "=" (0 | 1) "->" <expression> <set-suffix>

<constraint-sos> :==
    "S1::" (<identifier>":"<number>)+\n
  | "S2::" (<identifier>":"<number>)+\n

<constraint> :==
    <name> <expression> <set-suffix>
  | <name> <constraint-indicator>
  | <name> <sos-constraint>

<bound-expression> :==
    <identifier> i"free"
  | <identifier> <set-suffix>
  | <set-prefix> <identifier>
  | <set-prefix> <identifier> <set-suffix>
```

#### Differences

There are many differences in how solvers parse an LP file.

#### The "integer" section

Consider the section:

```
integers
x
```

Gurobi will interpret this as `x in MOI.Integer()`. FICO Xpress will interpret this as `x in MOI.ZeroOne()`.

FICO document this behavior, but they're an outlier.

**We choose to interpret `integers` as `MOI.Integer()`.**

#### Whether variables can have the same name as a keyword

Consider the file

```
min
st
st
st >= 0
end
```

or even the horrific

```
min st st st >= 0 end
```

Gurobi will complain up front that the keyword `st` appears twice, whereas Xpress will read the file as equivalent to :

```
minimize
x
subject to
x >= 0
end
```

**We choose to allow variables to be named as keywords, and we use context to disambiguate.**

#### Whitespace

Consider the file

```
minimize
  2x
end
```

Gurobi will interpret this as a single variable with the name 2x, where as Xpress will interpret this as the expression `2 * x`.

Gurobi document this behavior, saying that they require whitespace around all tokens, but they're an outlier.

**We choose to allow juxtaposed tokens without whitespace.**

#### Identifiers

In general, an identifier may contain the letters a-z, A-Z, the digits 0-9, and the characters `!"#\$%&()/,.;?@_'` `|~`.

Additional solvers put additional restrictions:

- In (all?) solvers except Gurobi, the identifier must not start with a digit or a `.` (in Gurobi, identifiers must be separated by whitespace)
- Identifiers in Mosek and lpsolve may not start with the letter e or E
- Keywords must not be used as names in Gurobi or Mosek, but they may in Xpress
- Many solvers actually support reading any UTF-8 string as the identifier, but they will normalize to the legal letters on write

**We choose to allow any valid UTF-8 names.**

**Brackets in identifiers**

In the CPLEX specification, `[]` are not allowed in identifiers. However, they are fairly common in names because of naming conventions like `x[1,2]`. Supporting `[` in an identifier when it is not the starting character is fairly trivial since it can't be ambiguous with any other syntax. However, deciding whether `]` is part of an identifier or represents a closing bracket from a quadratic term is quite tricky.

Consider the case: `[ x * x] + y]`. Is that the valid `"x"^2 + "y]"` or the corrupt `"x" * "x]" + "y"` where we're missing a multiplying variable on `"y"`?

Gurobi supports `[]` in identifiers because the require identifiers to be separated by whitespace. Since we allow juxtaposed tokens without whitespace, it follows that we cannot easily support `]` in identifiers.

**We choose not to allow `[]` in identifiers.**

# Chapter 29

# Nonlinear

## 29.1 Overview

### Nonlinear

The `Nonlinear` submodule contains data structures and functions for working with a nonlinear optimization problem in the form of an expression graph. This page explains the API and describes the rationale behind its design.

#### Standard form

Nonlinear programs (NLPs) are a class of optimization problems in which some of the constraints or the objective function are nonlinear:

$$\min_{x \in \mathbb{R}^n} f_0(x) \tag{29.1}$$

$$\text{s.t.} l_j \le f_j(x) \le u_j \qquad j = 1 \dots m \tag{29.2}$$

There may be additional constraints, as well as things like variable bounds and integrality restrictions, but we do not consider them here because they are best dealt with by other components of MathOptInterface.

#### API overview

The core element of the `Nonlinear` submodule is `Nonlinear.Model`:

```
julia> const Nonlinear = MOI.Nonlinear;

julia> model = Nonlinear.Model()
A Nonlinear.Model with:
 0 objectives
 0 parameters
 0 expressions
 0 constraints
```

`Nonlinear.Model` is a mutable struct that stores all of the nonlinear information added to the model.

### Decision variables

Decision variables are represented by `VariableIndex`es. The user is responsible for creating these using `MOI.VariableIndex(i)`, where `i` is the column associated with the variable.

### Expressions

The input data structure is a Julia `Expr`. The input expressions can incorporate `VariableIndex`es, but these must be interpolated into the expression with `$`:

```
julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> input = :(1 + sin($x)^2)
:(1 + sin(MathOptInterface.VariableIndex(1)) ^ 2)
```

There are a number of restrictions on the input `Expr`:

- It cannot contain macros
- It cannot contain broadcasting
- It cannot contain splatting (except in limited situations)
- It cannot contain linear algebra, such as matrix-vector products
- It cannot contain generator expressions, including `sum(i for i in S)`

Given an input expression, add an expression using `Nonlinear.add_expression`:

```
julia> expr = Nonlinear.add_expression(model, input)
MathOptInterface.Nonlinear.ExpressionIndex(1)
```

The return value, `expr`, is a `Nonlinear.ExpressionIndex` that can then be interpolated into other input expressions.

Looking again at `model`, we see:

```
julia> model
A Nonlinear.Model with:
 0 objectives
 0 parameters
 1 expression
 0 constraints
```

### Parameters

In addition to constant literals like `1` or `1.23`, you can create parameters. Parameters are placeholders whose values can change before passing the expression to the solver. Create a parameter using `Nonlinear.add_parameter`, which accepts a default value:

```
julia> p = Nonlinear.add_parameter(model, 1.23)
MathOptInterface.Nonlinear.ParameterIndex(1)
```

The return value, `p`, is a `Nonlinear.ParameterIndex` that can then be interpolated into other input expressions.

Looking again at `model`, we see:

```
julia> model
A Nonlinear.Model with:
 0 objectives
 1 parameter
 1 expression
 0 constraints
```

Update a parameter as follows:

```
julia> model[p]
1.23

julia> model[p] = 4.56
4.56

julia> model[p]
4.56
```

#### Objectives

Set a nonlinear objective using `Nonlinear.set_objective`:

```
julia> Nonlinear.set_objective(model, :($p + $expr + $x))

julia> model
A Nonlinear.Model with:
 1 objective
 1 parameter
 1 expression
 0 constraints
```

Clear a nonlinear objective by passing `nothing`:

```
julia> Nonlinear.set_objective(model, nothing)

julia> model
A Nonlinear.Model with:
 0 objectives
 1 parameter
 1 expression
 0 constraints
```

But we'll re-add the objective for later:

```
julia> Nonlinear.set_objective(model, :($p + $expr + $x));
```

### Constraints

Add a constraint using `Nonlinear.add_constraint`:

```
julia> c = Nonlinear.add_constraint(model, :(1 + sqrt($x)), MOI.LessThan(2.0))
MathOptInterface.Nonlinear.ConstraintIndex(1)

julia> model
A Nonlinear.Model with:
 1 objective
 1 parameter
 1 expression
 1 constraint
```

The return value, `c`, is a `Nonlinear.ConstraintIndex` that is a unique identifier for the constraint. Interval constraints are also supported:

```
julia> c2 = Nonlinear.add_constraint(model, :(1 + sqrt($x)), MOI.Interval(-1.0, 2.0))
MathOptInterface.Nonlinear.ConstraintIndex(2)

julia> model
A Nonlinear.Model with:
 1 objective
 1 parameter
 1 expression
 2 constraints
```

Delete a constraint using `Nonlinear.delete`:

```
julia> Nonlinear.delete(model, c2)

julia> model
A Nonlinear.Model with:
 1 objective
 1 parameter
 1 expression
 1 constraint
```

### User-defined operators

By default, `Nonlinear` supports a wide range of univariate and multivariate operators. However, you can also define your own operators by registering them.

#### Univariate operators

Register a univariate user-defined operator using `Nonlinear.register_operator`:

```
julia> f(x) = 1 + sin(x)^2
f (generic function with 1 method)

julia> Nonlinear.register_operator(model, :my_f, 1, f)
```

Now, you can use `:my_f` in expressions:

```
julia> new_expr = Nonlinear.add_expression(model, :(my_f($x + 1)))
MathOptInterface.Nonlinear.ExpressionIndex(2)
```

By default, `Nonlinear` will compute first- and second-derivatives of the registered operator using ForwardDiff.jl. Override this by passing functions which compute the respective derivative:

```
julia> f′(x) = 2 * sin(x) * cos(x)
f′ (generic function with 1 method)

julia> Nonlinear.register_operator(model, :my_f2, 1, f, f′)
```

or

```
julia> f′′(x) = 2 * (cos(x)^2 - sin(x)^2)
f′′ (generic function with 1 method)

julia> Nonlinear.register_operator(model, :my_f3, 1, f, f′, f′′)
```

#### Multivariate operators

Register a multivariate user-defined operator using `Nonlinear.register_operator`:

```
julia> g(x...) = x[1]^2 + x[1] * x[2] + x[2]^2
g (generic function with 1 method)

julia> Nonlinear.register_operator(model, :my_g, 2, g)
```

Now, you can use `:my_g` in expressions:

```
julia> new_expr = Nonlinear.add_expression(model, :(my_g($x + 1, $x)))
MathOptInterface.Nonlinear.ExpressionIndex(3)
```

By default, `Nonlinear` will compute the gradient of the registered operator using ForwardDiff.jl. (Hessian information is not supported.) Override this by passing a function to compute the gradient:

```
julia> function ∇g(ret, x...)
           ret[1] = 2 * x[1] + x[2]
           ret[2] = x[1] + 2 * x[2]
           return
       end
∇g (generic function with 1 method)

julia> Nonlinear.register_operator(model, :my_g2, 2, g, ∇g)
```

### MathOptInterface

MathOptInterface communicates the nonlinear portion of an optimization problem to solvers using concrete subtypes of `AbstractNLPEvaluator`, which implement the Nonlinear programming API.

Create an `AbstractNLPEvaluator` from `Nonlinear.Model` using `Nonlinear.Evaluator`.

`Nonlinear.Evaluator` requires an `Nonlinear.AbstractAutomaticDifferentiation` backend and an ordered list of the variables that are included in the model.

There following backends are available to choose from within MOI, although other packages may add more options by sub-typing `Nonlinear.AbstractAutomaticDifferentiation`:

- `Nonlinear.ExprGraphOnly`
- `Nonlinear.SparseReverseMode`.

```
julia> evaluator = Nonlinear.Evaluator(model, Nonlinear.ExprGraphOnly(), [x])
Nonlinear.Evaluator with available features:
  * :ExprGraph
```

The functions of the Nonlinear programming API implemented by `Nonlinear.Evaluator` depends upon the chosen `Nonlinear.AbstractAutomaticDifferentiation` backend.

The `:ExprGraph` feature means we can call `objective_expr` and `constraint_expr` to retrieve the expression graph of the problem. However, we cannot call gradient terms such as `eval_objective_gradient` because `Nonlinear.ExprGraphOnly` does not have the capability to differentiate a nonlinear expression.

If, instead, we pass `Nonlinear.SparseReverseMode`, then we get access to `:Grad`, the gradient of the objective function, `:Jac`, the Jacobian matrix of the constraints, `:JacVec`, the ability to compute Jacobian-vector products, and `:ExprGraph`.

```
julia> evaluator = Nonlinear.Evaluator(
           model,
           Nonlinear.SparseReverseMode(),
           [x],
       )
Nonlinear.Evaluator with available features:
  * :Grad
  * :Jac
  * :JacVec
  * :ExprGraph
```

However, before using the evaluator, we need to call `initialize`:

```
julia> MOI.initialize(evaluator, [:Grad, :Jac, :JacVec, :ExprGraph])
```

Now we can call methods like `eval_objective`:

```
julia> x = [1.0]
1-element Vector{Float64}:
 1.0
```

```
julia> MOI.eval_objective(evaluator, x)
7.268073418273571
```

and `eval_objective_gradient`:

```
julia> grad = [0.0]
1-element Vector{Float64}:
 0.0

julia> MOI.eval_objective_gradient(evaluator, grad, x)

julia> grad
1-element Vector{Float64}:
 1.909297426825682
```

Instead of passing `Nonlinear.Evaluator` directly to solvers, solvers query the `NLPBlock` attribute, which returns an `NLPBlockData`. This object wraps an `Nonlinear.Evaluator` and includes other information such as constraint bounds and whether the evaluator has a nonlinear objective. Create and set `NLPBlockData` as follows:

```
julia> block = MOI.NLPBlockData(evaluator);

julia> model = MOI.Utilities.UniversalFallback(MOI.Utilities.Model{Float64}());

julia> MOI.set(model, MOI.NLPBlock(), block);
```

**Warning**

Only call `NLPBlockData` once you have finished modifying the problem in `model`.

Putting everything together, you can create a nonlinear optimization problem in MathOptInterface as follows:

```
import MathOptInterface as MOI

function build_model(
    model::MOI.ModelLike;
    backend::MOI.Nonlinear.AbstractAutomaticDifferentiation,
)
    x = MOI.add_variable(model)
    y = MOI.add_variable(model)
    MOI.set(model, MOI.ObjectiveSense(), MOI.MIN_SENSE)
    nl_model = MOI.Nonlinear.Model()
    MOI.Nonlinear.set_objective(nl_model, :($x^2 + $y^2))
    evaluator = MOI.Nonlinear.Evaluator(nl_model, backend, [x, y])
    MOI.set(model, MOI.NLPBlock(), MOI.NLPBlockData(evaluator))
    return
end

# Replace `model` and `backend` with your optimizer and backend of choice.
```

```
model = MOI.Utilities.UniversalFallback(MOI.Utilities.Model{Float64}())
build_model(model; backend = MOI.Nonlinear.SparseReverseMode())
```

#### Expression-graph representation

`Nonlinear.Model` stores nonlinear expressions in `Nonlinear.Expression`s. This section explains the design of the expression graph data structure in `Nonlinear.Expression`.

Given a nonlinear function like `f(x) = sin(x)^2 + x`, a conceptual aid for thinking about the graph representation of the expression is to convert it into Polish prefix notation:

```
f(x, y) = (+ (^ (sin x) 2) x)
```

This format identifies each operator (function), as well as a list of arguments. Operators can be univariate, like `sin`, or multivariate, like `+`.

A common way of representing Polish prefix notation in code is as follows:

```
julia> x = MOI.VariableIndex(1);

julia> struct ExprNode
           op::Symbol
           children::Vector{Union{ExprNode,Float64,MOI.VariableIndex}}
       end

julia> expr = ExprNode(:+, [ExprNode(:^, [ExprNode(:sin, [x]), 2.0]), x]);
```

This data structure follows our Polish prefix notation very closely, and we can easily identify the arguments to an operator. However, it has a significant draw-back: each node in the graph requires a `Vector`, which is heap-allocated and tracked by Julia's garbage collector (GC). For large models, we can expect to have millions of nodes in the expression graph, so this overhead quickly becomes prohibitive for computation.

An alternative is to record the expression as a linear tape:

```
julia> expr = Any[:+, 2, :^, 2, :sin, 1, x, 2.0, x]
9-element Vector{Any}:
  :+
 2
  :^
 2
  :sin
 1
  MOI.VariableIndex(1)
 2.0
  MOI.VariableIndex(1)
```

The `Int` after each operator `Symbol` specifies the number of arguments.

This data-structure is a single vector, which resolves our problem with the GC, but each element is the abstract type, `Any`, and so any operations on it will lead to slower dynamic dispatch. It's also hard to identify the children of each operation without reading the entire tape.

To summarize, representing expression graphs in Julia has the following challenges:

- Nodes in the expression graph should not contain a heap-allocated object
- All data-structures should be concretely typed
- It should be easy to identify the children of a node

**Sketch of the design in Nonlinear**

`Nonlinear` overcomes these problems by decomposing the data structure into a number of different concrete-typed vectors.

First, we create vectors of the supported uni- and multivariate operators.

```
julia> const UNIVARIATE_OPERATORS = [:sin];

julia> const MULTIVARIATE_OPERATORS = [:+, :^];
```

In practice, there are many more supported operations than the ones listed here.

Second, we create an enum to represent the different types of nodes present in the expression graph:

```
julia> @enum(
           NodeType,
           NODE_CALL_MULTIVARIATE,
           NODE_CALL_UNIVARIATE,
           NODE_VARIABLE,
           NODE_VALUE,
       )
```

In practice, there are node types other than the ones listed here.

Third, we create two concretely typed structs as follows:

```
julia> struct Node
           type::NodeType
           parent::Int
           index::Int
       end

julia> struct Expression
           nodes::Vector{Node}
           values::Vector{Float64}
       end
```

For each node `node` in the `.nodes` field, if `node.type` is:

- `NODE_CALL_MULTIVARIATE`, we look up `MULTIVARIATE_OPERATORS[node.index]` to retrieve the operator
- `NODE_CALL_UNIVARIATE`, we look up `UNIVARIATE_OPERATORS[node.index]` to retrieve the operator
- `NODE_VARIABLE`, we create `MOI.VariableIndex(node.index)`
- `NODE_VALUE`, we look up `values[node.index]`

The `.parent` field of each node is the integer index of the parent node in `.nodes`. For the first node, the parent is `-1` by convention.

Therefore, we can represent our function as:

```
julia> expr = Expression(
           [
               Node(NODE_CALL_MULTIVARIATE, -1, 1),
               Node(NODE_CALL_MULTIVARIATE, 1, 2),
               Node(NODE_CALL_UNIVARIATE, 2, 1),
               Node(NODE_VARIABLE, 3, 1),
               Node(NODE_VALUE, 2, 1),
               Node(NODE_VARIABLE, 1, 1),
           ],
           [2.0],
       );
```

The ordering of the nodes in the tape must satisfy two rules:

- The children of a node must appear after the parent. This means that the tape is ordered topologically, so that a reverse pass of the nodes evaluates all children nodes before their parent
- The arguments for a `CALL` node are ordered in the tape based on the order in which they appear in the function call.

#### Design goals

This is less readable than the other options, but does this data structure meet our design goals?

Instead of a heap-allocated object for each node, we only have two `Vectors` for each expression, `nodes` and `values`, as well as two constant vectors for the `OPERATORS`. In addition, all fields are concretely typed, and there are no `Union` or `Any` types.

For our third goal, it is not easy to identify the children of a node, but it is easy to identify the parent of any node. Therefore, we can use `Nonlinear.adjacency_matrix` to compute a sparse matrix that maps parents to their children.

#### The design in practice

In practice, `Node` and `Expression` are exactly `Nonlinear.Node` and `Nonlinear.Expression`. However, `Nonlinear.NodeType` has more fields to account for comparison operators such as `:>=` and `:<=`, logic operators such as `:&&` and `:||`, nonlinear parameters, and nested subexpressions.

Moreover, instead of storing the operators as global constants, they are stored in `Nonlinear.OperatorRegistry`, and it also stores a vector of logic operators and a vector of comparison operators. In addition to `Nonlinear.DEFAULT_UNIVARIATE_` and `Nonlinear.DEFAULT_MULTIVARIATE_OPERATORS`, you can register user-defined functions using `Nonlinear.register_operat`

`Nonlinear.Model` is a struct that stores the `Nonlinear.OperatorRegistry`, as well as a list of parameters and subexpressions in the model.

#### ReverseAD

`Nonlinear.ReverseAD` is a submodule for computing derivatives of a nonlinear optimization problem using sparse reverse-mode automatic differentiation (AD).

This section does not attempt to explain how sparse reverse-mode AD works, but instead explains why MOI contains its own implementation, and highlights notable differences from similar packages.

**Warning**

Don't use the API in `ReverseAD` to compute derivatives. Instead, create a `Nonlinear.Evaluator` object with `Nonlinear.SparseReverseMode` as the backend, and then query the MOI API methods.

#### Design goals

The JuliaDiff organization maintains a list of packages for doing AD in Julia. At last count, there were at least ten packages——not including `ReverseAD`——for reverse-mode AD in Julia. `ReverseAD` exists because it has a different set of design goals.

- **Goal: handle scale and sparsity.** The types of nonlinear optimization problems that MOI represents can be large scale (10^5 or more functions across 10^5 or more variables) with very sparse derivatives. The ability to compute a sparse Hessian matrix is essential. To the best of our knowledge, `ReverseAD` is the only reverse-mode AD system in Julia that handles sparsity by default.
- **Goal: limit the scope to improve robustness.** Most other AD packages accept arbitrary Julia functions as input and then trace an expression graph using operator overloading. This means they must deal (or detect and ignore) with control flow, I/O, and other vagaries of Julia. In contrast, `ReverseAD` only accepts functions in the form of `Nonlinear.Expression`, which greatly limits the range of syntax that it must deal with. By reducing the scope of what we accept as input to functions relevant for mathematical optimization, we can provide a simpler implementation with various performance optimizations.
- **Goal: provide outputs which match what solvers expect.** Other AD packages focus on differentiating individual Julia functions. In contrast, `ReverseAD` has a very specific use-case: to generate outputs needed by the MOI nonlinear API. This means it needs to efficiently compute sparse Hessians, and it needs subexpression handling to avoid recomputing subexpressions that are shared between functions.

#### History

`ReverseAD` started life as ReverseDiffSparse.jl, development of which began in early 2014(!). This was well before the other AD packages started development. Because we had a well-tested, working AD in JuMP, there was less motivation to contribute to and explore other AD packages. The lack of historical interaction also meant that other packages were not optimized for the types of problems that JuMP is built for (that is, large-scale sparse problems). When we first created MathOptInterface, we kept the AD in JuMP to simplify the transition, and post-poned the development of a first-class nonlinear interface in MathOptInterface.

Prior to the introduction of `Nonlinear`, JuMP's nonlinear implementation was a confusing mix of functions and types spread across the code base and in the private `_Derivatives` submodule. This made it hard to swap the AD system for another. The main motivation for refactoring JuMP to create the `Nonlinear` submodule in MathOptInterface was to abstract the interface between JuMP and the AD system, allowing us to swap-in and test new AD systems in the future.

## 29.2 SymbolicAD

### SymbolicAD

The `Nonlinear.SymbolicAD` submodule contains data structures and functions for working with the symbolic derivatives of a nonlinear optimization problem. This page explains the API and describes the rationale behind its design.

### Background

The code in SymbolicAD is inspired by Hassan Hijazi's work on coin-or/gravity, a high-performance algebraic modeling language in C++.

Hassan made the following observations:

- For large scale models, symbolic differentiation is slower than other automatic differentiation techniques, such as the reverse mode algorithm implemented in `MOI.Nonlinear.ReverseAD`.
- However, most large-scale nonlinear programs have a lot of structure.
- Gravity asks the user to provide structure in the form of template constraints, where the user gives the symbolic form of the constraint as well as a set of data to convert from a symbolic form to the numerical form.
- Instead of differentiating each constraint in its numerical form, we can compute one symbolic derivative of the constraint in symbolic form, and then plug in the data in to get the numerical derivative of each function.
- As a final step, if users don't provide the structure, we can still infer it –perhaps with less accuracy–by comparing the expression tree of each constraint.

The symbolic differentiation approach of Gravity works well when the problem is large with few unique constraints. For example, a model like:

```
model = Model()
@variable(model, 0 <= x[1:10_000] <= 1)
@constraint(model, [i=1:10_000], sin(x[i]) <= 1)
@objective(model, Max, sum(x))
```

is ideal, because although the Jacobian matrix has 10,000 rows, we can compute the derivative of `sin(x[i])` as `cos(x[i])`, and then fill in the Jacobian by evaluating the derivative function instead of having to differentiation 10,000 expressions.

The symbolic differentiation approach of Gravity works poorly if there are a large number of unique constraints in the model (which would require a lot of expressions to be symbolically differentiated), or if the nonlinear functions contain a large number of nonlinear terms (which would make the symbolic derivative expensive to compute).

SymbolicAD started life as MathOptSymbolicAD.jl, development of which began in early 2022. This initial version of SymbolicAD used the `Symbolics.jl` package to compute the symbolic derivatives. In 2025, we rewrote MathOptSymbolicAD.jl to remove the dependence on `Symbolics.jl`, and, since the rewrite depended only on MathOptInterface, we added it to `MOI.Nonlinear` as a new submodule.

For more details on `MathOptSymbolicAD.jl`, see Oscar's JuMP-dev 2022 talk, although note that the syntax has changed since the original recording.

### Use SymbolicAD with JuMP

To use SymbolicAD with JuMP, set the `AutomaticDifferentiationBackend` attribute to `Nonlinear.SymbolicMode`:

```
using JuMP, Ipopt
model = Model(Ipopt.Optimizer)
set_attribute(
    model,
    MOI.AutomaticDifferentiationBackend(),
    MOI.Nonlinear.SymbolicMode(),
)
@variable(model, x[1:2])
@objective(model, Min, (1 - x[1])^2 + 100 * (x[2] - x[1]^2)^2)
optimize!(model)
```

To revert back to the default sparse reverse mode algorithm, set the `AutomaticDifferentiationBackend` attribute to `Nonlinear.SparseReverseMode`.

#### simplify

Use `Nonlinear.SymbolicAD.simplify` to simplify nonlinear expressions. The simplification algorithm performs simple rewrites such as lifting nested summations:

```
julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> f = MOI.ScalarNonlinearFunction(
           :+,
           Any[MOI.ScalarNonlinearFunction(:+, Any[1.0, x]), 2.0 * x + 3.0],
       )
+(+(1.0, MOI.VariableIndex(1)), 3.0 + 2.0 MOI.VariableIndex(1))

julia> MOI.Nonlinear.SymbolicAD.simplify(f)
4.0 + 3.0 MOI.VariableIndex(1)
```

and trivial identities such as $x^1 = x$:

```
julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> f = MOI.ScalarNonlinearFunction(:^, Any[x, 1])
^(MOI.VariableIndex(1), (1))

julia> MOI.Nonlinear.SymbolicAD.simplify(f)
MOI.VariableIndex(1)
```

The list of rewrites that will be made is intentionally limited to keep the codebase simple. `Nonlinear.SymbolicAD` is not a substitute for a Computer Algebraic System (CAS). For example, we do not detect the relationship $sin(x)^2 + cos(x)^2 = 1$:

```
julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> sin_x = MOI.ScalarNonlinearFunction(:sin, Any[x])
sin(MOI.VariableIndex(1))
```

```
julia> cos_x = MOI.ScalarNonlinearFunction(:cos, Any[x])
cos(MOI.VariableIndex(1))

julia> f = MOI.ScalarNonlinearFunction(
           :+,
           Any[
               MOI.ScalarNonlinearFunction(:^, Any[sin_x, 2]),
               MOI.ScalarNonlinearFunction(:^, Any[cos_x, 2]),
           ],
       )
+(^(sin(MOI.VariableIndex(1)), (2)), ^(cos(MOI.VariableIndex(1)), (2)))

julia> MOI.Nonlinear.SymbolicAD.simplify(f)
+(^(sin(MOI.VariableIndex(1)), (2)), ^(cos(MOI.VariableIndex(1)), (2)))
```

In addition to `Nonlinear.SymbolicAD.simplify`, there is an in-place version `Nonlinear.SymbolicAD.simplify!` that may make changes to the existing function.

#### variables

Use `Nonlinear.SymbolicAD.variables` to return a sorted list of the variables that appear in the function:

```
julia> x = MOI.VariableIndex.(1:3)
3-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(1)
 MOI.VariableIndex(2)
 MOI.VariableIndex(3)

julia> f = MOI.ScalarNonlinearFunction(:atan, Any[x[3], 2.0 * x[1]])
atan(MOI.VariableIndex(3), 0.0 + 2.0 MOI.VariableIndex(1))

julia> MOI.Nonlinear.SymbolicAD.variables(f)
2-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(1)
 MOI.VariableIndex(3)
```

#### derivative

Use `Nonlinear.SymbolicAD.derivative` to compute the symbolic derivative of a function with respect to a decision variable:

```
julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> f = MOI.ScalarNonlinearFunction(:sin, Any[x])
sin(MOI.VariableIndex(1))

julia> MOI.Nonlinear.SymbolicAD.derivative(f, x)
cos(MOI.VariableIndex(1))
```

Note that the resultant expression can often be simplified. Thus, in most cases you should call `Nonlinear.SymbolicAD.simplify` on the expression before using it in other places:

```
julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> f = MOI.ScalarNonlinearFunction(:sin, Any[x + 1.0])
sin(1.0 + 1.0 MOI.VariableIndex(1))

julia> df_dx = MOI.Nonlinear.SymbolicAD.derivative(f, x)
*(cos(1.0 + 1.0 MOI.VariableIndex(1)), 1.0)

julia> MOI.Nonlinear.SymbolicAD.simplify!(df_dx)
cos(1.0 + 1.0 MOI.VariableIndex(1))
```

#### gradient_and_hessian

In some cases, you may want to compute the gradient and (sparse) Hessian matrix of a function. One way to achieve this is by recursively calling `Nonlinear.SymbolicAD.derivative` on the result of `Nonlinear.SymbolicAD.derivative` for each variable in the list of `Nonlinear.SymbolicAD.variables`. But, to simplify the process, you should use `Nonlinear.SymbolicAD.gradient_and_hessian`:

```
julia> x = MOI.VariableIndex.(1:2);

julia> f = MOI.ScalarNonlinearFunction(:sin, Any[1.0 * x[1] + 2.0 * x[2]])
sin(0.0 + 1.0 MOI.VariableIndex(1) + 2.0 MOI.VariableIndex(2))

julia> y, ∇f, H, ∇²f = MOI.Nonlinear.SymbolicAD.gradient_and_hessian(f);

julia> y
2-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(1)
 MOI.VariableIndex(2)

julia> ∇f
2-element Vector{Any}:
 cos(0.0 + 1.0 MOI.VariableIndex(1) + 2.0 MOI.VariableIndex(2))
 *(cos(0.0 + 1.0 MOI.VariableIndex(1) + 2.0 MOI.VariableIndex(2)), 2.0)

julia> H
3-element Vector{Tuple{Int64, Int64}}:
 (1, 1)
 (1, 2)
 (2, 2)

julia> ∇²f
3-element Vector{Any}:
 -(sin(0.0 + 1.0 MOI.VariableIndex(1) + 2.0 MOI.VariableIndex(2)))
 *(-(sin(0.0 + 1.0 MOI.VariableIndex(1) + 2.0 MOI.VariableIndex(2))), 2.0)
 *(-(sin(0.0 + 1.0 MOI.VariableIndex(1) + 2.0 MOI.VariableIndex(2))), 4.0)
```

where:

- `y` is the list of variables that appear in `f`
- `∇f` is the first partial derivatives of `f` with respect to each variable in `y`

- `H` and $\nabla^2 f$ form a sparse Hessian matrix, were `H` is the `(row, column)` index of each element, and the corresponding element in $\nabla^2 f$ is the second partial derivative of `f` with respect to `y[row]` and `y[column]`.

Unlike `Nonlinear.SymbolicAD.derivative`, the gradient and Hessian expressions have already been simplified; you do not need to call `Nonlinear.SymbolicAD.simplify`.

## 29.3 API Reference

### Nonlinear Modeling

More information can be found in the Nonlinear section of the manual.

`MathOptInterface.Nonlinear.Model` – Type.

```
Model()
```

The core datastructure for representing a nonlinear optimization problem.

It has the following fields:

- `objective::Union{Nothing,Expression}` : holds the nonlinear objective function, if one exists, otherwise `nothing`.
- `expressions::Vector{Expression}` : a vector of expressions in the model.
- `constraints::OrderedDict{ConstraintIndex,Constraint}` : a map from `ConstraintIndex` to the corresponding `Constraint`. An `OrderedDict` is used instead of a `Vector` to support constraint deletion.
- `parameters::Vector{Float64}` : holds the current values of the parameters.
- `operators::OperatorRegistry` : stores the operators used in the model.

source

### Expressions

`MathOptInterface.Nonlinear.ExpressionIndex` – Type.

```
ExpressionIndex
```

An index to a nonlinear expression that is returned by `add_expression`.

Given `data::Model` and `ex::ExpressionIndex`, use `data[ex]` to retrieve the corresponding `Expression`.

source

`MathOptInterface.Nonlinear.add_expression` – Function.

```
add_expression(model::Model, expr)::ExpressionIndex
```

Parse `expr` into a `Expression` and add to `model`. Returns an `ExpressionIndex` that can be interpolated into other input expressions.

`expr` must be a type that is supported by `parse_expression`.

**Example**

```
julia> model = MOI.Nonlinear.Model();

julia> x = MOI.VariableIndex(1);

julia> ex = MOI.Nonlinear.add_expression(model, :($x^2 + 1))
MathOptInterface.Nonlinear.ExpressionIndex(1)

julia> MOI.Nonlinear.set_objective(model, :(sqrt($ex)))
```

source

### Parameters

`MathOptInterface.Nonlinear.ParameterIndex` – Type.

```
ParameterIndex
```

An index to a nonlinear parameter that is returned by `add_parameter`. Given `data::Model` and `p::ParameterIndex`, use `data[p]` to retrieve the current value of the parameter and `data[p] = value` to set a new value.

source

`MathOptInterface.Nonlinear.add_parameter` – Function.

```
add_parameter(model::Model, value::Float64)::ParameterIndex
```

Add a new parameter to `model` with the default value `value`. Returns a `ParameterIndex` that can be interpolated into other input expressions and used to modify the value of the parameter.

**Example**

```
julia> model = MOI.Nonlinear.Model()
A Nonlinear.Model with:
 0 objectives
 0 parameters
 0 expressions
 0 constraints

julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> p = MOI.Nonlinear.add_parameter(model, 1.2)
MathOptInterface.Nonlinear.ParameterIndex(1)

julia> c = MOI.Nonlinear.add_constraint(model, :($x^2 - $p), MOI.LessThan(0.0))
MathOptInterface.Nonlinear.ConstraintIndex(1)
```

source

### Objectives

MathOptInterface.Nonlinear.set_objective – Function.

```
set_objective(model::Model, obj)::Nothing
```

Parse obj into a Expression and set as the objective function of model.

obj must be a type that is supported by parse_expression.

To remove the objective, pass nothing.

**Example**

```
julia> model = MOI.Nonlinear.Model()
A Nonlinear.Model with:
 0 objectives
 0 parameters
 0 expressions
 0 constraints

julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> MOI.Nonlinear.set_objective(model, :($x^2 + 1))

julia> MOI.Nonlinear.set_objective(model, x)

julia> MOI.Nonlinear.set_objective(model, nothing)
```

source

### Constraints

MathOptInterface.Nonlinear.ConstraintIndex – Type.

```
ConstraintIndex
```

An index to a nonlinear constraint that is returned by add_constraint.

Given data::Model and c::ConstraintIndex, use data[c] to retrieve the corresponding Constraint.

source

MathOptInterface.Nonlinear.add_constraint – Function.

```
add_constraint(
    model::Model,
    func,
    set::Union{
```

```
        MOI.GreaterThan{Float64},
        MOI.LessThan{Float64},
        MOI.Interval{Float64},
        MOI.EqualTo{Float64},
    },
)
```

Parse `func` and `set` into a `Constraint` and add to `model`. Returns a `ConstraintIndex` that can be used to delete the constraint or query solution information.

**Example**

```
julia> model = MOI.Nonlinear.Model();

julia> x = MOI.VariableIndex(1);

julia> c = MOI.Nonlinear.add_constraint(model, :($x^2), MOI.LessThan(1.0))
MathOptInterface.Nonlinear.ConstraintIndex(1)
```

source

`MathOptInterface.Nonlinear.delete` – Function.

```
delete(model::Model, c::ConstraintIndex)::Nothing
```

Delete the constraint index `c` from `model`.

**Example**

```
julia> model = MOI.Nonlinear.Model()
A Nonlinear.Model with:
 0 objectives
 0 parameters
 0 expressions
 0 constraints

julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> c = MOI.Nonlinear.add_constraint(model, :($x^2), MOI.LessThan(1.0))
MathOptInterface.Nonlinear.ConstraintIndex(1)

julia> model
A Nonlinear.Model with:
 0 objectives
 0 parameters
 0 expressions
 1 constraint

julia> MOI.Nonlinear.delete(model, c)

julia> model
```

```
A Nonlinear.Model with:
 0 objectives
 0 parameters
 0 expressions
 0 constraints
```

source

**User-defined operators**

`MathOptInterface.Nonlinear.OperatorRegistry` – Type.

```
OperatorRegistry()
```

Create a new `OperatorRegistry` to store and evaluate univariate and multivariate operators.

source

`MathOptInterface.Nonlinear.DEFAULT_UNIVARIATE_OPERATORS` – Constant.

```
DEFAULT_UNIVARIATE_OPERATORS
```

The list of univariate operators that are supported by default.

**Example**

```
julia> MOI.Nonlinear.DEFAULT_UNIVARIATE_OPERATORS
73-element Vector{Symbol}:
 :+
 :-
 :abs
 :sign
 :sqrt
 :cbrt
 :abs2
 :inv
 :log
 :log10

 :airybi
 :airyaiprime
 :airybiprime
 :besselj0
 :besselj1
 :bessely0
 :bessely1
 :erfcx
 :dawson
```

source

`MathOptInterface.Nonlinear.DEFAULT_MULTIVARIATE_OPERATORS` – Constant.

```
DEFAULT_MULTIVARIATE_OPERATORS
```

The list of multivariate operators that are supported by default.

**Example**

```
julia> MOI.Nonlinear.DEFAULT_MULTIVARIATE_OPERATORS
9-element Vector{Symbol}:
 :+
 :-
 :*
 :^
 :/
 :ifelse
 :atan
 :min
 :max
```

source

`MathOptInterface.Nonlinear.register_operator` – Function.

```
register_operator(
    model::Model,
    op::Symbol,
    nargs::Int,
    f::Function,
    [∇f::Function],
    [∇²f::Function],
)
```

Register the user-defined operator `op` with `nargs` input arguments in `model`.

**Univariate functions**

- `f(x::T)::T` must be a function that takes a single input argument `x` and returns the function evaluated at `x`. If `∇f` and `∇²f` are not provided, `f` must support any `Real` input type `T`.
- `∇f(x::T)::T` is a function that takes a single input argument `x` and returns the first derivative of `f` with respect to `x`. If `∇²f` is not provided, `∇f` must support any `Real` input type `T`.
- `∇²f(x::T)::T` is a function that takes a single input argument `x` and returns the second derivative of `f` with respect to `x`.

**Multivariate functions**

- `f(x::T...)::T` must be a function that takes a `nargs` input arguments `x` and returns the function evaluated at `x`. If `∇f` and `∇²f` are not provided, `f` must support any `Real` input type `T`.
- `∇f(g::AbstractVector{T}, x::T...)::T` is a function that takes a cache vector `g` of length `length(x)`, and fills each element `g[i]` with the partial derivative of `f` with respect to `x[i]`.

- `∇²f(H::AbstractMatrix, x::T...)::T` is a function that takes a matrix `H` and fills the lower-triangular components `H[i, j]` with the Hessian of `f` with respect to `x[i]` and `x[j]` for `i >= j`.

**Notes for multivariate Hessians**

- `H` has `size(H) == (length(x), length(x))`, but you must not access elements `H[i, j]` for `i > j`.
- `H` is dense, but you do not need to fill structural zeros.

source

`MathOptInterface.Nonlinear.register_operator_if_needed` – Function.

```
register_operator_if_needed(
    registry::OperatorRegistry,
    op::Symbol,
    nargs::Int,
    f::Function;
)
```

Similar to `register_operator`, but this function warns if the function is not registered, and skips silently if it already is.

source

`MathOptInterface.Nonlinear.assert_registered` – Function.

```
assert_registered(registry::OperatorRegistry, op::Symbol, nargs::Int)
```

Throw an error if `op` is not registered in `registry` with `nargs` arguments.

source

`MathOptInterface.Nonlinear.check_return_type` – Function.

```
check_return_type(::Type{T}, ret::S) where {T,S}
```

Overload this method for new types `S` to throw an informative error if a user-defined function returns the type `S` instead of `T`.

source

`MathOptInterface.Nonlinear.eval_univariate_function` – Function.

```
eval_univariate_function(
    registry::OperatorRegistry,
    op::Union{Symbol,Integer},
    x::T,
) where {T}
```

Evaluate the operator `op(x)::T`, where `op` is a univariate function in `registry`.

If `op isa Integer`, then `op` is the index in `registry.univariate_operators[op]`.

**Example**

```
julia> r = MOI.Nonlinear.OperatorRegistry();

julia> MOI.Nonlinear.eval_univariate_function(r, :abs, -1.2)
1.2

julia> r.univariate_operators[3]
:abs

julia> MOI.Nonlinear.eval_univariate_function(r, 3, -1.2)
1.2
```

source

`MathOptInterface.Nonlinear.eval_univariate_gradient` – Function.

```
eval_univariate_gradient(
    registry::OperatorRegistry,
    op::Union{Symbol,Integer},
    x::T,
) where {T}
```

Evaluate the first-derivative of the operator `op(x)::T`, where `op` is a univariate function in `registry`.

If `op isa Integer`, then `op` is the index in `registry.univariate_operators[op]`.

**Example**

```
julia> r = MOI.Nonlinear.OperatorRegistry();

julia> MOI.Nonlinear.eval_univariate_gradient(r, :abs, -1.2)
-1.0

julia> r.univariate_operators[3]
:abs

julia> MOI.Nonlinear.eval_univariate_gradient(r, 3, -1.2)
-1.0
```

source

`MathOptInterface.Nonlinear.eval_univariate_function_and_gradient` – Function.

```
eval_univariate_function_and_gradient(
    registry::OperatorRegistry,
    op::Union{Symbol,Integer},
    x::T,
)::Tuple{T,T} where {T}
```

Evaluate the function and first-derivative of the operator `op(x)::T`, where `op` is a univariate function in `registry`.

If `op isa Integer`, then `op` is the index in `registry.univariate_operators[op]`.

**Example**

```
julia> r = MOI.Nonlinear.OperatorRegistry();

julia> MOI.Nonlinear.eval_univariate_function_and_gradient(r, :abs, -1.2)
(1.2, -1.0)

julia> r.univariate_operators[3]
:abs

julia> MOI.Nonlinear.eval_univariate_function_and_gradient(r, 3, -1.2)
(1.2, -1.0)
```

source

`MathOptInterface.Nonlinear.eval_univariate_hessian` – Function.

```
eval_univariate_hessian(
    registry::OperatorRegistry,
    op::Union{Symbol,Integer},
    x::T,
) where {T}
```

Evaluate the second-derivative of the operator `op(x)::T`, where `op` is a univariate function in `registry`.

If `op isa Integer`, then `op` is the index in `registry.univariate_operators[op]`.

**Example**

```
julia> r = MOI.Nonlinear.OperatorRegistry();

julia> MOI.Nonlinear.eval_univariate_hessian(r, :sin, 1.0)
-0.8414709848078965

julia> r.univariate_operators[16]
:sin

julia> MOI.Nonlinear.eval_univariate_hessian(r, 16, 1.0)
-0.8414709848078965

julia> -sin(1.0)
-0.8414709848078965
```

source

`MathOptInterface.Nonlinear.eval_multivariate_function` – Function.

```
eval_multivariate_function(
    registry::OperatorRegistry,
    op::Symbol,
    x::AbstractVector{T},
) where {T}
```

Evaluate the operator `op(x)::T`, where `op` is a multivariate function in `registry`.

source

`MathOptInterface.Nonlinear.eval_multivariate_gradient` – Function.

```
eval_multivariate_gradient(
    registry::OperatorRegistry,
    op::Symbol,
    g::AbstractVector{T},
    x::AbstractVector{T},
) where {T}
```

Evaluate the gradient of operator `g .= ∇op(x)`, where `op` is a multivariate function in `registry`.

source

`MathOptInterface.Nonlinear.eval_multivariate_hessian` – Function.

```
eval_multivariate_hessian(
    registry::OperatorRegistry,
    op::Symbol,
    H::AbstractMatrix,
    x::AbstractVector{T},
)::Bool where {T}
```

Evaluate the Hessian of operator `∇²op(x)`, where `op` is a multivariate function in `registry`.

The Hessian is stored in the lower-triangular part of the matrix `H`.

Returns a `Bool` indicating whether non-zeros were stored in the matrix.

**Note**

Implementations of the Hessian operators will not fill structural zeros. Therefore, before calling this function you should pre-populate the matrix `H` with `0`.

source

`MathOptInterface.Nonlinear.eval_logic_function` – Function.

```
eval_logic_function(
    registry::OperatorRegistry,
    op::Symbol,
```

```
    lhs::T,
    rhs::T,
)::Bool where {T}
```

Evaluate `(lhs op rhs)::Bool`, where `op` is a logic operator in `registry`.

source

`MathOptInterface.Nonlinear.eval_comparison_function` – Function.

```
eval_comparison_function(
    registry::OperatorRegistry,
    op::Symbol,
    lhs::T,
    rhs::T,
)::Bool where {T}
```

Evaluate `(lhs op rhs)::Bool`, where `op` is a comparison operator in `registry`.

source

#### Automatic-differentiation backends

`MathOptInterface.Nonlinear.Evaluator` – Type.

```
Evaluator(
    model::Model,
    backend::AbstractAutomaticDifferentiation,
    ordered_variables::Vector{MOI.VariableIndex},
)
```

Create `Evaluator`, a subtype of `MOI.AbstractNLPEvaluator`, from `Model`.

source

`MathOptInterface.Nonlinear.AbstractAutomaticDifferentiation` – Type.

```
AbstractAutomaticDifferentiation
```

An abstract type for extending `Evaluator`.

source

`MathOptInterface.Nonlinear.ExprGraphOnly` – Type.

```
ExprGraphOnly() <: AbstractAutomaticDifferentiation
```

The default implementation of `AbstractAutomaticDifferentiation`. The only supported feature is `:ExprGraph`.

source

`MathOptInterface.Nonlinear.SparseReverseMode` – Type.

```
SparseReverseMode() <: AbstractAutomaticDifferentiation
```

An implementation of `AbstractAutomaticDifferentiation` that uses sparse reverse-mode automatic differentiation to compute derivatives. Supports all features in the MOI nonlinear interface.

source

`MathOptInterface.Nonlinear.SymbolicMode` – Type.

```
SymbolicMode() <: AbstractAutomaticDifferentiation
```

A type for setting as the value of the `MOI.AutomaticDifferentiationBackend()` attribute to enable symbolic automatic differentiation.

source

**Data-structure**

`MathOptInterface.Nonlinear.Node` – Type.

```
struct Node
    type::NodeType
    index::Int
    parent::Int
end
```

A single node in a nonlinear expression tree. Used by `Expression`.

See the MathOptInterface documentation for information on how the nodes and values form an expression tree.

source

`MathOptInterface.Nonlinear.NodeType` – Type.

```
NodeType
```

An enum describing the possible node types. Each `Node` has a `.index` field, which should be interpreted as follows:

- `NODE_CALL_MULTIVARIATE`: the index into `operators.multivariate_operators`
- `NODE_CALL_UNIVARIATE`: the index into `operators.univariate_operators`
- `NODE_LOGIC`: the index into `operators.logic_operators`
- `NODE_COMPARISON`: the index into `operators.comparison_operators`
- `NODE_MOI_VARIABLE`: the value of `MOI.VariableIndex(index)` in the user's space of the model.

- `NODE_VARIABLE`: the 1-based index of the internal vector
- `NODE_VALUE`: the index into the `.values` field of `Expression`
- `NODE_PARAMETER`: the index into `data.parameters`
- `NODE_SUBEXPRESSION`: the index into `data.expressions`

source

`MathOptInterface.Nonlinear.Expression` – Type.

```
struct Expression
    nodes::Vector{Node}
    values::Vector{Float64}
end
```

The core type that represents a nonlinear expression. See the MathOptInterface documentation for information on how the nodes and values form an expression tree.

source

`MathOptInterface.Nonlinear.Constraint` – Type.

```
struct Constraint
    expression::Expression
    set::Union{
        MOI.LessThan{Float64},
        MOI.GreaterThan{Float64},
        MOI.EqualTo{Float64},
        MOI.Interval{Float64},
    }
end
```

A type to hold information relating to the nonlinear constraint `f(x) in S`, where `f(x)` is defined by `.expression`, and `S` is `.set`.

source

`MathOptInterface.Nonlinear.adjacency_matrix` – Function.

```
adjacency_matrix(nodes::Vector{Node})
```

Compute the sparse adjacency matrix describing the parent-child relationships in `nodes`.

The element `(i, j)` is `true` if there is an edge from `node[j]` to `node[i]`. Since we get a column-oriented matrix, this gives us a fast way to look up the edges leaving any node (that is, the children).

source

`MathOptInterface.Nonlinear.parse_expression` – Function.

```
parse_expression(data::Model, input)::Expression
```

Parse `input` into a `Expression`.

source

```
parse_expression(
    data::Model,
    expr::Expression,
    input::Any,
    parent_index::Int,
)::Expression
```

Parse `input` into a `Expression`, and add it to `expr` as a child of `expr.nodes[parent_index]`. Existing subexpressions and parameters are stored in `data`.

You can extend parsing support to new types of objects by overloading this method with a different type on `input::Any`.

source

`MathOptInterface.Nonlinear.convert_to_expr` – Function.

```
convert_to_expr(data::Model, expr::Expression)
```

Convert the `Expression` `expr` into a Julia `Expr`.

- subexpressions are represented by a `ExpressionIndex` object.
- parameters are represented by a `ParameterIndex` object.
- variables are represented by an `MOI.VariableIndex` object.

source

```
convert_to_expr(
    evaluator::Evaluator,
    expr::Expression;
    moi_output_format::Bool,
)
```

Convert the `Expression` `expr` into a Julia `Expr`.

If `moi_output_format = true`:

- subexpressions will be converted to Julia `Expr` and substituted into the output expression.
- the current value of each parameter will be interpolated into the expression
- variables will be represented in the form `x[MOI.VariableIndex(i)]`

If `moi_output_format = false`:

- subexpressions will be represented by a `ExpressionIndex` object.
- parameters will be represented by a `ParameterIndex` object.
- variables will be represented by an `MOI.VariableIndex` object.

**Warning**

To use `moi_output_format = true`, you must have first called `MOI.initialize` with `:ExprGraph` as a requested feature.

source

`MathOptInterface.Nonlinear.ordinal_index` – Function.

```
ordinal_index(evaluator::Evaluator, c::ConstraintIndex)::Int
```

Return the 1-indexed value of the constraint index `c` in `evaluator`.

**Example**

```
julia> model = MOI.Nonlinear.Model()
A Nonlinear.Model with:
 0 objectives
 0 parameters
 0 expressions
 0 constraints

julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> c1 = MOI.Nonlinear.add_constraint(model, :($x^2), MOI.LessThan(1.0))
MathOptInterface.Nonlinear.ConstraintIndex(1)

julia> c2 = MOI.Nonlinear.add_constraint(model, :($x^2), MOI.LessThan(1.0))
MathOptInterface.Nonlinear.ConstraintIndex(2)

julia> evaluator = MOI.Nonlinear.Evaluator(model)
Nonlinear.Evaluator with available features:
  * :ExprGraph

julia> MOI.initialize(evaluator, Symbol[])

julia> MOI.Nonlinear.ordinal_index(evaluator, c2)  # Returns 2
2

julia> MOI.Nonlinear.delete(model, c1)

julia> evaluator = MOI.Nonlinear.Evaluator(model)
Nonlinear.Evaluator with available features:
  * :ExprGraph

julia> MOI.initialize(evaluator, Symbol[])
```

```
julia> MOI.Nonlinear.ordinal_index(evaluator, c2)  # Returns 1
1
```

source

**SymbolicAD**

`MathOptInterface.Nonlinear.SymbolicAD.simplify` – Function.

```
simplify(f)
```

Return a simplified copy of the function `f`.

**Warning**

This function is not type stable by design.

**Example**

```
julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> f = MOI.ScalarNonlinearFunction(:^, Any[x, 1])
^(MOI.VariableIndex(1), (1))

julia> MOI.Nonlinear.SymbolicAD.simplify(f)
MOI.VariableIndex(1)
```

source

`MathOptInterface.Nonlinear.SymbolicAD.simplify!` – Function.

```
simplify!(f)
```

Simplify the function `f` in-place and return either the function `f` or a new object if `f` can be represented in a simpler type.

**Warning**

This function is not type stable by design.

**Example**

```
julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> f = MOI.ScalarNonlinearFunction(
```

```
            :+,
            Any[MOI.ScalarNonlinearFunction(:+, Any[1.0, x]), 2.0 * x + 3.0],
        )
+(+(1.0, MOI.VariableIndex(1)), 3.0 + 2.0 MOI.VariableIndex(1))

julia> MOI.Nonlinear.SymbolicAD.simplify!(f)
4.0 + 3.0 MOI.VariableIndex(1)

julia> f
+(1.0, MOI.VariableIndex(1), 3.0 + 2.0 MOI.VariableIndex(1))
```

source

```
simplify!(::Val{head}, f::MOI.ScalarNonlinearFunction)
```

Simplify the function `f` in-place and return either the function `f` or a new object if `f` can be represented in a simpler type.

**Val**

The head in `Val{head}` is taken from `f.head`. This function should be called as:

```
f = simplify!(Val(f.head), f)
```

Implementing a method that dispatches on head enables custom simplification rules for different operators without needing a giant switch statement.

**Note**

It is important that this function does not recursively call `simplify!`. Deal only with the immediate operator. The children arguments will already be simplified.

source

`MathOptInterface.Nonlinear.SymbolicAD.variables` – Function.

```
variables(f::Union{Real,MOI.AbstractScalarFunction})
```

Return a sorted list of the `MOI.VariableIndex` present in the function `f`.

**Example**

```
julia> x = MOI.VariableIndex.(1:3)
3-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(1)
 MOI.VariableIndex(2)
 MOI.VariableIndex(3)

julia> f = MOI.ScalarNonlinearFunction(:atan, Any[x[3], 2.0 * x[1]])
atan(MOI.VariableIndex(3), 0.0 + 2.0 MOI.VariableIndex(1))

julia> MOI.Nonlinear.SymbolicAD.variables(f)
```

```
2-element Vector{MathOptInterface.VariableIndex}:
 MOI.VariableIndex(1)
 MOI.VariableIndex(3)
```

source

`MathOptInterface.Nonlinear.SymbolicAD.derivative` – Function.

```
derivative(f::Union{Real,MOI.AbstractScalarFunction}, x::MOI.VariableIndex)
```

Return an expression representing the partial derivative of `f` with respect to `x`.

**Expression swelling**

With few exceptions, the algorithm used to compute the derivative does not perform simplifications. As a result, the returned expression may contain terms like `*(false, g)` that can be trivially simplified to `false`.

In most cases, you should call `simplify!(derivative(f, x))` to return a simplified expression of the derivative.

**Example**

```
julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> f = MOI.ScalarNonlinearFunction(:sin, Any[x])
sin(MOI.VariableIndex(1))

julia> df_dx = MOI.Nonlinear.SymbolicAD.derivative(f, x)
cos(MOI.VariableIndex(1))
```

source

`MathOptInterface.Nonlinear.SymbolicAD.gradient_and_hessian` – Function.

```
gradient_and_hessian(
    [filter_fn::Function = x -> true,]
    f::MOI.AbstractScalarFunction,
)
```

Compute the symbolic gradient and Hessian of `f`, and return the result as a tuple of four elements:

1. `x::Vector{MOI.VariableIndex}`: the list of variables that appear in `f`
2. `∇f::Vector{Any}`: a vector for the first partial derivative of `f` with respect to each element in `x`
3. `H::Vector{Tuple{Int,Int}}`: a vector of `(row, column)` tuples that list the non-zero entries in the Hessian of `f`
4. `∇²f::Vector{Any}`: a vector of expressions, in the same order as `H`, for the non-zero entries in the Hessian of `f`

**`filter_fn`**

This argument is a function, `filter_fn(::MOI.VariableIndex)::Bool` that returns `true` if the gradient and Hessian of `f` should be computed with respect to it.

Use this argument to filter out constant parameters from decision variables.

source

# Chapter 30

# Utilities

## 30.1 Overview

### The Utilities submodule

The Utilities submodule provides a variety of functions and datastructures for managing `MOI.ModelLike` objects.

#### Utilities.Model

`Utilities.Model` provides an implementation of a `ModelLike` that efficiently supports all functions and sets defined within MOI. However, given the extensibility of MOI, this might not cover all use cases.

Create a model as follows:

```
julia> model = MOI.Utilities.Model{Float64}()
MOIU.Model{Float64}
├ ObjectiveSense: FEASIBILITY_SENSE
├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
├ NumberOfVariables: 0
└ NumberOfConstraints: 0
```

#### Utilities.UniversalFallback

`Utilities.UniversalFallback` is a layer that sits on top of any `ModelLike` and provides non-specialized (slower) fallbacks for constraints and attributes that the underlying `ModelLike` does not support.

For example, `Utilities.Model` doesn't support some variable attributes like `VariablePrimalStart`, so JuMP uses a combination of Universal fallback and `Utilities.Model` as a generic problem cache:

```
julia> model = MOI.Utilities.UniversalFallback(MOI.Utilities.Model{Float64}())
MOIU.UniversalFallback{MOIU.Model{Float64}}
├ ObjectiveSense: FEASIBILITY_SENSE
├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
├ NumberOfVariables: 0
└ NumberOfConstraints: 0
```

**Warning**

Adding a UniversalFallback means that your `model` will now support all constraints, even if the inner-model does not. This can lead to unexpected behavior.

### Utilities.@model

For advanced use cases that need efficient support for functions and sets defined outside of MOI (but still known at compile time), we provide the `Utilities.@model` macro.

The `@model` macro takes a name (for a new type, which must not exist yet), eight tuples specifying the types of constraints that are supported, and then a `Bool` indicating the type is a subtype of `MOI.AbstractOptimizer` (if `true`) or `MOI.ModelLike` (if `false`).

The eight tuples are in the following order:

1. Un-typed scalar sets, for example, `Integer`
2. Typed scalar sets, for example, `LessThan`
3. Un-typed vector sets, for example, `Nonnegatives`
4. Typed vector sets, for example, `PowerCone`
5. Un-typed scalar functions, for example, `VariableIndex`
6. Typed scalar functions, for example, `ScalarAffineFunction`
7. Un-typed vector functions, for example, `VectorOfVariables`
8. Typed vector functions, for example, `VectorAffineFunction`

The tuples can contain more than one element. Typed-sets must be specified without their type parameter, for example, `MOI.LessThan`, not `MOI.LessThan{Float64}`.

Here is an example:

```
julia> MOI.Utilities.@model(
           MyNewModel,
           (MOI.Integer,),                 # Un-typed scalar sets
           (MOI.GreaterThan,),             # Typed scalar sets
           (MOI.Nonnegatives,),            # Un-typed vector sets
           (MOI.PowerCone,),               # Typed vector sets
           (MOI.VariableIndex,),           # Un-typed scalar functions
           (MOI.ScalarAffineFunction,),    # Typed scalar functions
           (MOI.VectorOfVariables,),       # Un-typed vector functions
           (MOI.VectorAffineFunction,),    # Typed vector functions
           true,                           # <:MOI.AbstractOptimizer?
       )
MathOptInterface.Utilities.GenericOptimizer{T, MathOptInterface.Utilities.ObjectiveContainer{T},
↪  MathOptInterface.Utilities.VariablesContainer{T}, MyNewModelFunctionConstraints{T}} where T

julia> model = MyNewModel{Float64}()
MOIU.GenericOptimizer{Float64, MOIU.ObjectiveContainer{Float64}, MOIU.VariablesContainer{Float64},
↪  MyNewModelFunctionConstraints{Float64}}
├ ObjectiveSense: FEASIBILITY_SENSE
```

```
├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
├ NumberOfVariables: 0
└ NumberOfConstraints: 0
```

**Warning**

MyNewModel supports every VariableIndex-in-Set constraint, as well as VariableIndex, ScalarAffineFunction, and ScalarQuadraticFunction objective functions. Implement MOI.supports as needed to forbid constraint and objective function combinations.

As another example, PATHSolver, which only supports VectorAffineFunction-in-Complements defines its optimizer as:

```
julia> MOI.Utilities.@model(
           PathOptimizer,
           (),  # Scalar sets
           (),  # Typed scalar sets
           (MOI.Complements,),  # Vector sets
           (),  # Typed vector sets
           (),  # Scalar functions
           (),  # Typed scalar functions
           (),  # Vector functions
           (MOI.VectorAffineFunction,),  # Typed vector functions
           true,  # is_optimizer
       )
MathOptInterface.Utilities.GenericOptimizer{T, MathOptInterface.Utilities.ObjectiveContainer{T},
↪  MathOptInterface.Utilities.VariablesContainer{T},
↪  MathOptInterface.Utilities.VectorOfConstraints{MathOptInterface.VectorAffineFunction{T},
↪  MathOptInterface.Complements}} where T
```

However, PathOptimizer does not support some VariableIndex-in-Set constraints, so we must explicitly define:

```
julia> function MOI.supports_constraint(
           ::PathOptimizer,
           ::Type{MOI.VariableIndex},
           ::Type{Union{<:MOI.Semiinteger,MOI.Semicontinuous,MOI.ZeroOne,MOI.Integer}}
       )
           return false
       end
```

Finally, PATH doesn't support an objective function, so we need to add:

```
julia> MOI.supports(::PathOptimizer, ::MOI.ObjectiveFunction) = false
```

**Warning**

This macro creates a new type, so it must be called from the top-level of a module, for example, it cannot be called from inside a function.

### Utilities.CachingOptimizer

A [`Utilities.CachingOptimizer`] is an MOI layer that abstracts the difference between solvers that support incremental modification (for example, they support adding variables one-by-one), and solvers that require the entire problem in a single API call (for example, they only accept the `A`, `b` and `c` matrices of a linear program).

It has two parts:

1. A cache, where the model can be built and modified incrementally
2. An optimizer, which is used to solve the problem

```
julia> model = MOI.Utilities.CachingOptimizer(
           MOI.Utilities.Model{Float64}(),
           PathOptimizer{Float64}(),
       )
MOIU.CachingOptimizer
├ state: EMPTY_OPTIMIZER
├ mode: AUTOMATIC
├ model_cache: MOIU.Model{Float64}
│ ├ ObjectiveSense: FEASIBILITY_SENSE
│ ├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
│ ├ NumberOfVariables: 0
│ └ NumberOfConstraints: 0
└ optimizer: MOIU.GenericOptimizer{Float64, MOIU.ObjectiveContainer{Float64},
↪  MOIU.VariablesContainer{Float64}, MOIU.VectorOfConstraints{MOI.VectorAffineFunction{Float64},
↪  MOI.Complements}}
  ├ ObjectiveSense: FEASIBILITY_SENSE
  ├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
  ├ NumberOfVariables: 0
  └ NumberOfConstraints: 0
```

A `Utilities.CachingOptimizer` may be in one of three possible states:

- `NO_OPTIMIZER`: The CachingOptimizer does not have any optimizer.
- `EMPTY_OPTIMIZER`: The CachingOptimizer has an empty optimizer, and it is not synchronized with the cached model. Modifications are forwarded to the cache, but not to the optimizer.
- `ATTACHED_OPTIMIZER`: The CachingOptimizer has an optimizer, and it is synchronized with the cached model. Modifications are forwarded to the optimizer. If the optimizer does not support modifications, and error will be thrown.

Use `Utilities.attach_optimizer` to go from `EMPTY_OPTIMIZER` to `ATTACHED_OPTIMIZER`:

```
julia> MOI.Utilities.attach_optimizer(model)

julia> MOI.Utilities.state(model)
ATTACHED_OPTIMIZER::CachingOptimizerState = 2
```

**Info**

You must be in ATTACHED_OPTIMIZER to use `optimize!`.

Use `Utilities.reset_optimizer` to go from ATTACHED_OPTIMIZER to EMPTY_OPTIMIZER:

```
julia> MOI.Utilities.reset_optimizer(model)

julia> MOI.Utilities.state(model)
EMPTY_OPTIMIZER::CachingOptimizerState = 1
```

**Info**

Calling `MOI.empty!(model)` also resets the state to EMPTY_OPTIMIZER. So after emptying a model, the modification will only be applied to the cache.

Use `Utilities.drop_optimizer` to go from any state to NO_OPTIMIZER:

```
julia> MOI.Utilities.drop_optimizer(model)

julia> MOI.Utilities.state(model)
NO_OPTIMIZER::CachingOptimizerState = 0

julia> model
MOIU.CachingOptimizer
├ state: NO_OPTIMIZER
├ mode: AUTOMATIC
├ model_cache: MOIU.Model{Float64}
│ ├ ObjectiveSense: FEASIBILITY_SENSE
│ ├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
│ ├ NumberOfVariables: 0
│ └ NumberOfConstraints: 0
└ optimizer: nothing
```

Pass an empty optimizer to `Utilities.reset_optimizer` to go from NO_OPTIMIZER to EMPTY_OPTIMIZER:

```
julia> MOI.Utilities.reset_optimizer(model, PathOptimizer{Float64}())

julia> MOI.Utilities.state(model)
EMPTY_OPTIMIZER::CachingOptimizerState = 1

julia> model
MOIU.CachingOptimizer
├ state: EMPTY_OPTIMIZER
├ mode: AUTOMATIC
├ model_cache: MOIU.Model{Float64}
│ ├ ObjectiveSense: FEASIBILITY_SENSE
│ ├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
│ ├ NumberOfVariables: 0
│ └ NumberOfConstraints: 0
└ optimizer: MOIU.GenericOptimizer{Float64, MOIU.ObjectiveContainer{Float64},
↪  MOIU.VariablesContainer{Float64}, MOIU.VectorOfConstraints{MOI.VectorAffineFunction{Float64},
↪  MOI.Complements}}
```

```
  ├ ObjectiveSense: FEASIBILITY_SENSE
  ├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
  ├ NumberOfVariables: 0
  └ NumberOfConstraints: 0
```

Deciding when to attach and reset the optimizer is tedious, and you will often write code like this:

```
try
    # modification
catch
    MOI.Utilities.reset_optimizer(model)
    # Re-try modification
end
```

To make this easier, `Utilities.CachingOptimizer` has two modes of operation:

- `AUTOMATIC`: The `CachingOptimizer` changes its state when necessary. Attempting to add a constraint or perform a modification not supported by the optimizer results in a drop to `EMPTY_OPTIMIZER` mode.
- `MANUAL`: The user must change the state of the `CachingOptimizer`. Attempting to perform an operation in the incorrect state results in an error.

By default, `AUTOMATIC` mode is chosen. However, you can create a `CachingOptimizer` in `MANUAL` mode as follows:

```
julia> model = MOI.Utilities.CachingOptimizer(
           MOI.Utilities.Model{Float64}(),
           MOI.Utilities.MANUAL,
       )
MOIU.CachingOptimizer
├ state: NO_OPTIMIZER
├ mode: MANUAL
├ model_cache: MOIU.Model{Float64}
│ ├ ObjectiveSense: FEASIBILITY_SENSE
│ ├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
│ ├ NumberOfVariables: 0
│ └ NumberOfConstraints: 0
└ optimizer: nothing

julia> MOI.Utilities.reset_optimizer(model, PathOptimizer{Float64}())

julia> model
MOIU.CachingOptimizer
├ state: EMPTY_OPTIMIZER
├ mode: MANUAL
├ model_cache: MOIU.Model{Float64}
│ ├ ObjectiveSense: FEASIBILITY_SENSE
│ ├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
│ ├ NumberOfVariables: 0
│ └ NumberOfConstraints: 0
└ optimizer: MOIU.GenericOptimizer{Float64, MOIU.ObjectiveContainer{Float64},
↪  MOIU.VariablesContainer{Float64}, MOIU.VectorOfConstraints{MOI.VectorAffineFunction{Float64},
↪  MOI.Complements}}
```

```
  ├ ObjectiveSense: FEASIBILITY_SENSE
  ├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
  ├ NumberOfVariables: 0
  └ NumberOfConstraints: 0
```

#### Printing

Use `print` to print the formulation of the model.

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model)
MOI.VariableIndex(1)

julia> MOI.set(model, MOI.VariableName(), x, "x_var")

julia> MOI.add_constraint(model, x, MOI.ZeroOne())
MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex, MathOptInterface.ZeroOne}(1)

julia> MOI.set(model, MOI.ObjectiveFunction{typeof(x)}(), x)

julia> MOI.set(model, MOI.ObjectiveSense(), MOI.MAX_SENSE)

julia> print(model)
Maximize VariableIndex:
 x_var

Subject to:

VariableIndex-in-ZeroOne
 x_var ∈ {0, 1}
```

Use `Utilities.latex_formulation` to display the model in LaTeX form:

```
julia> MOI.Utilities.latex_formulation(model)
$$ \begin{aligned}
\max\quad & x\_var \\
\text{Subject to}\\
 & \text{VariableIndex-in-ZeroOne} \\
 & x\_var \in \{0, 1\} \\
\end{aligned} $$
```

**Tip**

In IJulia, calling `print` or ending a cell with `Utilities.latex_formulation` will render the model in LaTeX.

#### Utilities.PenaltyRelaxation

Pass `Utilities.PenaltyRelaxation` to `modify` to relax the problem by adding penalized slack variables to the constraints. This is helpful when debugging sources of infeasible models.

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model);

julia> MOI.set(model, MOI.VariableName(), x, "x")

julia> c = MOI.add_constraint(model, 1.0 * x, MOI.LessThan(2.0));

julia> map = MOI.modify(model, MOI.Utilities.PenaltyRelaxation(Dict(c => 2.0)));

julia> print(model)
Minimize ScalarAffineFunction{Float64}:
 0.0 + 2.0 v[2]

Subject to:

ScalarAffineFunction{Float64}-in-LessThan{Float64}
 0.0 + 1.0 x - 1.0 v[2] <= 2.0

VariableIndex-in-GreaterThan{Float64}
 v[2] >= 0.0

julia> map[c]
0.0 + 1.0 MOI.VariableIndex(2)
```

You can also modify a single constraint using `Utilities.ScalarPenaltyRelaxation`:

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model);

julia> MOI.set(model, MOI.VariableName(), x, "x")

julia> c = MOI.add_constraint(model, 1.0 * x, MOI.LessThan(2.0));

julia> f = MOI.modify(model, c, MOI.Utilities.ScalarPenaltyRelaxation(2.0));

julia> print(model)
Minimize ScalarAffineFunction{Float64}:
 0.0 + 2.0 v[2]

Subject to:

ScalarAffineFunction{Float64}-in-LessThan{Float64}
 0.0 + 1.0 x - 1.0 v[2] <= 2.0

VariableIndex-in-GreaterThan{Float64}
 v[2] >= 0.0

julia> f
0.0 + 1.0 MOI.VariableIndex(2)
```

### Utilities.MatrixOfConstraints

The constraints of `Utilities.Model` are stored as a vector of tuples of function and set in a `Utilities.VectorOfConstraints`.

Other representations can be used by parameterizing the type `Utilities.GenericModel` (resp. `Utilities.GenericOptimizer`).

For example, if all non-`VariableIndex` constraints are affine, the coefficients of all the constraints can be stored in a single sparse matrix using `Utilities.MatrixOfConstraints`.

The constraints storage can even be customized up to a point where it exactly matches the storage of the solver of interest, in which case `copy_to` can be implemented for the solver by calling `copy_to` to this custom model.

For example, Clp.jl defines the following model:

```
julia> MOI.Utilities.@product_of_sets(
           SupportedSets,
           MOI.EqualTo{T},
           MOI.LessThan{T},
           MOI.GreaterThan{T},
           MOI.Interval{T},
       );

julia> const OptimizerCache = MOI.Utilities.GenericModel{
           Float64,
           MOI.Utilities.ObjectiveContainer{Float64},
           MOI.Utilities.VariablesContainer{Float64},
           MOI.Utilities.MatrixOfConstraints{
               Float64,
               MOI.Utilities.MutableSparseMatrixCSC{
                   # The data type of the coefficients
                   Float64,
                   # The data type of the variable indices
                   Cint,
                   # Can also be MOI.Utilities.OneBasedIndexing
                   MOI.Utilities.ZeroBasedIndexing,
               },
               MOI.Utilities.Hyperrectangle{Float64},
               SupportedSets{Float64},
           },
       };
```

Given the input model:

```
julia> src = MOI.Utilities.Model{Float64}();

julia> MOI.Utilities.loadfromstring!(
           src,
           """
           variables: x, y, z
           maxobjective: x + 2.0 * y + -3.1 * z
           x + y <= 1.0
           2.0 * y >= 3.0
           -4.0 * x + z == 5.0
           x in Interval(0.0, 1.0)
           y <= 10.0
```

```
            z == 5.0
            """,
        )
```

We can construct a new cached model and copy `src` to it:

```
julia> dest = OptimizerCache();

julia> index_map = MOI.copy_to(dest, src);
```

From `dest`, we can access the `A` matrix in sparse matrix form:

```
julia> A = dest.constraints.coefficients;

julia> A.n
3

julia> A.m
3

julia> A.colptr
4-element Vector{Int32}:
 0
 2
 4
 5

julia> A.rowval
5-element Vector{Int32}:
 0
 1
 1
 2
 0

julia> A.nzval
5-element Vector{Float64}:
 -4.0
  1.0
  1.0
  2.0
  1.0
```

The lower and upper row bounds:

```
julia> row_bounds = dest.constraints.constants;

julia> row_bounds.lower
3-element Vector{Float64}:
   5.0
 -Inf
   3.0
```

```
julia> row_bounds.upper
3-element Vector{Float64}:
  5.0
  1.0
 Inf
```

The lower and upper variable bounds:

```
julia> dest.variables.lower
3-element Vector{Float64}:
   0.0
 -Inf
   5.0

julia> dest.variables.upper
3-element Vector{Float64}:
  1.0
 10.0
  5.0
```

Because of larger variations between solvers, the objective can be queried using the standard MOI methods:

```
julia> MOI.get(dest, MOI.ObjectiveSense())
MAX_SENSE::OptimizationSense = 1

julia> F = MOI.get(dest, MOI.ObjectiveFunctionType())
MathOptInterface.ScalarAffineFunction{Float64}

julia> F = MOI.get(dest, MOI.ObjectiveFunction{F}())
0.0 + 1.0 MOI.VariableIndex(1) + 2.0 MOI.VariableIndex(2) - 3.1 MOI.VariableIndex(3)
```

Thus, Clp.jl implements `copy_to` methods similar to the following:

```
# This method copies from the cache to the `Clp.Optimizer` object.
function MOI.copy_to(dest::Optimizer, src::OptimizerCache)
    @assert MOI.is_empty(dest)
    A = src.constraints.coefficients
    row_bounds = src.constraints.constants
    Clp_loadProblem(
        dest,
        A.n,
        A.m,
        A.colptr,
        A.rowval,
        A.nzval,
        src.lower_bound,
        src.upper_bound,
        # (...) objective vector (omitted),
        row_bounds.lower,
        row_bounds.upper,
    )
```

```
    return MOI.Utilities.identity_index_map(src)
end

# This method copies from an arbitrary model to the optimizer, by the
# intermediate `OptimizerCache` representation.
function MOI.copy_to(dest::Optimizer, src::MOI.ModelLike)
    cache = OptimizerCache()
    index_map = MOI.copy_to(cache, src)
    MOI.copy_to(dest, cache)
    return index_map
end

# This is a special method that gets called in some cases when `OptimizerCache`
# is used as the backing data structure in a `MOI.Utilities.CachingOptimizer`.
# It is needed for performance, but not correctness.
function MOI.copy_to(
    dest::Optimizer,
    src::MOI.Utilities.UniversalFallback{OptimizerCache},
)
    MOI.Utilities.throw_unsupported(src)
    return MOI.copy_to(dest, src.model)
end
```

**Tip**

For other examples of `Utilities.MatrixOfConstraints`, see:

- Cbc.jl
- ECOS.jl
- SCS.jl

#### ModelFilter

Utilities provides `Utilities.ModelFilter` as a useful tool to copy a subset of a model. For example, given an infeasible model, we can copy the irreducible infeasible subsystem (for models implementing `ConstraintConflictStatus`) as follows:

```
my_filter(::Any) = true
function my_filter(ci::MOI.ConstraintIndex)
    status = MOI.get(dest, MOI.ConstraintConflictStatus(), ci)
    return status != MOI.NOT_IN_CONFLICT
end
filtered_src = MOI.Utilities.ModelFilter(my_filter, src)
index_map = MOI.copy_to(dest, filtered_src)
```

#### Fallbacks

The value of some attributes can be inferred from the value of other attributes.

For example, the value of `ObjectiveValue` can be computed using `ObjectiveFunction` and `VariablePrimal`.

When a solver gives direct access to an attribute, it is better to return this value. However, if this is not the case, `Utilities.get_fallback` can be used instead. For example:

```
function MOI.get(model::Optimizer, attr::MOI.ObjectiveFunction)
    return MOI.Utilities.get_fallback(model, attr)
end
```

#### DoubleDicts

When writing MOI interfaces, we often need to handle situations in which we map `ConstraintIndex`s to different values. For example, to a string for `ConstraintName`.

One option is to use a dictionary like `Dict{MOI.ConstraintIndex,String}`. However, this incurs a performance cost because the key is not a concrete type.

The DoubleDicts submodule helps this situation by providing two types main types `Utilities.DoubleDicts.DoubleDict` and `Utilities.DoubleDicts.IndexDoubleDict`. These types act like normal dictionaries, but internally they use more efficient dictionaries specialized to the type of the function-set pair.

The most common usage of a `DoubleDict` is in the `index_map` returned by `copy_to`. Performance can be improved, by using a function barrier. That is, instead of code like:

```
index_map = MOI.copy_to(dest, src)
for (F, S) in MOI.get(src, MOI.ListOfConstraintTypesPresent())
    for ci in MOI.get(src, MOI.ListOfConstraintIndices{F,S}())
        dest_ci = index_map[ci]
        # ...
    end
end
```

use instead:

```
function function_barrier(
    dest,
    src,
    index_map::MOI.Utilities.DoubleDicts.IndexDoubleDictInner{F,S},
) where {F,S}
    for ci in MOI.get(src, MOI.ListOfConstraintIndices{F,S}())
        dest_ci = index_map[ci]
        # ...
    end
    return
end

index_map = MOI.copy_to(dest, src)
for (F, S) in MOI.get(src, MOI.ListOfConstraintTypesPresent())
    function_barrier(dest, src, index_map[F, S])
end
```

## 30.2 API Reference

#### Utilities.Model

`MathOptInterface.Utilities.Model` – Type.

```
MOI.Utilities.Model{T}() where {T}
```

An implementation of `ModelLike` that supports all functions and sets defined in MOI. It is parameterized by the coefficient type.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}()
MOIU.Model{Float64}
├ ObjectiveSense: FEASIBILITY_SENSE
├ ObjectiveFunctionType: MOI.ScalarAffineFunction{Float64}
├ NumberOfVariables: 0
└ NumberOfConstraints: 0
```

source

### Utilities.UniversalFallback

`MathOptInterface.Utilities.UniversalFallback` – Type.

```
UniversalFallback
```

The `UniversalFallback` can be applied on a `MOI.ModelLike` `model` to create the model `UniversalFallback(model)` supporting any constraint and attribute. This allows to have a specialized implementation in `model` for performance critical constraints and attributes while still supporting other attributes with a small performance penalty. Note that `model` is unaware of constraints and attributes stored by `UniversalFallback` so this is not appropriate if `model` is an optimizer (for this reason, `MOI.optimize!` has not been implemented). In that case, optimizer bridges should be used instead.

source

### Utilities.@model

`MathOptInterface.Utilities.@model` – Macro.

```
macro model(
    model_name,
    scalar_sets,
    typed_scalar_sets,
    vector_sets,
    typed_vector_sets,
    scalar_functions,
    typed_scalar_functions,
    vector_functions,
    typed_vector_functions,
    is_optimizer = false
)
```

Creates a type `model_name` implementing the MOI model interface and supporting all combinations of the provided functions and sets.

Each `typed_ scalar/vector sets/functions` argument is a tuple of types. A type is "typed" if it has a coefficient {T} as the first type parameter.

**Tuple syntax**

To give no set/function, write `()`. To give one set or function X, write `(X,)`.

**`is_optimizer`**

If `is_optimizer = true`, the resulting struct is a of `GenericOptimizer`, which is a subtype of `MOI.AbstractOptimizer`, otherwise, it is a `GenericModel`, which is a subtype of `MOI.ModelLike`.

**VariableIndex**

- The function `MOI.VariableIndex` must not be given in `scalar_functions`.
- The model supports `MOI.VariableIndex`-in-S constraints where S is `MOI.EqualTo`, `MOI.GreaterThan`, `MOI.LessThan`, `MOI.Interval`, `MOI.Integer`, `MOI.ZeroOne`, `MOI.Semicontinuous` or `MOI.Semiinteger`.
- The sets supported with `MOI.VariableIndex` cannot be controlled from the macro; use `UniversalFallback` to support more sets.

**Example**

The model describing a linear program would be:

```
@model(
    LPModel,                                            # model_name
    (),                                                 # untyped scalar sets
    (MOI.EqualTo, MOI.GreaterThan, MOI.LessThan, MOI.Interval), #   typed scalar sets
    (MOI.Zeros, MOI.Nonnegatives, MOI.Nonpositives),    # untyped vector sets
    (),                                                 #   typed vector sets
    (),                                                 # untyped scalar functions
    (MOI.ScalarAffineFunction,),                        #   typed scalar functions
    (MOI.VectorOfVariables,),                           # untyped vector functions
    (MOI.VectorAffineFunction,),                        #   typed vector functions
    false,                                              # is_optimizer
)
```

source

`MathOptInterface.Utilities.GenericModel` – Type.

```
mutable struct GenericModel{T,O,V,C} <: AbstractModelLike{T}
```

Implements a model supporting coefficients of type T and:

- An objective function stored in `.objective::O`
- Variables and `VariableIndex` constraints stored in `.variable_bounds::V`
- F-in-S constraints (excluding `VariableIndex` constraints) stored in `.constraints::C`

All interactions take place via the MOI interface, so the types O, V, and C must implement the API as needed for their functionality.

source

`MathOptInterface.Utilities.GenericOptimizer` – Type.

```
mutable struct GenericOptimizer{T,O,V,C} <: AbstractOptimizer{T}
```

Implements a model supporting coefficients of type `T` and:

- An objective function stored in `.objective::O`
- Variables and `VariableIndex` constraints stored in `.variable_bounds::V`
- F-in-S constraints (excluding `VariableIndex` constraints) stored in `.constraints::C`

All interactions take place via the MOI interface, so the types `O`, `V`, and `C` must implement the API as needed for their functionality.

source

**`.objective`**

`MathOptInterface.Utilities.ObjectiveContainer` – Type.

```
ObjectiveContainer{T}
```

A helper struct to simplify the handling of objective functions in Utilities.Model.

source

**`.variables`**

`MathOptInterface.Utilities.VariablesContainer` – Type.

```
struct VariablesContainer{T} <: AbstractVectorBounds
    set_mask::Vector{UInt16}
    lower::Vector{T}
    upper::Vector{T}
end
```

A struct for storing variables and VariableIndex-related constraints. Used in `MOI.Utilities.Model` by default.

source

`MathOptInterface.Utilities.FreeVariables` – Type.

```
mutable struct FreeVariables <: MOI.ModelLike
    n::Int64
    FreeVariables() = new(0)
end
```

A struct for storing free variables that can be used as the `variables` field of `GenericModel` or `GenericModel`. It represents a model that does not support any constraint nor objective function.

**Example**

The following model type represents a conic model in geometric form. As opposed to `VariablesContainer`, `FreeVariables` does not support constraint bounds so they are bridged into an affine constraint in the `MOI.Nonnegatives` cone as expected for the geometric conic form.

```
julia> MOI.Utilities.@product_of_sets(
    Cones,
    MOI.Zeros,
    MOI.Nonnegatives,
    MOI.SecondOrderCone,
    MOI.PositiveSemidefiniteConeTriangle,
);

julia> const ConicModel{T} = MOI.Utilities.GenericOptimizer{
    T,
    MOI.Utilities.ObjectiveContainer{T},
    MOI.Utilities.FreeVariables,
    MOI.Utilities.MatrixOfConstraints{
        T,
        MOI.Utilities.MutableSparseMatrixCSC{
            T,
            Int,
            MOI.Utilities.OneBasedIndexing,
        },
        Vector{T},
        Cones{T},
    },
};

julia> model = MOI.instantiate(ConicModel{Float64}, with_bridge_type=Float64);

julia> x = MOI.add_variable(model)
MathOptInterface.VariableIndex(1)

julia> c = MOI.add_constraint(model, x, MOI.GreaterThan(1.0))
MathOptInterface.ConstraintIndex{MathOptInterface.VariableIndex,
↪  MathOptInterface.GreaterThan{Float64}}(1)

julia> MOI.Bridges.is_bridged(model, c)
true

julia> bridge = MOI.Bridges.bridge(model, c)
MathOptInterface.Bridges.Constraint.VectorizeBridge{Float64,
↪  MathOptInterface.VectorAffineFunction{Float64}, MathOptInterface.Nonnegatives,
↪  MathOptInterface.VariableIndex}(MathOptInterface.ConstraintIndex{MathOptInterface.VectorAffineFunction{Float64
↪  MathOptInterface.Nonnegatives}(1), 1.0)

julia> bridge.vector_constraint
MathOptInterface.ConstraintIndex{MathOptInterface.VectorAffineFunction{Float64},
↪  MathOptInterface.Nonnegatives}(1)

julia> MOI.Bridges.is_bridged(model, bridge.vector_constraint)
false
```

source

**.constraints**

MathOptInterface.Utilities.VectorOfConstraints – Type.

```
mutable struct VectorOfConstraints{
    F<:MOI.AbstractFunction,
    S<:MOI.AbstractSet,
} <: MOI.ModelLike
    constraints::CleverDicts.CleverDict{
        MOI.ConstraintIndex{F,S},
        Tuple{F,S},
        typeof(CleverDicts.key_to_index),
        typeof(CleverDicts.index_to_key),
    }
end
```

A struct storing F-in-S constraints as a mapping between the constraint indices to the corresponding tuple of function and set.

source

MathOptInterface.Utilities.StructOfConstraints – Type.

```
abstract type StructOfConstraints <: MOI.ModelLike end
```

A struct storing a subfields other structs storing constraints of different types.

See Utilities.@struct_of_constraints_by_function_types and Utilities.@struct_of_constraints_by_set_types.

source

MathOptInterface.Utilities.@struct_of_constraints_by_function_types – Macro.

```
Utilities.@struct_of_constraints_by_function_types(name, func_types...)
```

Given a vector of n function types (F1, F2,..., Fn) in func_types, defines a subtype of StructOfConstraints of name name and which type parameters {T, C1, C2, ..., Cn}. It contains n field where the ith field has type Ci and stores the constraints of function type Fi.

The expression Fi can also be a union in which case any constraint for which the function type is in the union is stored in the field with type Ci.

source

MathOptInterface.Utilities.@struct_of_constraints_by_set_types – Macro.

```
Utilities.@struct_of_constraints_by_set_types(name, func_types...)
```

Given a vector of n set types `(S1, S2,..., Sn)` in `func_types`, defines a subtype of `StructOfConstraints` of name `name` and which type parameters `{T, C1, C2, ..., Cn}`. It contains `n` field where the `i`th field has type `Ci` and stores the constraints of set type `Si`. The expression `Si` can also be a union in which case any constraint for which the set type is in the union is stored in the field with type `Ci`. This can be useful if `Ci` is a `MatrixOfConstraints` in order to concatenate the coefficients of constraints of several different set types in the same matrix.

source

`MathOptInterface.Utilities.struct_of_constraint_code` – Function.

```
struct_of_constraint_code(struct_name, types, field_types = nothing)
```

Given a vector of n `Union{SymbolFun,_UnionSymbolFS{SymbolFun}}` or `Union{SymbolSet,_UnionSymbolFS{SymbolSet}}` in `types`, defines a subtype of `StructOfConstraints` of name `name` and which type parameters `{T, F1, F2, ..., Fn}` if `field_types` is `nothing` and a `{T}` otherwise. It contains `n` field where the `i`th field has type `Ci` if `field_types` is `nothing` and type `field_types[i]` otherwise. If `types` is vector of `Union{SymbolFun,_UnionSymbolFS{SymbolFun}}` (resp. `Union{SymbolSet,_UnionSymbolFS{SymbolSet}}`) then the constraints of that function (resp. set) type are stored in the corresponding field.

This function is used by the macros `@model`, `@struct_of_constraints_by_function_types` and `@struct_of_constraints_b`

source

### Caching optimizer

`MathOptInterface.Utilities.CachingOptimizer` – Type.

```
CachingOptimizer
```

`CachingOptimizer` is an intermediate layer that stores a cache of the model and links it with an optimizer. It supports incremental model construction and modification even when the optimizer doesn't.

#### Mode

A `Utilities.CachingOptimizer` has two modes of operation: `Utilities.AUTOMATIC` and `Utilities.MANUAL`. See their docstrings for details.

Use `Utilities.mode` to query the mode of a `Utilities.CachingOptimizer`.

#### State

A `Utilities.CachingOptimizer` may be in one of three possible states: `NO_OPTIMIZER`, `Utilities.EMPTY_OPTIMIZER`, and `Utilities.ATTACHED_OPTIMIZER`. See their docstrings for details.

Use `Utilities.state` to query the state of a `Utilities.CachingOptimizer`.

#### Constructor with optimizer

```
    CachingOptimizer(cache::MOI.ModelLike, optimizer::AbstractOptimizer)
```

Creates a `CachingOptimizer` in `AUTOMATIC` mode, with the optimizer `optimizer`.

The type of the optimizer returned is `CachingOptimizer{typeof(optimizer),typeof(cache)}` so it does not support the function `Utilities.reset_optimizer(::CachingOptimizer, new_optimizer)` if the type of `new_optimizer` is different from the type of `optimizer`.

**Constructor without optimizer**

```
    CachingOptimizer(cache::MOI.ModelLike, mode::CachingOptimizerMode)
```

Creates a `CachingOptimizer` in the `NO_OPTIMIZER` `Utilities.CachingOptimizerState` and the `Utilities.CachingOptimiz` `mode`.

The type of the optimizer returned is `CachingOptimizer{MOI.AbstractOptimizer,typeof(cache)}` so it does support the function `reset_optimizer(::CachingOptimizer, new_optimizer)` if the type of `new_optimizer` is different from the type of `optimizer`.

source

`MathOptInterface.Utilities.CachingOptimizerState` – Type.

```
CachingOptimizerState
```

A `Utilities.CachingOptimizer` may be in one of three possible states.

**Values**

**`NO_OPTIMIZER`**

The CachingOptimizer does not have any optimizer.

**`EMPTY_OPTIMIZER`**

The CachingOptimizer has an optimizer. The optimizer is empty and it is not synchronized with the cached model.

**`ATTACHED_OPTIMIZER`**

The CachingOptimizer has an optimizer, and it is synchronized with the cached model.

source

`MathOptInterface.Utilities.NO_OPTIMIZER` – Constant.

```
NO_OPTIMIZER::CachingOptimizerState
```

An instance of the `CachingOptimizerState` enum.

**About**

The CachingOptimizer does not have any optimizer.

source

`MathOptInterface.Utilities.EMPTY_OPTIMIZER` – Constant.

```
EMPTY_OPTIMIZER::CachingOptimizerState
```

An instance of the `CachingOptimizerState` enum.

**About**

The CachingOptimizer has an optimizer. The optimizer is empty and it is not synchronized with the cached model.

source

`MathOptInterface.Utilities.ATTACHED_OPTIMIZER` – Constant.

```
ATTACHED_OPTIMIZER::CachingOptimizerState
```

An instance of the `CachingOptimizerState` enum.

**About**

The CachingOptimizer has an optimizer, and it is synchronized with the cached model.

source

`MathOptInterface.Utilities.state` – Function.

```
state(m::CachingOptimizer)::CachingOptimizerState
```

Returns the state of the CachingOptimizer `m`.

See `Utilities.CachingOptimizer`.

source

`MathOptInterface.Utilities.CachingOptimizerMode` – Type.

```
CachingOptimizerMode
```

A `Utilities.CachingOptimizer` has two modes of operation.

**Values**

**`MANUAL`**

The only methods that change the state of the `CachingOptimizer` are `Utilities.reset_optimizer`, `Utilities.drop_optimizer`, and `Utilities.attach_optimizer`.

Attempting to perform an operation in the incorrect state results in an error.

**`AUTOMATIC`**

The Utilities.CachingOptimizer changes its state when necessary. For example, MOI.optimize! will automatically call Utilities.attach_optimizer (an optimizer must have been previously set). Attempting to add a constraint or perform a modification not supported by the optimizer results in a drop to the EMPTY_OPTIMIZER state.

source

MathOptInterface.Utilities.AUTOMATIC – Constant.

```
AUTOMATIC::CachingOptimizerMode
```

An instance of the CachingOptimizerMode enum.

**About**

The Utilities.CachingOptimizer changes its state when necessary. For example, MOI.optimize! will automatically call Utilities.attach_optimizer (an optimizer must have been previously set). Attempting to add a constraint or perform a modification not supported by the optimizer results in a drop to the EMPTY_OPTIMIZER state.

source

MathOptInterface.Utilities.MANUAL – Constant.

```
MANUAL::CachingOptimizerMode
```

An instance of the CachingOptimizerMode enum.

**About**

The only methods that change the state of the CachingOptimizer are Utilities.reset_optimizer, Utilities.drop_optimizer, and Utilities.attach_optimizer.

Attempting to perform an operation in the incorrect state results in an error.

source

MathOptInterface.Utilities.mode – Function.

```
mode(m::CachingOptimizer)::CachingOptimizerMode
```

Returns the operating mode of the CachingOptimizer m.

See Utilities.CachingOptimizer.

source

MathOptInterface.Utilities.attach_optimizer – Function.

```
attach_optimizer(model::CachingOptimizer)
```

Attaches the optimizer to `model`, copying all model data into it. Can be called only from the `EMPTY_OPTIMIZER` state. If the copy succeeds, the `CachingOptimizer` will be in state `ATTACHED_OPTIMIZER` after the call, otherwise an error is thrown; see `MOI.copy_to` for more details on which errors can be thrown.

source

`MathOptInterface.Utilities.reset_optimizer` – Function.

```
reset_optimizer(m::CachingOptimizer, optimizer::MOI.AbstractOptimizer)
```

Sets or resets `m` to have the given empty optimizer `optimizer`.

Can be called from any state. An assertion error will be thrown if `optimizer` is not empty.

The `CachingOptimizer` `m` will be in state `EMPTY_OPTIMIZER` after the call.

source

```
reset_optimizer(m::CachingOptimizer)
```

Detaches and empties the current optimizer. Can be called from `ATTACHED_OPTIMIZER` or `EMPTY_OPTIMIZER` state. The `CachingOptimizer` will be in state `EMPTY_OPTIMIZER` after the call.

source

`MathOptInterface.Utilities.drop_optimizer` – Function.

```
drop_optimizer(m::CachingOptimizer)
```

Drops the optimizer, if one is present. Can be called from any state. The `CachingOptimizer` will be in state `NO_OPTIMIZER` after the call.

source

#### Mock optimizer

`MathOptInterface.Utilities.MockOptimizer` – Type.

```
MockOptimizer
```

`MockOptimizer` is a fake optimizer especially useful for testing. Its main feature is that it can store the values that should be returned for each attribute.

source

#### Printing

`MathOptInterface.Utilities.latex_formulation` – Function.

```
latex_formulation(model::MOI.ModelLike; kwargs...)
```

Wrap `model` in a type so that it can be pretty-printed as `text/latex` in a notebook like IJulia, or in Documenter.

To render the model, end the cell with `latex_formulation(model)`, or call `display(latex_formulation(model))` in to force the display of the model from inside a function.

Possible keyword arguments are:

- `simplify_coefficients` : Simplify coefficients if possible by omitting them or removing trailing zeros.
- `default_name` : The name given to variables with an empty name.
- `print_types` : Print the MOI type of each function and set for clarity.

source

**Copy utilities**

`MathOptInterface.Utilities.default_copy_to` – Function.

```
default_copy_to(dest::MOI.ModelLike, src::MOI.ModelLike)
```

A default implementation of `MOI.copy_to(dest, src)` for models that implement the incremental interface, that is, `MOI.supports_incremental_interface` returns `true`.

source

`MathOptInterface.Utilities.IndexMap` – Type.

```
IndexMap()
```

The dictionary-like object returned by `MOI.copy_to`.

source

`MathOptInterface.Utilities.identity_index_map` – Function.

```
identity_index_map(model::MOI.ModelLike)
```

Return an `IndexMap` that maps all variable and constraint indices of `model` to themselves.

source

`MathOptInterface.Utilities.ModelFilter` – Type.

```
ModelFilter(filter::Function, model::MOI.ModelLike)
```

A layer to filter out various components of `model`.

The filter function takes a single argument, which is each element from the list returned by the attributes below. It returns `true` if the element should be visible in the filtered model and `false` otherwise.

The components that are filtered are:

- Entire constraint types via:
  - `MOI.ListOfConstraintTypesPresent`
- Individual constraints via:
  - `MOI.ListOfConstraintIndices{F,S}`
- Specific attributes via:
  - `MOI.ListOfModelAttributesSet`
  - `MOI.ListOfConstraintAttributesSet`
  - `MOI.ListOfVariableAttributesSet`

**Warning**

The list of attributes filtered may change in a future release. You should write functions that are generic and not limited to the five types listed above. Thus, you should probably define a fallback `filter(::Any) = true`.

See below for examples of how this works.

**Note**

This layer has a limited scope. It is intended by be used in conjunction with `MOI.copy_to`.

**Example: copy model excluding integer constraints**

Use the `do` syntax to provide a single function.

```
filtered_src = MOI.Utilities.ModelFilter(src) do item
    return item != (MOI.VariableIndex, MOI.Integer)
end
MOI.copy_to(dest, filtered_src)
```

**Example: copy model excluding names**

Use type dispatch to simplify the implementation:

```
my_filter(::Any) = true  # Note the generic fallback
my_filter(::MOI.VariableName) = false
my_filter(::MOI.ConstraintName) = false
filtered_src = MOI.Utilities.ModelFilter(my_filter, src)
MOI.copy_to(dest, filtered_src)
```

**Example: copy irreducible infeasible subsystem**

```
my_filter(::Any) = true  # Note the generic fallback
function my_filter(ci::MOI.ConstraintIndex)
    status = MOI.get(dest, MOI.ConstraintConflictStatus(), ci)
    return status != MOI.NOT_IN_CONFLICT
end
filtered_src = MOI.Utilities.ModelFilter(my_filter, src)
MOI.copy_to(dest, filtered_src)
```

source

`MathOptInterface.Utilities.loadfromstring!` – Function.

```
loadfromstring!(model, s)
```

A utility function to aid writing tests.

**Warning**

This function is not intended for widespread use. It is mainly used as a tool to simplify writing tests in MathOptInterface. Do not use it as an exchange format for storing or transmitting problem instances. Use the FileFormats submodule instead.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> MOI.Utilities.loadfromstring!(model, """
       variables: x, y, z
       constrainedvariable: [a, b, c] in Nonnegatives(3)
       minobjective::Float64: 2x + 3y
       con1: x + y <= 1.0
       con2: [x, y] in Nonnegatives(2)
       x >= 0.0
       """)
```

**Notes**

Special labels are:

- variables
- minobjective
- maxobjectives

Everything else denotes a constraint with a name.

Append `::T` to use an element type of T when parsing the function.

Do not name `VariableIndex` constraints.

**Exceptions**

- `x - y` does NOT currently parse. Instead, write `x + -1.0 * y`.
- `x^2` does NOT currently parse. Instead, write `1 * x * x`.

source

### Penalty relaxation

`MathOptInterface.Utilities.PenaltyRelaxation` – Type.

```
PenaltyRelaxation(
    penalties = Dict{MOI.ConstraintIndex,Float64}();
    default::Union{Nothing,T} = 1.0,
    warn::Bool = true,
)
```

A problem modifier that, when passed to `MOI.modify`, destructively modifies the model in-place to create a penalized relaxation of the constraints.

**Warning**

This is a destructive routine that modifies the model in-place. If you don't want to modify the original model, use `JuMP.copy_model` to create a copy before calling `MOI.modify`.

#### Reformulation

See `Utilities.ScalarPenaltyRelaxation` for details of the reformulation.

For each constraint `ci`, the penalty passed to `Utilities.ScalarPenaltyRelaxation` is `get(penalties, ci, default)`. If the value is `nothing`, because `ci` does not exist in `penalties` and `default = nothing`, then the constraint is skipped.

#### Return value

`MOI.modify(model, PenaltyRelaxation())` returns a `Dict{MOI.ConstraintIndex,MOI.ScalarAffineFunction}` that maps each constraint index to the corresponding `y + z` as a `MOI.ScalarAffineFunction`. In an optimal solution, query the value of these functions to compute the violation of each constraint.

#### Relax a subset of constraints

To relax a subset of constraints, pass a `penalties` dictionary and set `default = nothing`.

#### Supported constraint types

The penalty relaxation is currently limited to modifying `MOI.ScalarAffineFunction` and `MOI.ScalarQuadraticFunction` constraints in the linear sets `MOI.LessThan`, `MOI.GreaterThan`, `MOI.EqualTo` and `MOI.Interval`.

It does not include variable bound or integrality constraints, because these cannot be modified in-place.

To modify variable bounds, rewrite them as linear constraints.

If a constraint cannot be modified, a warning is logged and the constraint is skipped. The warning can be disabled by setting `warn = false`.

#### Example

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model);

julia> c = MOI.add_constraint(model, 1.0 * x, MOI.LessThan(2.0));

julia> map = MOI.modify(model, MOI.Utilities.PenaltyRelaxation(default = 2.0));

julia> print(model)
Minimize ScalarAffineFunction{Float64}:
 0.0 + 2.0 v[2]

Subject to:

ScalarAffineFunction{Float64}-in-LessThan{Float64}
 0.0 + 1.0 v[1] - 1.0 v[2] <= 2.0

VariableIndex-in-GreaterThan{Float64}
 v[2] >= 0.0

julia> map[c] isa MOI.ScalarAffineFunction{Float64}
true
```

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model);

julia> c = MOI.add_constraint(model, 1.0 * x, MOI.LessThan(2.0));

julia> map = MOI.modify(model, MOI.Utilities.PenaltyRelaxation(Dict(c => 3.0)));

julia> print(model)
Minimize ScalarAffineFunction{Float64}:
 0.0 + 3.0 v[2]

Subject to:

ScalarAffineFunction{Float64}-in-LessThan{Float64}
 0.0 + 1.0 v[1] - 1.0 v[2] <= 2.0

VariableIndex-in-GreaterThan{Float64}
 v[2] >= 0.0

julia> map[c] isa MOI.ScalarAffineFunction{Float64}
true
```

source

`MathOptInterface.Utilities.ScalarPenaltyRelaxation` – Type.

```
ScalarPenaltyRelaxation(penalty::T) where {T}
```

A problem modifier that, when passed to `MOI.modify`, destructively modifies the constraint in-place to create a penalized relaxation of the constraint.

**Warning**

This is a destructive routine that modifies the constraint in-place. If you don't want to modify the original model, use `JuMP.copy_model` to create a copy before calling `MOI.modify`.

**Reformulation**

The penalty relaxation modifies constraints of the form $f(x) \in S$ into $f(x) + y - z \in S$, where $y, z \geq 0$, and then it introduces a penalty term into the objective of $a \times (y + z)$ (if minimizing, else $-a$), where $a$ is `penalty`

When S is `MOI.LessThan` or `MOI.GreaterThan`, we omit y or z respectively as a performance optimization.

**Return value**

`MOI.modify(model, ci, ScalarPenaltyRelaxation(penalty))` returns `y + z` as a `MOI.ScalarAffineFunction`. In an optimal solution, query the value of this function to compute the violation of the constraint.

**Example**

```
julia> model = MOI.Utilities.Model{Float64}();

julia> x = MOI.add_variable(model);

julia> c = MOI.add_constraint(model, 1.0 * x, MOI.LessThan(2.0));

julia> f = MOI.modify(model, c, MOI.Utilities.ScalarPenaltyRelaxation(2.0));

julia> print(model)
Minimize ScalarAffineFunction{Float64}:
 0.0 + 2.0 v[2]

Subject to:

ScalarAffineFunction{Float64}-in-LessThan{Float64}
 0.0 + 1.0 v[1] - 1.0 v[2] <= 2.0

VariableIndex-in-GreaterThan{Float64}
 v[2] >= 0.0

julia> f isa MOI.ScalarAffineFunction{Float64}
true
```

source

### MatrixOfConstraints

`MathOptInterface.Utilities.MatrixOfConstraints` – Type.

```
mutable struct MatrixOfConstraints{T,AT,BT,ST} <: MOI.ModelLike
    coefficients::AT
    constants::BT
```

```
    sets::ST
    caches::Vector{Any}
    are_indices_mapped::Vector{BitSet}
    final_touch::Bool
end
```

Represent `ScalarAffineFunction` and `VectorAffinefunction` constraints in a matrix form where the linear coefficients of the functions are stored in the `coefficients` field, the constants of the functions or sets are stored in the `constants` field. Additional information about the sets are stored in the `sets` field.

This model can only be used as the `constraints` field of a `MOI.Utilities.AbstractModel`.

When the constraints are added, they are stored in the `caches` field. They are only loaded in the `coefficients` and `constants` fields once `MOI.Utilities.final_touch` is called. For this reason, `MatrixOfConstraints` should not be used by an incremental interface. Use `MOI.copy_to` instead.

The constraints can be added in two different ways:

1. With `add_constraint`, in which case a canonicalized copy of the function is stored in `caches`.
2. With `pass_nonvariable_constraints`, in which case the functions and sets are stored themselves in `caches` without mapping the variable indices. The corresponding index in `caches` is added in `are_indices_mapped`. This avoids doing a copy of the function in case the getter of `CanonicalConstraintFunction` does not make a copy for the source model, for example, this is the case of `VectorOfConstraints`.

We illustrate this with an example. Suppose a model is copied from a `src::MOI.Utilities.Model` to a bridged model with a `MatrixOfConstraints`. For all the types that are not bridged, the constraints will be copied with `pass_nonvariable_constraints`. Hence the functions stored in `caches` are exactly the same as the ones stored in `src`. This is ok since this is only during the `copy_to` operation during which `src` cannot be modified. On the other hand, for the types that are bridged, the functions added may contain duplicates even if the functions did not contain duplicates in `src` so duplicates are removed with `MOI.Utilities.canonical`.

**Interface**

The `.coefficients::AT` type must implement:

- `AT()`
- `MOI.empty(::AT)!`
- `MOI.Utilities.add_column`
- `MOI.Utilities.set_number_of_rows`
- `MOI.Utilities.allocate_terms`
- `MOI.Utilities.load_terms`
- `MOI.Utilities.final_touch`

The `.constants::BT` type must implement:

- `BT()`
- `Base.empty!(::BT)`
- `Base.resize(::BT)`
- `MOI.Utilities.load_constants`

- MOI.Utilities.function_constants
- MOI.Utilities.set_from_constants

The .sets::ST type must implement:

- ST()
- MOI.is_empty(::ST)
- MOI.empty(::ST)
- MOI.dimension(::ST)
- MOI.is_valid(::ST, ::MOI.ConstraintIndex)
- MOI.get(::ST, ::MOI.ListOfConstraintTypesPresent)
- MOI.get(::ST, ::MOI.NumberOfConstraints)
- MOI.get(::ST, ::MOI.ListOfConstraintIndices)
- MOI.Utilities.set_types
- MOI.Utilities.set_index
- MOI.Utilities.add_set
- MOI.Utilities.rows
- MOI.Utilities.final_touch

source

**.coefficients**

MathOptInterface.Utilities.add_column – Function.

```
add_column(coefficients)::Nothing
```

Tell coefficients to pre-allocate datastructures as needed to store one column.

source

MathOptInterface.Utilities.allocate_terms – Function.

```
allocate_terms(coefficients, index_map, func)::Nothing
```

Tell coefficients that the terms of the function func where the variable indices are mapped with index_map will be loaded with load_terms.

The function func must be canonicalized before calling allocate_terms. See is_canonical.

source

MathOptInterface.Utilities.set_number_of_rows – Function.

```
set_number_of_rows(coefficients, n)::Nothing
```

Tell coefficients to pre-allocate datastructures as needed to store n rows.

source

`MathOptInterface.Utilities.load_terms` – Function.

```
load_terms(coefficients, index_map, func, offset)::Nothing
```

Loads the terms of `func` to `coefficients`, mapping the variable indices with `index_map`.

The `i`th dimension of `func` is loaded at the `(offset + i)`th row of `coefficients`.

The function must be allocated first with `allocate_terms`.

The function `func` must be canonicalized, see `is_canonical`.

source

`MathOptInterface.Utilities.final_touch` – Function.

```
final_touch(coefficients)::Nothing
```

Informs the `coefficients` that all functions have been added with `load_terms`. No more modification is allowed unless `MOI.empty!` is called.

```
final_touch(sets)::Nothing
```

Informs the `sets` that all functions have been added with `add_set`. No more modification is allowed unless `MOI.empty!` is called.

source

`MathOptInterface.Utilities.extract_function` – Function.

```
extract_function(coefficients, row::Integer, constant::T) where {T}
```

Return the `MOI.ScalarAffineFunction{T}` function corresponding to row `row` in `coefficients`.

```
extract_function(
    coefficients,
    rows::UnitRange,
    constants::Vector{T},
) where{T}
```

Return the `MOI.VectorAffineFunction{T}` function corresponding to rows `rows` in `coefficients`.

source

`MathOptInterface.Utilities.MutableSparseMatrixCSC` – Type.

```
mutable struct MutableSparseMatrixCSC{Tv,Ti<:Integer,I<:AbstractIndexing}
    indexing::I
    m::Int
    n::Int
    colptr::Vector{Ti}
    rowval::Vector{Ti}
    nzval::Vector{Tv}
    nz_added::Vector{Ti}
end
```

Matrix type loading sparse matrices in the Compressed Sparse Column format. The indexing used is `indexing`, see `AbstractIndexing`. The other fields have the same meaning than for `SparseArrays.SparseMatrixCSC` except that the indexing is different unless `indexing` is `OneBasedIndexing`. In addition, `nz_added` is used to cache the number of non-zero terms that have been added to each column due to the incremental nature of `load_terms`.

The matrix is loaded in 5 steps:

1. `MOI.empty!` is called.
2. `MOI.Utilities.add_column` and `MOI.Utilities.allocate_terms` are called in any order.
3. `MOI.Utilities.set_number_of_rows` is called.
4. `MOI.Utilities.load_terms` is called for each affine function.
5. `MOI.Utilities.final_touch` is called.

source

`MathOptInterface.Utilities.AbstractIndexing` – Type.

```
abstract type AbstractIndexing end
```

Indexing to be used for storing the row and column indices of `MutableSparseMatrixCSC`. See `ZeroBasedIndexing` and `OneBasedIndexing`.

source

`MathOptInterface.Utilities.ZeroBasedIndexing` – Type.

```
struct ZeroBasedIndexing <: AbstractIndexing end
```

Zero-based indexing: the `i`th row or column has index `i - 1`. This is useful when the vectors of row and column indices need to be communicated to a library using zero-based indexing such as C libraries.

source

`MathOptInterface.Utilities.OneBasedIndexing` – Type.

```
struct ZeroBasedIndexing <: AbstractIndexing end
```

One-based indexing: the `i`th row or column has index `i`. This enables an allocation-free conversion of `MutableSparseMatrixCSC` to `SparseArrays.SparseMatrixCSC`.

source

**.constants**

MathOptInterface.Utilities.load_constants – Function.

```
load_constants(constants, offset, func_or_set)::Nothing
```

This function loads the constants of `func_or_set` in `constants` at an offset of `offset`. Where `offset` is the sum of the dimensions of the constraints already loaded. The storage should be preallocated with `resize!` before calling this function.

This function should be implemented to be usable as storage of constants for `MatrixOfConstraints`.

The constants are loaded in three steps:

1. `Base.empty!` is called.
2. `Base.resize!` is called with the sum of the dimensions of all constraints.
3. `MOI.Utilities.load_constants` is called for each function for vector constraint or set for scalar constraint.

source

MathOptInterface.Utilities.function_constants – Function.

```
function_constants(constants, rows)
```

This function returns the function constants that were loaded with `load_constants` at the rows `rows`.

This function should be implemented to be usable as storage of constants for `MatrixOfConstraints`.

source

MathOptInterface.Utilities.set_from_constants – Function.

```
set_from_constants(constants, S::Type, rows)::S
```

This function returns an instance of the set `S` for which the constants where loaded with `load_constants` at the rows `rows`.

This function should be implemented to be usable as storage of constants for `MatrixOfConstraints`.

source

MathOptInterface.Utilities.modify_constants – Function.

```
modify_constants(constants, row::Integer, new_constant::T) where {T}
modify_constants(
    constants,
    rows::AbstractVector{<:Integer},
    new_constants::AbstractVector{T},
) where {T}
```

Modify `constants` in-place to store `new_constant` in the `row` row, or rows `rows`.

This function must be implemented to enable `MOI.ScalarConstantChange` and `MOI.VectorConstantChange` for `MatrixOfConstraints`.

source

`MathOptInterface.Utilities.Hyperrectangle` – Type.

```
struct Hyperrectangle{T} <: AbstractVectorBounds
    lower::Vector{T}
    upper::Vector{T}
end
```

A struct for the .constants field in MatrixOfConstraints.

source

**`.sets`**

`MathOptInterface.Utilities.set_index` – Function.

```
set_index(sets, ::Type{S})::Union{Int,Nothing} where {S<:MOI.AbstractSet}
```

Return an integer corresponding to the index of the set type in the list given by `set_types`.

If S is not part of the list, return `nothing`.

source

`MathOptInterface.Utilities.set_types` – Function.

```
set_types(sets)::Vector{Type}
```

Return the list of the types of the sets allowed in `sets`.

source

`MathOptInterface.Utilities.add_set` – Function.

```
add_set(sets, i::Int)::Int64
```

Add a scalar set of type index `i`.

```
add_set(sets, i::Int, dim::Int)::Int64
```

Add a vector set of type index `i` and dimension `dim`.

Both methods return a unique `Int64` of the set that can be used to reference this set.

source

`MathOptInterface.Utilities.rows` – Function.

```
rows(sets, ci::MOI.ConstraintIndex)::Union{Int,UnitRange{Int}}
```

Return the rows in `1:MOI.dimension(sets)` corresponding to the set of id `ci.value`.

For scalar sets, this returns an `Int`. For vector sets, this returns an `UnitRange{Int}`.

source

`MathOptInterface.Utilities.num_rows` – Function.

```
num_rows(sets::OrderedProductOfSets, ::Type{S}) where {S}
```

Return the number of rows corresponding to a set of type `S`. That is, it is the sum of the dimensions of the sets of type `S`.

source

`MathOptInterface.Utilities.set_with_dimension` – Function.

```
set_with_dimension(::Type{S}, dim) where {S<:MOI.AbstractVectorSet}
```

Returns the instance of `S` of `MOI.dimension` `dim`. This needs to be implemented for sets of type `S` to be usable with `MatrixOfConstraints`.

source

`MathOptInterface.Utilities.ProductOfSets` – Type.

```
abstract type ProductOfSets{T} end
```

Represents a cartesian product of sets of given types.

source

`MathOptInterface.Utilities.MixOfScalarSets` – Type.

```
abstract type MixOfScalarSets{T} <: ProductOfSets{T} end
```

Product of scalar sets in the order the constraints are added, mixing the constraints of different types.

Use `@mix_of_scalar_sets` to generate a new subtype.

source

`MathOptInterface.Utilities.@mix_of_scalar_sets` – Macro.

```
@mix_of_scalar_sets(name, set_types...)
```

Generate a new `MixOfScalarSets` subtype.

**Example**

```
julia> MOI.Utilities.@mix_of_scalar_sets(
           MixedIntegerLinearProgramSets,
           MOI.GreaterThan{T},
           MOI.LessThan{T},
           MOI.EqualTo{T},
           MOI.Integer,
       )
```

source

`MathOptInterface.Utilities.OrderedProductOfSets` – Type.

```
abstract type OrderedProductOfSets{T} <: ProductOfSets{T} end
```

Product of sets in the order the constraints are added, grouping the constraints of the same types contiguously.

Use `@product_of_sets` to generate new subtypes.

source

`MathOptInterface.Utilities.@product_of_sets` – Macro.

```
@product_of_sets(name, set_types...)
```

Generate a new `OrderedProductOfSets` subtype.

**Example**

```
julia> MOI.Utilities.@product_of_sets(
           LinearOrthants,
           MOI.Zeros,
           MOI.Nonnegatives,
           MOI.Nonpositives,
           MOI.ZeroOne,
       )
```

source

`MathOptInterface.Utilities.RuntimeProductOfSets` – Type.

```
RuntimeProductOfSets{T}()
```

An alternative to `Utilities.@product_of_sets`, in which the set types are decided at runtime using `Utilities.add_set_type`.

**Examples**

```
julia> import MathOptInterface as MOI

julia> sets = MOI.Utilities.RuntimeProductOfSets{Int}();

julia> MOI.Utilities.set_types(sets)
Type[]

julia> MOI.Utilities.add_set_type(sets, MOI.Nonpositives)

julia> MOI.Utilities.set_types(sets)
1-element Vector{Type}:
 MathOptInterface.Nonpositives
```

source

`MathOptInterface.Utilities.add_set_type` – Function.

```
add_set_type(
    set::RuntimeProductOfSets,
    ::Type{S},
) where {S<:MOI.AbstractSet}
```

Declare that the `Utilities.RuntimeProductOfSets` set supports the `MOI.AbstractSet` S.

**Examples**

```
julia> import MathOptInterface as MOI

julia> sets = MOI.Utilities.RuntimeProductOfSets{Int}();

julia> MOI.Utilities.set_types(sets)
Type[]

julia> MOI.Utilities.add_set_type(sets, MOI.Nonpositives)

julia> MOI.Utilities.set_types(sets)
1-element Vector{Type}:
 MathOptInterface.Nonpositives
```

source

### Fallbacks

`MathOptInterface.Utilities.get_fallback` – Function.

```
get_fallback(model::MOI.ModelLike, ::MOI.ObjectiveValue)
```

Compute the objective function value using the `VariablePrimal` results and the `ObjectiveFunction` value.

source

```
get_fallback(
    model::MOI.ModelLike,
    ::MOI.DualObjectiveValue,
    ::Type{T},
)::T where {T}
```

Compute the dual objective value of type `T` using the `ConstraintDual` results and the `ConstraintFunction` and `ConstraintSet` values.

Note that the nonlinear part of the model is ignored.

source

```
get_fallback(
    model::MOI.ModelLike,
    ::MOI.ConstraintPrimal,
    constraint_index::MOI.ConstraintIndex,
)
```

Compute the value of the function of the constraint of index `constraint_index` using the `VariablePrimal` results and the `ConstraintFunction` values.

source

```
get_fallback(
    model::MOI.ModelLike,
    attr::MOI.ConstraintDual,
    ci::MOI.ConstraintIndex{Union{MOI.VariableIndex,MOI.VectorOfVariables}},
    ::Type{T} = Float64,
) where {T}
```

Compute the dual of the constraint of index `ci` using the `ConstraintDual` of other constraints and the `ConstraintFunction` values.

Throws an error if some constraints are quadratic or if there is one another `MOI.VariableIndex`-in-`S` or `MOI.VectorOfVariables`-in-`S` constraint with one of the variables in the function of the constraint `ci`.

source

### Function utilities

The following utilities are available for functions:

`MathOptInterface.Utilities.eval_variables` – Function.

```
eval_variables(value_fn::Function, f::MOI.AbstractFunction)
```

Returns the value of function `f` if each variable index `vi` is evaluated as `value_fn(vi)`.

Note that `value_fn` must return a Number. See `substitute_variables` for a similar function where `value_fn` returns an `MOI.AbstractScalarFunction`.

**Warning**

The two-argument version of `eval_variables` is deprecated and may be removed in MOI v2.0.0. Use the three-argument method `eval_variables(::Function, ::MOI.ModelLike, ::MOI.AbstractFunction)` instead.

source

`MathOptInterface.Utilities.map_indices` – Function.

```
map_indices(index_map::Function, attr::MOI.AnyAttribute, x::X)::X where {X}
```

Substitute any `MOI.VariableIndex` (resp. `MOI.ConstraintIndex`) in `x` by the `MOI.VariableIndex` (resp. `MOI.ConstraintIndex`) of the same type given by `index_map(x)`.

**When to implement this method for new types `X`**

This function is used by implementations of `MOI.copy_to` on constraint functions, attribute values and submittable values. If you define a new attribute whose values `x::X` contain variable or constraint indices, you must also implement this function.

source

```
map_indices(
    variable_map::AbstractDict{T,T},
    x::X,
)::X where {T<:MOI.Index,X}
```

Shortcut for `map_indices(vi -> variable_map[vi], x)`.

**Note**

The returned value is not guaranteed to be a copy of `x`. For example, if `variable_map` is an `IndexMap` whose variable map is the identity, `x` may be returned without being copied. Therefore, it should not be modified unless `x` is not used anymore.

source

`MathOptInterface.Utilities.substitute_variables` – Function.

```
substitute_variables(variable_map::Function, x)
```

Substitute any `MOI.VariableIndex` in `x` by `variable_map(x)`. The `variable_map` function returns either `MOI.VariableIndex` or `MOI.ScalarAffineFunction`, see `eval_variables` for a similar function where `variable_map` returns a number.

This function is used by bridge optimizers on constraint functions, attribute values and submittable values when at least one variable bridge is used hence it needs to be implemented for custom types that are meant to be used as attribute or submittable value.

**Note**

When implementing a new method, don't use `substitute_variables(::Function`, because Julia will not specialize on it. Use instead `substitute_variables(::F, ...) where {F<:Function}`.

source

`MathOptInterface.Utilities.filter_variables` – Function.

```
filter_variables(keep::Function, f::AbstractFunction)
```

Return a new function `f` with the variable `vi` such that `!keep(vi)` removed.

WARNING: Don't define `filter_variables(::Function, ...)` because Julia will not specialize on this. Define instead `filter_variables(::F, ...) where {F<:Function}`.

source

`MathOptInterface.Utilities.remove_variable` – Function.

```
remove_variable(f::AbstractFunction, vi::VariableIndex)
```

Return a new function `f` with the variable `vi` removed.

source

```
remove_variable(
    f::MOI.AbstractFunction,
    s::MOI.AbstractSet,
    vi::MOI.VariableIndex,
)
```

Return a tuple `(g, t)` representing the constraint `f`-in-`s` with the variable `vi` removed. That is, the terms containing the variable `vi` in the function `f` are removed and the dimension of the set `s` is updated if needed (for example, when `f` is a `VectorOfVariables` with `vi` being one of the variables).

source

`MathOptInterface.Utilities.all_coefficients` – Function.

```
all_coefficients(p::Function, f::MOI.AbstractFunction)
```

Determine whether predicate `p` returns `true` for all coefficients of `f`, returning `false` as soon as the first coefficient of `f` for which `p` returns `false` is encountered (short-circuiting). Similar to `all`.

source

`MathOptInterface.Utilities.unsafe_add` – Function.

```
unsafe_add(t1::MOI.ScalarAffineTerm, t2::MOI.ScalarAffineTerm)
```

Sums the coefficients of `t1` and `t2` and returns an output `MOI.ScalarAffineTerm`. It is unsafe because it uses the `variable` of `t1` as the `variable` of the output without checking that it is equal to that of `t2`.

source

```
unsafe_add(t1::MOI.ScalarQuadraticTerm, t2::MOI.ScalarQuadraticTerm)
```

Sums the coefficients of `t1` and `t2` and returns an output `MOI.ScalarQuadraticTerm`. It is unsafe because it uses the `variable`'s of `t1` as the `variable`'s of the output without checking that they are the same (up to permutation) to those of `t2`.

source

```
unsafe_add(t1::MOI.VectorAffineTerm, t2::MOI.VectorAffineTerm)
```

Sums the coefficients of `t1` and `t2` and returns an output `MOI.VectorAffineTerm`. It is unsafe because it uses the `output_index` and `variable` of `t1` as the `output_index` and `variable` of the output term without checking that they are equal to those of `t2`.

source

`MathOptInterface.Utilities.isapprox_zero` – Function.

```
isapprox_zero(f::MOI.AbstractFunction, tol)
```

Return a `Bool` indicating whether the function `f` is approximately zero using `tol` as a tolerance.

**Important note**

This function assumes that `f` does not contain any duplicate terms, you might want to first call `canonical` if that is not guaranteed.

**Example**

```
julia> x = MOI.VariableIndex(1)
MOI.VariableIndex(1)

julia> f = MOI.ScalarAffineFunction(MOI.ScalarAffineTerm.([1, -1], [x, x]), 0)
```

```
(0) + (1) MOI.VariableIndex(1) - (1) MOI.VariableIndex(1)

julia> MOI.Utilities.isapprox_zero(f, 1e-8)
false

julia> MOI.Utilities.isapprox_zero(MOI.Utilities.canonical(f), 1e-8)
true
```

source

MathOptInterface.Utilities.modify_function – Function.

```
modify_function(f::AbstractFunction, change::AbstractFunctionModification)
```

Return a copy of the function `f`, modified according to change.

source

MathOptInterface.Utilities.zero_with_output_dimension – Function.

```
zero_with_output_dimension(::Type{T}, output_dimension::Integer) where {T}
```

Create an instance of type T with the output dimension `output_dimension`.

This is mostly useful in Bridges, when code needs to be agnostic to the type of vector-valued function that is passed in.

source

The following functions can be used to canonicalize a function:

MathOptInterface.Utilities.is_canonical – Function.

```
is_canonical(f::Union{ScalarAffineFunction, VectorAffineFunction})
```

Returns a Bool indicating whether the function is in canonical form. See canonical.

source

```
is_canonical(f::Union{ScalarQuadraticFunction, VectorQuadraticFunction})
```

Returns a Bool indicating whether the function is in canonical form. See canonical.

source

MathOptInterface.Utilities.canonical – Function.

```
canonical(f::MOI.AbstractFunction)
```

Returns the function in a canonical form, that is,

- A term appear only once.
- The coefficients are nonzero.
- The terms appear in increasing order of variable where there the order of the variables is the order of their value.
- For a `AbstractVectorFunction`, the terms are sorted in ascending order of output index.

The output of `canonical` can be assumed to be a copy of `f`, even for `VectorOfVariables`.

**Example**

```
julia> x, y, z = MOI.VariableIndex.(1:3);

julia> f = MOI.ScalarAffineFunction(
           MOI.ScalarAffineTerm.(Float64[2, 1, 3, -2, -3], [y, x, z, x, z]),
           5.0,
       )
5.0 + 2.0 MOI.VariableIndex(2) + 1.0 MOI.VariableIndex(1) + 3.0 MOI.VariableIndex(3) - 2.0
↪  MOI.VariableIndex(1) - 3.0 MOI.VariableIndex(3)

julia> MOI.Utilities.canonical(f)
5.0 - 1.0 MOI.VariableIndex(1) + 2.0 MOI.VariableIndex(2)
```

source

`MathOptInterface.Utilities.canonicalize!` – Function.

```
canonicalize!(f::Union{ScalarAffineFunction, VectorAffineFunction})
```

Convert a function to canonical form in-place, without allocating a copy to hold the result. See `canonical`.

source

```
canonicalize!(f::Union{ScalarQuadraticFunction, VectorQuadraticFunction})
```

Convert a function to canonical form in-place, without allocating a copy to hold the result. See `canonical`.

source

The following functions can be used to manipulate functions with basic algebra:

`MathOptInterface.Utilities.scalar_type` – Function.

```
scalar_type(F::Type{<:MOI.AbstractVectorFunction})
```

Type of functions obtained by indexing objects obtained by calling `eachscalar` on functions of type `F`.

source

`MathOptInterface.Utilities.scalarize` – Function.

```
scalarize(func::MOI.VectorOfVariables, ignore_constants::Bool = false)
```

Returns a vector of scalar functions making up the vector function in the form of a `Vector{MOI.SingleVariable}`.

See also `eachscalar`.

source

```
scalarize(func::MOI.VectorAffineFunction{T}, ignore_constants::Bool = false)
```

Returns a vector of scalar functions making up the vector function in the form of a `Vector{MOI.ScalarAffineFunction{T}}`.

See also `eachscalar`.

source

```
scalarize(func::MOI.VectorQuadraticFunction{T}, ignore_constants::Bool = false)
```

Returns a vector of scalar functions making up the vector function in the form of a `Vector{MOI.ScalarQuadraticFunction{T}`

See also `eachscalar`.

source

`MathOptInterface.Utilities.eachscalar` – Function.

```
eachscalar(f::MOI.AbstractVectorFunction)
```

Returns an iterator for the scalar components of the vector function.

See also `scalarize`.

source

```
eachscalar(f::MOI.AbstractVector)
```

Returns an iterator for the scalar components of the vector.

source

`MathOptInterface.Utilities.promote_operation` – Function.

```
promote_operation(
    op::Function,
    ::Type{T},
    ArgsTypes::Type{<:Union{T,AbstractVector{T},MOI.AbstractFunction}}...,
) where {T<:Number}
```

Compute the return type of the call `operate(op, T, args...)`, where the types of the arguments `args` are `ArgsTypes`.

One assumption is that the element type T is invariant under each operation. That is, `op(::T, ::T)::T` where op is a +, -, *, and /.

There are six methods for which we implement `Utilities.promote_operation`:

1. + a. `promote_operation(::typeof(+), ::Type{T}, ::Type{F1}, ::Type{F2})`
2. - a. `promote_operation(::typeof(-), ::Type{T}, ::Type{F})` b. `promote_operation(::typeof(-), ::Type{T}, ::Type{F1}, ::Type{F2})`
3. * a. `promote_operation(::typeof(*), ::Type{T}, ::Type{T}, ::Type{F})` b. `promote_operation(::typeof(*), ::Type{T}, ::Type{F}, ::Type{T})` c. `promote_operation(::typeof(*), ::Type{T}, ::Type{F1}, ::Type{F2})` where F1 and F2 are `VariableIndex` or `ScalarAffineFunction` d. `promote_operation(::typeof(*), ::Type{T}, ::Type{<:Diagonal{T}}, ::Type{F}`
4. / a. `promote_operation(::typeof(/), ::Type{T}, ::Type{F}, ::Type{T})`
5. vcat a. `promote_operation(::typeof(vcat), ::Type{T}, ::Type{F}...)`
6. imag a. `promote_operation(::typeof(imag), ::Type{T}, ::Type{F})` where F is `VariableIndex` or `VectorOfVariables`

In each case, F (or F1 and F2) is one of the ten supported types, with a restriction that the mathematical operation makes sense, for example, we don't define `promote_operation(-, T, F1, F2)` where F1 is a scalar-valued function and F2 is a vector-valued function. The ten supported types are:

1. ::T
2. ::VariableIndex
3. ::ScalarAffineFunction{T}
4. ::ScalarQuadraticFunction{T}
5. ::ScalarNonlinearFunction
6. ::AbstractVector{T}
7. ::VectorOfVariables
8. ::VectorAffineFunction{T}
9. ::VectorQuadraticFunction{T}
10. ::VectorNonlinearFunction

source

`MathOptInterface.Utilities.operate` – Function.

```
operate(
    op::Function,
    ::Type{T},
    args::Union{T,MOI.AbstractFunction}...,
)::MOI.AbstractFunction where {T<:Number}
```

Returns an `MOI.AbstractFunction` representing the function resulting from the operation `op(args...)` on functions of coefficient type T.

No argument can be modified.

**Methods**

1. `+` a. `operate(::typeof(+), ::Type{T}, ::F1)` b. `operate(::typeof(+), ::Type{T}, ::F1, ::F2)` c. `operate(::typeof(+), ::Type{T}, ::F1...)`
2. `-` a. `operate(::typeof(-), ::Type{T}, ::F)` b. `operate(::typeof(-), ::Type{T}, ::F1, ::F2)`
3. `*` a. `operate(::typeof(*), ::Type{T}, ::T, ::F)` b. `operate(::typeof(*), ::Type{T}, ::F, ::T)` c. `operate(::typeof(*), ::Type{T}, ::F1, ::F2)` where F1 and F2 are `VariableIndex` or `ScalarAffineFunction` d. `operate(::typeof(*), ::Type{T}, ::Diagonal{T}, ::F)`
4. `/` a. `operate(::typeof(/), ::Type{T}, ::F, ::T)`
5. `vcat` a. `operate(::typeof(vcat), ::Type{T}, ::F...)`
6. `imag` a. `operate(::typeof(imag), ::Type{T}, ::F)` where F is `VariableIndex` or `VectorOfVariables`

One assumption is that the element type T is invariant under each operation. That is, `op(::T, ::T)::T` where op is a `+`, `-`, `*`, and `/`.

In each case, F (or F1 and F2) is one of the ten supported types, with a restriction that the mathematical operation makes sense, for example, we don't define `promote_operation(-, T, F1, F2)` where F1 is a scalar-valued function and F2 is a vector-valued function. The ten supported types are:

1. ::T
2. ::VariableIndex
3. ::ScalarAffineFunction{T}
4. ::ScalarQuadraticFunction{T}
5. ::ScalarNonlinearFunction
6. ::AbstractVector{T}
7. ::VectorOfVariables
8. ::VectorAffineFunction{T}
9. ::VectorQuadraticFunction{T}
10. ::VectorNonlinearFunction

source

`MathOptInterface.Utilities.operate!` – Function.

```
operate!(
    op::Function,
    ::Type{T},
    args::Union{T,MOI.AbstractFunction}...,
)::MOI.AbstractFunction where {T<:Number}
```

Returns an `MOI.AbstractFunction` representing the function resulting from the operation `op(args...)` on functions of coefficient type T.

The first argument may be modified, in which case the return value is identical to the first argument. For operations which cannot be implemented in-place, this function returns a new object.

source

`MathOptInterface.Utilities.operate_output_index!` – Function.

```
operate_output_index!(
    op::Union{typeof(+),typeof(-)},
    ::Type{T},
    output_index::Integer,
    f::Union{AbstractVector{T},MOI.AbstractVectorFunction}
    g::Union{T,MOI.AbstractScalarFunction}...
) where {T<:Number}
```

Return an `MOI.AbstractVectorFunction` in which the scalar function in row `output_index` is the result of `op(f[output_index], g)`.

The functions at output index different to `output_index` are the same as the functions at the same output index in `func`. The first argument may be modified.

**Methods**

1. `+` a. `operate_output_index!(+, ::Type{T}, ::Int, ::VectorF, ::ScalarF)`
2. `-` a. `operate_output_index!(-, ::Type{T}, ::Int, ::VectorF, ::ScalarF)`

source

`MathOptInterface.Utilities.vectorize` – Function.

```
vectorize(x::AbstractVector{<:Number})
```

Returns x.

source

```
vectorize(x::AbstractVector{MOI.VariableIndex})
```

Returns the vector of scalar affine functions in the form of a `MOI.VectorAffineFunction{T}`.

source

```
vectorize(funcs::AbstractVector{MOI.ScalarAffineFunction{T}}) where T
```

Returns the vector of scalar affine functions in the form of a `MOI.VectorAffineFunction{T}`.

source

```
vectorize(funcs::AbstractVector{MOI.ScalarQuadraticFunction{T}}) where T
```

Returns the vector of scalar quadratic functions in the form of a `MOI.VectorQuadraticFunction{T}`.

source

### Constraint utilities

The following utilities are available for moving the function constant to the set for scalar constraints:

`MathOptInterface.Utilities.shift_constant` – Function.

```
shift_constant(set::MOI.AbstractScalarSet, offset)
```

Returns a new scalar set `new_set` such that `func-in-set` is equivalent to `func + offset-in-new_set`.

Use `supports_shift_constant` to check if the set supports shifting:

```
if MOI.Utilities.supports_shift_constant(typeof(set))
    new_set = MOI.Utilities.shift_constant(set, -func.constant)
    func.constant = 0
    MOI.add_constraint(model, func, new_set)
else
    MOI.add_constraint(model, func, set)
end
```

#### Note for developers

Only define this function if it makes sense and you have implemented `supports_shift_constant` to return `true`.

#### Example

```
julia> set = MOI.Interval(-2.0, 3.0)
MathOptInterface.Interval{Float64}(-2.0, 3.0)

julia> MOI.Utilities.supports_shift_constant(typeof(set))
true

julia> MOI.Utilities.shift_constant(set, 1.0)
MathOptInterface.Interval{Float64}(-1.0, 4.0)
```

source

`MathOptInterface.Utilities.supports_shift_constant` – Function.

```
supports_shift_constant(::Type{S}) where {S<:MOI.AbstractSet}
```

Return `true` if `shift_constant` is defined for set `S`.

See also `shift_constant`.

#### Example

```
julia> MOI.Utilities.supports_shift_constant(MOI.Interval{Float64})
true

julia> MOI.Utilities.supports_shift_constant(MOI.ZeroOne)
false
```

source

MathOptInterface.Utilities.normalize_and_add_constraint – Function.

```
normalize_and_add_constraint(
    model::MOI.ModelLike,
    func::MOI.AbstractScalarFunction,
    set::MOI.AbstractScalarSet;
    allow_modify_function::Bool = false,
)
```

Adds the scalar constraint obtained by moving the constant term in `func` to the set in `model`. If `allow_modify_function` is `true` then the function `func` can be modified.

source

MathOptInterface.Utilities.normalize_constant – Function.

```
normalize_constant(
    func::MOI.AbstractScalarFunction,
    set::MOI.AbstractScalarSet;
    allow_modify_function::Bool = false,
)
```

Return the `func`-in-`set` constraint in normalized form. That is, if `func` is `MOI.ScalarQuadraticFunction` or `MOI.ScalarAffineFunction`, the constant is moved to the set. If `allow_modify_function` is `true` then the function `func` can be modified.

source

The following utility identifies those constraints imposing bounds on a given variable, and returns those bound values:

MathOptInterface.Utilities.get_bounds – Function.

```
get_bounds(model::MOI.ModelLike, ::Type{T}, x::MOI.VariableIndex)
```

Return a tuple `(lb, ub)` of type `Tuple{T, T}`, where `lb` and `ub` are lower and upper bounds, respectively, imposed on `x` in `model`.

source

```
get_bounds(
    model::MOI.ModelLike,
    bounds_cache::Dict{MOI.VariableIndex,NTuple{2,T}},
    f::MOI.ScalarAffineFunction{T},
) where {T} --> Union{Nothing,NTuple{2,T}}
```

Return the lower and upper bound of `f` as a tuple. If the domain is not bounded, return `nothing`.

source

```
get_bounds(
    model::MOI.ModelLike,
    bounds_cache::Dict{MOI.VariableIndex,NTuple{2,T}},
    x::MOI.VariableIndex,
) where {T} --> Union{Nothing,NTuple{2,T}}
```

Return the lower and upper bound of `x` as a tuple. If the domain is not bounded, return `nothing`.

Similar to `get_bounds(::MOI.ModelLike, ::Type{T}, ::MOI.VariableIndex)`, except that the second argument is a cache which maps variables to their bounds and avoids repeated lookups.

source

The following utilities are useful when working with symmetric matrix cones.

`MathOptInterface.Utilities.is_diagonal_vectorized_index` – Function.

```
is_diagonal_vectorized_index(index::Base.Integer)
```

Return whether `index` is the index of a diagonal element in a `MOI.AbstractSymmetricMatrixSetTriangle` set.

source

`MathOptInterface.Utilities.side_dimension_for_vectorized_dimension` – Function.

```
side_dimension_for_vectorized_dimension(n::Integer)
```

Return the dimension `d` such that `MOI.dimension(MOI.PositiveSemidefiniteConeTriangle(d))` is `n`.

source

**Set utilities**

The following utilities are available for sets:

`MathOptInterface.Utilities.AbstractDistance` – Type.

```
abstract type AbstractDistance end
```

An abstract type used to enable dispatch of `Utilities.distance_to_set`.

source

`MathOptInterface.Utilities.ProjectionUpperBoundDistance` – Type.

```
ProjectionUpperBoundDistance() <: AbstractDistance
```

An upper bound on the minimum distance between `point` and the closest feasible point in `set`.

**Definition of distance**

The minimum distance is computed as:

$$d(x, \mathcal{K}) = \min_{y \in \mathcal{K}} ||x - y||$$

where $x$ is `point` and $\mathcal{K}$ is `set`. The norm is computed as:

$$||x|| = \sqrt{f(x, x, \mathcal{K})}$$

where $f$ is `Utilities.set_dot`.

In the default case, where the set does not have a specialized method for `Utilities.set_dot`, the norm is equivalent to the Euclidean norm $||x|| = \sqrt{\sum x_i^2}$.

**Why an upper bound?**

In most cases, `distance_to_set` should return the smallest upper bound, but it may return a larger value if the smallest upper bound is expensive to compute.

For example, given an epigraph from of a conic set, $\{(t, x) | f(x) \leq t\}$, it may be simpler to return $\delta$ such that $f(x) \leq t + \delta$, rather than computing the nearest projection onto the set.

If the distance is not the smallest upper bound, the docstring of the appropriate `distance_to_set` method must describe the way that the distance is computed.

source

`MathOptInterface.Utilities.distance_to_set` – Function.

```
distance_to_set(
    [d::AbstractDistance = ProjectionUpperBoundDistance()],]
    point::T,
    set::MOI.AbstractScalarSet,
) where {T}

distance_to_set(
    [d::AbstractDistance = ProjectionUpperBoundDistance(),]
    point::AbstractVector{T},
    set::MOI.AbstractVectorSet,
) where {T}
```

Compute the distance between `point` and `set` using the distance metric `d`. If `point` is in the set `set`, this function must return `zero(T)`.

If `d` is omitted, the default distance is `Utilities.ProjectionUpperBoundDistance`.

source

```
distance_to_set(::ProjectionUpperBoundDistance, x, ::MOI.RotatedSecondOrderCone)
```

Let `(t, u, y...) = x`. Return the 2-norm of the vector `d` such that in `x + d`, `u` is projected to 1 if `u <= 0`, and `t` is increased such that `x + d` belongs to the set.

source

```
distance_to_set(::ProjectionUpperBoundDistance, x, ::MOI.ExponentialCone)
```

Let `(u, v, w) = x`. If `v > 0`, return the epigraph distance `d` such that `(u, v, w + d)` belongs to the set.

If `v <= 0` return the 2-norm of the vector `d` such that `x + d = (u, 1, z)` where `z` satisfies the constraints.

source

```
distance_to_set(::ProjectionUpperBoundDistance, x, ::MOI.DualExponentialCone)
```

Let `(u, v, w) = x`. If `u < 0`, return the epigraph distance `d` such that `(u, v, w + d)` belongs to the set.

If `u >= 0` return the 2-norm of the vector `d` such that `x + d = (-1, v, z)` where `z` satisfies the constraints.

source

```
distance_to_set(::ProjectionUpperBoundDistance, x, ::MOI.GeometricMeanCone)
```

Let `(t, y...) = x`. If all `y` are non-negative, return the hypograph distance `d` such that `(t - d, y...)` belongs to the set.

If any `y` are strictly negative, return the 2-norm of the vector `d` that projects negative `y` elements to `0` and `t` to $\mathbb{R}_-$.

source

```
distance_to_set(::ProjectionUpperBoundDistance, x, ::MOI.DualGeometricMeanCone)
```

Let `(t, y...) = x`. If all `y` are non-negative, return the absolute value of `d` such that `(t + d, y...)` belongs to the set.

If any `y` are strictly negative, return the 2-norm of the vector `d` that projects negative `y` elements to `0` and `t` to `0`.

source

```
distance_to_set(::ProjectionUpperBoundDistance, x, ::MOI.PowerCone)
```

Let `(a, b, c) = x`. If `a` and `b` are non-negative, return the epigraph distance required to increase `c` such that the constraint is satisfied.

If `a` or `b` is strictly negative, return the 2-norm of the vector `d` such that in the vector `x + d`: `c`, and any negative `a` and `b` are projected to `0`.

source

```
distance_to_set(::ProjectionUpperBoundDistance, x, ::MOI.DualPowerCone)
```

Let `(a, b, c) = x`. If `a` and `b` are non-negative, return the epigraph distance required to increase `c` such that the constraint is satisfied.

If `a` or `b` is strictly negative, return the 2-norm of the vector `d` such that in the vector `x + d`: `c`, and any negative `a` and `b` are projected to `0`.

source

```
distance_to_set(::ProjectionUpperBoundDistance, x, ::MOI.NormOneCone)
```

Let `(t, y...) = x`. Return the epigraph distance `d` such that `(t + d, y...)` belongs to the set.

source

```
distance_to_set(::ProjectionUpperBoundDistance, x, ::MOI.NormInfinityCone)
```

Let `(t, y...) = x`. Return the epigraph distance `d` such that `(t + d, y...)` belongs to the set.

source

```
distance_to_set(::ProjectionUpperBoundDistance, x, ::MOI.RelativeEntropyCone)
```

Let `(u, v, w) = x`. If `v` and `w` are strictly positive, return the epigraph distance required to increase `u` such that the constraint is satisfied.

If any elements in `v` or `w` are non-positive, return the 2-norm of the vector `d` such that in the vector `x + d`: any non-positive elements in `v` and `w` are projected to 1, and `u` is projected such that the epigraph constraint holds.

source

```
distance_to_set(::ProjectionUpperBoundDistance, x, ::MOI.DualRelativeEntropyCone)
```

Let `(u, v, w) = x`. If `u` is strictly positive, return the 2-norm of the vector `d` such `(u, v + d, w)` satisfies the constraints.

If `u` is non-positive return the 2-norm of the vector `d` such that `x + d = (1, z, w)` and `z` is such that the constraints are satisfied.

source

```
distance_to_set(::ProjectionUpperBoundDistance, x, set::MOI.NormCone)
```

Let `(t, y...) = x`. Return the epigraph distance `d` such that `(t + d, y...)` belongs to the set.

source

```
distance_to_set(
    ::ProjectionUpperBoundDistance,
    x::AbstractVector,
    set::Union{
        MOI.PositiveSemidefiniteConeSquare,
        MOI.PositiveSemidefiniteConeTriangle,
    },
)
```

Let $X$ be x reshaped into the appropriate matrix. The returned distance is $||X - Y||_2^2$ where $Y$ is the eigen decomposition of $X$ with negative eigen values removed.

source

```
distance_to_set(
    ::ProjectionUpperBoundDistance,
    x::AbstractVector{T},
    set::MOI.VectorNonlinearOracle,
) where {T<:Real}
```

**Warning**

This is not an upper bound on the projection in x space; instead, it is a projection in `f(x)`-space.

source

```
distance_to_set(::ProjectionUpperBoundDistance, x, set::MOI.NormNuclearCone)
```

Let `(t, y...) = x`. Return the epigraph distance d such that `(t + d, y...)` belongs to the set.

source

```
distance_to_set(::ProjectionUpperBoundDistance, x, set::MOI.NormSpectralCone)
```

Let `(t, y...) = x`. Return the epigraph distance d such that `(t + d, y...)` belongs to the set.

source

```
distance_to_set(
    ::ProjectionUpperBoundDistance,
    x::AbstractVector,
    set::Union{MOI.RootDetConeSquare,MOI.RootDetConeTriangle},
)
```

Let $Y$ be y in `x = (t, y)`, reshaped into the appropriate matrix. The returned distance is $||Y - Z||_2^2$ where $Z$ is the eigen decomposition of $Y$ with negative eigen values removed, plus the epigraph distance in `t` needed to satisfy the root-determinant constraint.

source

```
distance_to_set(
    ::ProjectionUpperBoundDistance,
    x::AbstractVector,
    set::Union{MOI.LogDetConeSquare,MOI.LogDetConeTriangle},
)
```

Let $Y$ be y in `x = (t, y)`, reshaped into the appropriate matrix. The returned distance is $||Y/u - Z||_2^2$ where $Z$ is the eigen decomposition of $Y$ with negative eigen values removed, plus the epigraph distance in `t` needed to satisfy the log-determinant constraint.

source

```
distance_to_set(
    ::ProjectionUpperBoundDistance,
    x::AbstractVector,
    set::MOI.Scaled{S},
)
```

This is the distance in the un-scaled space.

source

`MathOptInterface.Utilities.set_dot` – Function.

```
set_dot(x::AbstractVector, y::AbstractVector, set::AbstractVectorSet)
```

Return the scalar product between a vector `x` of the set `set` and a vector `y` of the dual of the set `s`.

source

```
set_dot(x, y, set::AbstractScalarSet)
```

Return the scalar product between a number `x` of the set `set` and a number `y` of the dual of the set `s`.

source

#### DoubleDicts

`MathOptInterface.Utilities.DoubleDicts.DoubleDict` – Type.

```
DoubleDict{V}
```

An optimized dictionary to map `MOI.ConstraintIndex` to values of type `V`.

Works as a `AbstractDict{MOI.ConstraintIndex,V}` with minimal differences.

If `V` is also a `MOI.ConstraintIndex`, use `IndexDoubleDict`.

Note that `MOI.ConstraintIndex` is not a concrete type, opposed to `MOI.ConstraintIndex{MOI.VariableIndex, MOI.Integers}`, which is a concrete type.

When looping through multiple keys of the same Function-in-Set type, use

```
inner = dict[F, S]
```

to return a type-stable `DoubleDictInner`.

source

`MathOptInterface.Utilities.DoubleDicts.DoubleDictInner` – Type.

```
DoubleDictInner{F,S,V}
```

A type stable inner dictionary of `DoubleDict`.

source

`MathOptInterface.Utilities.DoubleDicts.IndexDoubleDict` – Type.

```
IndexDoubleDict
```

A specialized version of `[DoubleDict]` in which the values are of type `MOI.ConstraintIndex`

When looping through multiple keys of the same Function-in-Set type, use

```
inner = dict[F, S]
```

to return a type-stable `IndexDoubleDictInner`.

source

`MathOptInterface.Utilities.DoubleDicts.IndexDoubleDictInner` – Type.

```
IndexDoubleDictInner{F,S}
```

A type stable inner dictionary of `IndexDoubleDict`.

source

`MathOptInterface.Utilities.DoubleDicts.outer_keys` – Function.

```
outer_keys(d::AbstractDoubleDict)
```

Return an iterator over the outer keys of the `AbstractDoubleDict` `d`. Each outer key is a `Tuple{Type,Type}` so that a double loop can be easily used:

```
for (F, S) in DoubleDicts.outer_keys(dict)
    for (k, v) in dict[F, S]
        # ...
    end
end
```

For performance, it is recommended that the inner loop lies in a separate function to guarantee type-stability. Some outer keys `(F, S)` might lead to an empty `dict[F, S]`. If you want only nonempty `dict[F, S]`, use `nonempty_outer_keys`.

source

`MathOptInterface.Utilities.DoubleDicts.nonempty_outer_keys` – Function.

```
nonempty_outer_keys(d::AbstractDoubleDict)
```

Return a vector of outer keys of the `AbstractDoubleDict` `d`.

Only outer keys that have a nonempty set of inner keys will be returned.

Each outer key is a `Tuple{Type,Type}` so that a double loop can be easily used

```
for (F, S) in DoubleDicts.nonempty_outer_keys(dict)
    for (k, v) in dict[F, S]
        # ...
    end
end
For performance, it is recommended that the inner loop lies in a separate
function to guarantee type-stability.

If you want an iterator of all current outer keys, use [`outer_keys`](@ref).
```

source

# Chapter 31

# Test

## 31.1 Overview

### The Test submodule

The `Test` submodule provides tools to help solvers implement unit tests in order to ensure they implement the MathOptInterface API correctly, and to check for solver-correctness.

We use a centralized repository of tests, so that if we find a bug in one solver, instead of adding a test to that particular repository, we add it here so that all solvers can benefit.

#### How to test a solver

The skeleton below can be used for the wrapper test file of a solver named `FooBar`.

```
# ============================ /test/MOI_wrapper.jl ============================
module TestFooBar

import FooBar
using Test

import MathOptInterface as MOI

const OPTIMIZER = MOI.instantiate(
    MOI.OptimizerWithAttributes(FooBar.Optimizer, MOI.Silent() => true),
)

const BRIDGED = MOI.instantiate(
    MOI.OptimizerWithAttributes(FooBar.Optimizer, MOI.Silent() => true),
    with_bridge_type = Float64,
)

# See the docstring of MOI.Test.Config for other arguments.
const CONFIG = MOI.Test.Config(
    # Modify tolerances as necessary.
    atol = 1e-6,
    rtol = 1e-6,
    # Use MOI.LOCALLY_SOLVED for local solvers.
    optimal_status = MOI.OPTIMAL,
    # Pass attributes or MOI functions to `exclude` to skip tests that
    # rely on this functionality.
```

```
    exclude = Any[MOI.VariableName, MOI.delete],
)

"""
    runtests()

This function runs all functions in the this Module starting with `test_`.
"""
function runtests()
    for name in names(@__MODULE__; all = true)
        if startswith("$(name)", "test_")
            @testset "$(name)" begin
                getfield(@__MODULE__, name)()
            end
        end
    end
end

"""
    test_runtests()

This function runs all the tests in MathOptInterface.Test.

Pass arguments to `exclude` to skip tests for functionality that is not
implemented or that your solver doesn't support.
"""
function test_runtests()
    MOI.Test.runtests(
        BRIDGED,
        CONFIG,
        exclude = [
            "test_attribute_NumberOfThreads",
            "test_quadratic_",
        ],
        # This argument is useful to prevent tests from failing on future
        # releases of MOI that add new tests. Don't let this number get too far
        # behind the current MOI release though. You should periodically check
        # for new tests to fix bugs and implement new features.
        exclude_tests_after = v"0.10.5",
    )
    return
end

"""
    test_SolverName()

You can also write new tests for solver-specific functionality. Write each new
test as a function with a name beginning with `test_`.
"""
function test_SolverName()
    @test MOI.get(FooBar.Optimizer(), MOI.SolverName()) == "FooBar"
    return
end

end # module TestFooBar
```

```
# This line at tne end of the file runs all the tests!
TestFooBar.runtests()
```

Then modify your `runtests.jl` file to include the `MOI_wrapper.jl` file:

```
# ========================== /test/runtests.jl ============================

using Test

@testset "MOI" begin
    include("test/MOI_wrapper.jl")
end
```

**Info**

The optimizer `BRIDGED` constructed with `instantiate` automatically bridges constraints that are not supported by `OPTIMIZER` using the bridges listed in Bridges. It is recommended for an implementation of MOI to only support constraints that are natively supported by the solver and let bridges transform the constraint to the appropriate form. For this reason it is expected that tests may not pass if `OPTIMIZER` is used instead of `BRIDGED`.

**How to debug a failing test**

When writing a solver, it's likely that you will initially fail many tests. Some failures will be bugs, but other failures you may choose to exclude.

There are two ways to exclude tests:

- Exclude tests whose names contain a string using:

  ```
  MOI.Test.runtests(
      model,
      config;
      exclude = String["test_to_exclude", "test_conic_"],
  )
  ```

  This will exclude tests whose name contains either of the two strings provided.

- Exclude tests which rely on specific functionality using:

  ```
  MOI.Test.Config(exclude = Any[MOI.VariableName, MOI.optimize!])
  ```

  This will exclude tests which use the `MOI.VariableName` attribute, or which call `MOI.optimize!`.

Each test that fails can be independently called as:

```
model = FooBar.Optimizer()
config = MOI.Test.Config()
MOI.empty!(model)
MOI.Test.test_category_name_that_failed(model, config)
```

You can look-up the source code of the test that failed by searching for it in the `src/Test/test_category.jl` file.

**Tip**

Each test function also has a docstring that explains what the test is for. Use `?MOI.Test.test_category_name_that_failed` from the REPL to read it.

Periodically, you should re-run excluded tests to see if they now pass. The easiest way to do this is to swap the `exclude` keyword argument of `runtests` to `include`. For example:

```
MOI.Test.runtests(
    model,
    config;
    exclude = String["test_to_exclude", "test_conic_"],
)
```

becomes

```
MOI.Test.runtests(
    model,
    config;
    include = String["test_to_exclude", "test_conic_"],
)
```

#### How to add a test

To detect bugs in solvers, we add new tests to `MOI.Test`.

As an example, ECOS errored calling `optimize!` twice in a row. (See ECOS.jl PR #72.) We could add a test to ECOS.jl, but that would only stop us from re-introducing the bug to ECOS.jl in the future, but it would not catch other solvers in the ecosystem with the same bug. Instead, if we add a test to `MOI.Test`, then all solvers will also check that they handle a double optimize call.

For this test, we care about correctness, rather than performance. therefore, we don't expect solvers to efficiently decide that they have already solved the problem, only that calling `optimize!` twice doesn't throw an error or give the wrong answer.

#### Step 1

Install the `MathOptInterface` julia package in dev mode:

```
julia> ]
(@v1.6) pkg> dev MathOptInterface
```

#### Step 2

From here on, proceed with making the following changes in the `~/.julia/dev/MathOptInterface` folder (or equivalent dev path on your machine).

**Step 3**

Since the double-optimize error involves solving an optimization problem, add a new test to src/Test/test_solve.jl:

```
"""
    test_unit_optimize!_twice(model::MOI.ModelLike, config::Config)

Test that calling `MOI.optimize!` twice does not error.

This problem was first detected in ECOS.jl PR#72:
https://github.com/jump-dev/ECOS.jl/pull/72
"""
function test_unit_optimize!_twice(
    model::MOI.ModelLike,
    config::Config{T},
) where {T}
    # Use the `@requires` macro to check conditions that the test function
    # requires to run. Models failing this `@requires` check will silently skip
    # the test.
    @requires MOI.supports_constraint(
        model,
        MOI.VariableIndex,
        MOI.GreaterThan{Float64},
    )
    @requires _supports(config, MOI.optimize!)
    # If needed, you can test that the model is empty at the start of the test.
    # You can assume that this will be the case for tests run via `runtests`.
    # User's calling tests individually need to call `MOI.empty!` themselves.
    @test MOI.is_empty(model)
    # Create a simple model. Try to make this as simple as possible so that the
    # majority of solvers can run the test.
    x = MOI.add_variable(model)
    MOI.add_constraint(model, x, MOI.GreaterThan(one(T)))
    MOI.set(model, MOI.ObjectiveSense(), MOI.MIN_SENSE)
    MOI.set(
        model,
        MOI.ObjectiveFunction{MOI.VariableIndex}(),
        x,
    )
    # The main component of the test: does calling `optimize!` twice error?
    MOI.optimize!(model)
    MOI.optimize!(model)
    # Check we have a solution.
    @test MOI.get(model, MOI.TerminationStatus()) == MOI.OPTIMAL
    # There is a three-argument version of `Base.isapprox` for checking
    # approximate equality based on the tolerances defined in `config`:
    @test isapprox(MOI.get(model, MOI.VariablePrimal(), x), one(T), config)
    # For code-style, these tests should always `return` `nothing`.
    return
end
```

**Info**

Make sure the function is agnostic to the number type T; don't assume it is a `Float64` capable solver.

We also need to write a test for the test. Place this function immediately below the test you just wrote in the same file:

```
function setup_test(
    ::typeof(test_unit_optimize!_twice),
    model::MOI.Utilities.MockOptimizer,
    ::Config,
)
    MOI.Utilities.set_mock_optimize!(
        model,
        (mock::MOI.Utilities.MockOptimizer) -> MOIU.mock_optimize!(
            mock,
            MOI.OPTIMAL,
            (MOI.FEASIBLE_POINT, [1.0]),
        ),
    )
    return
end
```

Finally, you also need to implement `Test.version_added`. If we added this test when the latest released version of MOI was `v0.10.5`, define:

```
version_added(::typeof(test_unit_optimize!_twice)) = v"0.10.6"
```

#### Step 6

Commit the changes to git from `~/.julia/dev/MathOptInterface` and submit the PR for review.

**Tip**

If you need help writing a test, open an issue on GitHub, or ask the Developer Chatroom.

## 31.2 API Reference

### The Test submodule

Functions to help test implementations of MOI. See The Test submodule for more details.

`MathOptInterface.Test.Config` – Type.

```
Config(
    ::Type{T} = Float64;
    atol::Real = Base.rtoldefault(T),
    rtol::Real = Base.rtoldefault(T),
    optimal_status::MOI.TerminationStatusCode = MOI.OPTIMAL,
    infeasible_status::MOI.TerminationStatusCode = MOI.INFEASIBLE,
    exclude::Vector{Any} = Any[],
) where {T}
```

Return an object that is used to configure various tests.

**Configuration arguments**

- `atol::Real = Base.rtoldefault(T)`: Control the absolute tolerance used when comparing solutions.
- `rtol::Real = Base.rtoldefault(T)`: Control the relative tolerance used when comparing solutions.
- `optimal_status = MOI.OPTIMAL`: Set to `MOI.LOCALLY_SOLVED` if the solver cannot prove global optimality.
- `infeasible_status = MOI.INFEASIBLE`: Set to `MOI.LOCALLY_INFEASIBLE` if the solver cannot prove global infeasibility.
- `exclude = Vector{Any}`: Pass attributes or functions to `exclude` to skip parts of tests that require certain functionality. Common arguments include:
  - `MOI.delete` to skip deletion-related tests
  - `MOI.optimize!` to skip optimize-related tests
  - `MOI.ConstraintDual` to skip dual-related tests
  - `MOI.VariableName` to skip setting variable names
  - `MOI.ConstraintName` to skip setting constraint names

**Example**

For a nonlinear solver that finds local optima and does not support finding dual variables or constraint names:

```
julia> config = MOI.Test.Config(
           Float64;
           optimal_status = MOI.LOCALLY_SOLVED,
           exclude = Any[
               MOI.ConstraintDual,
               MOI.VariableName,
               MOI.ConstraintName,
               MOI.delete,
           ],
       );
```

source

`MathOptInterface.Test.runtests` – Function.

```
runtests(
    model::MOI.ModelLike,
    config::Config;
    include::Vector{Union{String,Regex}} = String[],
    exclude::Vector{Union{String,Regex}} = String[],
    warn_unsupported::Bool = false,
    exclude_tests_after::VersionNumber = v"999.0.0",
    verbose::Bool = false,
    test_module = MathOptInterface.Test,
)
```

Run all tests in `test_module`, which defaults to `MathOptInterface.Test`, on `model`.

**Configuration arguments**

- `config` is a `Test.Config` object that can be used to modify the behavior of tests.
- If `include` is not empty, only run tests if an element from `include` `occursin` the name of the test.
- If `exclude` is not empty, skip tests if an element from `exclude` `occursin` the name of the test.
- `exclude` takes priority over `include`.
- If `warn_unsupported` is `false`, `runtests` will silently skip tests that fail with a `MOI.NotAllowedError`, `MOI.UnsupportedError`, or `RequirementUnmet` error. (The latter is thrown when an `@requires` statement returns `false`.) When `warn_unsupported` is `true`, a warning will be printed. For most cases the default behavior, `false`, is what you want, since these tests likely test functionality that is not supported by `model`. However, it can be useful to run `warn_unsupported = true` to check you are not skipping tests due to a missing `supports_constraint` method or equivalent.
- `exclude_tests_after` is a version number that excludes any tests to MOI added after that version number. This is useful for solvers who can declare a fixed set of tests, and not cause their tests to break if a new patch of MOI is released with a new test.
- `verbose` is a `Bool` that controls whether the name of the test is printed before executing it. This can be helpful when debugging.
- `test_module` is a `Module` where all the functions starting with `test_` are considered as tests.

See also: `setup_test`.

**Example**

```
config = MathOptInterface.Test.Config()
MathOptInterface.Test.runtests(
    model,
    config;
    include = ["test_linear_", r"^test_model_Name$"],
    exclude = ["VariablePrimalStart"],
    warn_unsupported = true,
    verbose = true,
    exclude_tests_after = v"0.10.5",
)
```

source

`MathOptInterface.Test.setup_test` – Function.

```
setup_test(::typeof(f), model::MOI.ModelLike, config::Config)
```

Overload this method to modify `model` before running the test function `f` on `model` with `config`. You can also modify the fields in `config` (for example, to loosen the default tolerances).

This function should either return `nothing`, or return a function which, when called with zero arguments, undoes the setup to return the model to its previous state. You do not need to undo any modifications to `config`.

This function is most useful when writing new tests of the tests for MOI, but it can also be used to set test-specific tolerances, etc.

See also: runtests

**Example**

```
function MOI.Test.setup_test(
    ::typeof(MOI.Test.test_linear_VariablePrimalStart_partial),
    mock::MOIU.MockOptimizer,
    ::MOI.Test.Config,
)
    MOIU.set_mock_optimize!(
        mock,
        (mock::MOIU.MockOptimizer) -> MOIU.mock_optimize!(mock, [1.0, 0.0]),
    )
    mock.eval_variable_constraint_dual = false

    function reset_function()
        mock.eval_variable_constraint_dual = true
        return
    end
    return reset_function
end
```

source

MathOptInterface.Test.version_added – Function.

```
version_added(::typeof(function_name))
```

Returns the version of MOI in which the test function_name was added.

This method should be implemented for all new tests.

See the exclude_tests_after keyword of runtests for more details.

source

MathOptInterface.Test.@requires – Macro.

```
@requires(x)
```

Check that the condition x is true. Otherwise, throw an RequirementUnmet error to indicate that the model does not support something required by the test function.

**Example**

```
@requires MOI.supports(model, MOI.Silent())
@test MOI.get(model, MOI.Silent())
```

source

MathOptInterface.Test.RequirementUnmet – Type.

```
RequirementUnmet(msg::String) <: Exception
```

An error for throwing in tests to indicate that the model does not support some requirement expected by the test function.

source

`MathOptInterface.Test.HS071` – Type.

```
HS071(
    enable_hessian::Bool,
    enable_hessian_vector_product::Bool = false,
)
```

An `MOI.AbstractNLPEvaluator` for the problem:

$$\begin{aligned} \min \quad & x_1 * x_4 * (x_1 + x_2 + x_3) + x_3 \\ \text{subject to} \quad & x_1 * x_2 * x_3 * x_4 \geq 25 \\ & x_1^2 + x_2^2 + x_3^2 + x_4^2 = 40 \\ & 1 \leq x_1, x_2, x_3, x_4 \leq 5 \end{aligned}$$

The optimal solution is `[1.000, 4.743, 3.821, 1.379]`.

source

# Part VII

# Developer Docs

# Chapter 32

# Checklists

The purpose of this page is to collate a series of checklists for commonly performed changes to the source code of MathOptInterface.

In each case, copy the checklist into the description of the pull request.

## 32.1 Making a release

Use this checklist when making a release of the MathOptInterface repository.

```
## Basic

 - [ ] `version` field of `Project.toml` has been updated
       - If a breaking change, increment the MAJOR field and reset others to 0
       - If adding new features, increment the MINOR field and reset PATCH to 0
       - If adding bug fixes or documentation changes, increment the PATCH field

## Documentation

 - [ ] Add a new entry to `docs/src/changelog.md`, following existing style

## Tests

 - [ ] The `solver-tests.yml` GitHub action does not have unexpected failures.
       To run the action, go to:
       https://github.com/jump-dev/MathOptInterface.jl/actions/workflows/solver-tests.yml
       and click "Run workflow"
 - [ ] If new tests were added, ensure that `MOI.Test.version_added` is implemented.
```

## 32.2 Adding a new set

Use this checklist when adding a new set to the MathOptInterface repository.

```
## Basic

 - [ ] Add a new `AbstractScalarSet` or `AbstractVectorSet` to `src/sets.jl`
 - [ ] If `isbitstype(S) == false`, implement `Base.copy(set::S)`
 - [ ] If `isbitstype(S) == false`, implement `Base.:(==)(x::S, y::S)`
 - [ ] If an `AbstractVectorSet`, implement `dimension(set::S)`, unless the
```

```
       dimension is given by `set.dimension`.
 - [ ] Ensure the set does not contain references to any variables or constraints

## Utilities

 - [ ] If an `AbstractVectorSet`, implement `Utilities.set_dot`,
       unless the dot product between two vectors in the set is equivalent to
       `LinearAlgebra.dot`
 - [ ] If an `AbstractVectorSet`, implement `Utilities.set_with_dimension` in
       `src/Utilities/matrix_of_constraints.jl`
 - [ ] Add the set to the `@model` macro at the bottom of `src/Utilities/model.jl`

## Documentation

 - [ ] Add a docstring, which gives the mathematical definition of the set,
       along with an `## Example` block containing a `jldoctest`
 - [ ] Add the docstring to `docs/src/reference/standard_form.md`
 - [ ] Add the set to the relevant table in `docs/src/manual/standard_form.md`

## Tests

 - [ ] Define a new `_set(::Type{S})` method in `src/Test/test_basic_constraint.jl`
       and add the name of the set to the list at the bottom of that files
 - [ ] If the set has any checks in its constructor, add tests to `test/sets.jl`

## MathOptFormat

 - [ ] Open an issue at `https://github.com/jump-dev/MathOptFormat` to add
       support for the new set {{ replace with link to the issue }}

## Optional

 - [ ] Implement `dual_set(::S)` and `dual_set_type(::Type{S})`
 - [ ] Add new tests to the `Test` submodule exercising your new set
 - [ ] Add new bridges to convert your set into more commonly used sets
```

## 32.3 Adding a new bridge

Use this checklist when adding a new bridge to the MathOptInterface repository.

The steps are mostly the same, but locations depend on whether the bridge is a `Constraint`, `Objective`, or `Variable` bridge. In each case below, replace `XXX` with the appropriate type of bridge.

```
## Basic

 - [ ] Create a new file in `src/Bridges/XXX/bridges` named after the type of
       the bridge
 - [ ] Define the bridge, following existing examples. The name of the bridge
       struct must end in `Bridge`
 - [ ] Check if your bridge can be a subtype of [`MOI.Bridges.Constraint.SetMapBridge`](@ref)
 - [ ] Define a new `const` that is a `SingleBridgeOptimizer` wrapping the
       new bridge. The name of the const must be the name of the bridge, less
       the `Bridge` suffix
 - [ ] If the bridge should be enabled by default, add the bridge to
```

```
       `add_all_bridges` at the bottom of `src/Bridges/XXX/XXX.jl`

## Tests

 - [ ] Create a new file in the appropriate subdirectory of `tests/Bridges/XXX`
       named after the type of the bridge
 - [ ] Use `MOI.Bridges.runtests` to test various inputs and outputs of the
       bridge
 - [ ] If, after opening the pull request to add the bridge, some lines are not
       covered by the tests, add additional bridge-specific tests to cover the
       untested lines.

## Documentation

 - [ ] Add a docstring which uses the same template as existing bridges.

## Final touch

If the bridge depends on run-time values of other variables and constraints in
the model:

 - [ ] Implement `MOI.Utilities.needs_final_touch(::Bridge)`
 - [ ] Implement `MOI.Utilities.final_touch(::Bridge, ::MOI.ModelLike)`
 - [ ] Ensure that `final_touch` can be called multiple times in a row
```

## 32.4 Updating MathOptFormat

Use this checklist when updating the version of MathOptFormat.

```
## Basic

 - [ ] The file at `src/FileFormats/MOF/mof.schema.json` is updated
 - [ ] The constant `_SUPPORTED_VERSIONS` is updated in
       `src/FileFormats/MOF/MOF.jl`

## New sets

 - [ ] New sets are added to the `@model` in `src/FileFormats/MOF/MOF.jl`
 - [ ] New sets are added to the `@enum` in `src/FileFormats/MOF/read.jl`
 - [ ] `set_to_moi` is defined for each set in `src/FileFormats/MOF/read.jl`
 - [ ] `head_name` is defined for each set in `src/FileFormats/MOF/write.jl`
 - [ ] A new unit test calling `_test_model_equality` is added to
       `test/FileFormats/MOF/test_MOF.jl`

## Tests

 - [ ] The version field in `test/FileFormats/MOF/nlp.mof.json` is updated

## Documentation

 - [ ] The version fields are updated in `docs/src/submodules/FileFormats/overview.md`
```

# Chapter 33

# Contributing

This document explains how to contribute code to MathOptInterface.

## 33.1 Obtain the source code

The easiest way to obtain the source code for MathOptInterface is to run:

```
julia> import Pkg

julia> Pkg.dev("MathOptInterface")
```

This will download the MathOptInterface Git repository to `~/.julia/dev/MathOptInterface`. If you're on Windows, this will be `C:\\Users\\<my_name>\\.julia\\dev\\MathOptInterface`.

Alternatively, you can use `git` to clone the source to a directory of your choosing:

```
$ cd /some/local/path
$ git clone https://github.com/jump-dev/MathOptInterface.jl.git
```

## 33.2 Make changes

Code development in MathOptInterface follows typical git development practices.

First, go to https://github.com/jump-dev/MathOptInterface.jl and click the "Fork" button in the top-right corner. This will create a copy of MathOptInterface under your GitHub account.

Then, tell your local git about your new fork:

```
export GITHUB_ACCOUNT="odow"  # Replace with your name
cd ~/.julia/dev/MathOptInterface
git remote add ${GITHUB_ACCOUNT} https://github.com/${GITHUB_ACCOUNT}/MathOptInterface.jl.git
```

Before making changes to the source, ensure that you have the latest copy, and checkout a new branch:

```
git checkout master
git pull
git checkout -b my_new_branch
```

Now you can make changes by editing the local source code. See Running the tests for how to check if your code passes the tests.

Once you have finished making changes, make a commit and push your branch to GitHub:

```
git add .
git commit -m "A message describing what you changed"
git push -u $GITHUB_ACCOUNT my_new_branch
```

Finally, go to https://github.com/jump-dev/MathOptInterface.jl and follow the instructions that pop up to open a pull request.

## 33.3 Running the tests

There are lot of tests in MathOptInterface. Running them in their entirety can take a long time.

The easiest way to run the tests is to run:

```
julia> import Pkg

julia> Pkg.test("MathOptInterface")
```

The downside to the approach is that it will run all of the tests, and it will recompile MathOptInterface from scratch, even if you have made very trivial changes to the source code.

A faster approach is to use Revise.jl.

First, install Revise in your global package environment:

```
$ julia

julia> import Pkg

julia> Pkg.add("Revise")
```

Then, cd to the source code of MathOptInterface, and start Julia with the project set to `.` (for `~/.julia/dev/MathOptInterface/Pr`

```
$ cd ~/.julia/dev/MathOptInterface
$ julia --project=.
```

To run the tests, load Revise, and then include the relevant test file. For example:

```
julia> using Revise

julia> include("test/Nonlinear/test_Nonlinear.jl")

julia> # make changes to `src/Nonlinear`

julia> include("test/Nonlinear/test_Nonlinear.jl")
```

You can also run other tests such as `test/Bridges/Constraint/test_bridge.jl`, or `test/Utilities/test_distance_to_set.j`

There is one complication: `JSONSchema` is a test-time dependency that is not present in the default project. If you want to run `test/FileFormats/MOF/test_MOF.jl`, you will first need to install the package (`Pkg.test("MathOptInterface")` does this automatically):

```
julia> import Pkg; Pkg.add("JSONSchema")
```

If you have installed JSONSchema, you can also run all the tests with `include("test/runtests.jl")`.

Finally, running the tests locally is best practice, but it is not required. When you open a pull request, our automated CI will run all of the tests and highlight any failing tests that need to be fixed.

A comment from @odow: when I'm working on a feature, I make local changes, run the most relevant test file with `include`, and then open a PR. If the change caused a test to fail in some other part of the codebase, I then `include` the file with the failing test locally to debug my follow-up changes.

## 33.4 Building the documentation

Building the documentation follows a similar practice with Revise. First, `cd` to the source code of MathOptInterface, but this time, start Julia with `--project=docs`.

```
$ cd ~/.julia/dev/MathOptInterface

$ julia --project=docs
```

Then, ensure that MathOptInterface (in our current directory) has been added as a development dependency:

```
julia> import Pkg; Pkg.dev(".")
[...lines omitted...]
```

Finally, load `Revise` and then build the docs:

```
julia> using Revise

julia> include("docs/make.jl")
```

If you make changes to the docs or to the Julia source code (for example, the docstrings), you can re-run `include` to quickly rebuild the docs.

# Chapter 34

# Defining a new set

The easiest way to extend the behavior of MathOptInterface is to design a new set. The purpose of this tutorial is to explain how to define a new set and some of the associated nuances.

Defining a new function is an order of magnitude (if not two) harder than defining a new set. Don't consider it as an option unless you have already tried to support your behavior as a set.

As a motivation for this tutorial, we consider the LinMax constraint type, which appears often in constraint programming:

$$t = \max(f_1(x), f_2(x), \ldots, f_N(x))$$

The first step to design a new set for MathOptInterface is to define the mathematical relationship you want to model as a function-in-set $f(x) \in S$.

Your initial thought for representing the LinMax constraint in MathOptInterface may be to represent it as:

$$F(x) \in LinMax(t)$$

where $F(x)$ is the vector-valued function created by concatenating the $f_i$ functions. This formulation violates a basic rule of MathOptInterface:

**Rule**

Sets cannot contain references to decision variables or other constraints.

Instead, we could model the LinMax constraint as:

$$[t, f_1(x), f_2(x), \ldots, f_N(x)] \in LinMax(N+1)$$

In the language of MathOptInterface, this is a `AbstractVectorFunction` in the `LinMax` set of dimension $N+1$. The type of the function depends on the types of the component scalar functions, with a special convention that the first element in the function is interpretable as a `VariableIndex` `t`.

Now `LinMax` can be trivially defined as a new `AbstractVectorSet`:

```
julia> import MathOptInterface as MOI

julia> struct LinMax <: MOI.AbstractVectorSet
           dimension::Int
       end
```

and it can immediately be used in MathOptInterface:

```
julia> model = MOI.Utilities.UniversalFallback(MOI.Utilities.Model{Float64}());

julia> t = MOI.VariableIndex(1);

julia> x = MOI.VariableIndex.(2:3);

julia> F = 1.0 .* x .+ 2.0;

julia> g = MOI.Utilities.operate(vcat, Float64, t, F...);

julia> MOI.add_constraint(model, g, LinMax(3))
MathOptInterface.ConstraintIndex{MathOptInterface.VectorAffineFunction{Float64}, LinMax}(1)

julia> print(model)
Feasibility

Subject to:

VectorAffineFunction{Float64}-in-LinMax
 ┌             ┐
 │0.0 + 1.0 v[1]│
 │2.0 + 1.0 v[2]│
 │2.0 + 1.0 v[3]│
 └             ┘ ∈ LinMax(3)
```